

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ _____ ” _____ 2025 р.

Дипломний проєкт

на здобуття ступеня бакалавра

за освітньо-професійною програмою «Інженерія програмного забезпечення
інформаційних систем»

спеціальності «121 Інженерія програмного забезпечення»

на тему: Вебзастосунок для планування та нагадування про важливі події

Виконав студент IV курсу, групи ІП-12
(шифр групи)

Орищенко Ярослав Олександрович
(прізвище, ім'я, по батькові)

_____ (підпис)

Керівник старший викладач, Ковтунець О. В.
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Консультант старший викладач, Головченко М. М.
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Рецензент доцент кафедри ІСТ, к.т.н., Галушко Д. О.
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Засвідчую, що у цьому дипломному проєкті
немає запозичень з праць інших авторів без
відповідних посилань.

Студент _____
(підпис)

Київ – 2025

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ” _____ 2025 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Орищенко Ярославу Олександровичу

(прізвище, ім'я, по батькові)

Тема проєкту Вебзастосунок для планування та нагадування про важливі події

керівник проєкту Ковтунець Олесь Володимирович, старший викладач
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «23» травня 2025 р. №1705-с

2. Термін подання студентом проєкту «16» червня 2025 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Передпроектне обстеження області: постановка задачі дипломного проектування, аналіз предметної області, аналіз відомих програмних продуктів, аналіз існуючих алгоритмічних та технічних рішень, аналіз та моделювання бізнес-процесів.

2) Розроблення вимог до програмного забезпечення: варіанти використання програмного забезпечення, розроблення функціональних вимог, розроблення нефункціональних вимог, аналіз системних вимог, аналіз економічних показників програмного забезпечення, постановка завдання на розробку програмного забезпечення.

3) Конструювання та розроблення програмного забезпечення: Архітектура програмного забезпечення, архітектурні рішення та обґрунтування вибору засобів розробки, опис структури бази даних, аналіз безпеки даних.

- 4) Аналіз якості та тестування програмного забезпечення: аналіз якості ПЗ, опис процесів тестування, опис контрольного прикладу. _____
- 5) Розгортання та супровід програмного забезпечення: розгортання програмного забезпечення, супровід програмного забезпечення. _____

5. Перелік графічного матеріалу

- 1) Схема структурна варіантів використань _____
- 2) Схема структурна компонентів програмного забезпечення _____
- 3) Схема бази даних _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «15» березня 2025 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	15.03.2025	
2	Аналіз існуючих методів розв'язання задачі	24.03.2025	
3	Постановка та формалізація задачі	31.03.2025	
4	Розробка інформаційного забезпечення	07.04.2025	
5	Обґрунтування вибору використаних технічних засобів	14.04.2025	
6	Розробка програмного забезпечення	21.04.2025	
7	Налагодження програми	05.05.2025	
8	Виконання графічних документів	12.05.2025	
9	Оформлення пояснювальної записки	19.05.2025	
10	Подання ДП на попередній захист	02.06.2025	
11	Подання ДП рецензенту	10.06.2025	
12	Подання ДП на основний захист	16.06.2025	

Студент

(підпис)

Ярослав ОРИЩЕНКО

(ініціали, прізвище)

Керівник

(підпис)

Олесь КОВТУНЕЦЬ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з п'яти розділів, містить 57 таблиць, 56 рисунків та 27 джерел – загалом 108 сторінок.

Дипломний проєкт присвячений розробці вебзастосунку для планування подій та нагадувань.

Мета розробки полягає у спрощенні управління подіями, нагадуваннями та взаємодії між користувачами за рахунок створення зручного вебзастосунку для планування та організації діяльності у межах університету.

У розділі «Передпроектне обстеження предметної області» проаналізовано сферу планування подій, досліджено існуючі програмні продукти та визначено їх основні переваги та недоліги.

Розділ «Розробленні вимог до програмного забезпечення» присвячений формуванню варіантів використання, функціональних та нефункціональних вимог до застосунку. Також було проведено попередній аналіз економічних показників додатку.

Розділ «Конструювання та розроблення програмного забезпечення» обґрунтовує вибір засобів розробки, описує проектування архітектури програмного забезпечення та схеми бази даних.

Розділ «Аналіз якості та тестування програмного забезпечення» описує процес тестування ждя забезпечення якості, безпеки та стабільності продукту.

Розділ «Розгортання та супровід програмного забезпечення» розглядає процес впровадження застосунку в хмарному середовищі та методи підтримки його роботи.

КЛЮЧОВІ СЛОВА: ВЕБ-ЗАСТОСУНОК, ПОДІЇ, НАГАДУВАННЯ, БАЗА ДАНИХ, DJANGO, DJANGO REST FRAMEWORK, NEXTJS.

ABSTRACT

The explanatory note of the diploma project consists of five sections, contains 57 tables, 56 figures and 27 sources – in total 108 pages.

The purpose of the diploma project is to develop a web application for events planning and sending reminders.

The purpose of the project is to simplify the management of events, reminders, and user interaction by creating a convenient web application for planning and organizing activities within the university.

The section "Pre-project study of the subject area" analyzes the field of event planning, examines existing software solutions, and identifies their key advantages and disadvantages.

The section "Development of software requirements" is devoted to the formation of use cases, functional and non-functional requirements for the application. A preliminary analysis of the application economic indicators was also conducted.

The section "Design and development of the software" justifies the choice of development tools and describes the architecture of the software system and database schema.

The section "Quality analysis and software testing" describes the testing process to ensure the quality, security, and stability of the product.

The section "Deployment and maintenance of the software" examines the process of deploying the application in a cloud environment and the methods used to support its operation.

KEYWORDS: WEB-APPLICATION, EVENTS, REMINDERS, DATABASE, DJANGO, DJANGO REST FRAMEWORK, NEXTJS.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2025 р.

**ВЕБЗАСТОСУНОК ДЛЯ ПЛАНУВАННЯ ТА НАГАДУВАННЯ ПРО
ВАЖЛИВІ ПОДІЇ
Технічне завдання
КПІ.ІІ-1219.045440.01.91**

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Ярослав ОРИЩЕНКО

Київ – 2025

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ.....	4
2	ПІДСТАВА ДЛЯ РОЗРОБКИ.....	5
3	ПРИЗНАЧЕННЯ РОЗРОБКИ.....	6
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	7
4.1	Вимоги до функціональних характеристик	7
4.1.1	Користувацький інтерфейс	7
4.1.2	Керування профілем та взаємодія із користувачами	14
4.1.3	Керування подіями	15
4.1.4	Керування нагадуваннями	16
4.1.5	Керування командами та запрошеннями	17
4.1.6	Інтеграція зовнішніх сервісів для нагадувань	17
4.2	Вимоги до надійності	18
4.3	Умови експлуатації.....	18
4.3.1	Вид обслуговування	18
4.3.2	Обслуговуючий персонал	18
4.4	Вимоги до складу і параметрів технічних засобів	18
4.5	Вимоги до інформаційної та програмної сумісності	19
4.5.1	Вимоги до вхідних даних.....	19
4.5.2	Вимоги до вихідних даних	19
4.5.3	Вимоги до мови розробки.....	19
4.5.4	Вимоги до середовища розробки.....	19
4.5.5	Вимоги до представленню вихідних кодів	19
4.6	Вимоги до маркування та пакування.....	19
4.7	Вимоги до транспортування та зберігання	20
4.8	Спеціальні вимоги	20
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	21
5.1	Попередній склад програмної документації.....	21
5.2	Спеціальні вимоги до програмної документації	21
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ.....	22

7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	23
---	-------------------------------------	----

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Вебзастосунок для планування та нагадування про важливі події.

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення «Вебзастосунок для планування та нагадування про важливі події», котрий використовується для планування подій, створення нагадувань про них та призначений для внутрішнього використання працівниками університету.

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки вебзастосунку для планування та нагадування про важливі події є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для полегшення планування та організації особистого та робочого життя в умовах диджиталізації.

Метою розробки є спрощення управління подіями, нагадуваннями та взаємодії між користувачами за рахунок створення зручного вебзастосунку для планування та організації діяльності у межах університету.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Користувацький інтерфейс

- сторінка авторизації (рисунок 4.1);

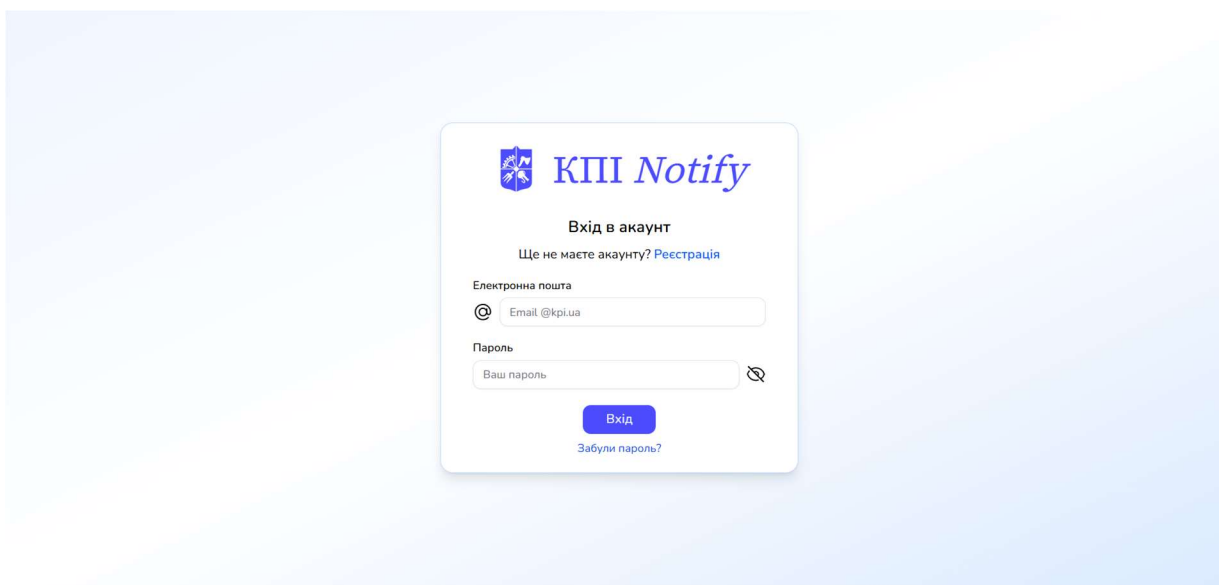


Рисунок 4.1 – Екранна форма сторінки авторизації

- реєстрація (рисунок 4.2);

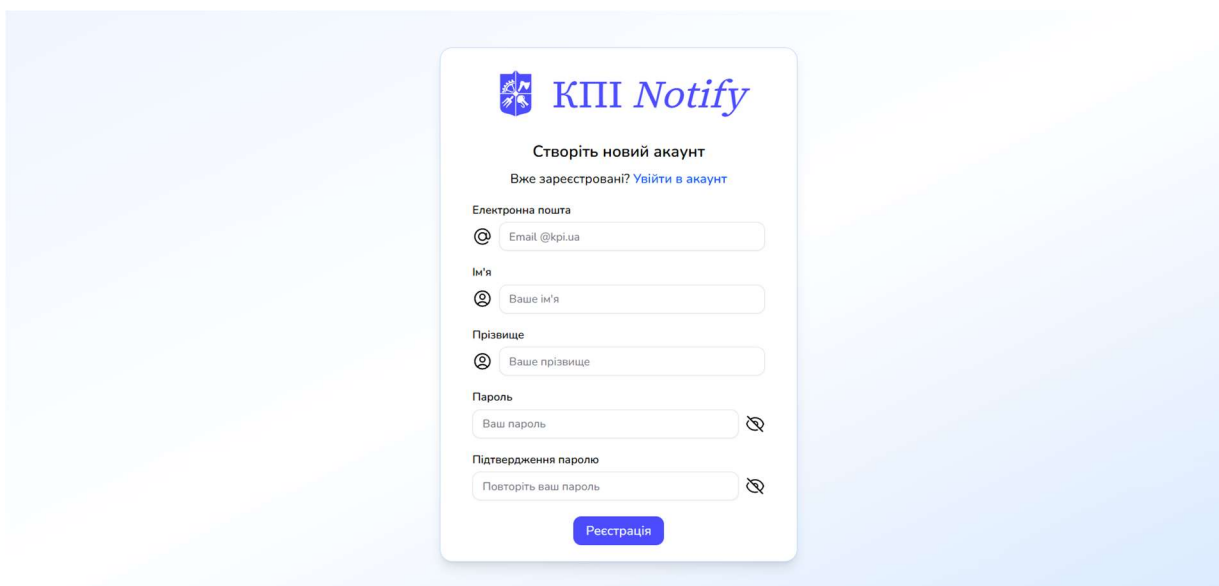


Рисунок 4.2 – Екранна форма сторінки реєстрації

- заповнення інформації про посаду (рисунки 4.3);

Рисунок 4.3 – Екранна форма налаштування профілю

- перегляд особистого профілю (рисунки 4.4);

Рисунок 4.4 – Екранна форма сторінки особистого профілю

- оновлення особистої інформації (рисунки 4.5);

Рисунок 4.5 – Екранна форма сторінки оновлення профілю

- видалення акаунту (рисунок 4.6);

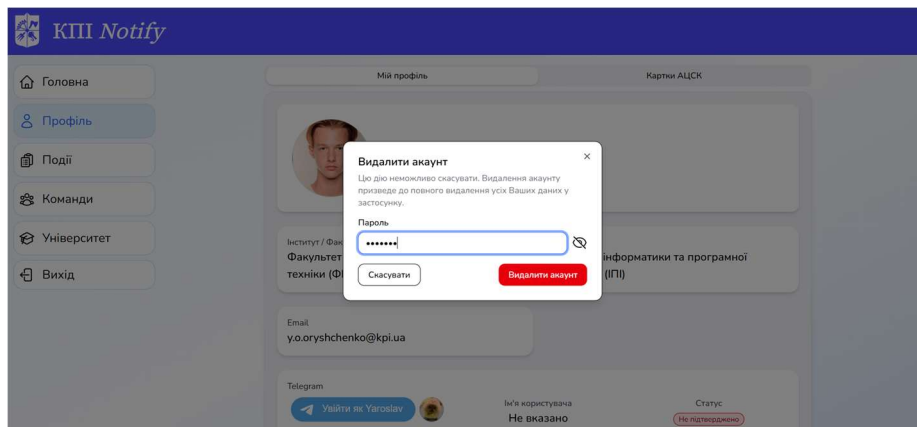


Рисунок 4.6 – Екранна форма вікна видалення акаунту

- перегляд списку збережених реєстраційних карток АЦСК (рисунок 4.7);

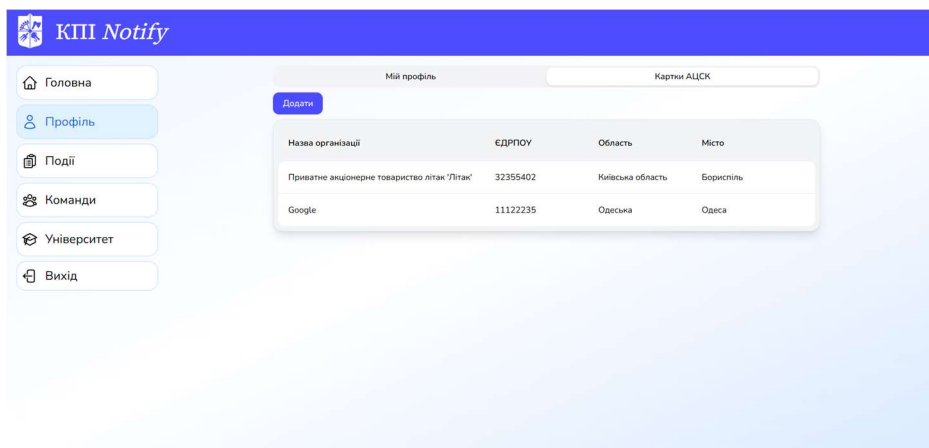


Рисунок 4.7 – Екранна форма сторінки збережених реєстраційних карток

- додавання реєстраційної картки АЦСК (рисунок 4.8);

Рисунок 4.8 – Екранна форма сторінки створення реєстраційної картки АЦСК

— перегляд деталей реєстраційної картки (рисунки 4.9, 4.10);

Реєстраційна картка АЦСК

Юридична особа

Назва організації
Приватне акціонерне товариство літак 'Літак'

Код ЄДРПОУ
32355402

Відомості про місцезнаходження

Область
Київська область

Населений пункт
Бориспіль

Дані заявника

ПІБ
Yaroslav Oryshchenko

Рисунок 4.9 – Екранна форма сторінки деталей реєстраційної картки АЦСК

Дівоче прізвище матері

Ключова фраза голосової аутентифікації
Суржико

Засоби зв'язку

Телефон
+380952146729

Електронна пошта
y.o.oryshchenko@gmail.com

Кваліфікований сертифікат електронної печатки

Назва електронної печатки
Хлопчаче прізвище матері

Ключова фраза до печатки
Блажко

Відати

Закрити

Рисунок 4.10 – Екранна форма сторінки деталей реєстраційної картки АЦСК

— перегляд списку подій (рисунки 4.11);

КПІ Notify

Ваші події

Пошук подій...

Додати подію

Майбутні Призначені Минулі

Назва	Опис	Дата і час	Пріоритет
Ранкова пробіжка	спорт здоров'я Прогіть навколо жк...	5 черв. 2025 р., 16:14	Середній
щотижнева звітність	важливо всім бути ...	6 черв. 2025 р., 18:20	Критичний
Кава з колегами	відпочинок колеги Зустрітись у кав'ярні після ро...	8 черв. 2025 р., 16:13	Низький
Зустрітись з керівником	навчання керівник Обговорити хід дипломної работ...	21 черв. 2025 р., 16:12	Високий
Забрати замовлення на НП	Це змінить моє життя...	20 лип. 2025 р., 20:02	Низький
важлива подія	важливо візна +1 дуже важлива подія яку обов'яз...	30 серп. 2025 р., 16:09	Високий

Рисунок 4.11 – Екранна форма сторінки списку подій

— перегляд деталей події (рисунок 4.12);

КПІ Notify

Деталі події

Кава з колегами
Зустрітись у кав'ярні після роботи

Дата та час: 8 черв. 2025 р., 16:13 - за 4 дні

Пріоритет: Низький

Місце: KPI Cafe

Повторювана подія: Ні

Нагадування: 7 черв. 2025 р., 22:00 - за 3 дні

Email: [input field] [Видалити]

Додати до Google Calendar

Вийти

Рисунок 4.12 – Екранна форма сторінки деталей події

— створення нової події (рисунки 4.13, 4.14);

КПІ Notify

Створіть нову подію

Назва події: [input field]

Опис: [input field]

Дата та час: [input field]

Пріоритет: Середній

Теги: [input field]

Місце: [input field]

Посилання: [input field]

Зображення: [input field]

Повторювана подія: ☒

Періодичність: Щотижня

Завершення повторень: [input field]

Нагадування: 15.06.2025 21:46 [input field] [Email] [Видалити]

Додати нагадування

Підтвердити

Рисунок 4.11 – Екранна форма сторінки створення нової події

Місце: [input field]

Посилання: [input field]

Зображення: [input field]

Повторювана подія: ☒

Періодичність: Щотижня

Завершення повторень: [input field]

Нагадування: 15.06.2025 21:46 [input field] [Email] [Видалити]

Додати нагадування

Підтвердити

Рисунок 4.12 – Екранна форма сторінки створення нової події

— перегляд списку команд (рисунок 4.13);

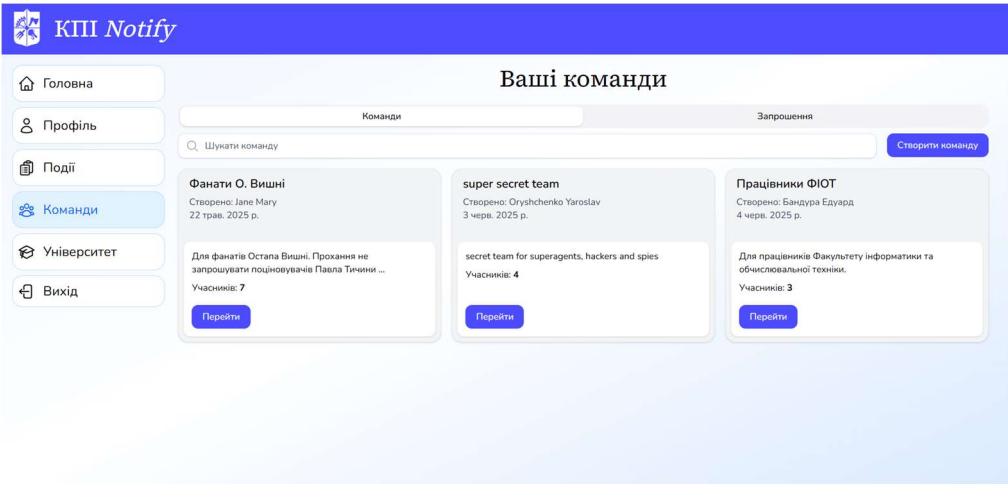


Рисунок 4.13 – Екранна форма сторінки списку команд

— перегляд деталей команди (рисунок 4.14);

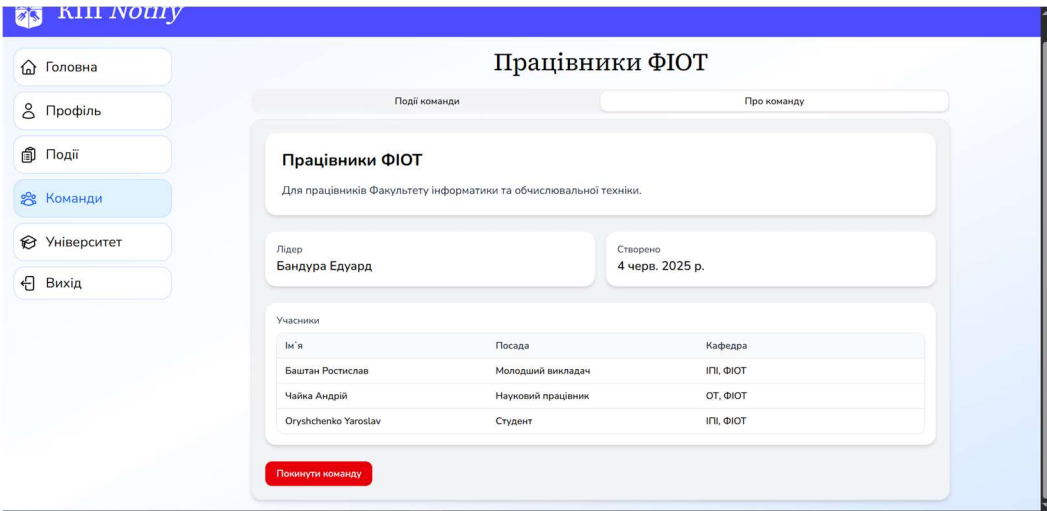


Рисунок 4.14 – Екранна форма сторінки деталей команди

— створення нової команди (рисунок 4.15);

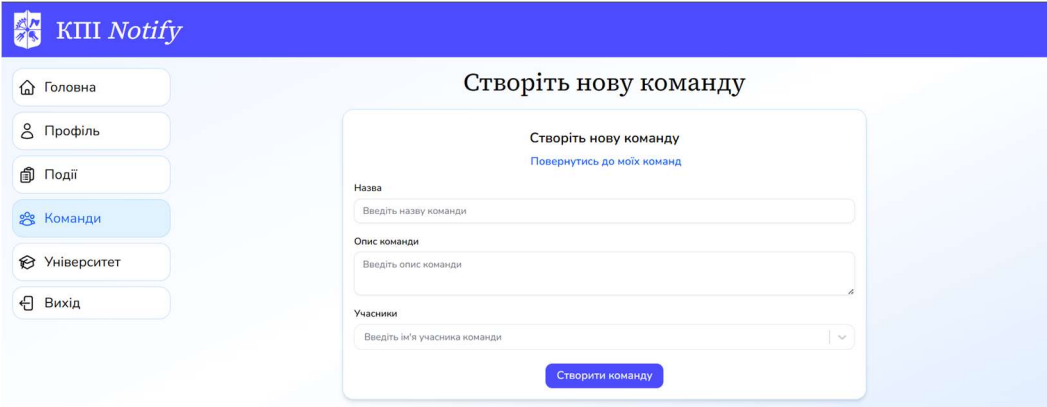


Рисунок 4.15 – Екранна форма сторінки створення нової команди

— перегляд списку запрошень у команди (рисунок 4.16);

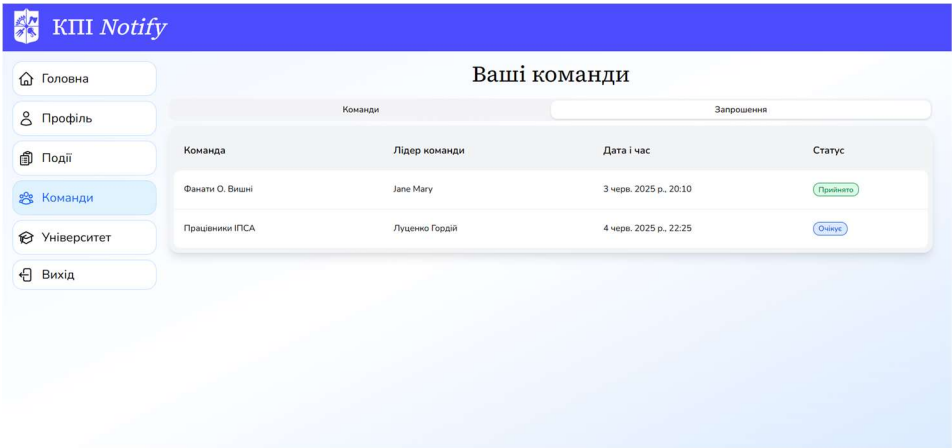


Рисунок 4.16 – Екранна форма сторінки списку запрошень у команди

— відповідь на запрошення (рисунок 4.17);

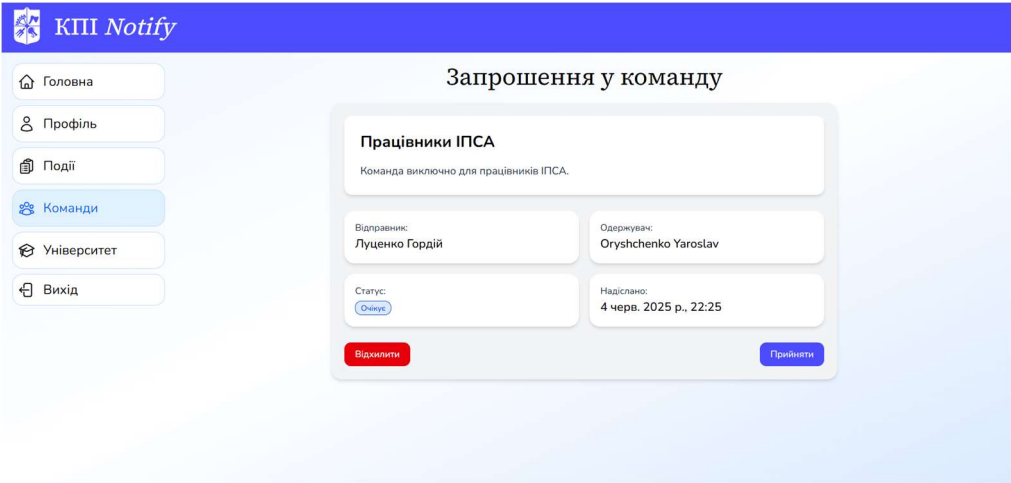


Рисунок 4.17 – Екранна форма сторінки деталей запрошення

— перегляд списку користувачів (рисунок 4.18);

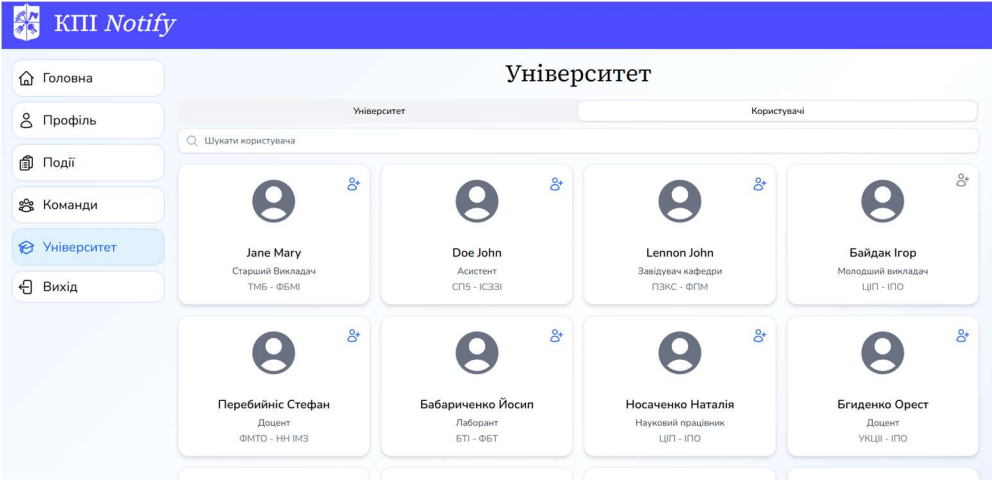


Рисунок 4.18 – Екранна форма сторінки списку користувачів

- запрошення користувача у команду (рисунок 4.19);

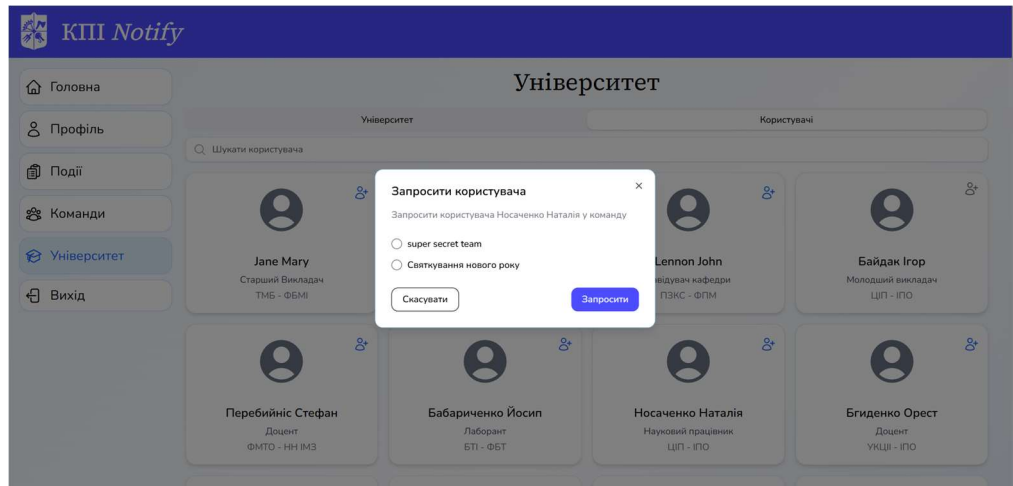


Рисунок 4.19 – Екранна форма вікна запрошення користувача у команду

4.1.2 Керування профілем та взаємодія із користувачами

- вхід в існуючий акаунт за допомогою електронної пошти та паролю;
- створення нового акаунту:
 - заповнення інформації про користувача: ім'я, прізвище;
 - заповнення даних для автентифікації: адреса електронної пошти, що має корпоративний домен університету, пароль та підтвердження паролю;
 - підтвердження створення акаунту через лист, надісланий на вказану електронну пошту;
 - заповнення даних про посаду: вибір факультету, вибір кафедри, що належить до обраного факультету, введення назви посади;
- управління профілем користувача:
 - встановлення фото профілю;
 - авторизація власного Telegram-акаунту для можливості отримувати нагадування через Telegram-бота;
 - видалення профілю та усіх даних, створених користувачем у застосунку;

- перегляд списку користувачів:
 - перегляд та пошук користувачів із усього університету;
 - перегляд та пошук користувачів із обраного інституту, факультету чи кафедри;
 - управління реєстраційними картками юридичної особи;
 - створення реєстраційної картки із інформацією про юридичну особу, її місце знаходження, дані заявника, контактна інформація, дані для формування кваліфікованого сертифікату відкритого ключа електронної печатки;
 - перегляд списку збережених реєстраційних карток;
 - перегляд деталей реєстраційної картки;
 - модифікація даних реєстраційної картки;
 - видалення реєстраційної картки.

4.1.3 Керування подіями

- створення нової події:
 - створення одноразової події;
 - створення повторюваної події із вказаною періодичністю повторення: щодня, щотижня, щомісяця, щороку;
 - заповнення основної інформації про подію: назва події, дата та час події в майбутньому, пріоритет події, обраний із чотирьох варіантів: «Низький», «Середній», «Високий» та «Критичний»;
 - заповнення додаткової інформації про подію: опис, зображення, локація, посилання, список тегів;
 - створення нагадувань для події;
- перегляд існуючих подій:
 - перегляд загального списку подій;
 - перегляд списку подій, що були призначені користувачу його керівниками;

- сортування списку подій за назвою, датою та пріоритетом події;
- пошук подій за назвою, описом та тегами події;
- перегляд деталей події:
 - перегляд повної інформації про подію та створених для неї нагадувань;
 - модифікація даних про подію;
 - перенесення події у Google Calendar;
 - видалення події, яка була створена користувачем;
 - відмова від події, яка була призначена керівником користувача.

4.1.4 Керування нагадуваннями

- створення нагадувань:
 - створення нагадувань для події під час створення події;
 - створення нагадувань для існуючої події у сторінці перегляду деталей події:
 - вибір дати та часу нагадування у майбутньому, раніше за дату та час, вказану в інформації про подію;
 - вибір методу нагадування: лист на корпоративну електронну пошту, адреса якої є логіном користувача, або повідомлення від Telegram-бота, якщо користувач авторизував свій Telegram-акаунт;
- перегляд створених нагадувань у сторінці деталей події;
- надсилання нагадувань у запланований користувачем час на електронну пошту;
- надсилання нагадувань у запланований користувачем час через Telegram-бота;
- видалення створених нагадувань у сторінці перегляду деталей події.

4.1.5 Керування командами та запрошеннями

- створення команди:
 - заповнення інформації про команду: назва та опис команди;
 - вибір списку користувачів, яких потрібно запросити у команду;
- перегляд списку команд користувача;
- пошук команди за назвою чи описом;
- перегляд деталей команди:
 - перегляд повної інформації про команду, її склад;
 - перегляд подій, призначених лідером для учасників команди;
 - перегляд надісланих запрошень для вступу у команду лідером команди;
 - модифікація інформації про команду лідером команди;
 - видалення учасників команди лідером команди;
 - виділення команди її лідером;
 - вихід учасника із команди;
- перегляд запрошень:
 - перегляд списку отриманих та надісланих запрошень;
 - перегляд деталей запрошення із інформацією про команду, адресата та адресанта, дати та часу створення, статусу запрошення;
 - видалення створеного користувачем запрошення;
 - зміна статусу запрошення користувачем, якому воно призначено: прийняття або відмова від запрошення;
- створення запрошень для користувачів із загального списку користувачів застосунку.

4.1.6 Інтеграція зовнішніх сервісів для нагадувань

- перенесення інформації про створену користувачем подію у Google Calendar;

- використання Telegram-бота для надсилання нагадувань:
 - авторизація Telegram-акаунта користувача за допомогою Telegram Login Widget;
 - надсилання повідомлення із нагадуванням про подію у запланований час у відповідний збережений Telegram-чат з користувачем, який пройшов авторизацію Telegram-акаунту.

4.2 Вимоги до надійності

Передбачити контроль введення інформації та захист від некоректних дій користувача. Забезпечити цілісність інформації в базі даних. Забезпечити відновлення запланованих завдань у разі збою системи.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Вимоги до виду обслуговування не висуваються.

4.3.2 Обслуговуючий персонал

Вимоги до обслуговуючого персоналу не висуваються.

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на IBM-сумісних персональних комп'ютерах.

Мінімальна конфігурація технічних засобів:

- Процесор: 1 гігагерц або швидший процесор чи SoC.
- Оперативна пам'ять: 1 гігабайт для 32-розрядної версії або 2 гігабайти для 64-розрядної версії.
- Обсяг дискового простору: 18 гігабайт для 32-розрядної ОС або 22 гігабайти для 64-розрядної ОС.

– Графічна плата: DirectX 9 або новішої версії з драйвером WDDM 1.0 або новіший.

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем сімейства WIN32 (Windows XP, Windows NT і т.д.) або Unix.

4.5.1 Вимоги до вхідних даних

Вимоги до вхідних даних не висуваються.

4.5.2 Вимоги до вихідних даних

Вимоги до вихідних даних не висуваються.

4.5.3 Вимоги до мови розробки

Серверну частину застосунку виконати на мові програмування Python із використанням фреймворку Django та бібліотеки Django Rest Framework. Клієнтську частину застосунку виконати на мові програмування TypeScript із використанням фреймворку Next.js.

4.5.4 Вимоги до середовища розробки

Розробку виконати за допомогою інтегрованого середовища розробки PyCharm.

4.5.5 Вимоги до представлення вихідних кодів

Вихідний код програми має бути представлений у вигляді готового проекту, завантаженого у публічний репозиторій на платформі GitHub.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8 Спеціальні вимоги

Спеціальні вимоги не висуваються.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- пояснювальна записка;
- технічне завдання;
- текст програми;
- програма та методика тестування;
- керівництво користувача.

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема структурна варіантів використання;
- схема структурна компонентів програмного забезпечення;
- схема бази даних;

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою проєкту	15.03	
2.	Розробка технічного завдання	31.03	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	07.04	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	14.04	Схема структурна програмного забезпечення та специфікація компонентів
5.	Програмна реалізація програмного забезпечення	21.04	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	05.05	Тести, результати тестування
7.	Розробка матеріалів текстової частини проєкту	12.05	Пояснювальна записка
8.	Розробка матеріалів графічної частини проєкту	19.05	Графічний матеріал проєкту
9.	Оформлення технічної документації проєкту	26.05	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

**Пояснювальна записка
до дипломного проєкту**

на тему: **Вебзастосунок для планування та нагадування про важливі події**

КПІ.ПІ-1219.045440.02.81

ЗМІСТ

ВСТУП	5
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1 Постановка завдання дипломного проєктування	6
1.2 Аналіз предметної області	6
1.3 Аналіз існуючих рішень.....	8
1.3.1 Аналіз відомих програмних продуктів.....	8
1.3.2 Аналіз відомих алгоритмічних та технічних рішень.....	11
1.4 Аналіз та моделювання бізнес-процесів	14
Висновки до розділу	19
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	20
2.1 Варіанти використання програмного забезпечення.....	20
2.2 Розроблення функціональних вимог	35
2.3 Розроблення нефункціональних вимог	40
2.4 Аналіз системних вимог.....	40
2.5 Аналіз економічних показників програмного забезпечення.....	41
2.6 Постановка завдання на розробку програмного забезпечення.....	50
Висновки до розділу	50
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	52
3.1 Архітектура програмного забезпечення.....	52
3.2 Архітектурні рішення та обґрунтування вибору засобів розробки.....	53
3.3 Конструювання програмного забезпечення.....	54
3.3.1 Опис структури бази даних	55
3.4 Аналіз безпеки даних	65
Висновки до розділу	66
4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	67
4.1 Аналіз якості ПЗ.....	67
4.2 Опис процесів тестування.....	70

4.3	Опис контрольного прикладу.....	84
	Висновки до розділу	96
5	РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	97
5.1	Розгортання програмного забезпечення.....	97
5.2	Супровід програмного забезпечення.....	102
	Висновки до розділу	103
	ВИСНОВКИ.....	104
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	105
	ДОДАТОК А. ЗВІТ ПОДІБНОСТІ.....	108

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment, інтегроване середовище розробки.
API	– Application Programming Interface, прикладний програмний Інтерфейс.
ER	– Entity-Relation diagram.
OC	– Операційна система.
БД	– База даних.
DRF	– Django Rest Framework.
ORM	– Object Relation Mapping.
MVC	– Model-View-Controller, архітектурний шаблон Модель-Вигляд-Контролер.
UC	– Use Case, варіант використання.
FR	– Functional Requirement, функціональна вимога.
BPMN	– Business Process Model and Notation, нотація і модель бізнес процесів.
CSRF	– Cross-Site Request Forgery, міжсайтова підробка запитів.
XSS	– Cross-Site Scripting, міжсайтовий скриптинг.

ВСТУП

У сучасному світі із кожним днем інформаційне навантаження на людину зростає. Дедалі важче стає тримати в голові усі події, обов'язки та дедлайни, що часто мають велике значення у житті. У результаті дослідження, проведеного Джорджем Міллером у 1956 році[1], було виявлено, що людина здатна тримати в робочій пам'яті в середньому від 5 до 9 об'єктів одночасно, що є критично низьким значенням, враховуючи наскільки велике число факторів здатне щосекунди відволікти людину та повністю змінити хід думок.

Ще однією важливою проблемою, яку ніяк не можна уникнути чи проігнорувати, є погіршення пам'яті з віком. Згідно дослідженню, проведеному в Університеті Короля Халіда[2], епізодична та короткочасна пам'ять є найвразливішою, та найбільше погіршується з віком. Втрата контролю над подіями та термінами призводить до стресу, професійних помилок та загального зниження ефективності.

Таким чином, стає актуальною потреба у використанні допоміжних засобів для планування, зберігання подій та отримання вчасних нагадувань про них. Використання відповідного програмного забезпечення особливо актуальним є для планування подій у робочому середовищі, де кожен працівник може мати багато паралельних обов'язків.

Університет є прикладом складноорганізованої структури, де працівники водночас можуть виконувати навчальні, наукові, адміністративні та організаційні функції. Кожна подія передбачає високий рівень відповідальності та дотримання чітких термінів. Впровадження програмного забезпечення, яке дозволить працівникам університету планувати події та вчасно отримувати про них нагадування, дозволить значно підвищити як особисту ефективність, так і командну координацію та злагодженість колективної праці.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Постановка завдання дипломного проєктування

Дипломне проєктування передбачає виконання наступних завдань:

- аналіз предметної області та опис її ключових бізнес-процесів, визначення загального завдання розробки у рамках ДП;
- аналіз існуючих рішень (як програмних продуктів, так і алгоритмічних чи технічних рішень) обраного завдання розробки у рамках ДП;
- аналіз та моделювання бізнес-процесів;
- розроблення функціональних, нефункціональних та системних вимог до програмного забезпечення;
- аналіз економічних показників програмного забезпечення ДП;
- постановка завдання на розробку програмного забезпечення ДП;
- розроблення архітектури програмного забезпечення та обґрунтування вибору засобів розробки програмного забезпечення;
- конструювання та розроблення програмного забезпечення;
- аналіз безпеки даних програмного забезпечення;
- аналіз якості та тестування програмного забезпечення;
- розгортання та супровід програмного забезпечення;
- створення супроводжувальної документації до розробленого програмного забезпечення.

1.2 Аналіз предметної області

Ефективне планування подій та задач є ключовою запорукою ефективної праці як окремо взятої людини, так і команди, що працює над спільною метою. Програмні засоби для управління подіями, задачами та нагадуваннями набувають все більшого поширення.

Першими реалізаціями цього функціоналу були «To-Do» застосунки. Вони дозволяли користувачам створювати прості списки задач із текстовим

описом та із можливістю позначення задач як виконаних. Досить швидко стало зрозуміло, що це є перспективним напрямком для розвитку. Почали з'являтися застосунки, що розширювали базовий функціонал списку задач. Користувачі отримали змогу присвоювати подіям допоміжні атрибути, прив'язувати подію до конкретного часу або періоду часу, сортувати та фільтрувати події.

Наступним етапом розвитку застосунків для планування подій стала командна взаємодія між користувачами. Подібні застосунки дуже часто інтегруються в робочий процес організацій та компаній із будь-яких сфер діяльності. Розподіл обов'язків та призначення задач учасникам команди у межах одного застосунку дозволяє гнучко контролювати хід роботи та забезпечувати ефективну комунікацію.

Існують як безкоштовні, так і платні застосунки для планування подій. Найбільш популярною є модель підписки, де користувач отримує доступ до покращеного функціоналу на певний період часу, зазвичай на місяць або на рік. Багаторазові грошові витрати на продовження підписки можуть здатись незначними, але у довгостроковій перспективі вони створюють суттєве фінансове навантаження. Робота у команді зазвичай є ще більш фінансово-затратною, оскільки подібні застосунки часто орієнтовані на бізнес-середовище, де планування подій розглядається як інструмент підвищення ефективності у комерційній діяльності компаній. Це значно ускладнює використання таких застосунків некомерційними установами, які працюють в умовах обмеженого бюджету.

Типовим стимулом для придбання платної версії застосунку є штучне обмеження базового функціоналу для звичайних користувачів, що унеможлиблює повноцінне користування застосунком. Подібними обмеженнями зазвичай є зменшення максимальної кількості запланованих подій, обмежений доступ до нагадувань, неможливість створювати команди.

В рамках дипломного проекту було обрано шлях розробки вебзастосунку для планування подій та нагадувань, який адаптовано під користування в межах університету. Це дозволить врахувати специфічну

структуру установи, забезпечити можливість організації особистих подій та командної роботи без додаткових обмежень. Отримання нагадувань через електронну пошту та Telegram-бота забезпечить покращений контроль над подіями та термінами.

1.3 Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні програмне забезпечення у даній області та технічні рішення, що допоможуть у реалізації вебзастосунку для планування та нагадування про важливі події. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

1.3.1 Аналіз відомих програмних продуктів

Google Calendar

Google Calendar[3] – один із найпопулярніших сервісів для планування подій та тайм-менеджменту, розроблений корпорацією Google (рис. 1.1). Додаток доступний у браузері та на мобільних платформах з 2009 року. Google Calendar дозволяє створювати події та нагадування, підтримує синхронізацію з іншими сервісами Google, такі як Google Classroom, Gmail чи Google Meet. Проте налаштування деталей події включають в себе лише основну інформацію про подію, а список подій не передбачає гнучкої фільтрації чи сортування. Додаток має можливість створювати групові події, проте участь у кожній події є добровільною та потребує підтвердження від отримувача, що не підходить для зручної роботи в команді.

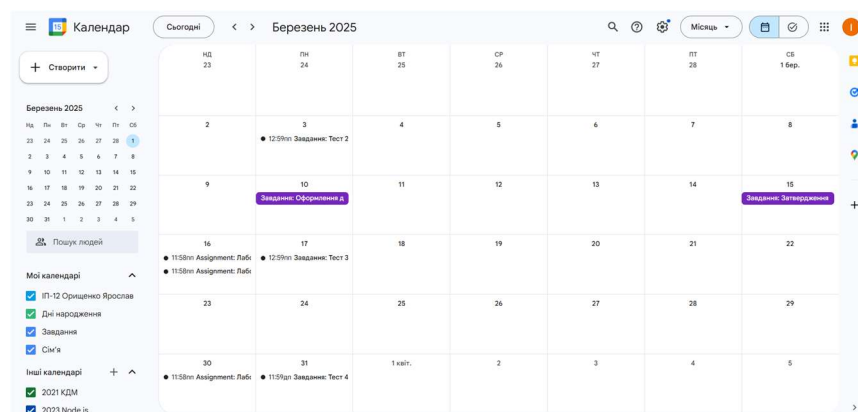


Рисунок 1.1 – Користувацький інтерфейс Google Calendar

Todoist

Todoist[4] – додаток для управління задачами та проектами (рис. 1.2). Додаток працює з 2007 року, і на даний момент має понад 30 мільйонів користувачів на різноманітних платформах. Додаток дозволяє групувати завдання у проекти, встановлювати пріоритети та додавати дедлайни, сортувати та фільтрувати завдання. Унікальною є функціональність «карми», яка нараховує бали за виконані завдання та стимулює підтримувати стабільний рівень продуктивності. Безкоштовна версія додатку має низку обмежень, такі як неможливість додавати нагадування про події, неповний перелік додаткової інформації про подію. Також Todoist підтримує повноцінну роботу в командах, проте цей функціонал є виключно платним.

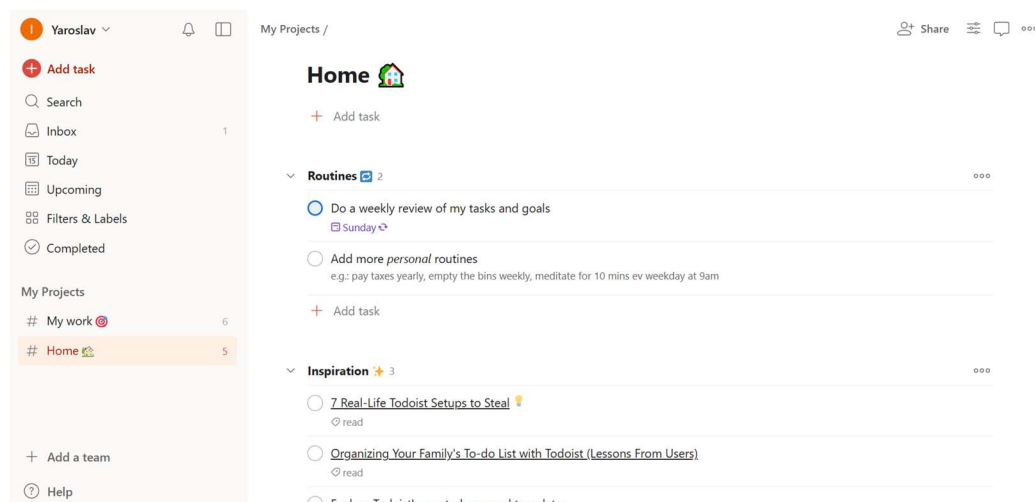


Рисунок 1.2 – Користувацький інтерфейс Todoist

Trello

Trello[5] – додаток для візуального управління задачами, що функціонує з 2011 року. Його головною структурною одиницею є «дошка» – організований простір, що складається із створених користувачем груп задач (рис. 1.3). Для кожної задачі можна додати детальний опис, прикріпити мітки та файли. До дошки можна запросити інших учасників та призначити їм конкретні завдання. Проте у Trello відсутня можливість створювати повторювані події та заплановувати нагадування для подій.

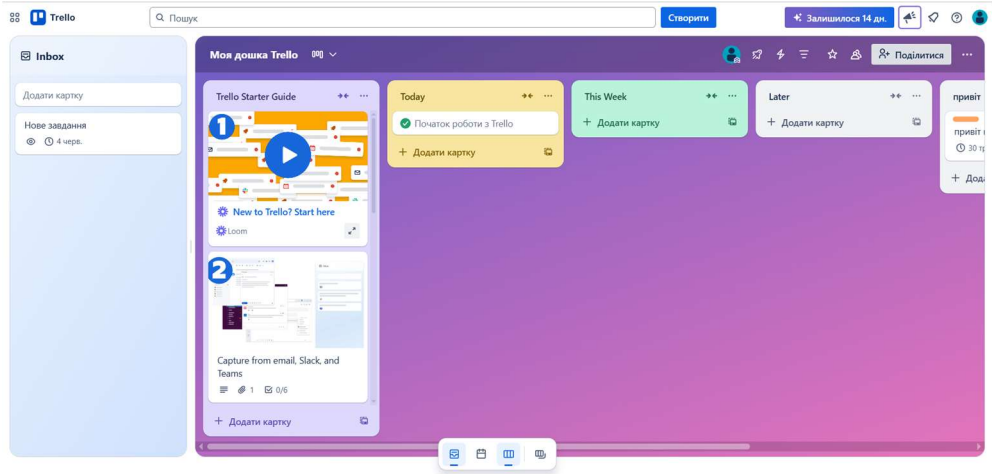


Рисунок 1.3 – Користувацький інтерфейс Trello

ClickUp

ClickUp[6] – платформа для управління проектами, завданнями та організацією роботи в команді (рис. 1.4). Застосунок дозволяє створювати події та нагадування про них, створювати проекти із групами задач. ClickUp дозволяє зручно працювати в командах, оскільки має розвинену систему ролей та прав доступу. Проте недоліком застосунку є обмеженість безкоштовного плану користування, у якому лімітується кількість створених задач, а максимальна кількість учасників команди становить лише дві людини.

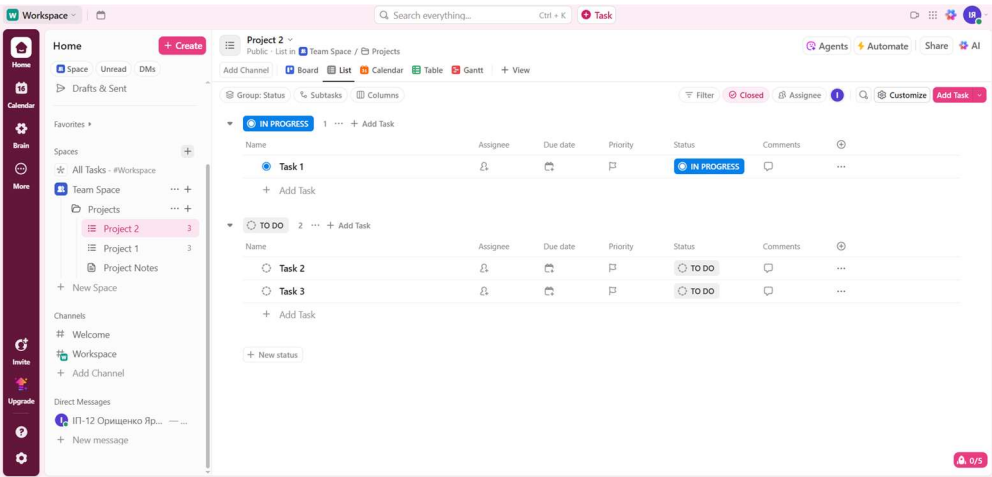


Рисунок 1.4 – Користувацький інтерфейс ClickUp

Для порівняння власного проекту з аналогами можна скористатись таблицею 1.1.

Таблиця 1.1 – Порівняння з аналогами

	Google Calendar	Todoist	Trello	ClickUp	KPI Notify (власний проект)
Створення нагадувань	Так	Ні	Ні	Так	Так
Створення команд	Ні	Ні	Так	Так	Так
Створення повторюваних подій	Так	Ні	Ні	Так	Так
Додавання детальної інформації про події	Ні	Ні	Так	Так	Так
Фільтрація та сортування подій	Ні	Так	Так	Так	Так
Отримання нагадувань через Telegram-бота	Ні	Ні	Ні	Ні	Так
Запрошення у команду із списку користувачів додатку	Ні	Ні	Ні	Ні	Так
Збереження даних реєстраційної картки юридичної особи	Ні	Ні	Ні	Ні	Так

1.3.2 Аналіз відомих алгоритмічних та технічних рішень

У результаті проведеного дослідження, було зроблено висновок, що розробка не потребує використання спеціальних алгоритмів.

Для розробки вебзастосунків типово використовується клієнт-серверна архітектура[7]. Основними її складовими є клієнт та сервер. Клієнтом може бути будь-який сервіс, програма чи інтерфейс, який ініціює запити до сервера

з метою отримання або обробки інформації. Клієнт слугує лише для відображення та взаємодії з інформацією, яка зберігається на сервері. Сервер – це компонент системи, який зберігає бізнес-логіку застосунку та взаємодіє із базою даних.

Переваги клієнт-серверної архітектури:

- Доступність додатку: більшість операцій, що потребують значних обчислювальних ресурсів, виконуються на сервері, що дозволяє знизити вимоги до технічних засобів, на яких встановлено клієнтську частину застосунку.
- Безпека даних: дані, що зберігаються на сервері, є надійно захищеними, оскільки звичайні користувачі можуть отримати доступ лише до тієї інформації, що була визначена розробником. Додатковий захист даних забезпечують шифрування трафіку, використання автентифікації та авторизації для контролю доступу до даних.

Недоліки клієнт-серверної архітектури:

- Висока залежність від одного компонента – сервера: якщо сервер припинить функціонування, то усі підключені до нього клієнти втратять доступ до застосунку.
- Висока вартість масштабування: серверне обладнання є значно дорожчим за звичайні комп'ютери, і потребує кваліфікованого обслуговування. Підтримка та розвиток застосунку можуть потребувати значних капіталовкладень.

Зважаючи на усі перераховані переваги та недоліки, було прийнято рішення використати клієнт-серверну архітектуру для розробки вебзастосунку.

Монолітна архітектура[8] – це традиційна модель архітектури програмного забезпечення, яка передбачає те, що усі компоненти, які реалізують функціонал застосунку, об'єднані в єдиній сутності – моноліті. Це спрощує взаємодію компонентів застосунку, які можуть спілкуватись одне з одним напряму.

Переваги монолітної архітектури:

- Простота розробки та розгортання: єдину базу коду значно простіше розробляти та підтримувати у порівнянні із розподіленою архітектурою, оскільки вона використовує менше окремих компонентів.
- Висока продуктивність: оскільки компоненти можуть легко взаємодіяти одне з одним напряму, без потреби виконання додаткових запитів, це значно підвищує загальну швидкодію застосунку.

Недоліки монолітної архітектури:

- Надійність: відмова одного компоненту може напряму вплинути на роботу усіх інших компонентів застосунку.
- Складність масштабування: через тісний зв'язок між компонентами, навіть незначні зміни у функціонал можуть потребувати повної перебудови системи.

Через простоту використання та ефективність для застосунків невеликого масштабу, було прийнято рішення обрати монолітну архітектуру.

Model View Controller (MVC)[9] – це патерн проектування програмного забезпечення, який розділяє логіку системи на три основні складові:

- Model (Модель): визначає структуру даних, які зберігає застосунок.
- View (Вигляд): визначає як дані повинні відображатись користувачеві.
- Controller (Контролер): визначає логіку взаємодії користувача із даними застосунку.

MVC дозволяє відокремити функціональність користувацького інтерфейсу від основної логіки застосунку, при цьому забезпечуючи швидку реакцію на дії користувача та зміни у даних.

1.4 Аналіз та моделювання бізнес-процесів

Для опису бізнес-процесів програмного забезпечення використовується BPMN модель (рисунки 1.5 – 1.8).

BMPN (Business Process Model and Notation)[10] – це система умовних позначень, що призначена для графічного моделювання бізнес-процесів та підтримки розуміння між бізнес-фахівцями, які визначають та управляють процесами, і технічними фахівцями, які реалізують ці процеси в інформаційних системах.

Опис моделі бізнес-процесу реєстрації користувача:

- користувач переходить на сторінку реєстрації;
- Frontend завантажує сторінку із формою для введення реєстраційних даних;
- користувач вводить ім'я та прізвище, адресу електронної пошти, пароль та підтвердження паролю;
- користувач натискає на кнопку «Зареєструватись»;
- дані форми валідуються на Frontend;
- якщо у формі знайдено невалідні дані, то відобразити відповідні помилки;
- якщо всі введені користувачем дані вірні, то надіслати дані форми реєстрації на Backend;
- якщо отримані дані на Backend не є валідними, або адреса пошти не є унікальною в базі даних, то повернути на Frontend відповідь із помилкою;
- відобразити помилку реєстрації користувачу;
- якщо отримані дані на Backend є валідними, а адреса пошти є унікальною в базі даних, то створити об'єкт користувача;
- надіслати підтвердження про реєстрацію на електронну пошту, вказану у формі реєстрації;
- відобразити повідомлення про прохання підтвердити реєстрацію через електронну пошту;

- користувач відкриває електронну пошту та переходить за посиланням у отриманому листі;
- Frontend надсилає дані активації на Backend;
- Backend активує користувача та повертає код успіху на Frontend;
- Frontend відображає повідомлення про успішну реєстрацію.

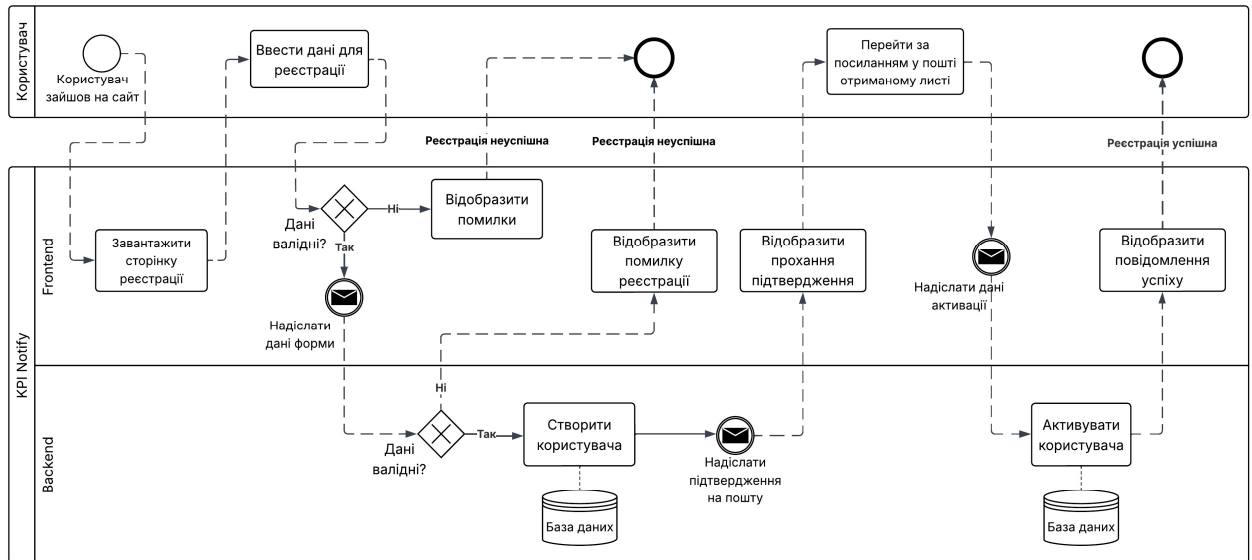


Рисунок 1.5 – Модель реєстрації користувача

Опис моделі бізнес-процесу отримання даних, доступних для автентифікованих користувачів:

- користувач відкриває сторінку, яка потребує автентифікації;
- Frontend надсилає запит на отримання потрібних даних, та передає токен доступу користувача;
- якщо токен доступу валідний, та час його дії не сплив, то отримати потрібну інформацію із бази даних;
- передати необхідні дані на Frontend;
- завантажити сторінку, на яку зайдеш користувач;
- якщо токен доступу не валідний, або час його дії сплив, то повернути на Frontend помилку автентифікації;
- Frontend надсилає запит на оновлення токена доступу за допомогою токена оновлення;

- якщо токен оновлення не валідний, або час його дії сплив, то повернути на Frontend помилку автентифікації;
- завантажити сторінку для входу в акаунт;
- якщо токен оновлення валідний, та час дії його не сплив, то оновити токен доступу та встановити його в Cookie;
- Frontend повторює спробу отримання даних, та передає оновлений токен доступу;
- Backend отримує потрібну інформацію із бази даних;
- передати отримані дані на Frontend;
- завантажити сторінку, на яку зайшов користувач.

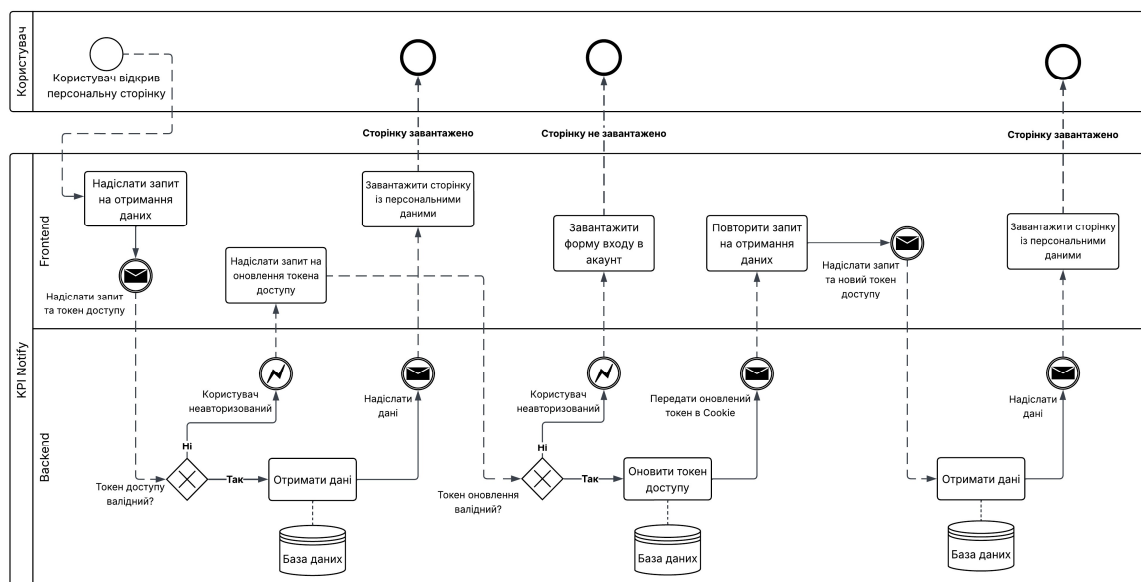


Рисунок 1.6 – Модель отримання даних для автентифікованих користувачів

Для зручності та запобігання багаторазового повторення, для наступних BPMN-моделей вважається, що користувач уже є автентифікованим, а завантаження сторінок за допомогою токенів доступу завершується успіхом.

Опис моделі бізнес-процесу створення події та нагадувань:

- користувач натискає на кнопку для створення нової події;
- Frontend завантажує сторінку із формою для створення події та нагадувань;

- користувач вказує інформацію про подію, обирає час та метод нагадувань, та підтверджує створення події;
- дані валідуються на Frontend;
- якщо у формі вказані невалідні дані, то відобразити відповідні помилки;
- якщо дані форми валідні, то Frontend передає дані на Backend;
- дані валідуються на Backend;
- якщо дані не валідні, то на Frontend повертається помилка;
- Frontend відображає помилку створення події;
- якщо дані події та нагадувань валідні, то подія додається у базу даних;
- для події створюються нагадування, заплановуються задачі для надсилання нагадувань обраним користувачем способом у вказаний час;
- повідомлення про успішне створення події та нагадувань передається на Frontend;
- користувачу завантажується повідомлення про успішне створення події та нагадувань.

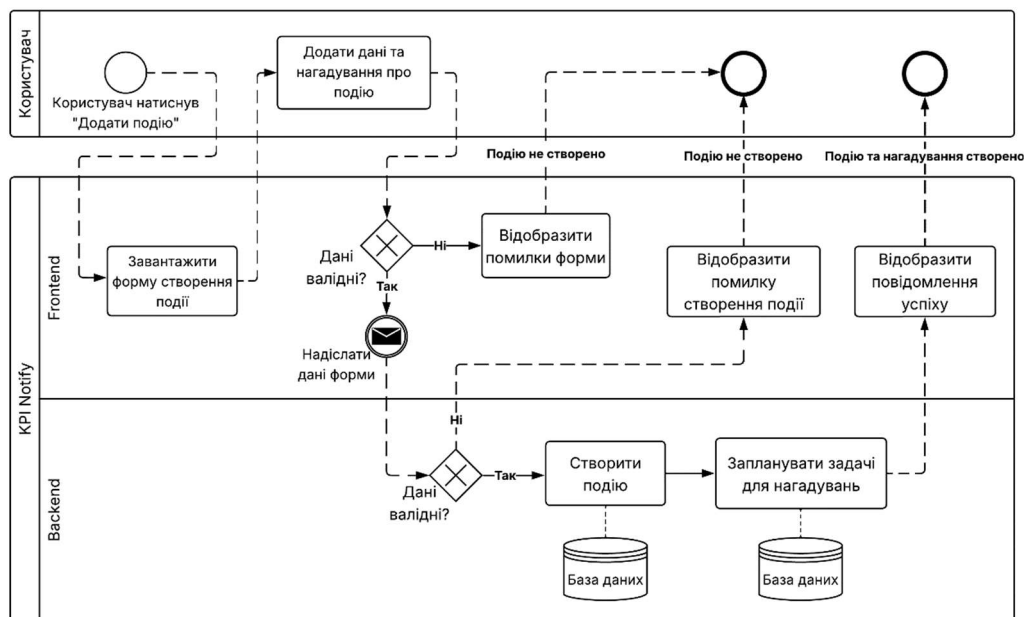


Рисунок 1.7 – Модель створення події та нагадувань

Опис моделі бізнес-процесу створення команди:

- користувач натискає на кнопку для створення нової події;
- Frontend надсилає запит на отримання списку користувачів, яких можна буде запросити у команду;
- Backend отримує дані про користувачів;
- Backend надсилає дані про користувачів на Frontend;
- Frontend завантажує сторінку із формою для створення команди;
- користувач вказує інформацію про команди, обирає учасників, яких він бажає запросити;
- дані валідуються на Frontend;
- якщо у формі вказані невалідні дані, то відобразити відповідні помилки;
- якщо дані форми валідні, то Frontend передає дані на Backend;
- дані валідуються на Backend;
- якщо дані не валідні, то на Frontend повертається помилка;
- Frontend відображає помилку створення команди;
- якщо дані валідні, то команда додається до бази даних;
- для команди та обраних користувачів створюються запрошення;
- листи із повідомленням про запрошення у команду надсилаються на електронну пошту обраним користувачам;
- повідомлення про успішне створення команди передається на Frontend;
- Frontend відображає повідомлення про успішне створення команди.

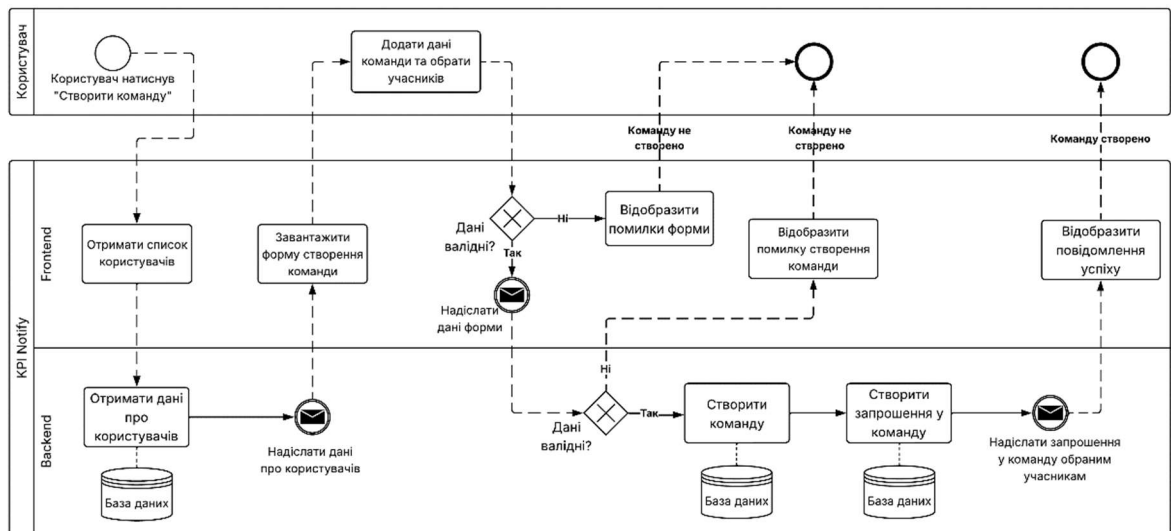


Рисунок 1.8 – Модель створення команди

Висновки до розділу

У розділі «Передпроектне обстеження предметної області» було детально досліджено предметну область застосунків для планування подій та нагадувань. Була розглянута проблематика даної області та обґрунтована актуальність розробок у цій сфері.

Було проаналізовано чотири відомих застосунків для планування подій, нагадувань та роботи в командах. У всіх застосунках було виявлено недоліки, переважна більшість яких полягає у критичному обмеженні функціоналу користувачів, які користуються застосунками на безоплатній основі. Після детального дослідження переваг та недоліків кожного додатку, було зроблено висновок про необхідність створення власного вебзастосунку для планування подій та нагадувань, призначений для внутрішнього використання в університеті, який матиме увесь необхідний функціонал, який не є реалізованим або не є доступним в уже існуючих програмних аналогах.

Було проаналізовано технічні та архітектурні рішення, які дозволяють розробити вебзастосунок, що відповідає сучасним вимогам та тенденціям.

За допомогою моделей BPMN були створені та описані схеми основних бізнес-процесів вебзастосунку.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Діаграма варіантів використання з ключовими акторами та основними функціями показана у графічному матеріалі, аркуш 1.

В таблицях 2.1 – 2.22 наведено опис варіантів використання програмного забезпечення.

Таблиця 2.1 – Варіант використання UC-01

Use case name	Реєстрація користувача
Use case ID	UC-01
Goals	Реєстрація нового користувача в системі
Actors	Незареєстрований користувач
Trigger	Користувач бажає зареєструватися
Pre-conditions	-
Flow of Events	Користувач переходить на сторінку реєстрації. В поля для реєстрації вводяться відповідні дані: пошта користувача, що має корпоративний домен університету, пароль в системі, та його повтор для підтвердження. Після заповнення даних користувач натискає кнопку реєстрації. Після цього з'являється повідомлення із проханням підтвердити створення акаунту через електронну пошту. Перейшовши за посиланням, користувач отримує повідомлення про успішну активацію акаунту, та перенаправляється на сторінку входу в акаунт.
Extension	Якщо якесь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Створення сторінки користувача, перехід на сторінку входу.

Таблиця 2.2 – Варіант використання UC-02

Use case name	Вхід в акаунт
Use case ID	UC-02
Goals	Автентифікація зареєстрованого користувача
Actors	Зареєстрований користувач
Trigger	Користувач бажає увійти в акаунт
Pre-conditions	-
Flow of Events	Користувач переходить на сторінку входу в акаунт. В поля вводяться дані електронної пошти, що містить корпоративний домен університету, та пароль. Після натиснення кнопки входу, користувач отримує повідомлення про успішний вхід в акаунт.
Extension	Якщо якесь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Вхід користувача в акаунт, перехід на сторінку застосунку.

Таблиця 2.3 – Варіант використання UC-03

Use case name	Заповнення даних про посаду
Use case ID	UC-03
Goals	Заповнити обов'язкові дані про посаду користувача
Actors	Зареєстрований користувач
Trigger	Користувач вперше входить в акаунт
Pre-conditions	Зареєстрований користувач вперше входить в акаунт
Flow of Events	Вперше увійшовши в свій акаунт, користувач перенаправляється на сторінку вводу даних про посаду: назва посади, інститут чи факультет та кафедра, де користувач працює. Натиснувши кнопку підтвердження, користувач отримує повідомлення про успішне збереження обов'язкових даних про посаду, та перенаправляється на сторінку застосунку.

Продовження таблиці 2.3

Extension	Якщо якийсь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Заповнення обов'язкових даних про посаду користувача, перехід на сторінку застосунку.

Таблиця 2.4 – Варіант використання UC-04

Use case name	Редагування профілю
Use case ID	UC-04
Goals	Зміна та доповнення особистої інформації
Actors	Зареєстрований користувач
Trigger	Користувач бажає змінити чи доповнити особисту інформацію у профілі.
Pre-conditions	Користувач увійшов у створений акаунт.
Flow of Events	Користувач у меню обирає розділ «Профіль». На сторінці із інформацією профілю користувач натискає на кнопку зміни профілю. Завантажується форма із усіма полями профілю, деякі або всі з яких вже заповнені поточною інформацією про користувача. Користувач заповнює або змінює дані свого імені та прізвища, статі, контактного номера телефону, посади, кафедри, імені користувача Telegram та номера телефону Telegram. Змінивши усі бажані дані, користувач натискає кнопку підтвердження, після чого отримує повідомлення про успішну зміну інформації профілю, та перенаправляється на сторінку із особистою інформацією.
Extension	Якщо якийсь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач оновив дані профілю.

Таблиця 2.5 – Варіант використання UC-05

Use case name	Створення події
Use case ID	UC-05

Продовження таблиці 2.5

Goals	Створити нову подію
Actors	Зареєстрований користувач
Trigger	Користувач бажає запланувати нову подію.
Pre-conditions	Користувач увійшов у створений акаунт.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком подій, користувач натискає на кнопку «Створити». Завантажується форма для створення події, де користувач заповнює обов'язкові дані: назва події, пріоритет, дата та час події. Користувач може додати необов'язкові дані: опис, зображення, теги, посилання, локація, періодичність повторення події, нагадування. Натиснувши на кнопку створення, користувач отримає повідомлення про успішне створення події, та буде перенаправлений на сторінку із деталями події.
Extension	Якщо якесь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач створив подію.

Таблиця 2.6 – Варіант використання UC-06

Use case name	Редагування події
Use case ID	UC-06
Goals	Змінити інформацію про створену подію
Actors	Зареєстрований користувач
Trigger	Користувач бажає змінити інформацію про подію
Pre-conditions	Користувач увійшов у створений акаунт, користувач має не менше однієї створеної події.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком подій, користувач натискає на подію, яку бажає редагувати. Потрапивши на сторінку із повною інформацією про подію, користувач натискає кнопку «Редагувати».

Продовження таблиці 2.6

Flow of Events	У завантаженій формі, що заповнена наявними даними про подію, користувач змінює чи додає потрібну інформацію. Після натиснення кнопки підтвердження, користувач отримує повідомлення про успішне редагування події та перенаправляється на сторінку із повною інформацією про подію.
Extension	Якщо якесь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач редагував дані про подію.

Таблиця 2.7 – Варіант використання UC-07

Use case name	Додавання події до Google Calendar.
Use case ID	UC-07
Goals	Перенести створену подію у Google Calendar.
Actors	Зареєстрований користувач
Trigger	Користувач бажає перенести дані про створену подію у Google Calendar.
Pre-conditions	Користувач увійшов у створений акаунт, користувач має не менше однієї створеної події.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком подій, користувач натискає на подію, яку бажає редагувати. Потрапивши на сторінку із повною інформацією про подію, користувач натискає кнопку «Додати до Google Calendar». Користувач перенаправляється на сторінку застосунку Google Calendar, на якій відкрита форма із заповненими даними про подію, яку він бажав перенести. Користувач натискає кнопку «Зберегти», після чого подія з'являється у календарі.
Extension	
Post-Condition	Користувач додав подію до Google Calendar.

Таблиця 2.8 – Варіант використання UC-08

Use case name	Видалення події
Use case ID	UC-08
Goals	Видалення існуючої події
Actors	Зареєстрований користувач
Trigger	Користувач бажає видалити подію.
Pre-conditions	Користувач увійшов у створений акаунт, користувач має не менше однієї створеної події.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком подій, користувач натискає на подію, яку бажає видалити. Потрапивши на сторінку із повною інформацією про подію, користувач натискає кнопку «Видалити». У відкритому вікні користувач підтверджує що хоче видалити подію, після чого отримує повідомлення про успішне видалення, та перенаправляється на сторінку із загальним списком подій.
Extension	
Post-Condition	Користувач видалив подію.

Таблиця 2.9 – Варіант використання UC-09

Use case name	Створення команди
Use case ID	UC-09
Goals	Створити нову команду
Actors	Зареєстрований користувач
Trigger	Користувач бажає створити нову команду
Pre-conditions	Користувач увійшов у створений акаунт.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком команд, учасником яких він є, користувач натискає на кнопку «Створити». Після перенаправлення на сторінку із формою для створення команди, користувач вводить у відповідні поля назву та опис команди, а також обирає із списку користувачів, яких він бажає запросити у команду.

Продовження таблиці 2.9

Flow of Events	Натиснувши кнопку «Створити», користувач отримує повідомлення про успішне створення та перенаправляється на сторінку із повною інформацією про команду.
Extension	Якщо якийсь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач створив команду, обраним учасникам були надіслані запрошення у команду.

Таблиця 2.10 – Варіант використання UC-10

Use case name	Редагування команди
Use case ID	UC-10
Goals	Змінити інформацію про команду
Actors	Зареєстрований користувач
Trigger	Користувач бажає змінити інформацію про команду.
Pre-conditions	Користувач увійшов у створений акаунт, користувач є власником не менш ніж однієї команди.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком команд, користувач натискає на команду, лідером якої він є. На завантажній сторінці із інформацією про команду користувач натискає кнопку «Редагувати». Після перенаправлення на сторінку із формою, яка містить інформацію про команду, користувач модифікує бажані поля. Після натиснення кнопки підтвердження, користувач отримує повідомлення про успішне редагування та перенаправляється на сторінку із повною інформацією про команду.
Extension	Якщо якийсь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач редагував інформацію про команду

Таблиця 2.11 – Варіант використання UC-11

Use case name	Запрошення учасника до команди
Use case ID	UC-11
Goals	Створити запрошення для стороннього користувача
Actors	Зареєстрований користувач
Trigger	Користувач бажає запросити нового учасника до своєї команди
Pre-conditions	Користувач увійшов у створений акаунт, користувач є власником не менш ніж однієї команди.
Flow of Events	Користувач обирає у меню розділ «Університет». Через пошук або у списку користувачів, що належать до певного відділу університету, користувач знаходить того, кого він бажає запросити у команду. Користувач натискає на кнопку «Запросити до команди». У відкритому вікні користувач обирає до якої команди він бажає запросити користувача та натискає на кнопку підтвердження. Після цього користувач отримує повідомлення про успішне запрошення учасника до команди.
Extension	
Post-Condition	Користувач створив нове запрошення

Таблиця 2.12 – Варіант використання UC-12

Use case name	Видалення команди
Use case ID	UC-12
Goals	Видалення існуючої команди
Actors	Зареєстрований користувач, користувач є власником не менш ніж однієї команди.
Trigger	Користувач бажає видалити команду, власником якої він є.
Pre-conditions	Користувач увійшов у створений акаунт, користувач має не менше однієї створеної команди.

Продовження таблиці 2.12

Flow of Events	Користувач обирає у меню розділ «Команди». Після перенаправлення на сторінку із загальним списком команд, користувач натискає на команду, яку бажає видалити. Потрапивши на сторінку із повною інформацією про команду, користувач натискає кнопку «Видалити». У відкритому вікні користувач підтверджує що хоче видалити команду, після чого отримує повідомлення про успішне видалення, та перенаправляється на сторінку із загальним списком команд.
Extension	
Post-Condition	Користувач видалив команду.

Таблиця 2.13 – Варіант використання UC-13

Use case name	Вихід із команди
Use case ID	UC-13
Goals	Вийти із команди у яку приєднався користувач
Actors	Зареєстрований користувач
Trigger	Користувач бажає вийти із команди, у яку він приєднався.
Pre-conditions	Користувач увійшов у створений акаунт, користувач є учасником не менш ніж однієї команди.
Flow of Events	Користувач обирає у меню розділ «Команди». Після перенаправлення на сторінку із загальним списком команд, користувач натискає на команду, яку бажає покинути. Потрапивши на сторінку із повною інформацією про команду, користувач натискає кнопку «Покинути команду». У відкритому вікні користувач підтверджує що хоче покинути команду, після чого отримує повідомлення про успішний вихід, та перенаправляється на сторінку із загальним списком команд.
Extension	
Post-Condition	Користувач вийшов із команди

Таблиця 2.14 – Варіант використання UC-14

Use case name	Відповідь на отримане запрошення
Use case ID	UC-14
Goals	Прийняти або відхилити отримане запрошення
Actors	Зареєстрований користувач
Trigger	Користувач бажає прийняти або відхилити отримане запрошення у команду.
Pre-conditions	Користувач увійшов у створений акаунт, користувач отримав не менш ніж одне запрошення, стан якого «Очікує відповіді».
Flow of Events	Користувач обирає у меню розділ «Команди». Після перенаправлення на сторінку із загальним списком команд, користувач обирає вкладку «Запрошення», де завантажується загальний список запрошень. Користувач обирає запрошення, на яке він бажає відповісти. На завантаженій сторінці із деталями запрошення користувач натискає кнопку «Прийняти» або «Відмовити». Користувач отримує повідомлення про успішну відповідь на запрошення, та перенаправляється на сторінку деталей команди, якщо запрошення було прийняте, або на сторінку із списком запрошень, якщо воно було відхилено.
Extension	
Post-Condition	Користувач відповів на отримане запрошення

Таблиця 2.15 – Варіант використання UC-15

Use case name	Видалення надісланого запрошення
Use case ID	UC-15
Goals	Видалити створене запрошення у команду
Actors	Зареєстрований користувач
Trigger	Користувач бажає видалити створене ним запрошення у команду.
Pre-conditions	Користувач увійшов у створений акаунт, користувач є власником не менш ніж одієї команди, та створив не менш ніж одне запрошення, стан якого «Очікує відповіді».

Продовження таблиці 2.15

Flow of Events	Користувач обирає у меню розділ «Команди». Після перенаправлення на сторінку із загальним списком команд, користувач обирає вкладку «Запрошення», де завантажується загальний список запрошень. Користувач натискає на створене ним запрошення, яке він бажає видалити. На завантаженій сторінці із деталями запрошення користувач натискає кнопку «Видалити». Користувач отримує повідомлення про успішне видалення запрошення, та перенаправляється на сторінку із списком запрошень.
Extension	
Post-Condition	Користувач видалив створене ним запрошення.

Таблиця 2.16 – Варіант використання UC-16

Use case name	Створення події для підлеглих
Use case ID	UC-16
Goals	Створити нову подію для підлеглих
Actors	Зареєстрований користувач
Trigger	Користувач бажає створити подію для підлеглих.
Pre-conditions	Користувач увійшов у створений акаунт, користувач є власником команди, яка має не менш ніж одного учасника.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком подій, користувач натискає на кнопку «Створити». Завантажується форма для створення події, де користувач заповнює обов'язкові дані та необов'язкові дані.
Flow of Events	Із списку користувачів, які є підлеглими, користувач обирає одного або декількох користувачів. Натиснувши на кнопку створення, користувач отримає повідомлення про успішне створення події, та буде перенаправлений на сторінку із деталями події.
Extension	Якщо якесь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач створив подію.

Таблиця 2.17 – Варіант використання UC-17

Use case name	Відмова від події
Use case ID	UC-17
Goals	Відмова від призначеної події
Actors	Зареєстрований користувач
Trigger	Користувач бажає видалити із списку подій призначену йому подію.
Pre-conditions	Користувач увійшов у створений акаунт, користувач є учасником не менш ніж однієї команди, користувач має не менше однієї призначеної йому події.
Flow of Events	Користувач обирає у меню розділ «Події». Після перенаправлення на сторінку із загальним списком подій, користувач натискає на призначену йому подію, від якої від бажає відмовитись. Потрапивши на сторінку із повною інформацією про подію, користувач натискає кнопку «Відмовитись». У відкритому вікні користувач підтверджує що хоче відмовитись від події, після чого отримує повідомлення про успішну відмову, та перенаправляється на сторінку із загальним списком подій.
Extension	
Post-Condition	Користувач відмовився від події.

Таблиця 2.18 – Варіант використання UC-18

Use case name	Створення реєстраційної картки
Use case ID	UC-18
Goals	Створити реєстраційну картку юридичної особи
Actors	Зареєстрований користувач
Trigger	Користувач бажає створити реєстраційну картку юридичної особи
Pre-conditions	Користувач увійшов у створений акаунт.

Продовження таблиці 2.18

Flow of Events	Користувач у меню обирає розділ «Профіль». На сторінці із інформацією профілю користувач обирає вкладку «Реєстраційні картки», де завантажується загальний список реєстраційних карток. Користувач натискає кнопку «Створити», після чого завантажується форма, яка містить поля із інформацією про юридичну особу, місце розташування, дані заявника, контактну інформацію, дані для формування кваліфікованого сертифікату відкритого ключа електронної печатки. Заповнивши бажані поля, користувач натискає «Створити», після чого отримує повідомлення про успішне створення реєстраційної картки, та перенаправляється на загальний список реєстраційних карток.
Extension	Якщо якийсь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач створив реєстраційну картку.

Таблиця 2.19 – Варіант використання UC-19

Use case name	Редагування реєстраційної картки
Use case ID	UC-19
Goals	Редагувати реєстраційну картку юридичної особи
Actors	Зареєстрований користувач
Trigger	Користувач бажає редагувати реєстраційну картку юридичної особи
Pre-conditions	Користувач увійшов у створений акаунт, користувач створив принаймні одну реєстраційну картку.
Flow of Events	Користувач у меню обирає розділ «Профіль». На сторінці із інформацією профілю користувач обирає вкладку «Реєстраційні картки», де завантажується загальний список реєстраційних карток. Користувач обирає потрібну картку, після чого завантажується сторінка із повною інформацією про реєстраційну картку.

Продовження таблиці 2.19

Flow of Events	Користувач натискає на кнопку «Редагувати», після чого завантажується форма із заповненими полями. Відредагувавши чи заповнивши бажані поля, користувач натискає «Редагувати», після чого отримує повідомлення про успішне оновлення реєстраційної картки, та перенаправляється на загальний список реєстраційних карток.
Extension	Якщо якийсь конкретне поле введено некоректно, воно виділяється червоним і вказується опис помилки.
Post-Condition	Користувач редагував реєстраційну картку.

Таблиця 2.20 – Варіант використання UC-20

Use case name	Вихід з акаунту
Use case ID	UC-20
Goals	Вийти із облікового запису
Actors	Зареєстрований користувач
Trigger	Користувач бажає вийти із свого облікового запису
Pre-conditions	Користувач увійшов у свій обліковий запис.
Flow of Events	Користувач у меню обирає опцію «Вийти». Натиснувши на цю кнопку, користувач отримує повідомлення про успішний вихід із акаунту, та перенаправляється на сторінку входу в акаунт.
Extension	
Post-Condition	Користувач вийшов із облікового запису.

Таблиця 2.21 – Варіант використання UC-21

Use case name	Прив'язка Telegram-акаунту
Use case ID	UC-21
Goals	Прив'язати Telegram-акаунт користувача до застосунку.
Actors	Зареєстрований користувач

Продовження таблиці 2.21

Trigger	Користувач бажає прив'язати свій Telegram-акаунт до застунку, щоб мати можливість отримувати нагадування про події через Telegram-бота.
Pre-conditions	Користувач увійшов у створений акаунт.
Flow of Events	Користувач у меню обирає розділ «Профіль». На сторінці із інформацією профілю у полі Telegram користувач натискає на кнопку «Увійти через Telegram». У відкритій сторінці «Авторизація Telegram» користувач вводить номер телефону, на який зареєстровано його Telegram-акаунт. Натиснувши на кнопку «Далі», користувач отримує повідомлення від офіційного акаунту Telegram із інформацією про отриманий запит на авторизацію. Користувач натискає кнопку «Підтвердити» та повертається до сторінки «Авторизація Telegram». На сторінці, де відображена інформація Telegram-акаунту користувача та Telegram-бота KPI Notify, користувач обирає пункт «Дозволити KPI Notify надсилати вам повідомлення в Telegram», та натискає кнопку «Прийняти». Користувач отримує повідомлення про авторизацію від офіційного акаунту Telegram, а також повідомлення від Telegram-боту KPI Notify.
Extension	
Post-Condition	Користувач прив'язав свій Telegram-акаунт.

Таблиця 2.22 – Варіант використання UC-22

Use case name	Видалення акаунту
Use case ID	UC-22
Goals	Вийти акаунт користувача
Actors	Зареєстрований користувач
Trigger	Користувач бажає видалити свій акаунт та усі створені ним дані.
Pre-conditions	Користувач увійшов у свій обліковий запис.

Продовження таблиці 2.22

Flow of Events	Користувач у меню обирає розділ «Профіль». На сторінці із інформацією профілю користувач натискає на кнопку «Видалити акаунт». У відкритому вікні для підтвердження видалення акаунту, користувач вводить пароль від свого облікового запису та натискає «Видалити акаунт». Користувач отримує повідомлення про успішне видалення акаунту та перенаправляється на вітальну сторінку.
Extension	Якщо було введено неправильний пароль від акаунту, то користувач отримає повідомлення про помилку під час видалення акаунту.
Post-Condition	Користувач видалив свій акаунт та усі створені ним дані із застосунку.

2.2 Розроблення функціональних вимог

Програмне забезпечення розділене на модулі. Кожен модуль має свій певний набір функцій. В таблиці 2.23 наведено загальну модель вимог, а в таблицях 2.24 – 2.25 наведений опис функціональних вимог до програмного забезпечення. Матрицю трасування вимог можна побачити на рисунку 2.1.

Таблиця 2.23 – Загальна модель вимог

№	Назва	ID Вимоги	Пріоритети	Ризики
	Реєстрація користувача	FR-1	Високий	Високий
	Вхід в акаунт	FR-2	Високий	Високий
	Управління профілем користувача	FR-3	Низький	Низький
	Управління реєстраційними картками	FR-4	Середній	Середній
	Перегляд та пошук користувачів	FR-5	Низький	Низький
6	Управління подіями	FR-6	Високий	Середній
7	Створення повторюваних подій	FR-7	Високий	Середній

Продовження таблиці 2.23

8	Додавання події до Google Calendar	FR-8	Низький	Низький
9	Фільтрування та сортування подій	FR-9	Високий	Середній
10	Управління нагадуваннями про події	FR-10	Високий	Середній
11	Отримання нагадувань через Email	FR-11	Середній	Середній
12	Отримання нагадувань через Telegram	FR-12	Середній	Середній
13	Управління командами	FR-13	Високий	Середній
14	Управління запрошеннями до команд	FR-14	Середній	Низький
15	Вихід з акаунту	FR-15	Низький	Низький
	Видалення акаунту	FR-16	Низький	Низький

Таблиця 2.24 – Перелік функціональних вимог

ІД	Назва	Опис
FR-1	Реєстрація користувача	Вебзастосунок повинен надавати можливість реєстрації користувачу шляхом введення електронної пошти з корпоративним доменом університету, ім'я та прізвища, паролю та підтвердження паролю.
FR-2	Вхід в акаунт	Вебзастосунок повинен надавати можливість користувачу входу в створений акаунт за допомогою електронної пошти та паролю, із збереженням усіх даних акаунта.

Продовження таблиці 2.24

FR-3	Управління профілем користувача	Вебзастосунок повинен надавати можливість переглядати та модифікувати інформацію у профілі, який містить особисту інформацію, посаду, дані Telegram.
FR-4	Управління реєстраційними картками	Вебзастосунок повинен надавати можливість створювати, переглядати, модифікувати та видаляти реєстраційні картки юридичної особи.
FR-5	Перегляд та пошук користувачів	Вебзастосунок повинен надавати можливість перегляду та пошуку в загальному списку користувачів та відфільтрованому списку за відділом університету, у якому працюють користувачі.
FR-6	Управління подіями	Вебзастосунок повинен надавати можливість створювати, редагувати та видаляти особисті події та події для учасників команди, вказуючи перелік інформації про подію.
FR-7	Створення повторюваних подій	Вебзастосунок повинен надавати можливість вказувати періодичність багаторазового повторення події.
FR-8	Додавання події до Google Calendar	Вебзастосунок повинен надавати можливість перенести основні дані створеної події в Google Calendar.

Продовження таблиці 2.24

FR-9	Фільтрування та сортування подій	Вебзастосунок повинен надавати можливість виконувати фільтрацію та сортування списку подій.
FR-10	Управління нагадуваннями про події	Вебзастосунок повинен надавати можливість створювати, переглядати та видаляти нагадування про події, як під час створення події, так і для вже існуючої події.
FR-11	Отримання нагадувань через Email	Вебзастосунок повинен надавати можливість отримувати нагадування через вказаний у профілі Email.
FR-12	Отримання нагадувань через Telegram	Вебзастосунок повинен надавати можливість отримувати нагадування через Telegram-бота, в якому користувач пройшов верифікацію, попередньо вказавши ім'я користувача чи номер телефону Telegram в інформації профілю.
FR-13	Управління командами	Вебзастосунок повинен надавати можливість створювати, переглядати, редагувати та видаляти команди, що складаються із лідера та учасників, що є йому підпорядковані.
FR-14	Управління запрошеннями до команд	Вебзастосунок повинен надавати можливість створювати, переглядати, та видаляти запрошення у команду.

Продовження таблиці 2.24

FR-14	Управління запрошеннями до команд	Створити запрошення можна як під час створення команди, так і для існуючої команди, обравши бажаного користувача у списку. Запрошений користувач може прийняти або відхилити запрошення.
FR-15	Вихід з акаунту	Вебзастосунок повинен надавати можливість вийти із свого акаунту.
FR-16	Видалення акаунту	Вебзастосунок повинен надавати можливість видалити свій акаунт разом із усіма створеними даними.

	UC-1	UC-2	UC-3	UC-4	UC-5	UC-6	UC-7	UC-8	UC-9	UC-10	UC-11	UC-12	UC-13	UC-14	UC-15	UC-16	UC-17	UC-18	UC-19	UC-20	UC-21	UC-22
FR-1	+																					
FR-2		+																				
FR-3			+	+																	+	
FR-4																		+	+			
FR-5									+	+	+					+						
FR-6					+	+	+	+								+	+					
FR-7																						
FR-8							+															
FR-9						+		+														
FR-10					+	+										+						
FR-11					+	+										+						
FR-12					+	+										+					+	
FR-13									+	+	+	+	+			+						
FR-14									+		+			+	+							
FR-15																				+		
FR-16																						+

Рисунок 2.1 – Матриця трасування вимог

2.3 Розроблення нефункціональних вимог

Нефункціональні вимоги – це вимоги, які визначають якість роботи програмного забезпечення, та не стосуються конкретного функціоналу чи функцій програми. Нефункціональні вимоги визначають атрибут якості програмного забезпечення.

Таблиця 2.25 – Нефункціональні вимоги

І Д	Назва	Опис
NFR-1	Зручність користувачького інтерфейсу	Веб-застосунок повинен бути простим, зручним та інтуїтивно зрозумілим.
NFR-2	Сумісність	Веб-застосунок повинен бути сумісним із різними браузерами для гарантування правильної роботи та відображення для широкого кола користувачів.
NFR-3	Безпека	Веб-застосунок повинен забезпечувати безпеку від CSRF-атак, XSS-атак, SQL-ін'єкцій, а також захищати конфіденційні дані від вільного доступу.
NFR-4	Надійність	Веб-застосунок повинен реагувати на виникнення помилок спричинених користувачем, не допускати виконання помилкових дій, та надавати інформативні повідомлення користувачеві про виниклі помилки.

2.4 Аналіз системних вимог

Оскільки розроблюваний програмний продукт є вебзастосунком, то доступ до нього здійснюється через будь-який сучасний браузер. Найпопулярнішим браузером на сьогодні є Google Chrome, який

підтримується більшістю операційних систем. Google Chrome сумісний із комп'ютерами з операційною системою Windows 7 та вище. Таким чином, далі наведено мінімальні системні вимоги для роботи ОС Windows 7 на персональному комп'ютері.

- Процесор: 1 гігагерц або швидший процесор чи SoC.
- Оперативна пам'ять: 1 гігабайт для 32-розрядної версії або 2 гігабайти для 64-розрядної версії.
- Обсяг дискового простору: 18 гігабайт для 32-розрядної ОС або 22 гігабайти для 64-розрядної ОС.
- Графічна плата: DirectX 9 або новішої версії з драйвером WDDM 1.0 або новіший.

2.5 Аналіз економічних показників програмного забезпечення

Функціональні точки[11] – це метрика, призначена для вимірювання розміру програмного забезпечення, що сфокусована на функціонал, який реалізує продукт. Метод функціональних точок має п'ять основних складових:

- ILF(Internal Logic Files): логічно пов'язана група даних, яка використовується програмою та зберігається у межах програми.
- EIF (External Interface Files): логічно пов'язана група даних, яка використовується програмою та зберігаються повністю за межами програми.
- EI (External Inputs): операції, ініційовані користувачем, які модифікують дані в системі.
- EO (External Outputs): операції формування та виведення даних, що включають в себе певну логіку або обчислення.
- EQ (External Queries): операції виведення даних, які не модифікують дані, а лише повертають результат.

Складність функцій, що пропонує застосунок, обчислюється за допомогою DET (Data Element Type) – унікального запису даних, та RET (Record Element Type) – підгрупа записів даних, що утворює логічну структуру.

Обчислимо кількість ILF, EIF, EI, EO та EQ для розроблюваного застосунку.

Внутрішні логічні файли застосунку (ILF):

- Користувачі – дані про зареєстрованих користувачів, інформація профілю.
- Реєстраційні картки юридичної особи – дані для формування сертифікатів та реєстрації АЦСК.
- Факультети та кафедри – додаткова інформація для вибору посадової приналежності.
- Події – дані про створені події, їх час, пріоритет, опис тощо.
- Нагадування – дані про нагадування з інформацією про подію, час доставки та метод доставки.
- Команди – дані про команди, учасників, керівників і опис.
- Запрошення – інформація про запрошення до команд.

Зовнішні інтерфейсні файли (EIF): застосунок не використовує зовнішніх джерел даних.

Зовнішні входи (EI):

- Реєстрація користувача.
- Авторизація користувача.
- Заповнення або оновлення профілю.
- Видалення профілю.
- Створення події.
- Редагування події.
- Видалення події.
- Створення нагадування.
- Видалення нагадування.
- Створення команди.

- Редагування команди.
- Видалення команди.
- Створення запрошення у команду.
- Відповідь на запрошення у команду.
- Видалення запрошення у команду.
- Створення реєстраційної картки.
- Редагування реєстраційної картки.
- Видалення реєстраційної картки.
- Авторизація через Telegram-акаунт.

Зовнішні виходи (EO):

- Надсилання листа для підтвердження реєстрації.
- Надсилання нагадування на електронну пошту.
- Надсилання нагадування через Telegram.
- Перенесення події у Google Calendar.
- Надсилання листа про запрошення у команду.

Зовнішні запити(EQ):

- Перегляд особистого профілю.
- Перегляд реєстраційних карток.
- Перегляд списку подій.
- Перегляд деталей події.
- Перегляд списку команд.
- Перегляд списку запрошень.
- Перегляд списку користувачів.

Наступний етап – підрахунок нескоригованих функціональних точок (UFP), пов'язаних з даними. На рисунках 2.2 та 2.3 наведені схеми для обчислення складності ILF та EIF, та відповідній їм кількості UFP.

	1-19 DET	20-50 DET	50+ DET
1 RET	Низький	Низький	Середній
2-5 RET	Низький	Середній	Високий
6+ RET	Середній	Високий	Високий

Рисунок 2.2 – Визначення складності ILF та EIF

Складність даних	Кількість UFP (ILF)	Кількість UFP (EIF)
Низький	7	5
Середній	10	7
Високий	15	10

Рисунок 2.3 – Визначення кількості UFP для ILF та EIF

У таблиці 2.26 наведено розрахунки кількості UFP для ILF.

Таблиця 2.26 – розрахунок UFP для ILF.

ILF	RET	DET	Складність	UFP
Користувачі	3	23	Середня	10
Реєстраційні картки	1	15	Низька	7
Факультети та кафедри	2	7	Низька	7
Події	2	21	Середня	10
Нагадування	1	7	Низька	7
Команди	1	6	Низька	7
Запрошення	1	7	Низька	7

Загалом, для ILF застосунку було отримано 55 ненормованих функціональних точок.

Далі йде обчислення кількості UFP для EI, EO та EQ. На рисунках 2.4 – 2.6 наведено схеми обчислення складності та кількості UFP.

EI	1-4 DET	5-15 DET	16+DET
0-1 FTR	Низький	Низький	Середній
2 FTR	Низький	Середній	Високий
3+ FTR	Середній	Високий	Високий

Рисунок 2.4 – Визначення складності EI

EO&EQ	1-5 DET	6-19 DET	20+ DET
0-1	Низький	Низький	Середній
2-3	Низький	Середній	Високий
4+	Середній	Високий	Високий

Рисунок 2.5 – Визначення складності EO та EQ

Складність транзакцій	Кількість UFP (EI, EQ)	Кількість UFP (EO)
Низький	3	4
Середній	4	5
Високий	6	7

Рисунок 2.6 – Визначення кількості UFP для EI, EO та EQ

У таблиці 2.27 наведено розрахунки кількості UFP для EI.

Таблиця 2.27 – розрахунок UFP для EI.

EI	FRT	DET	Складність	UFP
Реєстрація користувача	1	5	Низька	3
Авторизація користувача	1	2	Низька	3
Оновлення профілю	2	7	Середня	4
Видалення профілю	2	7	Середня	4

Продовження таблиці 2.27

Створення події	2	21	Висока	6
Редагування події	2	17	Висока	6
Видалення події	2	21	Висока	6
Створення нагадування	1	7	Низька	3
Редагування нагадування	1	5	Низька	3
Видалення нагадування	1	7	Низька	3
Створення команди	1	6	Низька	3
Редагування команди	1	4	Низька	3
Видалення команди	1	6	Низька	3
Створення запрошення	1	7	Низька	3
Редагування запрошення	1	5	Низька	3
Видалення запрошення	1	7	Низька	3
Створення реєстраційної картки	1	15	Середня	4

Продовження таблиці 2.27

Редагування реєстраційної картки	1	13	Низька	3
Видалення реєстраційної картки	1	15	Середня	4
Авторизація -акаунту	1	5	Низька	3

Загалом, для ЕІ застосунку було отримано 73 ненормованих функціональних точок. У таблиці 2.28 наведено розрахунки кількості UFP для ЕО.

Таблиця 2.28 – розрахунок UFP для ЕО.

ЕО	FRT	DET	Складність	UFP
Надсилання листа підтвердження реєстрації	1	1	Низька	4
Надсилання нагадування на електронну пошту	3	28	Висока	7
Надсилання нагадування через Telegram	3	28	Висока	7
Перенесення події у Google Calendar	1	3	Низька	4

Продовження таблиці 2.28

Надсилення листа про запрошення у команду	3	10	Середня	5
--	---	----	---------	---

Загалом, для ЕО застосунку було отримано 27 ненормованих функціональних точок. У таблиці 2.29 наведено розрахунки кількості UFP для EQ.

Таблиця 2.29 – розрахунок UFP для EQ.

EQ	FRT	DET	Складність	UFP
Перегляд особистого профілю	3	8	Середня	4
Перегляд реєстраційних карток	1	15	Низька	3
Перегляд списку подій	2	20	Висока	6
Перегляд деталей події	3	25	Висока	6
Перегляд списку команд	3	6	Низька	4
Перегляд списку запрошень	3	6	Низька	4
Перегляд списку користувачів	2	6	Середня	5

Загалом, для EQ застосунку було отримано 29 ненормованих функціональних точок.

У сумі, кількість нескоригованих функціональних точок становить 184.

Для обчислення нормованої кількості функціональних точок, визначимо основні характеристики системи:

- Обмін даними – 0. Продукт є автономним додатком.
- Розподілена обробка даних – 0. Додаток не потребує розподіленої обробки даних.
- Продуктивність – 3. Додаток має забезпечувати короткий час відгуку для підтримання швидкодії навігації та вчасне виконання завдань з надсилання нагадувань.
- Обмеження по апаратним ресурсам – 0. Немає обмежень.
- Транзакційне навантаження – 2. Значне навантаження не передбачається, проте варто застосувати кешування даних.
- Інтенсивність взаємодії з користувачем – 3. Продукт має достатньо велику кількість інтерактивних операцій.
- Ергономіка – 3. Продукт має бути зручним та зрозумілим користувачам.
- Інтенсивність зміни даних (ILF) користувачами – 3. Застосунок передбачає постійну генерацію та редагування даних.
- Складність обробки даних – 3. Базова обробка даних, фільтрування, сортування.
- Повторне використання – 0. Не вимагається.
- Зручність інсталяції – 0. Сайт працює як веб-застосунок, тому спеціальних вимог до інсталяції немає.
- Зручність адміністрування – 1. Адміністрування на даний момент не вимагається.
- Можливість портування – 1. Портування на даний момент не вимагається.
- Гнучкість – 0. Не вимагається.

Підсумовуючи основні характеристики системи, отримуємо значення підсумкового ступеня впливу TDI (Total Degree of Influence) = 19.

Обчислимо нормуючий фактор VAF:

$$VAF = (TDI * 0.01) + 0.65 = 0.84.$$

Тепер можна отримати нормовану кількість функціональних точок, домноживши кількість ненормованих функціональних точок на нормуючий фактор:

$$FP = UFPC * VAF = 184 * 0.84 = 154.4.$$

Кількість функціональних точок 154.5 свідчить про застосунок від малого до середнього розміру. В сучасних мовах програмування реалізація однієї функціональної точки займає близько 50 рядків коду, що дозволяє оцінити розмір вихідного продукту в 7725 ліній коду.

2.6 Постановка завдання на розробку програмного забезпечення

У ході виконання даної роботи необхідно реалізувати вебзастосунок для планування та нагадування про важливі події, який повинен містити функціонал створення одноразових та повторюваних подій із можливістю додавати додаткову інформацію про подію, створення команд та зручне запрошення користувачів у команди, зберігання даних реєстраційної картки юридичної особи. Вебзастосунок повинен бути зручним та зрозумілим у використанні, доступним, безпечним, та стійким до помилок.

Метою розробки є спрощення управління подіями, нагадуваннями та взаємодії між користувачами за рахунок створення зручного вебзастосунку для планування та організації діяльності у межах університету.

Висновки до розділу

У розділі «Розроблення вимог до програмного забезпечення» було детально проаналізовано можливі варіанти використання застосунку,

результат яких було відображено у діаграмі варіантів використання та таблицях із їх описом.

Було сформовано функціональні та нефункціональні вимоги до застосунку та створено матрицю трасування вимог, яка детально ілюструє який основний функціонал застосовується при кожному варіанті використання.

На основі сформованих варіантів використання та функціональних вимог було проведено дослідження економічних показників проекту за допомогою методу Функціональних точок. У результаті дослідження була отримана кількість функціональних точок 154.5, що дозволяє оцінити потенційний розмір програмного забезпечення в 7725 ліній коду.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Розроблюваний застосунок побудований на основі клієнт-серверної архітектури, серверна частина якої має монолітну архітектуру. Застосунок складається із сервісів, які розгорнуті у єдиній мережі за допомогою Docker Compose[12]. Такий метод розгортання забезпечує ізоляцію робочого середовища, зручність при масштабуванні, та запускати застосунок на будь-якому комп'ютері чи сервері із гарантією, що всюди він працюватиме однаково.

Застосунок складається із таких сервісів:

- api: Backend застосунку, реалізує основні бізнес-процеси;
- client: Frontend застосунку, надає користувацький інтерфейс для взаємодії із застосунком;
- nginx: зворотній проксі між api та client, додатково виконує кешування медіа даних;
- postgres: база даних застосунку;
- mailpit: локальний поштовий сервер, який дозволяє тестувати надсилання та отримання Email-повідомлень у ході розробки;
- redis: брокер повідомлень для асинхронних Celery задач;
- celery_worker: виконавць асинхронних задач Celery;
- flower: моніторинг стану Celery задач;
- telegram_bot: Telegram-бот для надсилання нагадувань.

UML-діаграма розгортання, яка ілюструє взаємодію між контейнерами застосунку, наведена у графічному матеріалі, аркуш 2.

3.2 Архітектурні рішення та обґрунтування вибору засобів розробки

Для розробки веб-застосунку можуть бути використані різні мови програмування, такі як C# із використанням фреймворку ASP.NET, Python із використанням Django або Flask, Java із використанням Spring Framework. Для розробки вебзастосунку було вирішено використати мову програмування Python[13], оскільки вона є однією із найбільш популярних на сьогодні, має простий та зручний синтаксис, а головною перевагою є наявність великої кількості бібліотек та фреймворків, які дозволяють значно спростити процес розробки.

Для розробки Backend частини застосунку на мові програмування Python було обрано фреймворк Django[14]. Django є високорівневим фреймворком, що надає вбудовані рішення для багатьох аспектів розробки веб-додатків, таких як автентифікація, адміністративний інтерфейс тощо, завдяки чому він дозволяє швидко створювати великі та функціональні застосунки. У той час Flask є більш низькорівневим та надає менше готових рішень, що надає більшій гнучкості та контролю, проте ускладнює процес розробки.

Для розробки API було обрано Django Rest Framework[15], що є потужним інструментом для розширення функціоналу Django, що дозволяє швидко, гнучко та безпечно реалізовувати всю необхідну додатку логіку.

Для розробки Frontend частини застосунку було обрано фреймворк Next.js[16], що побудований на базі React[17], та значно розширює його можливості. Значною перевагою React є можливість створювати компоненти – шаблонні об'єкти сторінки, які можна повторно використовувати у багатьох місцях застосунку. Next.js дозволяє організовувати роутинг додатку, оптимізує зображення, посилання та шрифти, забезпечує серверний та клієнтський рендеринг, та підтримує інтеграцію з TypeScript та Tailwind.

Як середовище для розробки застосунку було обрано PyCharm від компанії JetBrains. Його головною перевагою є адаптованість під розробку саме на мові програмування Python, він містить багато вбудованих функцій та

плагінів, які значно спрощують розробку. Так само добре PyCharm підтримує і розробку за допомогою React та Next.js.

3.3 Конструювання програмного забезпечення

Основою арі компонента є архітектурний патерн MVC, проте Django використовує його модифіковану версію – MVT (Model-View-Template)[13]. У даному патерні контролер було замінено на шаблон, функціонал контролера тепер виконує вигляд, а шаблон відповідає за представлення даних та сторінок користувачу. Django Rest Framework ще далі модифікує цю концепцію, і замість Template тепер використовується Serializer (серіалізатор). Серіалізатори призначені для перетворення даних із Python-об'єктів у JSON об'єкти. Таким чином, коли клієнт виконує запит на отримання даних, то йому повернуться оброблені серіалізатором дані у вигляді JSON об'єкту.

Кожен проект Django складається із окремих додатків – apps, кожен з яких реалізовує окремий набір функціональностей застосунку. Кожен додаток має власний набір моделей, виглядів та серіалізаторів та посилань, через які можна взаємодіяти із даними. Розроблюваний застосунок складається із 8 додатків, опис яких наведено у таблиці 3.1.

Таблиця 3.1 – Додатки проекту Django

Назва додатку	Опис
common	Додаток містить універсальний функціонал, який можна застосовувати у будь яких інших додатках.
units	Додаток реалізує функціонал взаємодії із підрозділами університету – інститутами чи факультетами, та кафедрами.
users	Додаток реалізує функціонал базового користувача Django, реєстрацію та автентифікацію.

Продовження таблиці 3.1

profiles	Додаток реалізує функціонал особистого профілю користувача.
registration_cards	Додаток реалізує функціонал управління реєстраційними картками юридичної особи.
events	Додаток реалізує функціонал управління одноразовими та багаторазовими подіями.
notifications	Додаток реалізує управління нагадування про події та їх надсилання користувачам.
teams	Додаток реалізує управління командами та запрошеннями в команди.

3.3.1 Опис структури бази даних

В якості системи управління базами даних використовується PostgreSQL. Django повністю підтримує роботу із PostgreSQL, що дозволяє спростити управління об'єктами бази даних за рахунок використання ORM(Object-Relational Mapping)[18]. Такий підхід дозволяє оперувати таблицями бази даних як класами, а полями бази даних – як атрибутами об'єктів класу. Також Django ORM дозволяє створювати таблиці із зв'язком багато-до-багатьох напрямку, без потреби вручну створювати проміжку таблицю.

База даних серверу призначена для зберігання користувачів та їх профілів, даних про заплановані події та нагадування, даних про команди та запрошення, дані реєстраційної картки юридичної особи. Опис таблиць бази даних наведено у таблицях 3.2-3.11. Модель бази даних наведена у графічному матеріалі, аркуш 3.

Таблиця 3.2 – Опис таблиці User

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор користувача у форматі UUID.
email	varchar	Електронна пошта користувача, яка використовується як логін.
password	archar	Пароль користувача
first_name	varchar	Ім'я користувача
last_name	varchar	Прізвище користувача

Таблиця 3.3 – Опис таблиці Profile

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор профіля у форматі UUID.
user	varchar	Користувач, якому належить профіль.
position	varchar	Посада користувача.
department	varchar	Кафедра, на якій працює користувач.
avatar	varchar	Адреса у файловому сховищі, за яким зберігається зображення аватара користувача.
created_at	datetime	Момент часу створення сутності Profile.
updated_at	datetime	Момент часу останнього оновлення сутності Profile.

Таблиця 3.4 – Опис таблиці TelegramData

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор таблиці із даними про Telegram-користувача у форматі UUID.
profile	varchar	Профіль користувача, якому належать дані Telegram акаунта.
telegram_username	varchar	Унікальне ім'я користувача акаунту Telegram.
telegram_first_name	varchar	Ім'я користувача в Telegram.
telegram_last_name	varchar	Прізвище користувача в Telegram.
telegram_user_id	bigint	Унікальний ідентифікатор Telegram акаунту.
is_verified	boolean	Чи було авторизовано користувача у Telegram-боті.
created_at	datetime	Момент часу створення сутності TelegramData.
updated_at	datetime	Момент часу останнього оновлення сутності TelegramData.

Таблиця 3.5 – Опис таблиці Faculty

Назва поля	Тип даних	Опис
id	int	Унікальний ідентифікатор таблиці факультету.

Продовження таблиці 3.5

name	varchar	Назва факультету.
abbreviation	varchar	Абревіатура факультету.

Таблиця 3.6 – Опис таблиці Department

Назва поля	Тип даних	Опис
id	int	Унікальний ідентифікатор таблиці кафедри.
faculty	int	Факультет, до якого належить кафедра.
name	varchar	Назва кафедри.
abbreviation	varchar	Абревіатура кафедри.

Таблиця 3.7 – Опис таблиці Event

Назва поля	Тип даних	Опис
	varchar	Унікальний ідентифікатор події у форматі UUID.
created_by	varchar	Користувач, яким було створено подію.
assigned_to	varchar	Список користувачів, яким призначена подія (ManyToMany).
title	varchar	Назва події.
start_datetime	datetime	Момент початку події.
description	varchar	Опис події.

Продовження таблиці 3.7

location	varchar	Додаткова локаційна мітка для події.
link	varchar	Додаткове посилання для події.
priority	smallint	Пріоритет події.
image	varchar	Адреса у файловому сховищі, за яким зберігається зображення, додане до події.
is_recurring	boolean	Чи є подія повторюваною.
created_at	datetime	Момент часу створення сутності Event.
updated_at	datetime	Момент часу останнього оновлення сутності Event.

Таблиця 3.8 – Опис таблиці RecurringEvent

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор повторюваної події у форматі UUID.
event	varchar	Основна подія.
recurrence_rule	varchar	Періодичність повторюваності події.
recurrence_end_datetime	datetime	Момент завершення повторюваності події.
created_at	datetime	Момент часу створення сутності RecurringEvent.
updated_at	datetime	Момент часу останнього оновлення сутності RecurringEvent.

Таблиця 3.9 – Опис таблиці Notification

Назва поля	Тип даних	Опис
id	int	Унікальний ідентифікатор нагадування.
event	varchar	Подія, для якої створено нагадування.
created_by	varchar	Користувач, якому потрібно надіслати нагадування.
notification_method	varchar	Метод нагадування.
notification_datetime	datetime	Момент нагадування.
celery_task_id	varchar	Ідентифікатор запланованої асинхронної задачі Celery для відправлення нагадування.
is_sent	boolean	Чи було надіслано нагадування.

Таблиця 3.10 – Опис таблиці RegistrationCard

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор картки реєстрації у форматі UUID.
user	varchar	Користувач, що створив реєстраційну картку.
organization_name	varchar	Назва організації.
edrpou_code	varchar	ЄДРПОУ код організації.
region	varchar	Область, де розташована організація.

Продовження таблиці 3.10

city	varchar	Населений пункт, де розташована організація.
full_name	varchar	Повне ім'я заявника.
id_number	varchar	Ідентифікаційний код заявника.
keyword_phrase	varchar	Питання до ключової фрази
voice_phrase	varchar	Ключова фраза голосової аутентифікації
email	varchar	Електронна адреса заявника.
electronic_seal_name	varchar	Назва електронної печатки.
electronic_seal_keyword	varchar	Ключова фраза до печатки.
created_at	datetime	Момент часу створення сутності RegistrationCard.
updated_at	datetime	Момент часу останнього оновлення сутності RegistrationCard.

Таблиця 3.11 – Опис таблиці Team

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор команди у форматі UUID.
created_by	varchar	Користувач, який створив команду.

Продовження таблиці 3.11

members	varchar	Користувачі, які є учасниками команди (ManyToMany).
name	varchar	Назва команди.
description	varchar	Опис команди.
created_at	datetime	Момент часу створення сутності RegistrationCard.
updated_at	datetime	Момент часу останнього оновлення сутності RegistrationCard.

Таблиця 3.12 – Опис таблиці Invitation

Назва поля	Тип даних	Опис
id	varchar	Унікальний ідентифікатор запрошення у форматі UUID.
created_by	varchar	Користувач, який створив запрошення у команду.
sent_to	varchar	Користувач, якому було надіслано запрошення у команду.
team	varchar	Команда, у яку запрошують користувача.
status	varchar	Статус запрошення.
created_at	datetime	Момент часу створення сутності RegistrationCard.

Продовження таблиці 3.12

updated_at	datetime	Момент часу останнього оновлення сутності RegistrationCard.
------------	----------	---

Опис утиліт, бібліотек та іншого стороннього програмного забезпечення, що використовується у розробці наведено в таблиці 3.13.

Таблиця 3.13 – Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	PyCharm	Головне середовище розробки програмного забезпечення.
	Django	Фреймворк, на основі якого побудовано застосунок.
3	Django Rest Framework	Бібліотека для проектування API інтерфейсів, за допомогою якої було побудовано Backend застосунку.
4	Django Taggit	Додавання тегів до подій.
5	Django Filter	Фільтрація об'єктів при запитах.
6	Django PhoneNumber Field	Поле для зберігання номерів телефонів.
7	Phonenumbers	Бібліотека для валідації та форматування номерів телефонів.
8	Redis	Брокер повідомлень для асинхронних задач та сховище даних для боту Telegram.
9	Celery	Виконання асинхронних задач.
10	Flower	Моніторинг стану Celery-задач.
11	Django Celery Email	Відправка листів на електронну пошту через Celery.

Продовження таблиці 3.13

12	Djoser	Реалізація API для реєстрації, автентифікації та управління користувачами.
13	Python Dotenv	Зчитування змінних середовища з .env файлів.
14	DRF yasg	Генерування API документації на основі створеного функціоналу.
15	Argon2	Додаткове хешування паролів.
16	Python Magic	Визначення MIME типу зображень.
17	Python Dateutil	Генерація відносних часових проміжків.
18	Django Encrypted Model Field	Поле для бази даних, яке зберігає дані у зашифрованому вигляді.
19	Aiogram	Бібліотека для створення Telegram-бота.
20	React	Фреймворк для створення користувацького інтерфейсу застосунку.
21	Next	Основний фреймворк, за допомогою якого працює клієнтська частина застосунку.
22	Zod	Валідація форм та типів на Frontend.
23	Redux	Глобальне управління станом у React застосунку, побудова API запитів та застосування кешування даних.
24	Shadcn	Бібліотека для побудови компонентів користувацького інтерфейсу.
25	Cookie Next	Читання даних, збережених у Cookie.
26	React Hook Form	Обробка та валідація форм на Frontend.
27	React Select	Поле для випадаючих списків у формах із розширеним функціоналом.
28	React Tag Input	Поле для вводу тегів у формі.

Продовження таблиці 3.13

29	React Toastify	Відображення інтерактивних повідомлень у відповідь на дії користувача.
30	Tailwind	Стилізація користувацького інтерфейсу.
31	Date FNS	Форматування дати та часу.
32	Postman	Програмне забезпечення, що використовувалось для тестування API інтерфейсів, та клієнтських запитів.

Тексти програмного коду наведені в окремому документі «Текст програми».

3.4 Аналіз безпеки даних

Застосунок використовує стандартні механізми захисту даних, які надають фреймворки Django та Next.js.

Захист від міжсайтового скриптингу (XSS)[19]. XSS-атаки дозволяють зловмисникам впроваджувати сценарії на стороні клієнта в браузері інших користувачів. Зазвичай це досягається шляхом зберігання шкідливих сценаріїв у базі даних, де вони будуть отримані та відображені іншим користувачем. Використання React дозволяє екранувати вміст JSX файлів, що мінімізує ризики XSS атак.

Захист від підробки міжсайтових запитів (CSRF)[20]. Атаки CSRF дозволяють зловмисникам виконувати дії з використанням облікових даних іншого користувача без відома або згоди цього користувача. Django забезпечує захист за допомогою механізму CSRF-токенів, які додаються до кожного POST запиту, та які перевіряються на сервері.

Захист від SQL-ін'єкцій[21]. Це тип атаки, коли зловмисник може виконати довільний код SQL у базі даних. Це може призвести до видалення записів або витіку даних. У застосунку усі запити виконуються через Django

ORM, який автоматично параметризує запити, що запобігає можливості ін'єкції стороннього SQL коду.

Застосунок забезпечує безпечну авторизацію через Telegram-акаунт користувача, що є необхідним для можливості отримання нагадувань через Telegram-бота. Для цього використовується Telegram Login Widget[22], що є офіційним методом надання застосунку та Telegram-боту даних Telegram-акаунту користувача та дозволу отримувати повідомлення від бота.

Висновки до розділу

У розділі «Конструювання та розроблення програмного забезпечення» було детально розглянуто процес проектування та реалізації вебзастосунку для планування та нагадування про події.

Було обгрунтовано вибір клієнт-серверної архітектури з використанням монолітного бекенду на базі фреймворку Django та фронтенду на базі Next.js. Бекенд застосунку складається із окремих логічних модулів – додатків, кожен з яких реалізує окрему функціональність: керування профілем, подіями, нагадуваннями, командами, реєстраційними картками тощо.

Базу даних застосунку було розроблено із використанням PostgreSQL, опираючись на Django ORM, що спрощує управління даними та надає додатковий шар безпеки.

Для інтеграції Telegram було використано Telegram Login Widget, що забезпечує офіційний механізм авторизації користувача за допомогою Telegram-акаунту. Це надає можливість користувачам створювати та отримувати нагадування через спроектованого Telegram-бота.

Усі компоненти застосунку організовано у вигляді окремих контейнерів та розгорнуто за допомогою Docker Compose, що забезпечує їх ізоляцію та зручність масштабування проекту.

Було проаналізовано безпеку даних та методи, якими застосунок її забезпечує. Застосунок забезпечує безпеку проти міжсайтового скриптингу, підробки міжсайтових запитів та SQL-ін'єкцій.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Аналіз якості ПЗ

Було здійснено аналіз якості коду Back-End складової вебзастосунку. Якісний код забезпечує простоту його читання, аналізу та пошуку помилок, безпеку даних та швидкодію застосунку.

Для аналізу коду на логічні, синтаксичні помилки та його відповідність стандарту PEP8 було використано бібліотеку flake8. Часткові результати первинного аналізу проекту із використанням flake8 наведені на рисунках 4.1 та 4.2.

```
(.venv) PS C:\Users\Ярослав\PycharmProjects\EventsReminder\api> flake8 --ignore=E501
.\apps\common\admin.py:1:1: F401 'django.contrib.admin' imported but unused
.\apps\common\cookie_auth.py:3:1: F401 'rest_framework_simplejwt.authentication.AuthUser' imported but unused
.\apps\common\cookie_auth.py:5:1: F401 'rest_framework_simplejwt.tokens.Token' imported but unused
.\apps\common\cookie_auth.py:11:1: W293 blank line contains whitespace
.\apps\common\cookie_auth.py:24:1: W293 blank line contains whitespace
.\apps\common\permissions.py:1:1: F401 'rest_framework.permissions.BasePermission' imported but unused
.\apps\common\renderers.py:19:1: W293 blank line contains whitespace
.\apps\common\tests\test_validators.py:1:1: F401 'apps.common.validators.validate_image_type' imported but unused
.\apps\common\tests\test_validators.py:1:1: F401 'apps.common.validators.validate_image_size' imported but unused
.\apps\common\tests\test_validators.py:2:1: F401 'pytest' imported but unused
.\apps\common\validators.py:17:31: E201 whitespace after '('
.\apps\common\views.py:1:1: F401 'django.shortcuts.render' imported but unused
.\apps\events\apps.py:9:1: W293 blank line contains whitespace
.\apps\events\apps.py:11:9: F401 'apps.events.signals' imported but unused
.\apps\events\management\commands\reschedule_recurring_events.py:6:1: F401 'celery.result.AsyncResult' imported but unused
.\apps\events\permissions.py:13:1: W293 blank line contains whitespace
.\apps\events\permissions.py:23:50: W291 trailing whitespace
.\apps\events\serializers.py:11:1: F401 'celery.result.AsyncResult' imported but unused
.\apps\events\signals.py:1:1: F401 'django.db.models.signals.post_save' imported but unused
.\apps\events\signals.py:4:1: F401 '.recurring.reschedule_recurring_event' imported but unused
```

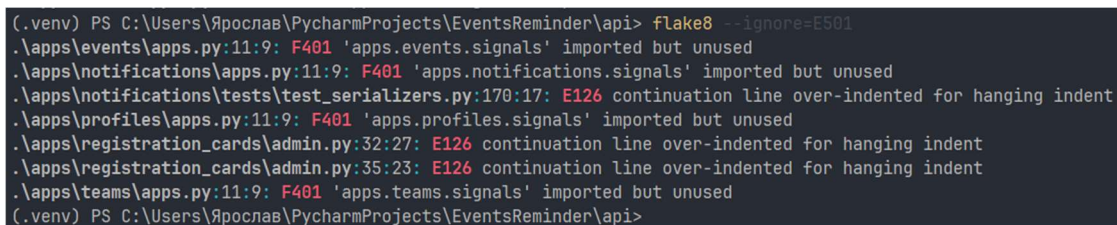
Рисунок 4.1 – Аналіз коду за допомогою flake8

```
.\apps\profiles\views.py:1:1: F401 'django.shortcuts.get_object_or_404' imported but unused
.\apps\profiles\views.py:16:1: F401 'time' imported but unused
.\apps\registration_cards\admin.py:13:1: E302 expected 2 blank lines, found 1
.\apps\registration_cards\admin.py:26:1: W293 blank line contains whitespace
.\apps\registration_cards\admin.py:31:54: W291 trailing whitespace
.\apps\registration_cards\admin.py:32:55: W291 trailing whitespace
.\apps\registration_cards\admin.py:50:1: E305 expected 2 blank lines after class or function definition, found 1
.\apps\registration_cards\models.py:14:1: W293 blank line contains whitespace
.\apps\registration_cards\models.py:32:74: W291 trailing whitespace
.\apps\registration_cards\models.py:34:84: W291 trailing whitespace
.\apps\registration_cards\models.py:43:1: W293 blank line contains whitespace
.\apps\registration_cards\serializers.py:11:1: W293 blank line contains whitespace
.\apps\registration_cards\urls.py:8:58: W291 trailing whitespace
.\apps\registration_cards\urls.py:10:58: W291 trailing whitespace
.\apps\registration_cards\views.py:15:1: W293 blank line contains whitespace
.\apps\teams\admin.py:20:1: W293 blank line contains whitespace
.\apps\teams\apps.py:11:9: F401 'apps.teams.signals' imported but unused
.\apps\teams\models.py:31:1: W293 blank line contains whitespace
.\apps\teams\models.py:36:1: W293 blank line contains whitespace
.\apps\teams\permissions.py:13:1: W293 blank line contains whitespace
.\apps\teams\permissions.py:24:1: W293 blank line contains whitespace
.\apps\teams\serializers.py:5:1: F401 'djoser.serializers.UserSerializer' imported but unused
.\apps\teams\tests\test_emails.py:3:1: F401 'apps.profiles.tests.factories.UserWithProfileFactory' imported but unused
.\apps\teams\tests\test_serializers.py:2:1: F401 'apps.teams.models.Team' imported but unused
.\apps\teams\tests\test_tasks.py:1:1: F401 'apps.teams.models.Invitation' imported but unused
.\apps\units\admin.py:2:1: F401 'django.utils.translation.gettext_lazy as _' imported but unused
```

Рисунок 4.2 – Аналіз коду за допомогою flake8

Основними скаргами бібліотеки є імпортування функцій, яких після цього не було використано у коді, а також наявність зайвих пробілів в кінці ліній або на місці порожніх ліній.

Результати повторного аналізу коду із використанням flake8 після виправлення основних зауважень наведено на рисунку 3.3.



```
(.venv) PS C:\Users\Ярослав\PycharmProjects\EventsReminder\api> flake8 --ignore=E501
.\apps\events\apps.py:11:9: F401 'apps.events.signals' imported but unused
.\apps\notifications\apps.py:11:9: F401 'apps.notifications.signals' imported but unused
.\apps\notifications\tests\test_serializers.py:170:17: E126 continuation line over-indented for hanging indent
.\apps\profiles\apps.py:11:9: F401 'apps.profiles.signals' imported but unused
.\apps\registration_cards\admin.py:32:27: E126 continuation line over-indented for hanging indent
.\apps\registration_cards\admin.py:35:23: E126 continuation line over-indented for hanging indent
.\apps\teams\apps.py:11:9: F401 'apps.teams.signals' imported but unused
(.venv) PS C:\Users\Ярослав\PycharmProjects\EventsReminder\api>
```

Рисунок 4.3 – Повторний аналіз коду за допомогою flake8

Повторний аналіз коду відображає невикористані імпорти із модулів signals, що насправді не є помилкою, оскільки вони дозволяють відслідковувати сигнали в Django та не можуть бути видалені. Інші помилки вказують на неправильно використані відступи при переносі багаторядкових виразів. Загалом, отримані у результаті повторного аналізу попередження виникають через специфіку роботи Django проекту та автоматичного форматування коду середовища розробки PyCharm.

Для більш повної оцінки якості ПЗ, було застосовано наступні метрики:

- Цикломатична складність: дозволяє оцінити складність коду на основі кількості використаних конструкцій та операторів;
- Індекс підтримуваності коду: дозволяє визначити наскільки складно буде працювати із кодом в майбутньому. Ґрунтується на кількості рядків вихідного коду, цикломатичній складності та метриці Холстеда.

Обчислення описаних метрик забезпечує бібліотека radon[23], результати роботи якої наведено на рисунках 4.4 – 4.6.

```

apps\teams\permissions.py
  C 19:0 IsOwnerOrMember - A (4)
  C 8:0 IsOwner - A (3)
  M 25:4 IsOwnerOrMember.has_object_permission - A (3)
  M 14:4 IsOwner.has_object_permission - A (2)
apps\teams\serializers.py
  C 9:0 TeamCreateSerializer - A (5)
  M 20:4 TeamCreateSerializer.validate_members_ids - A (5)
  M 92:4 InvitationCreateSerializer.validate - A (5)
  C 68:0 InvitationCreateSerializer - A (3)
  C 125:0 InvitationRespondSerializer - A (3)
  M 37:4 TeamCreateSerializer.create - A (2)
  M 78:4 InvitationCreateSerializer.validate_team - A (2)
  M 85:4 InvitationCreateSerializer.validate_sent_to - A (2)
  M 132:4 InvitationRespondSerializer.validate - A (2)
  M 140:4 InvitationRespondSerializer.update - A (2)
  C 51:0 TeamUpdateSerializer - A (1)
  C 57:0 TeamDetailSerializer - A (1)
  M 107:4 InvitationCreateSerializer.create - A (1)
  C 112:0 InvitationDetailSerializer - A (1)
apps\teams\signals.py
  F 8:0 create_celery_invitation_task - A (2)
apps\teams\tasks.py
  F 10:0 send_team_invitation_email_task - A (2)

```

Рисунок 4.4 – Часткові результати цикломатичної складності

```

512 blocks (classes, functions, methods) analyzed.
Average complexity: A (2.556640625)

```

Рисунок 4.5 – Середній результат цикломатичної складності проекту

```

apps\common\apps.py - A (100.00)
apps\common\celery.py - A (69.72)
apps\common\cookie_auth.py - A (61.48)
apps\common\models.py - A (100.00)
apps\common\permissions.py - A (100.00)
apps\common\renderers.py - A (84.62)
apps\common\uploads.py - A (100.00)
apps\common\validators.py - A (60.50)
apps\common\__init__.py - A (100.00)
apps\common\migrations\__init__.py - A (100.00)
apps\events\admin.py - A (61.89)
apps\events\apps.py - A (100.00)
apps\events\filters.py - A (100.00)
apps\events\models.py - A (64.89)
apps\events\permissions.py - A (87.33)
apps\events\recurring.py - A (100.00)
apps\events\serializers.py - A (43.67)
apps\events\signals.py - A (57.56)
apps\events\tasks.py - A (66.45)
apps\events\urls.py - A (100.00)
apps\events\views.py - A (65.72)
apps\events\__init__.py - A (100.00)
apps\events\management\commands\reschedule_recurring_events.py - A (62.96)

```

Рисунок 4.6 – Часткові результати обчислення індексу підтримуваності коду

Середній результат цикломатичної складності 2.55/100 означає, що використані у проекті процедури є не складними та мають мінімальний ризик виникнення несправностей.

Середній результат індексу підтримуваності коду становить 83.63/100, що вказує на те, що файли проекту мають дуже високий рівень підтримуваності.

4.2 Опис процесів тестування

Тестування виконується згідно документу «Програма та методика тестування».

Для тестування проектів застосовується концепція піраміди тестування[24], визначає оптимальний спосіб організації тестів в процесі розробки програмного забезпечення. Вона зазвичай складається із таких основних рівнів:

- Нижчий рівень – Unit-Tests. Unit-тести перевіряють окремі компоненти програмного забезпечення (наприклад, функції, методи чи класи). Unit-тести повинні бути легкими, швидкими та автоматизованими.
- Середній рівень – інтеграційне тестування. На середньому рівні здійснюється інтеграційне тестування, яке перевіряє взаємодію між різними компонентами або модулями програми. Ці тести можуть охоплювати взаємодію класів або служб, обмін даними між ними тощо.
- Верхній рівень – End-to-End Tests. Це тести рівня кінцевого користувача, які перевіряють, чи весь програмний продукт працює правильно як єдина система. End-to-End тести можуть містити тестування користувацького інтерфейсу та сценарії взаємодії з програмою від початку до кінця.

Було вирішено застосувати для підходи до тестування: unit-тести та мануальне тестування на рівні End-To-End.

Для unit-тестування було використано бібліотеки `pytest`, `factoryboy`, `faker` та `pytest-cov`. На рисунку 4.7 наведено початковий звіт із покриття функціоналу застосунку тестами.

Coverage report: 20%

Files Functions Classes

coverage.py v7.8.2, created at 2025-06-10 20:53 +0300

File	statements ▼	missing	excluded	coverage
apps/teams/views.py	151	151	0	0%
apps/events/serializers.py	137	137	0	0%
apps/teams/serializers.py	102	102	0	0%
apps/events/views.py	90	90	0	0%
apps/profiles/views.py	85	85	0	0%
apps/events/models.py	69	19	0	72%
apps/profiles/serializers.py	64	64	0	0%

Рисунок 4.7 – Початкове покриття коду тестами

На початковому етапі розробки unit-тестів, лише 20% коду покрито тестами, що не є достатнім для гарантування стабільної роботи застосунку. На рисунку 4.8 наведено фрагмент коду, який потребує тестування. Це клас, який забезпечує отримання списку подій користувачем, та містить логіку перевірки права доступу до даних, фільтрації, сортування.

```

27 class MyEventsListAPIView(generics.ListAPIView):
28     ...
29     API view to retrieve a list of events created by
30     or assigned to the authenticated user.
31     ...
32     queryset = Event.objects.all().prefetch_related('tags', 'assigned_to')
33     serializer_class = EventDetailSerializer
34     permission_classes = [IsAuthenticated, IsOwnerOrAssignedTo]
35     pagination_class = StandardResultsSetPagination
36     filter_backends = [DjangoFilterBackend, drf_filters.SearchFilter]
37     filterset_class = EventFilter
38     search_fields = ['title', 'description', 'tags__name']
39     renderer_classes = [JSONRenderer]
40     object_label = 'events'
41
42     def get_queryset(self):
43         created_events = Event.objects.filter(created_by=self.request.user)
44         assigned_events = Event.objects.filter(assigned_to=self.request.user)
45         return (created_events | assigned_events).distinct().order_by('start_datetime')

```

Рисунок 4.8 – Фрагмент коду, що потребує тестування

На рисунку 4.9 наведено тест, у якому перевіряється, що користувач у запиті на отримання подій отримує лише ті події, які були ним створені, а також події, які були йому призначені. Для цього створюються одноразові об'єкти подій, імітується запит від користувача, та перевіряється успішність запиту і те, що користувач отримав лише ті події, на які він має право доступу.

```
@pytest.mark.django_db
def test_my_events_list(client, normal_user):
    EventFactory.create_batch(3, created_by=normal_user)
    EventFactory.create_batch(2, assigned_to=[normal_user])
    EventFactory.create_batch(2)

    response = client.get(reverse('my-events-list'))

    assert response.status_code == 200
    assert len(response.data['results']) == 5
```

Рисунок 4.9 – Приклад тесту для перевірки отримання списку подій

Для перевірки коректності роботи інших частин застосунку, було створено unit-тести, які охоплюють такі ключові функції:

- коректну обробку запитів до створених ендпоїнтів;
- перевірку доступу користувача до даних;
- створення, оновлення та видалення об'єктів із відповідною бізнес-логікою;
- валідацію даних та обробку некоректних значень;
- створення та виконання асинхронних задач.

У результаті було створено 191 тест, за допомогою яких було досягнуто покриття коду на рівні 96%. На рисунку 4.10 наведено результат запуску тестів, а на рисунку 4.11 наведено фрагмент звіту про покриття коду тестами.

```
===== test session starts =====
platform linux -- Python 3.12.2, pytest-7.4.4, pluggy-1.6.0
django: settings: config.settings.local (from int)
rootdir: /app
configfile: pytest.ini
testpaths: apps
plugins: anyio-4.9.0, Faker-37.3.0, factoryboy-2.5.1, cov-4.0.0, django-4.5.2
collected 191 items

apps/events/tests/test_models.py ..... [ 4%]
apps/events/tests/test_permissions.py ..... [ 8%]
apps/events/tests/test_serializers.py ..... [ 19%]
apps/events/tests/test_tasks.py ... [ 21%]
apps/events/tests/test_views.py ..... [ 28%]
apps/notifications/tests/test_emails.py . [ 29%]
apps/notifications/tests/test_models.py . [ 29%]
apps/notifications/tests/test_permissions.py .. [ 30%]
apps/notifications/tests/test_serializers.py ..... [ 36%]
apps/notifications/tests/test_signals.py ... [ 37%]
apps/notifications/tests/test_tasks.py .. [ 38%]
apps/notifications/tests/test_views.py .. [ 39%]
apps/profiles/tests/test_models.py ..... [ 43%]
apps/profiles/tests/test_serializers.py ..... [ 47%]
apps/profiles/tests/test_tasks.py . [ 48%]
apps/profiles/tests/test_views.py ..... [ 53%]
apps/registration_cards/tests/test_models.py ..... [ 56%]
apps/registration_cards/tests/test_permissions.py .. [ 57%]
apps/registration_cards/tests/test_serializers.py ... [ 58%]
apps/registration_cards/tests/test_views.py .... [ 60%]
apps/teams/tests/test_emails.py . [ 61%]
apps/teams/tests/test_models.py .. [ 62%]
apps/teams/tests/test_serializers.py ..... [ 70%]
apps/teams/tests/test_tasks.py .. [ 71%]
apps/teams/tests/test_views.py ..... [ 85%]
apps/units/tests/test_models.py .... [ 87%]
apps/units/tests/test_serializers.py ... [ 89%]
apps/units/tests/test_views.py . [ 89%]
apps/users/tests/test_forms.py .. [ 90%]
apps/users/tests/test_models.py ..... [ 97%]
apps/users/tests/test_serializers.py .. [ 98%]
apps/users/tests/test_views.py .. [100%]

----- coverage: platform linux, python 3.12.2-final-0 -----
Coverage HTML written to dir htmlcov

===== 191 passed in 68.79s (0:01:08) =====
```

Рисунок 4.10 – Результат виконання unit-тестів

Coverage report: 96%

Files

Functions

Classes

coverage.py v7.8.2, created at 2025-06-10 21:44 +0300

File	statements ▼	missing	excluded	coverage
apps/teams/views.py	151	6	0	96%
apps/events/serializers.py	137	0	0	100%
apps/teams/serializers.py	102	0	0	100%
apps/events/views.py	90	0	0	100%
apps/profiles/views.py	85	0	0	100%
apps/events/models.py	69	1	0	99%
apps/profiles/serializers.py	64	0	0	100%

Рисунок 4.11 – Результуюче покриття коду тестами

Результати мануального тестування основного функціоналу програмного забезпечення наведено у таблицях 4.1 – 4.15.

Таблиця 4.1 – Тест 1.1 Реєстрація користувача

Початковий стан системи	Користувач знаходиться на сторінці реєстрації
Вхідні дані	Ім'я, прізвище, електронна пошта, пароль, підтвердження паролю.
Опис проведення тесту	У відповідні поля користувач вводить: ім'я, прізвище, коректну адресу корпоративної електронної пошти, яка раніше не була зареєстрована у системі, пароль, довжина якого становить не менше 6 символів, та підтвердження паролю, яке співпадає із введеним паролем. Користувач натискає кнопку «Реєстрація».
Очікуваний результат	Користувач отримує повідомлення про успішну реєстрацію. На вказану електронну пошту надходить лист для підтвердження реєстрації.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.2 – Тест 1.2 Авторизація користувача

Початковий стан системи	Користувач знаходиться на сторінці авторизації
Вхідні дані	Електронна пошта, пароль
Опис проведення тесту	У відповідні поля користувач вводить коректну адресу електронної пошти та пароль, які зареєстровані у системі, після чого натискає на кнопку «Вхід».
Очікуваний результат	Користувач отримує повідомлення про успішну авторизацію, завантажується головна сторінка застосунку.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.3 – Тест 1.3 Прив'язка Telegram-акаунту

Початковий стан системи	Авторизований користувач знаходиться на сторінці профілю
Вхідні дані	Номер телефону
Опис проведення тесту	Користувач натискає на віджет «Увійти через Telegram». У відкритому вікні авторизації у відповідне поле користувач вводить номер телефону, на який зареєстровано акаунт Telegram, та натискає «Далі». У отриманому повідомленні в застосунку Telegram користувач підтверджує запит на авторизацію. У вікні авторизації користувач обирає пункт «Дозволити KPI Notify надсилати вам повідомлення в Telegram» та натискає «Прийняти».
Очікуваний результат	Користувач отримує повідомлення від Telegram-бота KPI Notify про успішну авторизацію, у вебзастосунку на сторінці профілю з'являється ім'я користувача Telegram, статус змінюється із «Не підтверджено» на «Підтверджено».
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 2.1 Створення події

Початковий стан системи	Користувач знаходиться на сторінці списку подій
Вхідні дані	Інформація про подію: назва, пріоритет, дата та час, додаткові дані.
Опис проведення тесту	Користувач натискає на кнопку «Додати подію», після чого потрапляє на сторінку для створення нової події.

Продовження таблиці 4.4

Опис проведення тесту	У відповідні поля користувач вводить назву події, дату та час події, які обов'язково є у майбутньому, обирає пріоритет події із випадającego списку. За необхідності, користувач у відповідні поля вводить додаткову інформацію про подію: опис події; теги, які не повторюються між собою для цієї події; місце події; коректне посилання; зображення типу png або jpeg, розміром не більше 5Мб. Користувач натискає кнопку «Підтвердити».
Очікуваний результат	Користувач отримує повідомлення про успішне створення події, завантажується сторінка із інформацією про створену подію, яка відповідає даним, введеним користувачем.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 2.2 Пошук події

Початковий стан системи	Користувач знаходиться на сторінці списку подій
Вхідні дані	Пошуковий запит
Опис проведення тесту	У відповідне поле для пошуку користувач вводить пошуковий запит, текст якого може знаходитись у назві події, описі чи тегах.
Очікуваний результат	Список подій змінюється, відображаються лише ті події, які мають у тексті назви, опису або тегах введене користувачем слово.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.6 – Тест 2.3 Сорткування списку подій

Початковий стан системи	Користувач знаходиться на сторінці списку подій
Вхідні дані	
Опис проведення тесту	У списку подій користувач натискає на заголовок «Назва», «Дата і час» або «Пріоритет».
Очікуваний результат	Таблиця сортується за обраним параметром: при натисненні на «Назва», події сортуються за назвою події у алфавітному порядку; при натисненні на «Дата і час», події сортуються від найближньої події до найпізнішої; при натисненні на «Пріоритет» події сортуються від найменш важливих до найбільш важливих. При повторному натисненні на той самий заголовок, події сортуються у зворотньому порядку.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.7 – Тест 2.4 Створення повторюваної події

Початковий стан системи	Користувач знаходиться на сторінці списку подій
Вхідні дані	Інформація про подію, періодичність повторювання, дата та час завершення повторювання.
Опис проведення тесту	Користувач натискає на кнопку «Додати подію», після чого потрапляє на сторінку для створення нової події. У відповідні поля користувач вводить дані про подію. Користувач ставить позначку біля поля «Повторювана подія», після чого у відповідних полях, що з'явилися, із випадаючого списку обирає періодичність повторення події, дату та час завершення повторення.

Продовження таблиці 4.7

Опис проведення тесту	Момент завершення повторення є у майбутньому, пізніше за вказані попередньо дату та час настання події. Користувач натискає кнопку «Підтвердити».
Очікуваний результат	Користувач отримує повідомлення про успішне створення події, завантажується сторінка із інформацією про створену подію, яка відповідає даним, введеним користувачем. Після моменту настання події, дата та час наступного моменту настання події оновлюються, відповідно до вказаної періодичності повторення. Якщо наступний момент настання повторюваної події є пізніше за момент завершення повторення події, то дата та час настання події не оновлюються.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.8 – Тест 3.1 Створення нагадування через електронну пошту

Початковий стан системи	Користувач знаходиться на сторінці списку подій
Вхідні дані	Дата та час нагадування, метод нагадування.
Опис проведення тесту	Користувач натискає на подію, після чого завантажується сторінка із повною інформацією про обрану подію. У пункті «Нагадування» у відповідні поля користувач обрає дату та час нагадування, які є раніше за момент настання події, із випадаючого списку обирає метод нагадування, після чого натискає кнопку «Додати».

Продовження таблиці 4.8

Очікуваний результат	Користувач отримує повідомлення про успішне створення нагадування, створене нагадування з'являється у пункті «Нагадування». У момент настання нагадування, користувач отримує електронний лист на корпоративну електронну адресу, вказану при реєстрації, із нагадуванням про обрану подію.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.9 – Тест 3.2 Створення нагадування через Telegram

Початковий стан системи	Користувач знаходиться на сторінці списку подій
Вхідні дані	Дата та час нагадування, метод нагадування.
Опис проведення тесту	Користувач натискає на подію, після чого завантажується сторінка із повною інформацією про обрану подію. У пункті «Нагадування» у відповідні поля користувач обрає дату та час нагадування, які є раніше за момент настання події, із випадаючого списку обирає метод нагадування Telegram», після чого натискає кнопку «Додати».
Очікуваний результат	Якщо користувач не прив'язав свій акаунт Telegram, то він отримає повідомлення про неможливість створити нагадування. Якщо користувач успішно прив'язав свій акаунт Telegram, то він отримає повідомлення про успішне створення нагадування. У момент настання нагадування, користувач отримує повідомлення від Telegram-бота KPI Notify із нагадуванням про обрану подію.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.10 – Тест 4.1 Створення команди

Початковий стан системи	Користувач знаходиться на сторінці списку команд
Вхідні дані	Назва та опис команди.
Опис проведення тесту	Користувач натискає на кнопку «Створити команду», після чого потрапляє на сторінку для створення нової команди. У відповідні поля користувач вводить назву та опис для команди. У полі «Учасники» користувач шукає за іменем та обирає із випадаючого списку інших користувачів, яких він бажає запросити у нову команду. Користувач натискає на кнопку «Створити команду».
Очікуваний результат	Користувач отримує повідомлення про успішне створення команди, після чого завантажується сторінка із повною інформацією про команду, яка відповідає даним, введеним користувачем. У вкладці «Запрошення у команду» відображається список із запрошень із статусом «Очікує» для обраних у полі «Учасники» користувачів. Обраним користувачам на електронні адреси, вказані при реєстрації, надійшли листи про запрошення у команду.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.11 – Тест 4.2 Запрошення користувача до команди

Початковий стан системи	Користувач знаходиться на сторінці університету
Вхідні дані	
Опис проведення тесту	Користувач натискає на інститут або факультет, працівника якого він бажає запросити у команду, лідером якої він є.

Продовження таблиці 4.11

Опис проведення тесту	На завантаженій сторінці із списком кафедр обраного інституту або факультету, користувач обирає кафедру, працівника якої він бажає запросити. На завантаженій сторінці користувачів, що є працівниками обраної кафедри, користувач знаходить того, кого він бажає запросити до команди, та натискає на кнопку «Запросити до команди». У відкритому вікні із списком команд, лідером яких є користувач, він обирає потрібну команду та натискає «Запросити».
Очікуваний результат	Користувач отримує повідомлення про успішне створення запрошення. Обраному користувачеві на електронну адресу, вказану при реєстрації, надійшов лист із повідомленням про запрошення у команду.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.12 – Тест 4.3 Прийняття запрошення до команди

Початковий стан системи	Користувач знаходиться на сторінці списку команд
Вхідні дані	
Опис проведення тесту	На сторінці списку команд користувач обирає вкладку «Запрошення». Завантажується список запрошень у команди, отриманих від інших користувачів. Користувач обирає запрошення із статусом «Очікує», після чого завантажується сторінка із деталями про запрошення та команду. Користувач натискає кнопку «Прийняти».

Продовження таблиці 4.12

Очікуваний результат	Користувач отримує повідомлення про успішне прийняття запрошення, після чого потрапляє на сторінку із інформацією про команду. Користувач з'являється у списку учасників команди. Статус запрошення змінюється на «Прийнято».
Фактичний результат	Збігається з очікуваним.

Таблиця 4.13 – Тест 4.4 Створення події для команди

Початковий стан системи	Користувач знаходиться на сторінці списку команд
Вхідні дані	Назва, пріоритет, дата та час події.
Опис проведення тесту	Користувач обирає команду, лідером якої він є, та натискає кнопку «Перейти». На завантаженій сторінці із списком подій команди користувач натискає на кнопку «Додати подію», після чого потрапляє на сторінку для створення командної події. У відповідні поля користувач вводить назву події, дату та час події, які обов'язково є у майбутньому, обирає пріоритет події із випадального списку. За необхідності користувач додає додаткову інформацію про подію. У полі «Призначити подію», із випадального списку користувач обирає членів команди, яким він бажає призначити цю подію, після чого натискає кнопку «Підтвердити».
Очікуваний результат	Користувач отримує повідомлення про успішне створення події, після чого потрапляє на сторінку із інформацією про подію, яка відповідає введеним користувачем даним.

Продовження таблиці 4.13

Очікуваний результат	У пункті «Призначено для» відображено перелік членів команди, яким було призначено подію. На сторінці подій команди відображається створена подія.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.14 – Тест 5.1 Створення реєстраційної картки юридичної особи

Початковий стан системи	Авторизований користувач знаходиться на сторінці профілю
Вхідні дані	Дані юридичної особи, дані заявника, засоби зв'язку із заявником, дані електронної печатки.
Опис проведення тесту	На сторінці профілю користувач натискає на вкладку «Картки АЦСК», після чого потрапляє на сторінку із списком збережених карток юридичних осіб. Користувач натискає на кнопку «Додати», після чого потрапляє на сторінку для створення картки АЦСК. Користувач вводить дані про юридичну особу: назву організації, код ЄДРПОУ, який складається із 8 цифр, область та місто місцезнаходження юридичної особи; дані про заявника: ім'я заявника, ідентифікаційний номер заявника, який складається із 10 цифр, коректну адресу електронної пошти, номер телефону у міжнародному форматі, питання ключової фрази та ключову фразу; дані для формування електронної печатки: питання ключової фрази та ключова фраза. Користувач натискає кнопку «Створити».

Продовження таблиці 4.14

Очікуваний результат	Користувач отримує повідомлення про успішне створення електронної печатки, та потрапляє на сторінку із інформацією реєстраційної картки, яка відповідає введеним користувачем даним.
Фактичний результат	Збігається з очікуванням.

4.3 Опис контрольного прикладу

У контрольному прикладі розглянуто основний функціонал застосунку, описано процес реєстрації та авторизації, створення команди, запрошення користувачів у команду, створення подій та нагадувань.

Користувач заходить на вебсайт та потрапляє на вітальну сторінку (рисунок 4.12). Користувач натискає кнопку «Реєстрація».

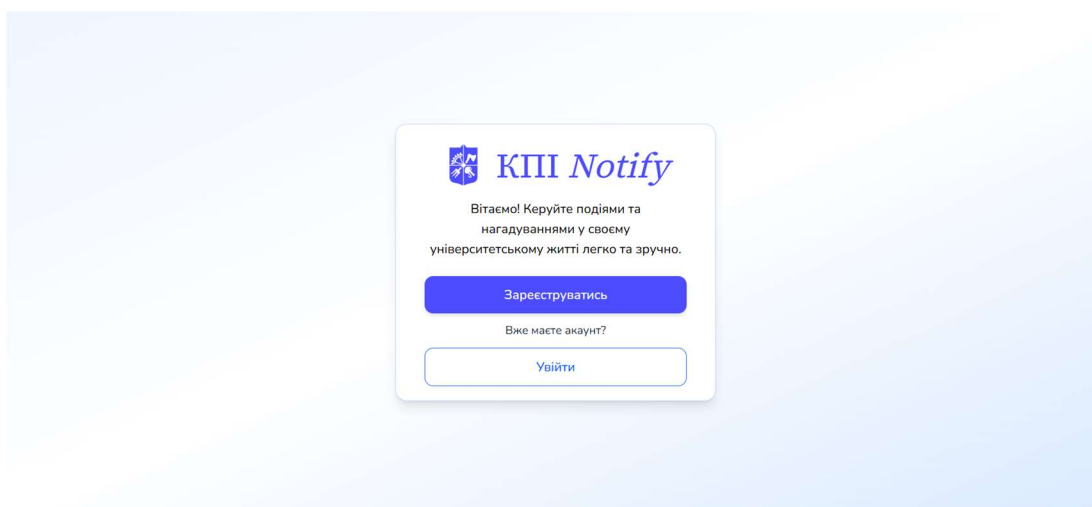


Рисунок 4.12 – Вітальна сторінка застосунку

Завантажується форма для реєстрації (рисунок 4.13). Користувач вводить дані у відповідні поля та натискає кнопку «Реєстрація».

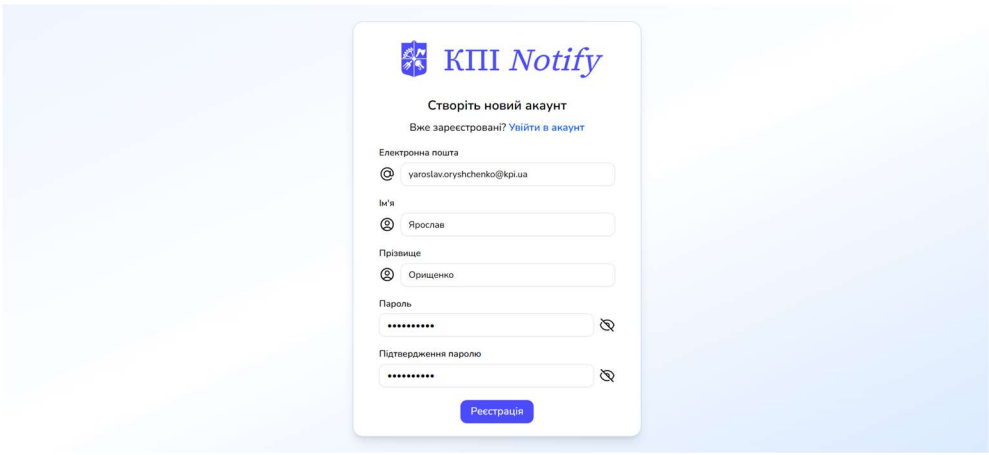


Рисунок 4.13 – Сторінка реєстрації

На вказану електронну адресу надходить лист із посиланням для активації акаунту (рисунок 4.14). Користувач переходить за отриманим посиланням.

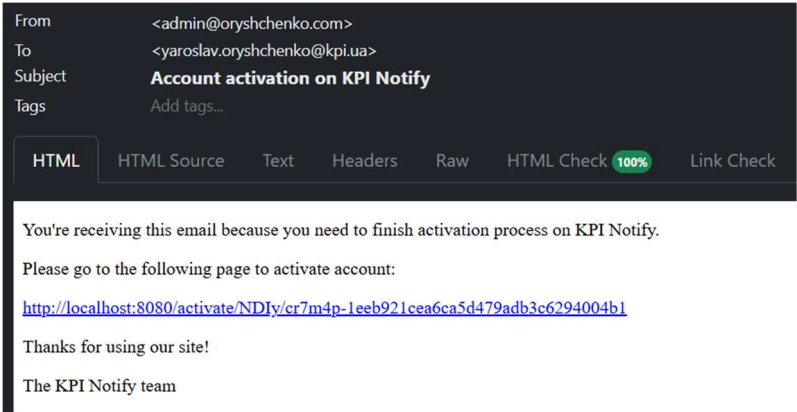


Рисунок 4.14 – Лист активації акаунту

Користувач отримує повідомлення про успішну активацію акаунту (рисунок 4.15).



Рисунок 4.15 – Сторінка активації акаунту

Завантажується сторінка входу в акаунт (рисунок 4.16). У відповідні поля користувач вводить електронну адресу та пароль.

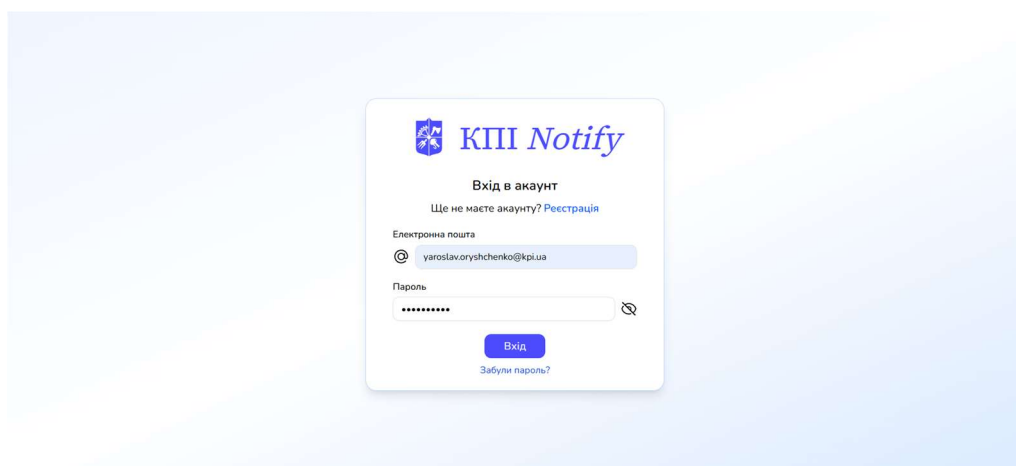


Рисунок 4.16 – Сторінка входу в акаунт

Після входу в акаунт завантажується сторінка для налаштування профілю (рисунок 4.17). Користувач вводить свою посаду, обирає факультет, після чого обирає кафедру на обраному факультеті, та натискає «Підтвердити».

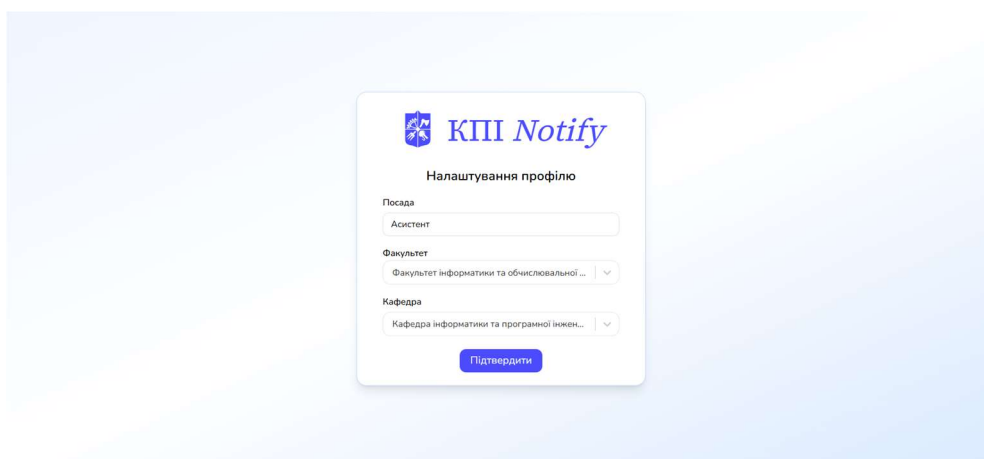


Рисунок 4.17 – Сторінка налаштування профілю

Після успішного входу в акаунт завантажується головна сторінка застосунку (рисунок 4.18). У навігаційному меню користувач натискає на «Профіль».

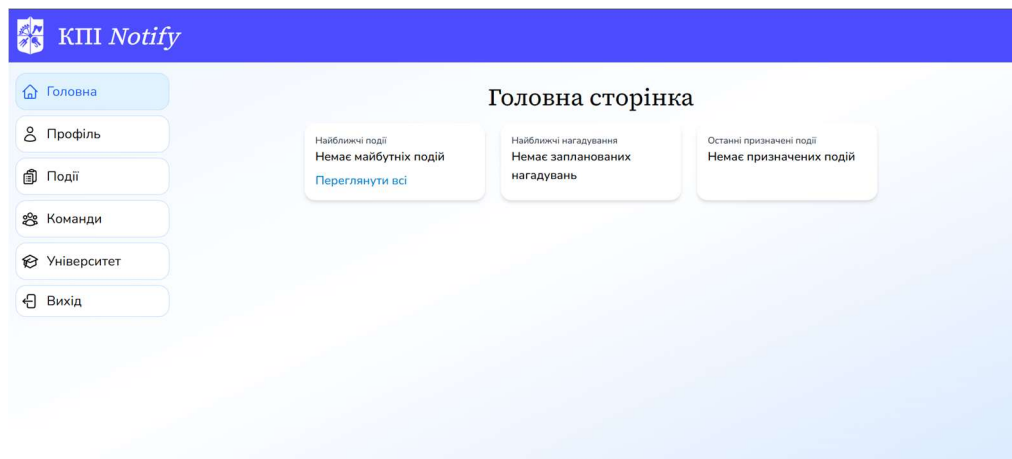


Рисунок 4.18 – Головна сторінка застосунку

Завантажується сторінка із інформацією про користувача (рисунок 4.19). У розділі «Telegram» користувач натискає на віджет «Увійти через Telegram».

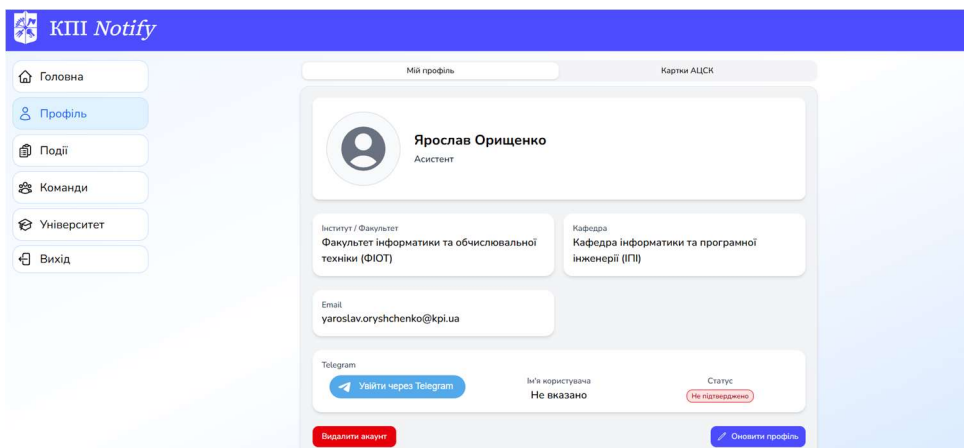


Рисунок 4.19 – Сторінка профілю користувача

Відкривається вікно авторизації Telegram (рисунок 4.20). Користувач вводить свій номер телефону та натискає «Далі».

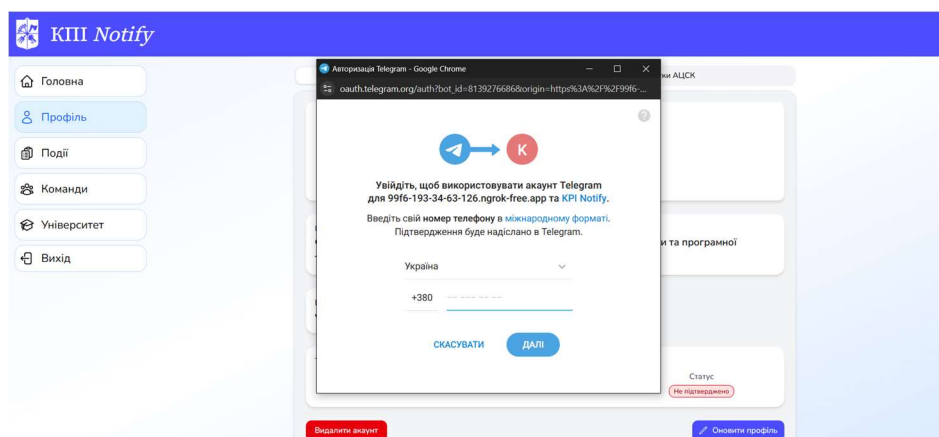


Рисунок 4.20 – Вікно авторизації Telegram-акаунту

Акаунт із вказаним номером телефону отримує повідомлення від Telegram для підтвердження авторизації (рисунок 4.21). Користувач натискає «Підтвердити».

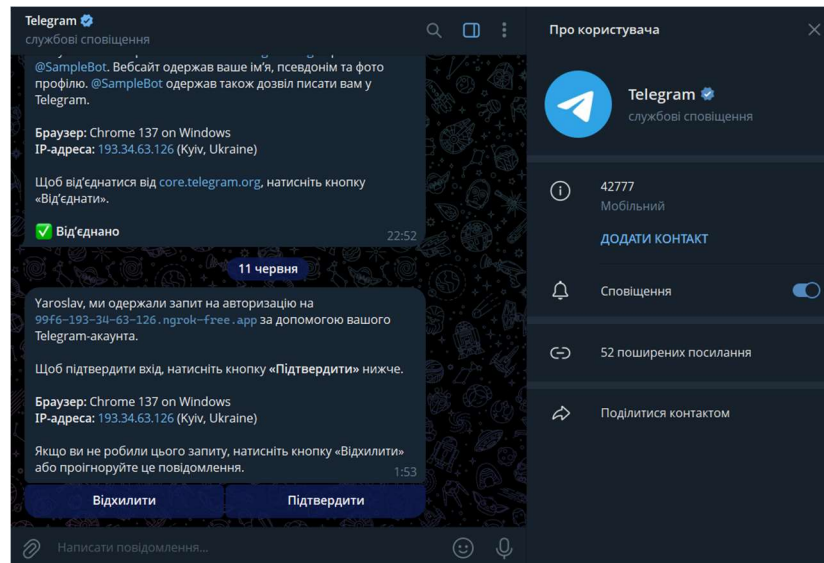


Рисунок 4.21 – Підтвердження авторизації через Telegram

У вікні авторизації Telegram користувач натискає на кнопку «Прийняти», надаючи застосунку дані свого Telegram-акаунту (рисунок 4.22).

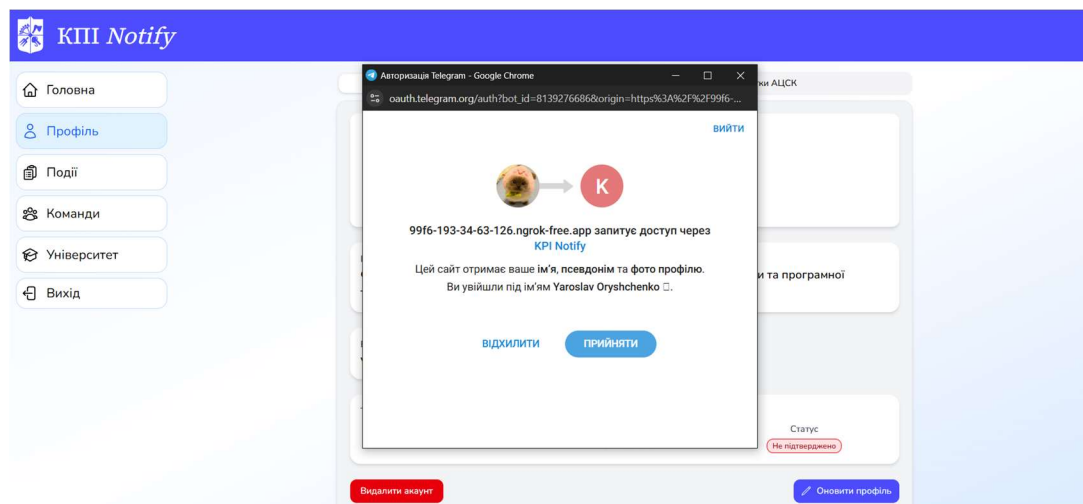


Рисунок 4.22 – Прив'язка Telegram-акаунту до застосунку

Користувач отримує повідомлення про успішну авторизацію від Telegram (рисунок 4.23), а також від бота KPI Notify (рисунок 4.24).

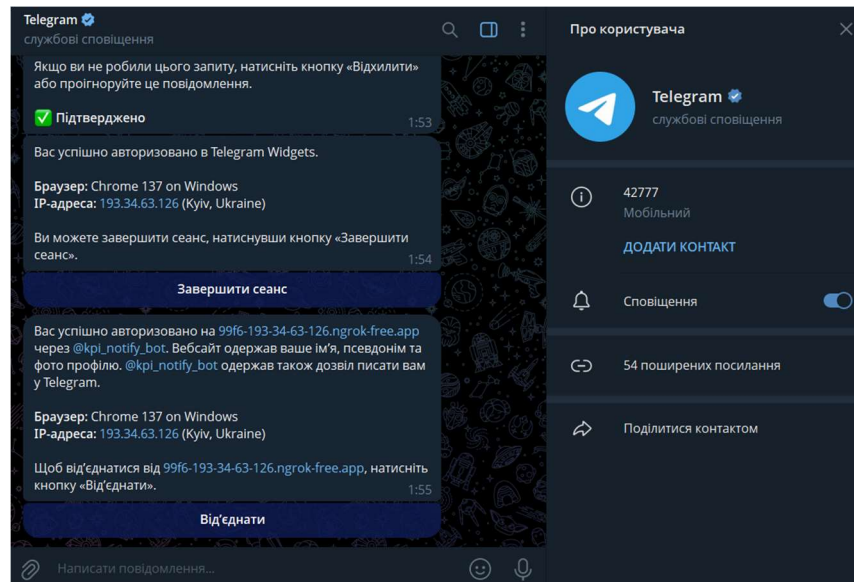


Рисунок 4.23 – Повідомлення про успішну авторизацію від Telegram

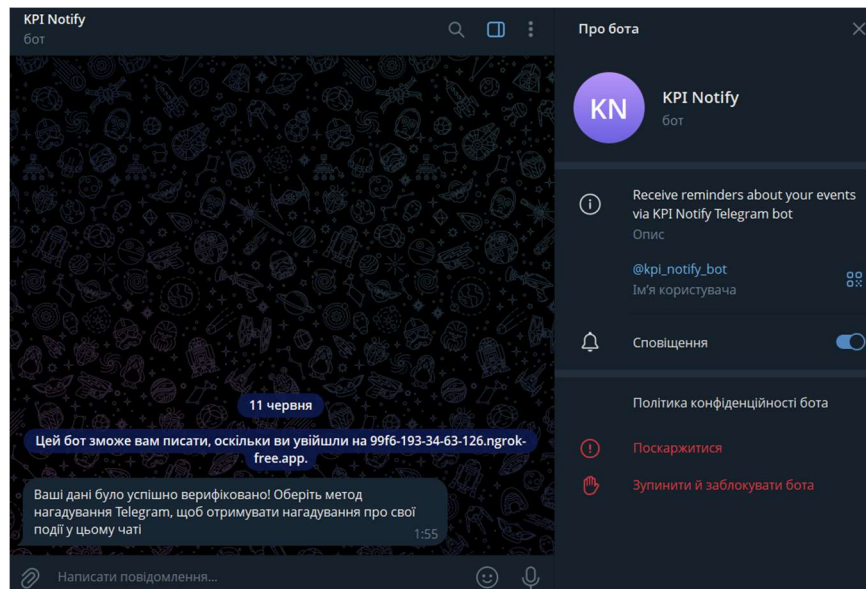


Рисунок 4.24 – Повідомлення про успішну авторизацію від бота

У вебзастосунку користувач натискає на розділ «Команди». Завантажується сторінка із списком команд, до яких належить користувач (рисунок 4.25). Користувач натискає кнопку «Створити команду».

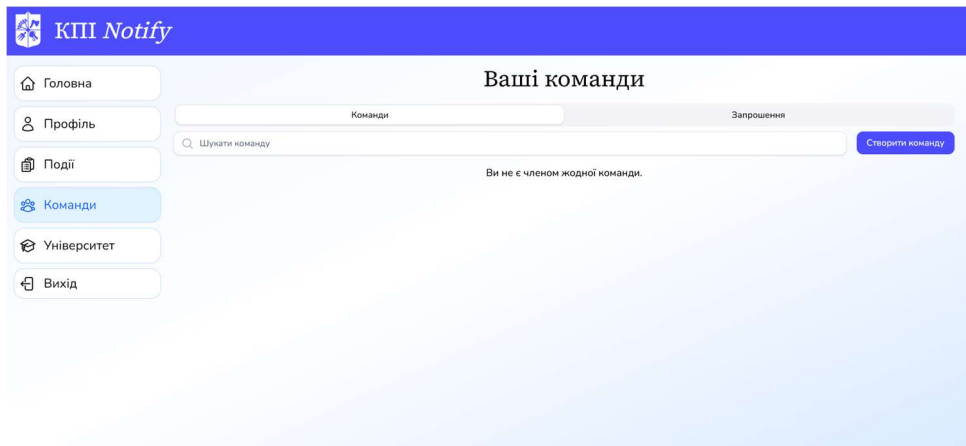


Рисунок 4.25 – Сторінка команд користувача

Завантажується сторінка для створення нової команди (рисунок 4.26). Користувач вводить назву команди та її опис, знаходить та обирає користувачів, яких він бажає запросити у команду, та натискає кнопку «Створити команду».

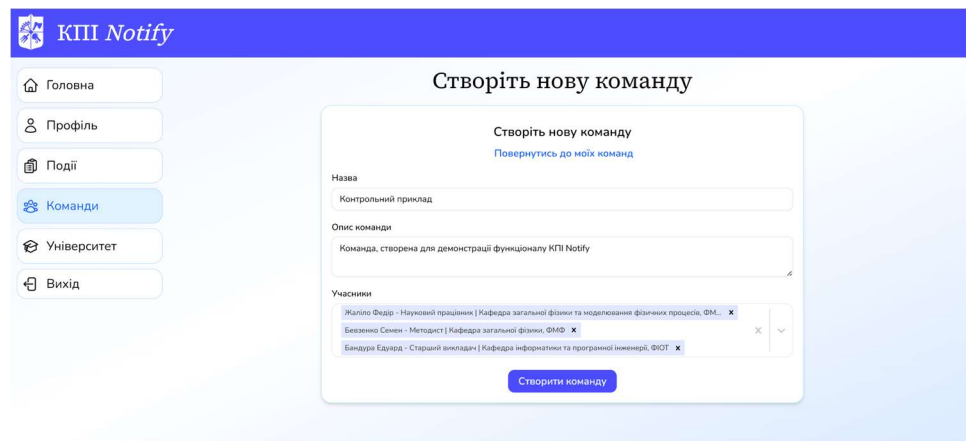


Рисунок 4.26 – Сторінка створення команди

Завантажується сторінка із повною інформацією про команду (рисунок 4.27). Після того, як усі запрошені користувачі прийняли запрошення, на цій сторінці можна переглянути склад команди (рисунок 4.28).

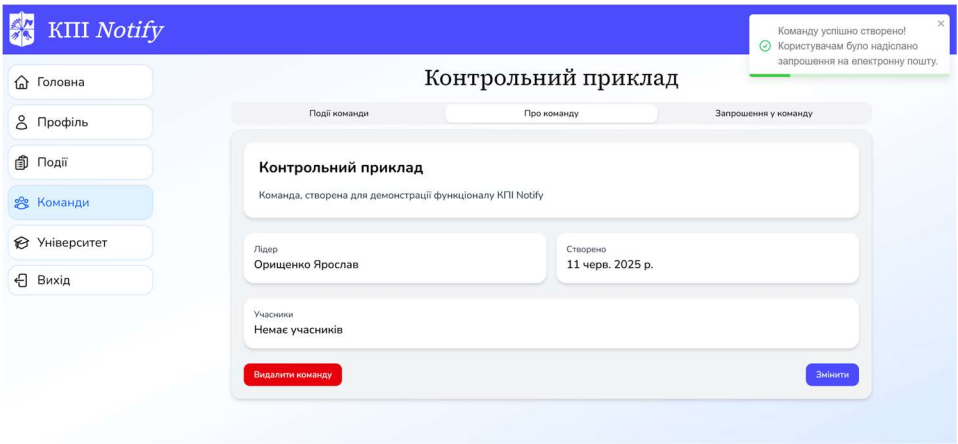


Рисунок 4.27 – Сторінка інформації про команду

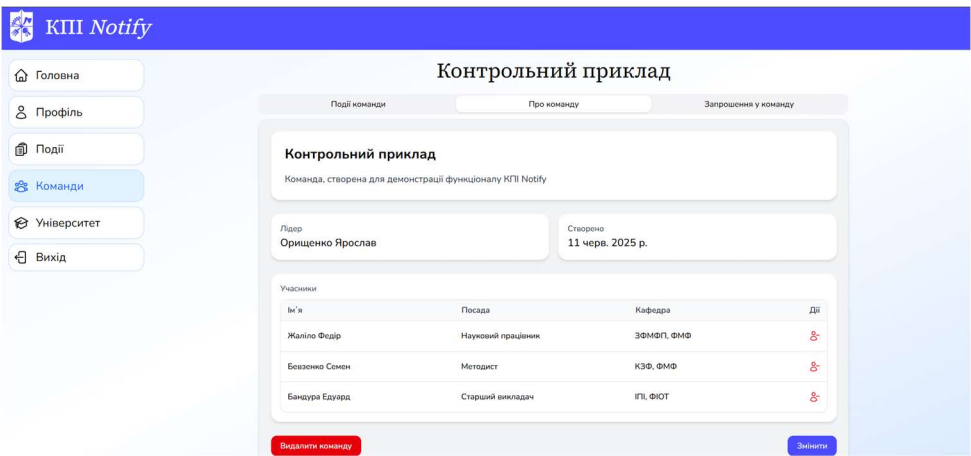


Рисунок 4.28 – Сторінка інформації про команду після доєднання до неї учасників

Користувач натискає на розділ «Університет», після чого завантажується сторінка із списком інститутів та факультетів університету (рисунок 4.29).

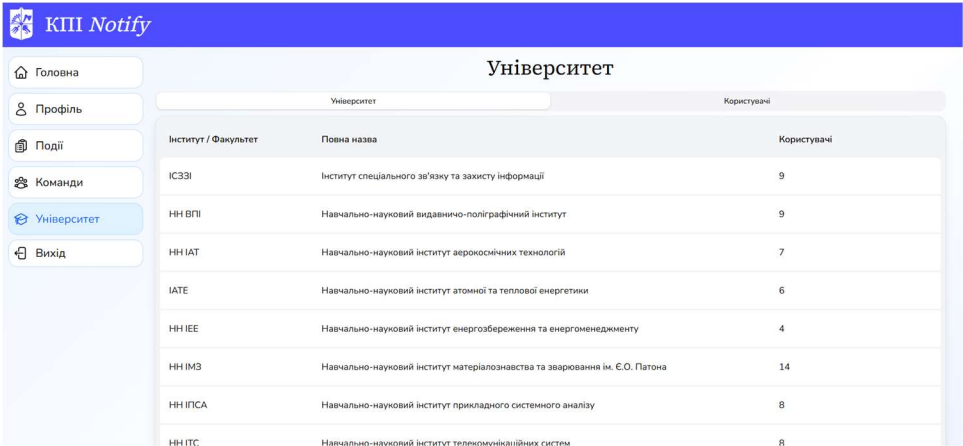


Рисунок 4.29 – Сторінка списку підрозділів університету

Користувач обирає вкладку «Користувачі», де завантажується список усіх користувачів застосунку. У пошуковому рядку користувач вводить пошуковий запит, після чого завантажуються ті користувачі, ім'я яких містить введений текст (рисунок 4.30).

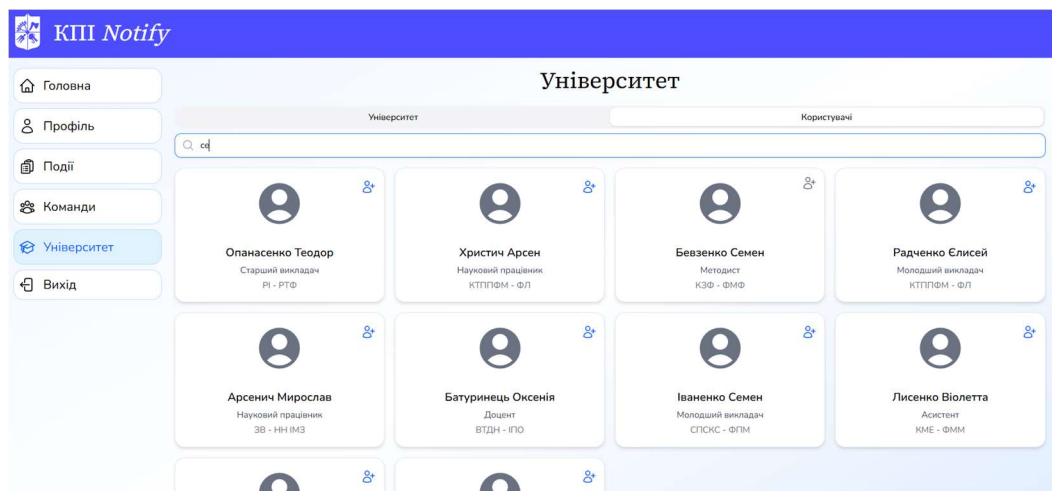


Рисунок 4.30 – Сторінка списку користувачів

Користувач обирає того, кого він бажає запросити у команду, та натискає на кнопку запрошення. Завантажується із списком команд, куди користувач може запросити обрану людину (рисунок 4.31). Користувач обирає команду та натискає «Запросити».

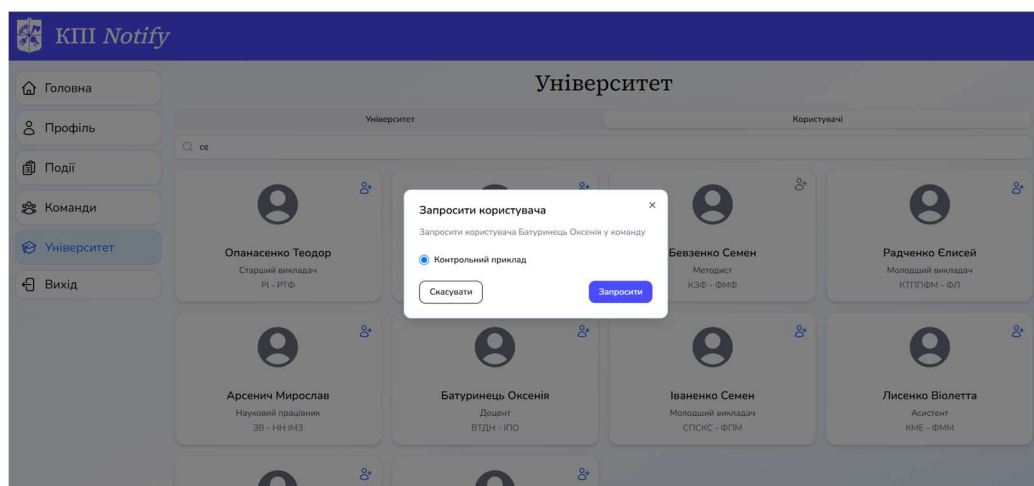


Рисунок 4.31 – Вікно запрошення користувача в команду

Користувач натискає на розділ «Команди», де тепер відображається створена команда із основною інформацією про неї (рисунок 4.32). Користувач натискає на кнопку «Перейти».

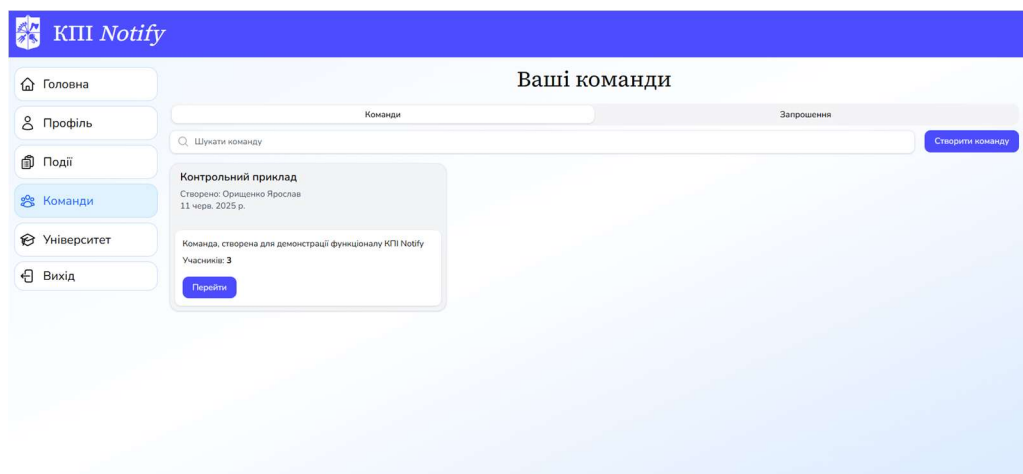


Рисунок 4.32 – Сторінка списку команд користувача

Завантажується сторінка із подіями команди (рисунок 4.33). Користувач натискає на кнопку «Додати подію».

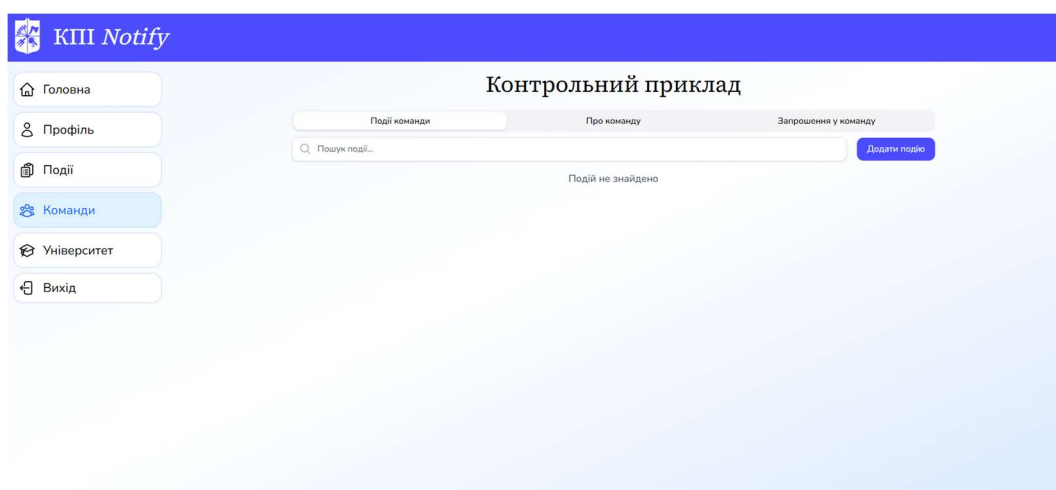


Рисунок 4.33 – Сторінка із подіями команди

Завантажується сторінка для створення нової події (рисунки 4.34 – 3.35). Користувач вводить інформацію про подію, обирає пріоритет, дату та час настання події. У полі «Призначити подію» користувач обирає членів команди, яким він бажає призначити дану подію. Щоб задати повторюваність події, користувач обирає пункт «Потворювана подія», після чого у завантажених полях обирає періодичність повтору події та момент, після якого потрібно завершити повтор події. Користувач натискає на кнопку «Додати нагадування», щоб одразу створити нагадування для події. Користувач обирає

дату та час нагадування і метод нагадування. Користувач натискає на кнопку «Підтвердити».

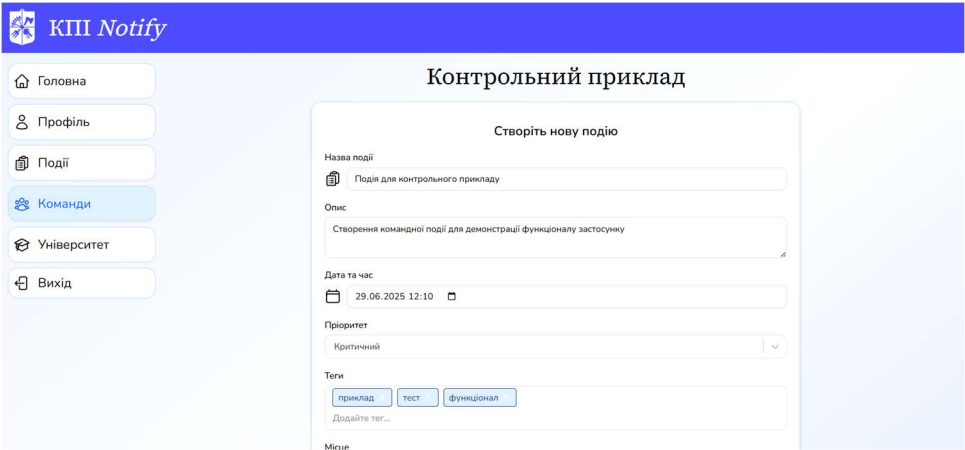


Рисунок 4.34 – Сторінка створення події для команди

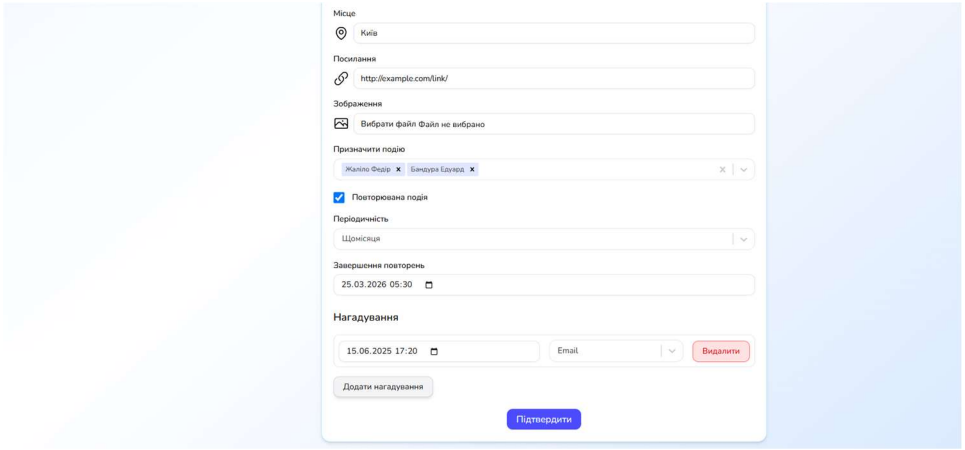


Рисунок 4.35 – Сторінка створення події для команди

Після створення події завантажується сторінка із деталями про подію (рисунки 3.36 – 3.37).

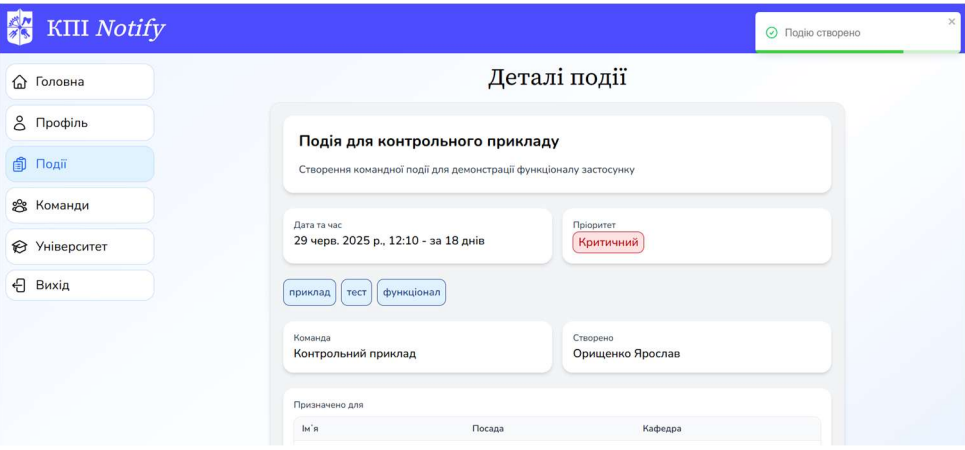


Рисунок 4.36 – Сторінка із деталями про подію

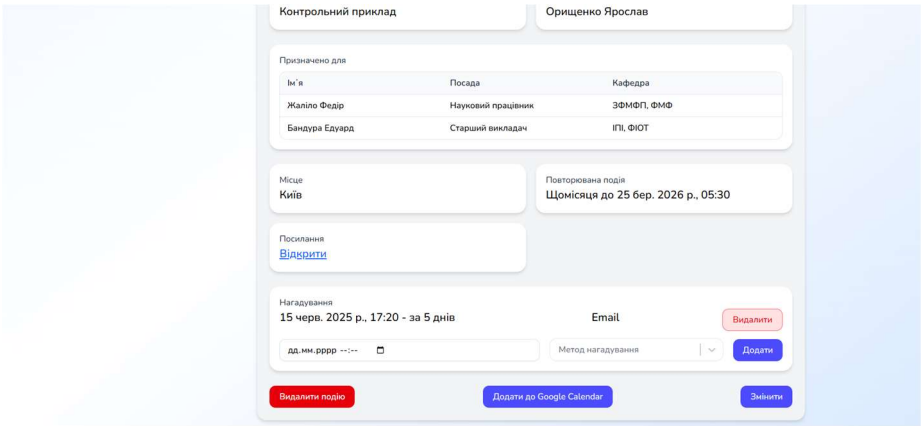


Рисунок 4.37 – Сторінка із деталями про подію

Користувач натискає на кнопку «Додати до Google Calendar», після чого він перенаправляється на сторінку для створення події у Google Calendar із перенесеними даними про подію (рисунок 4.38).

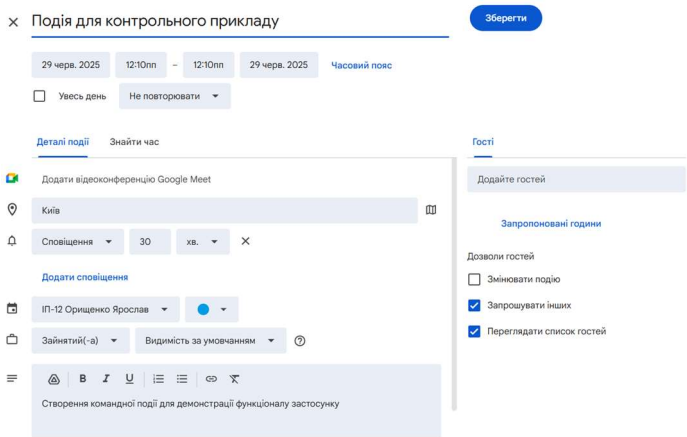


Рисунок 4.38 – Додавання події до Google Calendar

Користувач повертається до застосунку та обирає розділ «Події». Завантажується сторінка із усіма подіями, які були створені користувачем або призначені йому (рисунок 4.39).

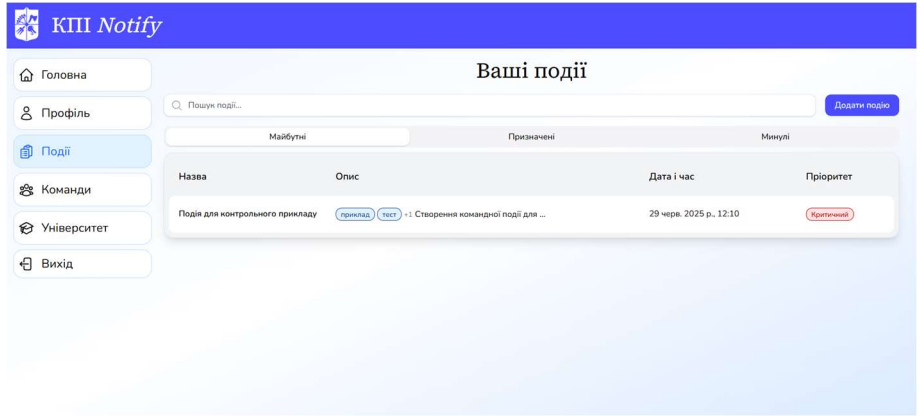


Рисунок 4.39 – Сторінка списку подій користувача

Висновки до розділу

У даному розділі було здійснено аналіз якості програмного забезпечення та проведено його тестування.

Код було проаналізовано на помилки, неточності та невідповідності стандартним правилам стилю коду на мові програмування Python. У коді було виявлено невикористані імпорти та наявність. Була виявлена певна кількість імпортів, які після цього не використовувались у коді, також наявність зайвих пробілів в кінці ліній або на місці порожніх ліній, після чого було проведено оптимізацію коду. Було оцінено метрики цикломатичної складності коду та індексу його підтримуваності, у результаті чого було отримано високі позитивні результати.

Було проаналізовано піраміду тестування, після чого було описано unit-тестування застосунку та виконання мануального тестування основних функцій на рівні користувача. Коректне функціонування веб-застосунку було продемонстровано у контрольному прикладі.

5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 Розгортання програмного забезпечення

Для забезпечення зручного, ізольованого та масштабованого розгортання програмного забезпечення, було використано технології Docker та Docker Compose. Це дозволяє уникнути проблем, пов'язаних із відмінностями локального середовища розробки та середовища розгортання, залежностями та несумісністю версій.

Для повноцінного розгортання розроблюваного застосунку ключовими є три компоненти:

- Доменне ім'я. Домен є обов'язковою складовою, він забезпечує публічний доступ користувачів до застосунку через безпечне HTTPS з'єднання. Також наявність публічного домену є обов'язковою для можливості використання офіційного віджета авторизації Telegram.

- SMTP-сервер. Simple Mail Transfer Protocol [25] – це мережевий протокол для надсилання листів на електронну пошту. Для коректної роботи SMTP-сервера необхідною є наявність доменного імені, що дозволяє верифікувати відправника електронних листів, та коректно налаштувати DNS-записи для уникнення блокування або потрапляння листів у спам на боці одержувачів.

- Сервер. Це обчислювальний ресурс, на якому безпосередньо розгортається застосунок. Сервер пов'язується із доменним іменем, обробляє та передає отримані запити відповідним компонентам системи, а також взаємодіє із SMTP-сервером, формуючи запити на надсилання електронних листів.

Для початку необхідно зареєструвати домен. Для цього можна скористатись такими сервісами, як Namecheap, Cloudflare, Squarespace, тощо. Зареєстрований домен буде використовуватись для налаштування SSL-сертифікатів та SMTP-сервісу, прив'язки Telegram-бота та публічного доступу до застосунку.

Для можливості використання офіційної авторизації через Telegram необхідно вказати зареєстрований домен у налаштуваннях Telegram-бота, використовуючи BotFather. Це є обов'язковою умовою для коректної роботи Telegram Login Widget.

Наступним кроком є створення сервера, де буде розгорнуто застосунок. Для цього можна використати такі рішення, як Amazon Web Services, Microsoft Azure, Google Cloud Platform, DigitalOcean тощо. Для розгортання проекту було обрано DigitalOcean.

Після реєстрації у DigitalOcean у панелі керування потрібно створити проект, який міститиме усі ресурси застосунку. У проекті необхідно створити Droplet – це віртуальна машина, яка надає доступ до відокремленої операційної системи із власними ресурсами, на якій можна запускати необхідне програмне забезпечення.

При створенні Droplet необхідно вказати регіон, де фізично буде розташовано сервер – найкраще обирати той регіон, який знаходиться найближче до потенційних користувачів. Далі потрібно вказати образ операційної системи, найбільш поширеним варіантом є Linux Ubuntu. Можна обрати як чистий образ, так і попередньо налаштований, наприклад, із завантаженими Docker та Docker Compose. Також необхідно вказати конфігурацію апаратного забезпечення серверу: тип процесора, об'єм оперативної пам'яті та розмір жорсткого диску.

Для можливості безпечного віддаленого підключення до сервера, потрібно додати SSH-ключ. Згенерувати публічний та приватний SSH-ключі можна у терміналі за допомогою команди **ssh-keygen**, або за допомогою спеціального програмного забезпечення, наприклад PuTTY. У налаштуваннях Droplet потрібно додати публічний SSH-ключ, в той час як приватний зберігається на локальному комп'ютері. При спробі віддаленого підключення до сервера, приватний ключ генерує підпис, який перевіряється сервером за допомогою публічного ключа, що забезпечує безпечний та авторизований доступ до сервера.

Після успішного створення Droplet, до проекту DigitalOcean необхідно додати зареєстрований домен. Це дозволить серверу коректно направляти трафік до відповідних компонент застосунку. Для доданого домену автоматично було згенеровано базові DNS-записи до основного імені (рисунок 5.1). Для забезпечення коректної роботи та можливості управління станом застосунку, потрібно додатково створити DNS-записи із піддоменами `www.<основний домен>`, `portainer.<основний домен>` та `flower.<основний домен>`.

Create new record [Learn](#)

A AAAA CNAME MX TXT NS SRV CAA

Use @ to create the record at the root of the domain or enter a hostname to create it elsewhere. A records are for IPv4 addresses only and tell a request where your domain should direct to.

HOSTNAME WILL DIRECT TO TTL (SECONDS)

Enter @ or hostname Select resource or enter custom IP Enter TTL 3600 Create Record

DNS records

Type	Hostname	Value	TTL (seconds)
NS	webapp.lol	directs to ns1.digitalocean.com.	600 More
NS	webapp.lol	directs to ns2.digitalocean.com.	600 More
NS	webapp.lol	directs to ns3.digitalocean.com.	600 More

Рисунок 5.1 – згенеровані DNS-записи до основного імені домену

Як SMTP-сервер було обрано Mailgun. Після реєстрації, на сервісі також необхідно додати створений домен, з якого надсилатимуться електронні листи. Для цього потрібно вказати імені домену та його регіон. Після цього будуть згенеровані DNS-записи із піддоменами `pic._domainkey.mg.<основний домен>`, `mg.<основний домен>` та `email.mg.<основний домен>`, які повністю забезпечують функціонал надсилання листів на електронну пошту. Ці записи потрібно вручну додати до налаштувань домену у проекті DigitalOcean, після чого у Mailgun статус DNS-запитів зміниться на «Active» та «Verified» (рисунок 5.2).

На рисунку 5.3 наведено повний список DNS-запитів для забезпечення роботи застосунку.

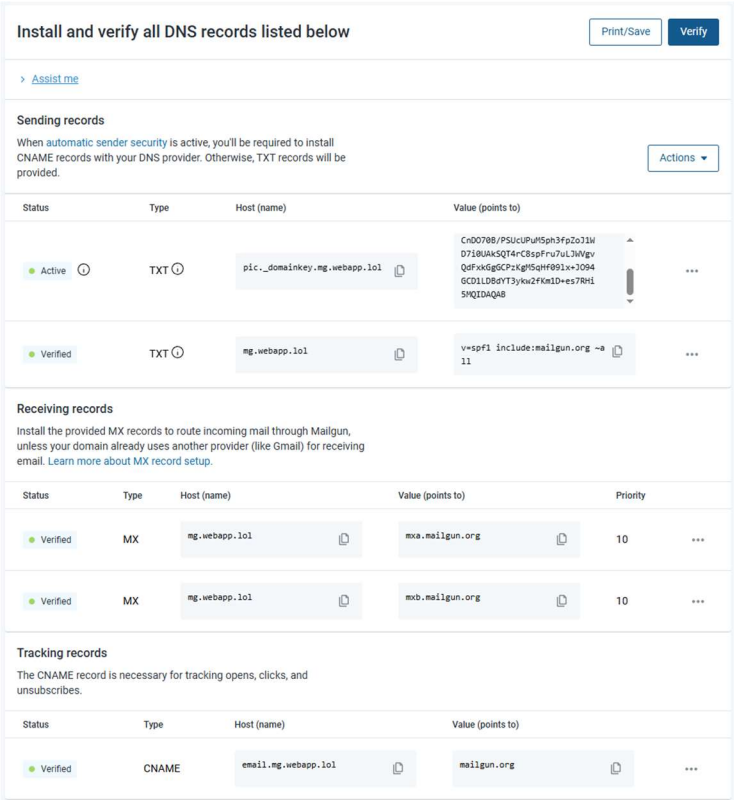


Рисунок 5.2 – DNS-записи Mailgun для надсилання електронних листів

DNS records

Type	Hostname	Value	TTL (seconds)
CNAME	email.mg.webapp.lol	is an alias of mailgun.org.	600
MX	mg.webapp.lol	mail handled by mxb.mailgun.org.	10 600
MX	mg.webapp.lol	mail handled by mxa.mailgun.org.	10 600
TXT	pic._domainkey.mg.webapp.lol	returns k=rsa; p=MIGIMA0GCSqGSib3DQEBA...	600
TXT	mg.webapp.lol	returns v=spf1 include:mailgun.org ~all	600
A	flower.webapp.lol	directs to 165.227.149.193	600
A	portainer.webapp.lol	directs to 165.227.149.193	600
A	www.webapp.lol	directs to 165.227.149.193	600
NS	webapp.lol	directs to ns1.digitalocean.com.	600
NS	webapp.lol	directs to ns2.digitalocean.com.	600
NS	webapp.lol	directs to ns3.digitalocean.com.	600
A	webapp.lol	directs to 165.227.149.193	600

Рисунок 5.3 – DNS-записи для забезпечення роботи застосунку

Фінальним етапом розгортання є налаштування сервера, перенесення та запуск файлів застосунку. Спочатку необхідно підключитись до сервера через

термінал за допомогою команди **ssh root@<ip адреса сервера>**. Після успішного підключення, щоб перед початком встановлення програмного забезпечення переконатися, що система використовує актуальні пакети із офіційних джерел, потрібно виконати команду **apt-get update && apt-get upgrade**.

Користуючись офіційною документацією [26], потрібно встановити Docker та необхідні для нього пакети на сервер. Після успішного завершення процесу інсталяції, коректність роботи Docker можна за допомогою команди **systemctl status docker**. Якщо Docker встановлено правильно, то результат виконання команди буде містити статус **active (running)**.

Використовуючи офіційну документацію від DigitalOcean [27], необхідно інтегрувати сервіс Let's Encrypt, який автоматично генерує SSL-сертифікати. Це забезпечує підтримку захищеного HTTPS з'єднання між клієнтом та сервером.

Щоб контейнери Docker мали можливість спілкуватись між собою, потрібно створити мережу за допомогою команди **docker network create kpi_notify_nw**.

За допомогою веб-інтерфейсу Portainer можна керувати станом контейнерів, запущених на сервері. Щоб Portainer зберігав налаштування та дані про стан контейнерів після перезапуску сервера, потрібно створити volume за допомогою команди **docker volume create portainer_data**. Після створення volume потрібно сконфігурувати та запустити сам контейнер Portainer за допомогою команди: **docker run -d -p 8000:8000 -p 9443:9443 --name portainer --network=kpi_notify_nw -v /var/run/docker:sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce:latest**. Перевірити успішність створення контейнера можна за допомогою команди **docker ps**, вивід якої повинен містити інформацію про працюючий контейнер **portainer**.

Після виконання усіх цих етапів, залишається лише перенести дані застосунку із локального комп'ютера на сервер. У локальному терміналі

потрібно встановити змінну середовища, яка є IP-адресою DigitalOcean сервера, за допомогою команди **export DIGITAL_OCEAN_IP_ADDRESS=<ip-адреса>**. Після цього, потрібно запустити скрипт **server_deploy.sh**, що знаходиться у кореневій директорії проекту, який підключається до сервера, переносить на нього усі дані застосунку, будує та запускає контейнери згідно з конфігураційним файлом **production.yml**.

Після завершення виконання скрипта, застосунок є успішно розгорнутим та публічно доступним за зареєстрованим доменом. Контролювати стан запущених на сервері контейнерів можна через веб-інтерфейс Portainer, доступний за посиланням **https://portainer.<основний домен>/**. Для моніторингу асинхронних задач Celery використовується Flower, доступний за адресою **https://flower.<основний домен>/**.

5.2 Супровід програмного забезпечення

Інструкція користувача наведена в окремому документі.

Супровід програмного забезпечення передбачає підтримку стабільної роботи застосунку, оновлення функціоналу та оперативне реагування на можливі помилки.

Після впровадження нового функціоналу або виправлення помилок, застосунок проходить обов'язковий етап локального тестування за допомогою автоматизованих unit-тестів та мануального тестування важливих сценаріїв. У разі успішного проходження всіх тестів, нова версія застосунку завантажується на сервер за допомогою скрипта **server_deploy.sh**. Цей скрипт створює архів із актуального вмісту репозиторію проекту, переносить його на сервер та перезапускає контейнери, не впливаючи на дані, що зберігаються у базі даних або інших volume.

Для моніторингу стану контейнерів використовується Portainer – веб-інтерфейс для керування Docker контейнерами, доступний лише для

авторизованих адміністраторів. Portainer дозволяє переглядати поточний статус контейнерів та їх логи, слідкувати за використанням ресурсів тощо.

Для контролю виконання асинхронних задач використовується Flower – веб-інтерфейс для моніторингу задач Celery. Flower дозволяє відслідковувати поточний стан черги задач, їх статус, помилки та результати виконання.

Висновки до розділу

У даному розділі було детально описано процес розгортання та супроводу розробленого програмного забезпечення. Для забезпечення стабільної та масштабованої роботи застосунку було використано технології Docker та Docker Compose.

Описано повний цикл налаштування серверної інфраструктури: реєстрація доменного імені, підключення SMTP-сервера Mailgun, розгортання на платформі DigitalOcean із забезпеченням захищеного HTTPS-з'єднання через Let's Encrypt.

Процес оновлення застосунку автоматизовано за допомогою скрипта `server_deploy.sh`, який забезпечує перенесення проекту на сервер і перезапуск контейнерів без втрати даних, збережених у базі даних.

Для контролю над поточним станом системи впроваджено веб-інтерфейси Portainer, що забезпечує управління контейнерами, та Flower що дозволяє слідкувати за асинхронними задачами Celery.

ВИСНОВКИ

У ході виконання дипломного проекту було розроблено вебзастосунок, призначений для планування подій та нагадувань, орієнтований на внутрішнє використання працівниками університету.

Були проаналізовані застосунки для планування подій, нагадувань, та роботи в команді. У більшості застосунків були виявлені такі ключові недоліки: орієнтованість на комерційне використання, неадаптованість під конкретне середовище використання, обмеженість методів отримання нагадувань.

Грунтуючись на проведеному дослідженні, були сформульовані вимоги для нового застосунку: управління профілем та зберігання даних реєстраційної картки юридичної особи, управління подіями, створення нагадувань через електронну пошту або Telegram-бота, управління командами користувачів, адаптація застосунку для використання працівниками університету.

Було спроектовано архітектуру застосунку, реалізовано клієнт-серверну модель з використанням фреймворків Django і Next.js, створено базу даних на основі PostgreSQL та розгорнуто контейнеризоване середовище за допомогою Docker Compose.

Для тестування вебзастосунку були розроблені автоматизовані unit-тести та проведені мануальні тести основного функціоналу, завдяки чому було доведено коректну та безпечну роботу проекту.

Було описано повний процес розгортання застосунку за допомогою хмарного провайдера DigitalOcean, що включає реєстрацію доменного імені, генерацію SSL-сертифікатів, налаштування серверної інфраструктури та інтеграцію SMTP-серверу Mailgun.

Таким чином, було досягнуто мети розробки, яка полягала у є спрощенні управління подіями, нагадуваннями та взаємодії між користувачами за рахунок створення зручного вебзастосунку для планування та організації діяльності у межах університету.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Miller G. Magic Number Seven. URL: <https://labs.la.utexas.edu/gilden/files/2016/04/MagicNumberSeven-Miller1956.pdf>.
2. Al-Qahtani A. The Effect of Age on Memory. URL: https://www.researchgate.net/publication/337394916_The_Effect_of_Age_on_Memory.
3. Get started with Google Calendar. Google Help. URL: https://support.google.com/calendar/answer/2465776?hl=en&ref_topic=10509740&sjid=10511347306953588384-EU.
4. Todoist | A To-Do List to Organize Your Work & Life. Todoist. URL: <https://www.todoist.com/home>.
5. Capture, organize, and tackle your to-dos from anywhere | Trello. Trello. URL: <https://trello.com/home>.
6. ClickUp | The everything app for work. ClickUp. URL: <https://clickup.com/>.
7. Client-Server Architecture Explained with Examples, Diagrams, and Real-World Applications. Medium. URL: <https://medium.com/nerd-for-tech/client-server-architecture-explained-with-examples-diagrams-and-real-world-applications-407e9e04e2d1>.
8. What is Monolithic Architecture? | IBM. IBM - United States. URL: <https://www.ibm.com/think/topics/monolithic-architecture>.
9. MVC. MDN Web Docs. URL: <https://developer.mozilla.org/en-US/docs/Glossary/MVC>.
10. Business process model and notation 2.0+ (BPMN). Custom Software Development Company OTAKOYI. URL: <https://otakoyi.software/blog/business-process-model-and-notation-2-0-bpmn>.
11. Functionpoints Course. URL: http://www.thimoty.it/softeng/functionpoints_course.pdf.

12. Docker Compose. Docker Documentation. URL: <https://docs.docker.com/compose/>.
13. Python. Python.org. URL: <https://www.python.org/doc/>.
14. Django overview. Django Project. URL: <https://www.djangoproject.com/start/overview/>.
15. Django REST framework. URL: <https://www.django-rest-framework.org/>.
16. Next.js. Next.js by Vercel - The React Framework. URL: <https://nextjs.org/docs>.
17. React. React. URL: <https://react.dev/>.
18. Django ORM (Querysets). HonKit. URL: https://tutorial.djangogirls.org/en/django_orm/.
19. Cross Site Scripting (XSS). OWASP Foundation. URL: <https://owasp.org/www-community/attacks/xss/>.
20. Cross Site Request Forgery (CSRF). OWASP Foundation. URL: [https://owasp.org/www-community/attacks/csrf#:~:text=Cross-Site%20Request%20Forgery%20\(CSRF,which%20they're%20currently%20authenticated.](https://owasp.org/www-community/attacks/csrf#:~:text=Cross-Site%20Request%20Forgery%20(CSRF,which%20they're%20currently%20authenticated.)
21. SQL Injection: Attacks & Defenses. Medium. URL: <https://blog.gatewayd.io/sql-injection-attacks-and-defense-8c0fab2abd36>.
22. Telegram Login for Websites. URL: <https://telegram.org/blog/login>
23. <https://radon.readthedocs.io/en/latest/commandline.html>.
24. Singh R. The Testing Pyramid: The Key to Efficient Software Testing. HeadSpin. URL: <https://www.headspin.io/blog/the-testing-pyramid-simplified-for-one-and-all>.
25. Simple Mail Transfer Protocol (SMTP) Explained. Mailgun. URL: <https://www.mailgun.com/blog/deliverability/what-is-smtp/>.
26. Ubuntu. Docker Documentation. URL: <https://docs.docker.com/engine/install/ubuntu/>.

27. Garnett A. How To Secure Nginx with Let's Encrypt on Ubuntu. DigitalOcean | Cloud Infrastructure for Developers. URL: <https://www.digitalocean.com/community/tutorials/how-to-secure-nginx-with-lets-encrypt-on-ubuntu-22-04>.

ДОДАТОК А. ЗВІТ ПОДІБНОСТІ



Дата звіту 6/12/2025
Дата редагування 6/12/2025

Документ прийнятий

Звіт подібності

метадані

Назва організації
National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute
Заголовок
ІП-12_Орищенко_ПЗ
Автор
Науковий керівник / Експерт
ІП-12_ОрищенкоКовтунець О.В.
підрозділ
ФІОТ, К-а інформатики та програмної інженерії

Обсяг знайдених подібностей

Коефіцієнт подібності визначає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Зверніть увагу, що високі значення коефіцієнта не автоматично означають плагіат. Звіт має аналізувати компетентна / уповноважена особа.



КП 1

10

Довжина фрази для коефіцієнта подібності 2

14680

Кількість слів

111567

Кількість символів

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2025 р.

**ВЕБЗАСТОСУНОК ДЛЯ ПЛАНУВАННЯ ТА НАГАДУВАННЯ ПРО
ВАЖЛИВІ ПОДІЇ**

Текст програми

КПІ.ІП-1219.045440.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

Максим ГОЛОВЧЕНКО

Виконавець:

Ярослав ОРИЩЕНКО

Посилання на репозиторій з повним текстом програмного коду
<https://github.com/butuz0/EventsReminder>

Файл users/models.py

Реалізація моделі користувача

```
from django.db import models
from django.contrib.auth.models import AbstractBaseUser, PermissionsMixin
from django.utils.translation import gettext_lazy as _
from .managers import UserManager
import uuid

class User(AbstractBaseUser, PermissionsMixin):
    pkid = models.BigAutoField(primary_key=True, editable=False)
    id = models.UUIDField(default=uuid.uuid4, editable=False, unique=True)
    first_name = models.CharField(verbose_name=_('First Name'), max_length=50)
    last_name = models.CharField(verbose_name=_('Last Name'), max_length=50)
    email = models.EmailField(verbose_name=_('Email Address'), db_index=True,
unique=True)

    is_staff = models.BooleanField(default=False)
    is_active = models.BooleanField(default=True)

    USERNAME_FIELD = 'email'

    REQUIRED_FIELDS = ['first_name', 'last_name']

    objects = UserManager()

    class Meta:
        verbose_name = _('user')
        verbose_name_plural = _('users')

    def __str__(self) -> str:
        return self.full_name

    @property
    def full_name(self) -> str:
        return f'{self.first_name.title()} {self.last_name.title()}'
```

Файл users/managers.py

Реалізація інтерфейсу менеджера для моделі користувача

```

from django.contrib.auth.base_user import BaseUserManager
from django.core.exceptions import ValidationError
from django.core.validators import validate_email
from django.contrib.auth.hashers import make_password
from django.utils.translation import gettext_lazy as _

def custom_validate_email(email: str):
    try:
        validate_email(email)
    except ValidationError:
        raise ValidationError(_('Enter a valid email address.'))

    if not email.endswith('@kpi.ua'):
        raise ValidationError(_('Enter your corporate @kpi.ua email address.'))

class UserManager(BaseUserManager):
    def _create_user(self, first_name: str, last_name: str, email: str,
password: str | None, is_admin: bool = False,
        **extra_fields):
        if not first_name:
            raise ValueError(_('Users must have a first name.'))
        if not last_name:
            raise ValueError(_('Users must have a last name.'))
        if not email:
            raise ValueError(_('Users must have an email address.'))

        email = self.normalize_email(email)
        if not is_admin:
            custom_validate_email(email)
        else:
            validate_email(email)

        user = self.model(first_name=first_name,
                        last_name=last_name,
                        email=email,
                        **extra_fields)
        user.password = make_password(password)
        user.save(using=self._db)
        return user

    def create_user(self, first_name: str, last_name: str, email: str,
password: str | None = None, **extra_fields):
        extra_fields.setdefault('is_staff', False)
        extra_fields.setdefault('is_superuser', False)

        return self._create_user(first_name=first_name,
                        last_name=last_name,
                        email=email,
                        password=password,
                        **extra_fields)

    def create_superuser(self, first_name: str, last_name: str, email: str,
password: str | None = None,
        **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)

        if extra_fields.get('is_staff') is not True:
            raise ValueError(_('Superuser must have is_staff=True.'))

        if extra_fields.get('is_superuser') is not True:
            raise ValueError(_('Superuser must have is_superuser=True.'))

```

```
return self._create_user(first_name=first_name,
                          last_name=last_name,
                          email=email,
                          password=password,
                          is_admin=True,
                          **extra_fields)
```

Файл `users/forms.py`

Реалізація форм для створення та редагування користувача

```
from django import forms
from django.contrib.auth import get_user_model, forms as auth_forms

User = get_user_model()

class UserCreationForm(auth_forms.UserCreationForm):
    class Meta(auth_forms.UserCreationForm.Meta):
        model = User
        fields = ['first_name', 'last_name', 'email']

    error_messages = {
        'email_already_exists': 'A user with this email already exists.',
    }

    def clean_email(self):
        email = self.cleaned_data['email']
        if User.objects.filter(email=email).exists():
            raise
        forms.ValidationError(self.error_messages['email_already_exists'])
        return email

class UserChangeForm(auth_forms.UserChangeForm):
    class Meta(auth_forms.UserChangeForm.Meta):
        model = User
        fields = ['first name', 'last name', 'email']
```

Файл profiles/models.py

Реалізація моделі профілю користувача

```
from django.db import models
from django.utils.translation import gettext_lazy as _
from django.contrib.auth import get_user_model
from apps.common.models import TimeStampedModel
from apps.common.validators import image_validator
from apps.common.uploads import upload_avatar
from apps.units.models import Department

User = get_user_model()

class Profile(TimeStampedModel):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related name='profile')
```

```

        department = models.ForeignKey(Department, on_delete=models.SET_NULL,
related_name='employees', null=True)

        position = models.CharField(verbose_name=_('Position'), max_length=250,
null=True)

        avatar = models.ImageField(upload_to=upload_avatar,
validators=[image_validator], blank=True, null=True,
                                verbose_name=_('Avatar'))

class Meta:
    verbose_name = _('Profile')
    verbose_name_plural = _('Profiles')
    indexes = [
        models.Index(fields=['position']),
    ]

    def __str__(self) -> str:
        return f'{self.user.full_name} - {self.position or 'Unknown
position'}'

    def is_telegram_verified(self) -> bool:
        return hasattr(self, 'telegram') and self.telegram.is_verified

class TelegramData(TimeStampedModel):
    profile = models.OneToOneField(Profile, on_delete=models.CASCADE,
related_name='telegram')

    # set by telegram login widget
    telegram_username = models.CharField(max_length=50, null=True,
unique=True, verbose_name=_('Telegram Username'))
    telegram_first_name = models.CharField(max_length=50, null=True,
verbose_name=_('Telegram First Name'))
    telegram_last_name = models.CharField(max_length=50, null=True,
verbose_name=_('Telegram Last Name'))
    telegram_user_id = models.BigIntegerField(unique=True, null=True,
verbose_name=_('Telegram User Id'))

    # set by telegram bot
    is_verified = models.BooleanField(default=False, verbose_name=_('Is
Verified'))

class Meta:
    verbose_name = _('Telegram Data')

```

```

        verbose_name_plural = _('Telegram Data')

    def __str__(self) -> str:
        return self.telegram_username or f'Unconnected user
{self.profile.user}'

```

Файл `profiles/serializers.py`

Реалізація сереалізаторів для моделі профілю користувача

```

from apps.units.models import Department
from .models import Profile, TelegramData
from rest_framework import serializers

class BaseProfileSerializer(serializers.ModelSerializer):
    first_name = serializers.CharField(source='user.first_name')
    last_name = serializers.CharField(source='user.last_name')
    avatar = serializers.ImageField(write_only=True, required=False)
    avatar_url = serializers.SerializerMethodField()

    class Meta:
        model = Profile
        fields = ['first_name', 'last_name', 'position',
                  'avatar', 'avatar_url']

    def get_avatar_url(self, obj: Profile) -> str | None:
        request = self.context.get('request')
        if obj.avatar and request:
            url = request.build_absolute_uri(obj.avatar.url)
            if url.startswith('http://localhost/'):
                return url.replace('http://localhost/',
                                   'http://localhost:8080/')
            return url
        return None

class TelegramDataSerializer(serializers.ModelSerializer):
    class Meta:
        model = TelegramData
        fields = ['telegram_username', 'telegram_first_name',
                  'telegram_last_name', 'is_verified']

class ProfileRetrieveSerializer(BaseProfileSerializer):
    id = serializers.UUIDField(source='user.id', read_only=True)
    email = serializers.ReadOnlyField(source='user.email')
    department_name = serializers.SerializerMethodField()
    department_abbreviation = serializers.SerializerMethodField()
    faculty_name = serializers.SerializerMethodField()
    faculty_abbreviation = serializers.SerializerMethodField()
    telegram = TelegramDataSerializer()

    class Meta(BaseProfileSerializer.Meta):
        fields = (BaseProfileSerializer.Meta.fields + ['id', 'email',
        'department_name', 'department_abbreviation',
        'department_abbreviation', 'faculty_name',
        'faculty_abbreviation', 'telegram'])

```

```

def get_department_name(self, obj: Profile) -> str | None:
    if obj.department:
        return obj.department.department_name
    return None

def get_department_abbreviation(self, obj: Profile) -> str | None:
    if obj.department:
        return obj.department.department_abbreviation
    return None

def get_faculty_name(self, obj: Profile) -> str | None:
    if obj.department and obj.department.faculty:
        return obj.department.faculty.faculty_name
    return None

def get_faculty_abbreviation(self, obj: Profile) -> str | None:
    if obj.department and obj.department.faculty:
        return obj.department.faculty.faculty_abbreviation
    return None

class ProfileUpdateSerializer(BaseProfileSerializer):
    department = serializers.PrimaryKeyRelatedField(queryset=Department.objects.all())

    class Meta(BaseProfileSerializer.Meta):
        fields = BaseProfileSerializer.Meta.fields + ['department']

    def update(self, instance: Profile, validated_data: dict) -> Profile:
        user_data = validated_data.pop('user', {})

        # update standard User model fields
        for attr, value in user_data.items():
            setattr(instance.user, attr, value)
            instance.user.save()

        return super().update(instance, validated_data)

class ProfileSetupSerializer(serializers.ModelSerializer):
    department = serializers.PrimaryKeyRelatedField(queryset=Department.objects.all())

    class Meta:
        model = Profile
        fields = ['position', 'department']

```

Файл profiles/signals.py

Реалізація сигналів для моделі профілю користувача

```

from django.db.models.signals import post_save
from django.dispatch import receiver
from config.settings.base import AUTH_USER_MODEL
from django.db.models.base import Model
from .models import Profile, TelegramData
from typing import Any, Type

@receiver(post_save, sender=AUTH_USER_MODEL)
def create_user_profile(sender: Type[Model], instance: Model, created: bool,
**kwargs: Any) -> None:
    if created:
        Profile.objects.create(user=instance)

```

```
@receiver(post_save, sender=Profile)
def create_telegram_data_model(sender, instance: Profile, created: bool,
**kwargs):
    if created and not hasattr(instance, 'telegram'):
        TelegramData.objects.create(profile=instance)
```

Файл profiles/views.py

Реалізація виглядів для моделі профілю користувача

```
from django.contrib.auth import get_user_model
from django.utils.timezone import now
from django.conf import settings
from apps.common.renderers import JSONRenderer
from .models import Profile, TelegramData
from .serializers import ProfileRetrieveSerializer, ProfileUpdateSerializer,
ProfileSetupSerializer
from .tasks import send_telegram_greeting
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import filters, generics, status, permissions
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.pagination import PageNumberPagination
import hashlib
import hmac
```

```
User = get_user_model()
```

```
class StandardResultsSetPagination(PageNumberPagination):
    page_size = 20
    page_size_query_param = 'page_size'
    max_page_size = 100
```

```
class ProfileListAPIView(generics.ListAPIView):
    queryset = Profile.objects.all()
    serializer_class = ProfileRetrieveSerializer
    pagination_class = StandardResultsSetPagination
    filter_backends = [DjangoFilterBackend, filters.SearchFilter]
    filterset_fields = ['department', 'department__faculty']
    search_fields = ['user__first_name', 'user__last_name', 'position']
    renderer_classes = [JSONRenderer]
    object_label = 'profiles'
```

```
def get_queryset(self):
    return (self.queryset
            .exclude(user=self.request.user)
            .exclude(user__is_staff=True)
            .exclude(user__is_superuser=True))
```

```
class ProfileDetailAPIView(generics.RetrieveAPIView):
    serializer_class = ProfileRetrieveSerializer
    renderer_classes = [JSONRenderer]
    object_label = 'profile'
    lookup_field = 'user__id'
    lookup_url_kwarg = 'user_id'

    def get_queryset(self):
        return Profile.objects.select_related('user', 'telegram',
        'department__faculty')
```

```

class MyProfileAPIView(generics.RetrieveAPIView):
    serializer_class = ProfileRetrieveSerializer
    renderer_classes = [JSONRenderer]
    object_label = 'profile'

    def get_object(self) -> Profile:
        profile, _ = Profile.objects.get_or_create(user=self.request.user)
        return profile

class ProfileUpdateAPIView(generics.UpdateAPIView):
    serializer_class = ProfileUpdateSerializer
    renderer_classes = [JSONRenderer]
    object_label = 'profile'

    def get_object(self) -> Profile:
        profile, _ = Profile.objects.get_or_create(user=self.request.user)
        return profile

class ProfileSetupView(APIView):
    def post(self, request):
        profile, _ = Profile.objects.get_or_create(user=request.user)

        serializer = ProfileSetupSerializer(profile, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response({'message': 'Profile set up successfully'},
                            status=status.HTTP_200_OK)

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

class TelegramAuthAPIView(APIView):
    permission_classes = [permissions.IsAuthenticated]

    def post(self, request):
        """
        Method for authorizing Telegram user.
        Telegram Login Widget sends id, first_name, last_name,
        username, photo_url, auth_date and hash fields.
        """
        data = request.data

        received_hash = data.get('hash')
        if not received_hash:
            return Response({'detail': 'Hash missing'},
                            status=status.HTTP_400_BAD_REQUEST)

        # Data-check-string is a concatenation of all received fields,
        # sorted in alphabetical order, in the format key=<value> with
        # a line feed character ('\n', 0x0A) used as separator
        auth_data = {key: value for key, value in data.items() if key != 'hash'}
        data_check_string = '\n'.join(f'{k}={auth_data[k]}' for k in
sorted(auth_data))

        # SHA256 hash of the bot's token used as a secret key
        secret_key =
hashlib.sha256(settings.TELEGRAM_BOT_TOKEN.encode()).digest()

        # hexadecimal representation of the HMAC-SHA-256
        # signature of the data-check-string
        calculated_hash = hmac.new(secret_key, data_check_string.encode(),
hashlib.sha256).hexdigest()

```

```

# verify received hash
if not hmac.compare_digest(received_hash, calculated_hash):
    return Response({'detail': 'Invalid Telegram auth data'},
                    status=status.HTTP_403_FORBIDDEN)

# check auth_date expiration (10 min)
auth_date = int(data.get('auth_date', 0))
if now().timestamp() - auth_date > 600:
    return Response({'detail': 'Telegram auth data expired'},
                    status=status.HTTP_403_FORBIDDEN)

# update TelegramData model
profile = request.user.profile
telegram_data, _ = TelegramData.objects.get_or_create(profile=profile)

send_telegram_greeting.delay(data.get('id'))

telegram_data.telegram_user_id = data.get('id')
telegram_data.telegram_username = data.get('username')
telegram_data.telegram_first_name = data.get('first_name')
telegram_data.telegram_last_name = data.get('last_name')
telegram_data.save()

return Response({'detail': 'Telegram account connected successfully'})

```

Файл `profiles/tasks.py`

Реалізація функції для надсилання повідомлення привітання після авторизації через Telegram

```

from .models import TelegramData
from telegram_bot.utils.send_message import send_message
from telegram_bot.utils.update_auth_state import set_user_verified_state
from telegram_bot.messages.verification import MESSAGES
from celery import shared_task

@shared_task(bind=True, max_retries=10, default_retry_delay=2)
def send_telegram_greeting(self, chat_id: int):
    try:
        send_message(chat_id, MESSAGES['verification_successful'])
        set_user_verified_state(chat_id)

        TelegramData.objects.filter(telegram_user_id=chat_id).update(is_verified=True)
    except Exception as e:
        raise self.retry(exc=e)

```

Файл `registration_cards/models.py`

Реалізація моделі реєстраційної картки АЦСК

```

from django.db import models
from django.contrib.auth import get_user_model
from django.utils.translation import gettext_lazy as _
from django.core.exceptions import ValidationError
from apps.common.models import TimeStampedModel
from phonenumber_field.modelfields import PhoneNumberField
from encrypted_model_fields.fields import EncryptedCharField

User = get_user_model()

```

```

class RegistrationCard(TimeStampedModel):
    created_by = models.ForeignKey(User, on_delete=models.CASCADE,
    verbose_name=_('Created By'))

    # Legal Entity
    organization_name = models.CharField(max_length=255, verbose_name=_('Full
    Organization Name'))
    edrpou_code = models.CharField(max_length=8, verbose_name=_('EDRPOU
    Code'))
    region = models.CharField(max_length=100, blank=True,
    verbose_name=_('Region'))
    city = models.CharField(max_length=100, blank=True,
    verbose_name=_('City'))

    # Applicant
    full_name = EncryptedCharField(max_length=255, blank=True,
    verbose_name=_('Applicant Full Name'))
    id_number = EncryptedCharField(max_length=20, blank=True,
    verbose_name=_('IPN or series/number of document'))
    keyword_phrase = EncryptedCharField(max_length=255, blank=True,
    verbose_name=_('Keyword Phrase'))
    voice_phrase = EncryptedCharField(max_length=255, blank=True,
    verbose_name=_('Voice Phrase'))

    # Contacts
    email = models.EmailField(blank=True, verbose_name=_('Email'))
    phone_number = PhoneNumberField(max_length=20, blank=True,
    verbose_name=_('Phone Number'))

    # Electronic Seal
    electronic_seal_name = EncryptedCharField(max_length=255, blank=True,
    verbose_name=_('Electronic Seal
    Name'))
    electronic_seal_keyword_phrase = EncryptedCharField(max_length=255,
    blank=True,
    verbose_name=_('Electronic Seal Keyword Phrase'))

    class Meta:
        verbose_name = 'Registration Card'
        verbose_name_plural = 'Registration Cards'

    def __str__(self):
        return self.organization_name

    def clean(self):
        if self.edrpou_code and not self.edrpou_code.isdigit():
            raise ValidationError(_('edrpou_code': _('EDRPOU Code must contain
    only digits.')))
        if self.edrpou_code and len(self.edrpou_code) != 8:
            raise ValidationError(_('edrpou_code': _('EDRPOU Code must be 8
    digits long.')))

```

Файл `regitration_cars/serializers.py`

Реалізація сереалізатора для моделі реєстраційної картки АЦСК

```

from rest_framework import serializers
from rest_framework.exceptions import ValidationError
from .models import RegistrationCard

```

```

class RegistrationCardSerializer(serializers.ModelSerializer):
    class Meta:
        model = RegistrationCard

```

```

fields = '__all__'
read_only_fields = ('created_by', 'created_at', 'updated_at')

def validate_edrpou_code(self, value):
    if not value.isdigit():
        raise ValidationError({'edrpou_code': 'EDRPOU Code must contain
only digits.'})
    if len(value) != 8:
        raise ValidationError({'edrpou_code': 'EDRPOU Code must be 8 digits
long.'})
    return value

```

Файл registration_cards/views.py

Реалізація виглядів для моделі реєстраційної картки АЦСК

```

from apps.common.renderers import JSONRenderer
from .models import RegistrationCard
from .serializers import RegistrationCardSerializer
from .permissions import IsOwner
from rest_framework import generics
from rest_framework.permissions import IsAuthenticated

class RegistrationCardListCreateAPIView(generics.ListCreateAPIView):
    queryset = RegistrationCard.objects.all()
    serializer_class = RegistrationCardSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'registration_cards'

    def perform_create(self, serializer):
        serializer.save(created_by=self.request.user)

    def get_queryset(self):
        return self.queryset.filter(created_by=self.request.user)

class RegistrationCardAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = RegistrationCard.objects.all()
    serializer_class = RegistrationCardSerializer
    permission_classes = [IsAuthenticated, IsOwner]
    lookup_field = 'id'

```

Файл events/models.py

Реалізація моделі події

```

from django.db import models
from django.utils.translation import gettext_lazy as _
from django.contrib.auth import get_user_model
from django.utils import timezone
from apps.teams.models import Team
from apps.common.models import TimeStampedModel
from apps.common.validators import image_validator
from apps.common.uploads import upload_event_image
from taggit.managers import TaggableManager
from dateutil.relativedelta import relativedelta
from datetime import datetime, timedelta

User = get_user_model()

class Event(TimeStampedModel):
    class Priority(models.IntegerChoices):

```

```

        LOW = 1, _('Low')
        MEDIUM = 2, _('Medium')
        HIGH = 3, _('High')
        CRITICAL = 4, _('Critical')

    # Required fields
    created_by = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='created_events', verbose_name=_('Created By'))
    title = models.CharField(max_length=255, verbose_name=_('Title'))
    priority = models.PositiveSmallIntegerField(choices=Priority.choices,
default=Priority.MEDIUM, verbose_name=_('Priority'))
    start_datetime = models.DateTimeField(verbose_name=_('Start Time'))

    # Optional fields
    team = models.ForeignKey(Team, on_delete=models.CASCADE,
related_name='events', null=True, blank=True, verbose_name=_('Team'))
    assigned_to = models.ManyToManyField(User, related_name='assigned_events',
verbose_name=_('Assigned To'), blank=True)
    description = models.TextField(blank=True, verbose_name=_('Description'))
    location = models.CharField(max_length=255, blank=True,
verbose_name=_('Location'))
    link = models.URLField(blank=True, null=True, verbose_name=_('Event Link'))
    image = models.ImageField(upload_to=upload_event_image,
validators=[image_validator], blank=True, null=True,
verbose_name=_('Image'))
    tags = TaggableManager(blank=True, verbose_name=_('Tags'))

    # If event is recurring, RecurringEvent model will be created
    is_recurring = models.BooleanField(default=False, verbose_name=_('Is
Recurring'))

    class Meta:
        verbose_name = _('Event')
        verbose_name_plural = _('Events')
        ordering = ['start_datetime']

    def __str__(self) -> str:
        return self.title

class RecurringEvent(TimeStampedModel):
    """
    Extends Event model with additional fields for recurring events.
    """

    class RecurrenceRule(models.TextChoices):
        DAILY = 'd', _('Daily')
        WEEKLY = 'w', _('Weekly')
        MONTHLY = 'm', _('Monthly')
        YEARLY = 'y', _('Yearly')

    event = models.OneToOneField(Event, on_delete=models.CASCADE,
related_name='recurring_event', verbose_name=_('Recurring Event'))
    recurrence_rule = models.CharField(verbose_name=_('Recurrence Rule'),
max_length=10, choices=RecurrenceRule.choices,
default=RecurrenceRule.WEEKLY)
    recurrence_end_datetime = models.DateTimeField(null=True, blank=True,
verbose_name=_('End time'))

```

```

# celery task for updating event.start_datetime
celery_task_id = models.CharField(max_length=255, blank=True, null=True)

class Meta:
    verbose_name = _('Recurring Event')
    verbose_name_plural = _('Recurring Events')

def __str__(self) -> str:
    return f'{self.event.title} - {self.get_recurrence_rule_display()}'

def calculate_next_occurrence(self) -> datetime | None:
    """
    Calculate the next occurrence of the event based on the recurrence rule.
    Returns None if:
        - recurrence is finished (next occurrence > end_datetime);
        - recurrence rule is unrecognized.
    """
    event_datetime = self.event.start_datetime

    if timezone.is_naive(event_datetime):
        event_datetime = timezone.make_aware(event_datetime)

    if event_datetime >= timezone.now():
        return event_datetime

    if self.recurrence_rule == self.RecurrenceRule.DAILY:
        next_occurrence = event_datetime + timedelta(days=1)
    elif self.recurrence_rule == self.RecurrenceRule.WEEKLY:
        next_occurrence = event_datetime + timedelta(weeks=1)
    elif self.recurrence_rule == self.RecurrenceRule.MONTHLY:
        next_occurrence = event_datetime + relativedelta(months=1)
    elif self.recurrence_rule == self.RecurrenceRule.YEARLY:
        next_occurrence = event_datetime + relativedelta(years=1)
    else:
        return None

    if self.recurrence_end_datetime and next_occurrence >= self.recurrence_end_datetime:
        return None
    return next_occurrence

```

Файл events/serializers.py

Реалізація сереалізаторів для моделі події

```

from django.utils.timezone import now
from django.contrib.auth import get_user_model
from .models import Event, RecurringEvent
from .recurring import reschedule_recurring_event
from apps.teams.models import Team
from apps.users.serializers import CustomUserSerializer
from rest_framework import serializers
from taggit.serializers import TagListSerializerField, TaggitSerializer
from djoser.serializers import UserSerializer
from datetime import datetime

User = get_user_model()

```

```

class RecurringEventSerializer(serializers.ModelSerializer):
    class Meta:
        model = RecurringEvent
        fields = [
            'id', 'recurrence_rule', 'recurrence_end_datetime',
            'created_at', 'updated_at'

```

```

    ]
    read_only_fields = ['id', 'created_at', 'updated_at']

    def validate_recurrence_end_datetime(self, value: datetime) -> datetime:
        if value and value < now():
            raise serializers.ValidationError('End date must be in the
future.')
        return value

    def validate(self, data: dict) -> dict:
        if self.instance:
            event = self.instance.event # if object already exists
        else:
            event = self.context.get('event') # if object is being created

        if event and data.get('recurrence_end_datetime') and
data['recurrence_end_datetime'] < event.start_datetime:
            raise serializers.ValidationError('End date must be after event
start date.')
        return data

    def create(self, validated_data: dict) -> RecurringEvent:
        event = validated_data.pop('event', None) or self.context.get('event')
        if not event:
            raise serializers.ValidationError('Event must be provided.')

        recurring = RecurringEvent.objects.create(event=event,
**validated_data)
        reschedule_recurring_event(recurring)
        return recurring

    def update(self, instance: RecurringEvent, validated_data: dict) ->
RecurringEvent:
        recurring = super().update(instance, validated_data)
        reschedule_recurring_event(instance)
        return recurring

class BaseEventSerializer(TaggitSerializer, serializers.ModelSerializer):
    assigned_to_ids = serializers.ListField(child=serializers.UUIDField(),
write_only=True, required=False)
    assigned_to = CustomUserSerializer(many=True, read_only=True)
    tags = TagListSerializerField(required=False)

    class Meta:
        model = Event
        fields = ['id', 'title', 'description', 'start_datetime',
'location', 'link', 'priority', 'image',
'tags', 'assigned_to_ids', 'assigned_to',
'is_recurring']

    def validate_start_datetime(self, value: datetime) -> datetime:
        if value < now():
            raise serializers.ValidationError('Event start time cannot be in
the past.')
        return value

    def validate_assigned_to_ids(self, value: list[str]) -> list[str]:
        user = self.context['request'].user

        if user.id in value:
            raise serializers.ValidationError('You cannot assign the event to
yourself.')

```

```

    # Check if any user IDs are duplicates
    user_ids = set(value)
    if len(user_ids) != len(value):
        raise serializers.ValidationError('Duplicate user IDs are not
allowed.')

    # Check if all users exist
    existing_ids = set(
        User.objects
        .filter(id__in=user_ids)
        .values_list('id', flat=True)
    )
    missing_ids = user_ids - existing_ids
    if missing_ids:
        raise serializers.ValidationError(f'Users with the following IDs do
not exist: {list(missing_ids)}')

    return value

class EventCreateSerializer(BaseEventSerializer):
    team = serializers.UUIDField(required=False)

    class Meta(BaseEventSerializer.Meta):
        fields = BaseEventSerializer.Meta.fields + ['team']

    def validate_team(self, value: str) -> str:
        try:
            team = Team.objects.get(id=value)
        except Team.DoesNotExist:
            raise serializers.ValidationError('Team does not exist.')

        if self.context['request'].user != team.created_by:
            raise serializers.ValidationError('Only team owner can create
events.')

        return value

    def validate(self, attrs: dict) -> dict:
        team_id = attrs.get('team')
        if not team_id:
            return attrs

        # Check if all users are members of the provided team
        team = Team.objects.get(id=team_id)

        assigned_to_ids = set(attrs.get('assigned_to_ids'))
        team_member_ids = set(team.members.values_list('id', flat=True))

        if not assigned_to_ids.issubset(team_member_ids):
            invalid_ids = assigned_to_ids - team_member_ids
            raise serializers.ValidationError(
                f'The following users are not members of the selected team:
{list(invalid_ids)}'
            )
        return attrs

    def create(self, validated_data: dict) -> Event:
        tags = validated_data.pop('tags', [])
        assigned_to = validated_data.pop('assigned_to_ids', [])
        team_id = validated_data.pop('team', None)
        team = Team.objects.get(id=team_id) if team_id else None
        user = self.context['request'].user

```

```

        event = Event.objects.create(created_by=user, team=team,
**validated_data)
        event.assigned_to.set(User.objects.filter(id__in=assigned_to))
        event.tags.set(tags)

    return event

class EventUpdateSerializer(BaseEventSerializer):
    class Meta(BaseEventSerializer.Meta):
        fields = BaseEventSerializer.Meta.fields

    def update(self, instance: Event, validated_data: dict) -> Event:
        tags = validated_data.pop('tags', None)
        assigned_to = validated_data.pop('assigned_to_ids', None)
        old_datetime = instance.start_datetime
        new_datetime = validated_data.get('start_datetime', old_datetime)

        # Delete RecurringEvent instance if is_recurring is set from True to
False
        if not validated_data.get('is_recurring', True) and hasattr(instance,
'recurring_event'):
            instance.recurring_event.delete()

        # Update standard model fields
        for attr, value in validated_data.items():
            setattr(instance, attr, value)
        instance.save()

        if instance.is_recurring and hasattr(instance, 'recurring_event') and
old_datetime != new_datetime:
            reschedule_recurring_event(instance.recurring_event)

        # Update tags field
        if tags is not None:
            instance.tags.set(tags)

        # Update assigned_to field
        if assigned_to is not None:
            assigned_to_users = User.objects.filter(id__in=assigned_to)
            instance.assigned_to.set(assigned_to_users)

    return instance

class EventDetailSerializer(serializers.ModelSerializer):
    tags = TagListSerializerField()
    created_by = UserSerializer(read_only=True)
    assigned_to = CustomUserSerializer(many=True, read_only=True)
    recurring_event = RecurringEventSerializer(read_only=True)
    image_url = serializers.SerializerMethodField()
    team = serializers.SerializerMethodField()

    class Meta:
        model = Event
        fields = ['id', 'created_by', 'title', 'description',
                  'start_datetime', 'location', 'link',
                  'priority', 'image_url', 'tags', 'assigned_to',
                  'is_recurring', 'recurring_event', 'team', 'created_at']

    def get_team(self, obj: Event) -> dict | None:
        if obj.team:
            return {
                'id': str(obj.team.id),

```

```

        'name': obj.team.name
    }
    return None

def get_image_url(self, obj: Event) -> str | None:
    request = self.context.get('request')
    if obj.image and request:
        url = request.build_absolute_uri(obj.image.url)
        if url.startswith('http://localhost/'):
            return url.replace('http://localhost/',
                                'http://localhost:8080/')
        return url
    return None

```

Файл events/filters.py

Реалізація фільтрів для моделі події

```

from taggit.models import Tag
from .models import Event
import django_filters

class EventFilter(django_filters.FilterSet):
    from_date = django_filters.DateTimeFilter(field_name='start_datetime',
                                              lookup_expr='gte')
    to_date = django_filters.DateTimeFilter(field_name='start_datetime',
                                           lookup_expr='lte')

    priority = django_filters.NumberFilter(field_name='priority')

    tags = django_filters.ModelMultipleChoiceFilter(field_name='tags__name',
                                                    to_field_name='name',

queryset=Tag.objects.all(),
                                                    lookup_expr='icontains')

    is_recurring = django_filters.BooleanFilter(field_name='is_recurring')

    team = django_filters.UUIDFilter(field_name='team__id')

    ordering = django_filters.OrderingFilter(
        fields=['start_datetime', 'priority', 'title', 'created_at']
    )

    class Meta:
        model = Event
        fields = ['priority', 'tags', 'from_date', 'to_date', 'is_recurring',
'team']
        ordering = ['start_datetime']

```

Файл events/permissions.py

Реалізація дозволів для моделі події

```

from .models import Event, RecurringEvent
from apps.common.permissions import IsAdminPermissionMixin
from rest_framework import permissions
from rest_framework.request import Request
from rest_framework.views import View

class IsOwner(IsAdminPermissionMixin, permissions.BasePermission):
    """
    Custom permission to allow only owners of an event to edit it.

```

```

'''
message = 'You do not have permission to edit or delete this event.'

def has_object_permission(self, request: Request, view: View, obj: Event |
RecurringEvent) -> bool:
    if isinstance(obj, Event):
        return obj.created_by == request.user or self.is_admin(request)
    elif isinstance(obj, RecurringEvent):
        return obj.event.created_by == request.user or
self.is_admin(request)

class IsOwnerOrAssignedTo(IsAdminPermissionMixin, permissions.BasePermission):
    '''IsOwnerOrAssignedTo
    Custom permission to allow owners of an event
    and users assigned to the event to view it.
    '''

    def has_object_permission(self, request: Request, view: View, obj: Event)
-> bool:
        user = request.user
        return user == obj.created_by or
obj.assigned_to.filter(id=user.id).exists() or self.is_admin(request)

```

Файл events/signals.py

Реалізація сигналів для моделі події

```

from django.db.models.signals import pre_save, post_delete
from django.dispatch import receiver
from .models import Event, RecurringEvent
from celery.result import AsyncResult

@receiver(pre_save, sender=Event)
def delete_old_image_on_update(sender, instance: Event, **kwargs):
    if not instance.id:
        return

    try:
        old_image = Event.objects.only('image').get(pk=instance.pk).image
    except Event.DoesNotExist:
        return

    new_image = instance.image

    if old_image and old_image != new_image:
        old_image.delete(save=False)

@receiver(post_delete, sender=Event)
def delete_image_on_delete(sender, instance: Event, **kwargs):
    if instance.image:
        instance.image.delete(save=False)

@receiver(post_delete, sender=RecurringEvent)
def set_recurring_flag_false(sender, instance: RecurringEvent, **kwargs):
    event = instance.event
    if event.is_recurring:
        event.is_recurring = False
        event.save(update_fields=['is_recurring'])

@receiver(post_delete, sender=RecurringEvent)

```

```
def delete_event_rescheduling_celery_task(sender, instance: RecurringEvent,
**kwargs):
    if instance.celery_task_id:
        AsyncResult(instance.celery_task_id).revoke(terminate=True)
```

Файл events/views.py

Реалізація виглядів для моделі події

```
from django.shortcuts import get_object_or_404
from apps.common.renderers import JSONRenderer
from .models import Event, RecurringEvent
from .serializers import (
    EventCreateSerializer,
    EventUpdateSerializer,
    EventDetailSerializer,
    RecurringEventSerializer
)
from .permissions import IsOwner, IsOwnerOrAssignedTo
from .filters import EventFilter
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import generics, status, filters as drf_filters
from rest_framework.permissions import IsAuthenticated
from rest_framework.exceptions import PermissionDenied
from rest_framework.pagination import PageNumberPagination
from rest_framework.response import Response
from rest_framework.views import APIView

class StandardResultsSetPagination(PageNumberPagination):
    page_size = 20
    page_size_query_param = 'page_size'
    max_page_size = 100

class MyEventsListAPIView(generics.ListAPIView):
    """
    API view to retrieve a list of events created by
    or assigned to the authenticated user.
    """
    queryset = Event.objects.all().prefetch_related('tags', 'assigned_to')
    serializer_class = EventDetailSerializer
    permission_classes = [IsAuthenticated, IsOwnerOrAssignedTo]
    pagination_class = StandardResultsSetPagination
    filter_backends = [DjangoFilterBackend, drf_filters.SearchFilter]
    filterset_class = EventFilter
    search_fields = ['title', 'description', 'tags__name']
    renderer_classes = [JSONRenderer]
    object_label = 'events'

    def get_queryset(self):
        created_events = Event.objects.filter(created_by=self.request.user)
        assigned_events = Event.objects.filter(assigned_to=self.request.user)
        return (created_events
                | assigned_events).distinct().order_by('start_datetime')

class EventDetailAPIView(generics.RetrieveAPIView):
    queryset = Event.objects.all()
    serializer_class = EventDetailSerializer
    permission_classes = [IsAuthenticated, IsOwnerOrAssignedTo]
    renderer_classes = [JSONRenderer]
    object_label = 'event'
    lookup_field = 'id'
```

```

class EventCreateAPIView(generics.CreateAPIView):
    serializer_class = EventCreateSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'event'

class EventUpdateAPIView(generics.UpdateAPIView):
    serializer_class = EventUpdateSerializer
    permission_classes = [IsAuthenticated, IsOwner]
    renderer_classes = [JSONRenderer]
    object_label = 'event'
    lookup_field = 'id'

    def get_queryset(self):
        return Event.objects.filter(created_by=self.request.user)

class EventDeleteAPIView(generics.DestroyAPIView):
    """
    API view to delete an event created by the user.
    """
    serializer_class = EventUpdateSerializer
    permission_classes = [IsAuthenticated, IsOwner]
    lookup_field = 'id'

    def get_queryset(self):
        return Event.objects.filter(created_by=self.request.user)

class EventLeaveAPIView(APIView):
    """
    API view to delete yourself from assigned_to field of an event.
    """
    permission_classes = [IsAuthenticated]

    def delete(self, request, id):
        event = get_object_or_404(Event, id=id)

        if event.assigned_to.filter(id=request.user.id).exists():
            event.assigned_to.remove(request.user)
            return Response(status=status.HTTP_204_NO_CONTENT)

        raise PermissionDenied('You cannot leave the event because you have
never been assigned to it.')

class RecurringEventCreateAPIView(generics.CreateAPIView):
    serializer_class = RecurringEventSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'recurring_event'

    def perform_create(self, serializer):
        event_id = self.kwargs['event_id']
        event = get_object_or_404(Event, id=event_id)

        if event.created_by != self.request.user:
            raise PermissionDenied('Only event creator can create a recurring
event.')

        event.is_recurring = True
        event.save(update_fields=['is_recurring'])

```

```

        serializer.save(event=event)

class RecurringEventUpdateAPIView(generics.UpdateAPIView):
    queryset = RecurringEvent.objects.all()
    serializer_class = RecurringEventSerializer
    permission_classes = [IsAuthenticated, IsOwner]
    renderer_classes = [JSONRenderer]
    object_label = 'recurring_event'

    def get_object(self):
        event_id = self.kwargs['event_id']
        event = get_object_or_404(Event, id=event_id)
        obj = get_object_or_404(RecurringEvent, event=event)
        self.check_object_permissions(self.request, obj)
        return obj

```

Файл notifications/models.py

Реалізація моделі нагадування

```

from django.db import models
from django.utils.translation import gettext_lazy as _
from django.contrib.auth import get_user_model
from apps.events.models import Event

User = get_user_model()

class Notification(models.Model):
    class NotificationMethod(models.TextChoices):
        EMAIL = 'email', _('Email')
        TELEGRAM = 'tg', _('Telegram')

    event = models.ForeignKey(Event, on_delete=models.CASCADE,
        related_name='notifications', verbose_name=_('Event'))
    created_by = models.ForeignKey(User, on_delete=models.CASCADE,
        related_name='notifications',
        verbose_name=_('Created By'))
    notification_method = models.CharField(max_length=10,
        choices=NotificationMethod.choices,
        default=NotificationMethod.EMAIL,
        verbose_name=_('Notification Method'))
    notification_datetime = models.DateTimeField(verbose_name=_('Notification
        Datetime'))
    celery_task_id = models.CharField(max_length=255, blank=True, null=True,
        verbose_name=_('Celery Task ID'))
    is_sent = models.BooleanField(default=False, blank=True,
        verbose_name=_('Is Sent'))

    class Meta:
        verbose_name = _('Notification')
        verbose_name_plural = _('Notifications')
        ordering = ['-notification_datetime']

    def __str__(self) -> str:
        return f'{self.event.title} - {self.get_notification_method_display()}'

```

Файл notifications/serializers.py

Реалізація сереалізаторів для моделі нагадування

```

from django.utils.timezone import now

```

```

from rest_framework import serializers
from apps.events.models import Event
from .models import Notification

class NotificationSerializer(serializers.ModelSerializer):
    event = serializers.UUIDField()
    created_by = serializers.SerializerMethodField()

    class Meta:
        model = Notification
        fields = [
            'id', 'event', 'notification_method', 'created_by',
            'notification_datetime', 'is_sent'
        ]
        read_only_fields = ['id', 'created_by', 'is_sent']

    def get_created_by(self, obj: Notification) -> str:
        return str(obj.created_by.id)

    def validate_notification_datetime(self, value):
        if value < now():
            raise serializers.ValidationError('Notification datetime cannot be
in the past.')

        event = self.initial_data.get('event')
        if not event:
            raise serializers.ValidationError('Event is required.')

        try:
            event = Event.objects.select_related('recurring_event').get(id=event)
        except Event.DoesNotExist:
            raise serializers.ValidationError('Event does not exist.')

        if value > event.start_datetime:
            raise serializers.ValidationError('Notification must be before
event start time.')

        if event.is_recurring and hasattr(event, 'recurring_event'):
            recurrence_end = event.recurring_event.recurrence_end_datetime
            if recurrence_end and value > recurrence_end:
                raise serializers.ValidationError('Notification must be before
recurring event end time.')

        return value

    def validate_event(self, value):
        if not Event.objects.filter(id=value).exists():
            raise serializers.ValidationError('Event does not exist.')
        return value

    def validate(self, attrs):
        event_id = attrs.get('event')
        event = Event.objects.get(id=event_id)
        user = self.context['request'].user

        if event.created_by != user and not
event.assigned_to.filter(id=user.id).exists():
            raise serializers.ValidationError('You do not have permission to
create notifications for this event.')

        if (attrs.get(

```

```

        'notification_method')
Notification.NotificationMethod.TELEGRAM
user.profile.is_telegram_verified():
        raise serializers.ValidationError('Your Telegram account is not
connected yet.')

    return attrs

    def create(self, validated_data):
        event_id = validated_data.pop('event')
        event = Event.objects.get(id=event_id)
        user = self.context['request'].user

        return Notification.objects.create(event=event,
**validated_data)

```

Файл .py

Реалізація асинхронних задач для відправлення нагадувань

```

from django.contrib.auth import get_user_model
from apps.events.models import Event
from .models import Notification
from .emails import send_notification_email
from telegram_bot.utils.generate_reminder import generate_event_reminder_text
from telegram_bot.utils.send_message import send_message
from celery import shared_task

```

```
User = get_user_model()
```

```
@shared_task
```

```
def send_notification_email_task(event_id, user_id, notification_id):
    event = Event.objects.get(id=event_id)
    user = User.objects.get(id=user_id)

```

```
    send_notification_email(user, event)

```

```
    notification = Notification.objects.get(id=notification_id)
    notification.is_sent = True
    notification.save(update_fields=['is_sent'])

```

```
@shared_task
```

```
def send_notification_telegram_message_task(event_id, user_id,
notification_id):
    event = Event.objects.get(id=event_id)
    user = User.objects.get(id=user_id)

```

```
    text = generate_event_reminder_text(event)
    user_id = user.profile.telegram.telegram_user_id

```

```
    send_message(user_id, text)

```

```
    notification = Notification.objects.get(id=notification_id)
    notification.is_sent = True
    notification.save(update_fields=['is_sent'])

```

Файл teams.py

Реалізація моделей команди та запрошення у команду

```

from django.db import models
from django.contrib.auth import get_user_model
from django.utils.translation import gettext_lazy as _

```

```

from apps.common.models import TimeStampedModel

User = get_user_model()

class Team(TimeStampedModel):
    name = models.CharField(max_length=255)
    created_by = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='created_teams',
verbose_name=_('Created By'))
    members = models.ManyToManyField(User, related_name='teams', blank=True,
verbose_name=_('Members'))
    description = models.TextField(blank=True, null=True,
verbose_name=_('Description'))

    def __str__(self):
        return f'{self.name} - {self.created_by}'

    class Meta:
        verbose_name = 'Team'
        verbose_name_plural = 'Teams'
        indexes = [
            models.Index(fields=['created_by']),
        ]

class Invitation(TimeStampedModel):
    class Status(models.TextChoices):
        PENDING = 'p', 'Pending'
        ACCEPTED = 'a', 'Accepted'
        REJECTED = 'r', 'Rejected'

    created_by = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='sent_invitations',
verbose_name=_('Created By'))
    team = models.ForeignKey(Team, on_delete=models.CASCADE,
related_name='invitations', verbose_name=_('Team'))
    sent_to = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='received_invitations',
verbose_name=_('Sent To'))
    status = models.CharField(max_length=5, choices=Status.choices,
default=Status.PENDING, blank=True,
verbose_name=_('Status'))

    def __str__(self):
        return f'{self.sent_to} - {self.team}'

    class Meta:
        verbose_name = 'Invitation'
        verbose_name_plural = 'Invitations'

```

Файл teams/serializers.py

Реалізація сереалізаторів для моделей команди та запрошення у команду

```

from django.contrib.auth import get_user_model
from .models import Team, Invitation
from apps.users.serializers import CustomUserSerializer
from rest_framework import serializers

User = get_user_model()

class TeamCreateSerializer(serializers.ModelSerializer):

```

```

members_ids = serializers.ListField(child=serializers.UUIDField(),
write_only=True, required=False)

class Meta:
    model = Team
    fields = [
        'id', 'created_by', 'name', 'description', 'members_ids',
        'members', 'created_at', 'updated_at'
    ]
    read_only_fields = ['id', 'created_by', 'members', 'created_at',
'updated_at']

    def validate_members_ids(self, value: list[str]) -> list[str]:
        if not value:
            return []
        if self.context['request'].user.id in value:
            raise serializers.ValidationError('You cannot add yourself to the
team since you are the creator of it.')

        user_ids = set(value)
        if len(user_ids) != len(value):
            raise serializers.ValidationError('Duplicate user IDs are not
allowed.')

        existing_ids =
set(User.objects.filter(id__in=user_ids).values_list('id', flat=True))
        missing_ids = user_ids - existing_ids
        if missing_ids:
            raise serializers.ValidationError(f'Users with the following IDs do
not exist: {list(missing_ids)}')

        return value

    def create(self, validated_data: dict) -> Team:
        user = self.context['request'].user
        members_ids = validated_data.pop('members_ids', [])

        team = Team.objects.create(created_by=user, **validated_data)

        # Create invitation for each member
        members = User.objects.filter(id__in=members_ids)
        for member in members:
            Invitation.objects.create(created_by=user, team=team,
sent_to=member)

        return team

class TeamUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = Team
        fields = ['name', 'description']

class TeamDetailSerializer(serializers.ModelSerializer):
    created_by = CustomUserSerializer(read_only=True)
    members = CustomUserSerializer(many=True, read_only=True)

    class Meta:
        model = Team
        fields = ['id', 'name', 'description', 'created_by',
'members', 'created_at', 'updated_at']
        read_only_fields = fields

```

```

class InvitationCreateSerializer(serializers.ModelSerializer):
    team = serializers.UUIDField()
    sent_to = serializers.UUIDField()

    class Meta:
        model = Invitation
        fields = ['id', 'team', 'created_by', 'sent_to',
                  'status', 'created_at', 'updated_at']
        read_only_fields = ['id', 'created_by']

    def validate_team(self, value: str) -> Team:
        try:
            team = Team.objects.get(id=value)
        except Team.DoesNotExist:
            raise serializers.ValidationError('Team does not exist.')
        return team

    def validate_sent_to(self, value: str):
        try:
            user = User.objects.get(id=value)
        except User.DoesNotExist:
            raise serializers.ValidationError('User does not exist.')
        return user

    def validate(self, attrs: dict) -> dict:
        user = self.context['request'].user
        team = attrs.get('team')
        sent_to = attrs.get('sent_to')

        if user != team.created_by:
            raise serializers.ValidationError('Only team owner can send
            invitations.')
        if sent_to == user:
            raise serializers.ValidationError('You cannot invite yourself.')
        if sent_to in team.members.all():
            raise serializers.ValidationError('User is already a member of the
            team.')
        if Invitation.objects.filter(team=team, sent_to=sent_to,
            status=Invitation.Status.PENDING).exists():
            raise serializers.ValidationError('Invitation already pending.')
        return attrs

    def create(self, validated_data: dict) -> Invitation:
        validated_data['created_by'] = self.context['request'].user
        return super().create(validated_data)

class InvitationDetailSerializer(serializers.ModelSerializer):
    team_name = serializers.CharField(source='team.name')
    team_description = serializers.CharField(source='team.description')
    sent_to = CustomUserSerializer()
    created_by = CustomUserSerializer()

    class Meta:
        model = Invitation
        fields = ['id', 'sent_to', 'created_by', 'team_name',
                  'team_description', 'status',
                  'created_at', 'updated_at']

class InvitationRespondSerializer(serializers.ModelSerializer):
    status = serializers.ChoiceField(choices=Invitation.Status.choices,
        required=True)

```

```

class Meta:
    model = Invitation
    fields = ['status']

def validate(self, attrs: dict) -> dict:
    instance = self.instance

    if instance.status != Invitation.Status.PENDING:
        raise serializers.ValidationError('Invitation already responded.')

    return attrs

def update(self, instance: Invitation, validated_data: dict) -> Invitation:
    instance.status = validated_data['status']
    instance.save()

    if instance.status == Invitation.Status.ACCEPTED:
        instance.team.members.add(instance.sent_to)

    return instance

```

Файл teams/views.py

Реалізація виглядів для моделей команди та запрошення у команду

```

from django.db.models import Q
from django.shortcuts import get_object_or_404
from django.contrib.auth import get_user_model
from apps.common.renderers import JSONRenderer
from apps.events.models import Event
from apps.events.serializers import EventDetailSerializer
from apps.events.filters import EventFilter
from .permissions import IsOwner, IsOwnerOrMember
from .models import Team, Invitation
from .serializers import (
    TeamCreateSerializer,
    TeamUpdateSerializer,
    TeamDetailSerializer,
    InvitationCreateSerializer,
    InvitationDetailSerializer,
    InvitationRespondSerializer
)
from rest_framework import generics, status
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from rest_framework.exceptions import PermissionDenied
from rest_framework import filters as drf_filters
from django_filters.rest_framework import DjangoFilterBackend
from djoser.serializers import UserSerializer

User = get_user_model()

class TeamListAPIView(generics.ListAPIView):
    queryset = Team.objects.all()
    serializer_class = TeamDetailSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [drf_filters.SearchFilter]
    search_fields = ('name', 'description', 'created_by__first_name',
                    'created_by__last_name')
    renderer_classes = [JSONRenderer]
    object_label = 'teams'

```

```

    def get_queryset(self):
        user = self.request.user
        return self.queryset.filter(Q(created_by=user)
Q(members=user)).distinct()

class TeamCreateAPIView(generics.CreateAPIView):
    queryset = Team.objects.all()
    serializer_class = TeamCreateSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'team'

class TeamUpdateAPIView(generics.UpdateAPIView):
    queryset = Team.objects.all()
    serializer_class = TeamUpdateSerializer
    permission_classes = [IsAuthenticated, IsOwner]
    renderer_classes = [JSONRenderer]
    object_label = 'team'
    lookup_field = 'id'

class TeamDetailAPIView(generics.RetrieveAPIView):
    queryset = Team.objects.all()
    serializer_class = TeamDetailSerializer
    permission_classes = [IsAuthenticated, IsOwnerOrMember]
    renderer_classes = [JSONRenderer]
    object_label = 'team'
    lookup_field = 'id'

class TeamDeleteAPIView(generics.DestroyAPIView):
    queryset = Team.objects.all()
    serializer_class = TeamDetailSerializer
    permission_classes = [IsAuthenticated, IsOwner]
    lookup_field = 'id'

class RemoveTeamMemberView(APIView):
    permission_classes = [IsAuthenticated]

    def delete(self, request, team_id, user_id):
        team = get_object_or_404(Team, id=team_id, created_by=request.user)
        user = get_object_or_404(User, id=user_id)

        if user not in team.members.all():
            return Response({'detail': 'User is not a member of this team.'},
                            status=status.HTTP_400_BAD_REQUEST)

        team.members.remove(user)
        return Response({'detail': 'Member deleted successfully.'},
                        status=status.HTTP_204_NO_CONTENT)

class TeamLeaveAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def delete(self, request, id):
        user = request.user
        team = get_object_or_404(Team, id=id)

        if team.created_by == user:

```

```

        return Response({'detail': 'You cannot leave a team that you have
created.'},
                        status=status.HTTP_403_FORBIDDEN)

    if not team.members.filter(id=user.id).exists():
        raise PermissionDenied('You are not a member of this team.')

    team.members.remove(user)
    return Response({'detail': 'You have left the team.'},
                    status=status.HTTP_204_NO_CONTENT)

class InvitationListCreateAPIView(generics.ListCreateAPIView):
    queryset = Invitation.objects.all()
    serializer_class = InvitationCreateSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'invitations'

class InvitationListAPIView(generics.ListCreateAPIView):
    queryset = Invitation.objects.all()
    serializer_class = InvitationDetailSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'invitations'

    def get_queryset(self):
        user = self.request.user
        return self.queryset.filter(sent_to=user)

class InvitationDetailAPIView(generics.RetrieveAPIView):
    queryset = Invitation.objects.all()
    serializer_class = InvitationDetailSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'invitations'
    lookup_field = 'id'

    def get_queryset(self):
        user = self.request.user
        return self.queryset.filter(Q(created_by=user) | Q(sent_to=user))

class InvitationRespondAPIView(generics.UpdateAPIView):
    queryset = Invitation.objects.all()
    serializer_class = InvitationRespondSerializer
    permission_classes = [IsAuthenticated]
    lookup_field = 'id'

    def get_queryset(self):
        user = self.request.user
        return self.queryset.filter(sent_to=user)

class InvitationDeleteAPIView(generics.DestroyAPIView):
    queryset = Invitation.objects.all()
    serializer_class = InvitationDetailSerializer
    permission_classes = [IsAuthenticated]
    lookup_field = 'id'

    def get_queryset(self):
        user = self.request.user

```

```

        return self.queryset.filter(created_by=user)

class TeamMembersListAPIView(generics.ListAPIView):
    serializer_class = UserSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        team = get_object_or_404(Team, id=self.kwargs['team_id'],
                                   created_by=self.request.user)
        return team.members.all()

class TeamEventsListAPIView(generics.ListAPIView):
    queryset = Event.objects.all()
    serializer_class = EventDetailSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [DjangoFilterBackend, drf_filters.SearchFilter]
    filterset_class = EventFilter
    search_fields = ['title', 'description', 'location', 'tags__name']
    renderer_classes = [JSONRenderer]
    object_label = 'events'

    def get_queryset(self):
        team_id = self.kwargs['team_id']
        team = get_object_or_404(Team, id=team_id)

        permission = IsOwnerOrMember()
        if not permission.has_object_permission(self.request, self, team):
            raise PermissionDenied(detail=permission.message)

        return Event.objects.filter(team=team)

class TeamInvitationsListAPIView(generics.ListAPIView):
    queryset = Invitation.objects.all()
    serializer_class = InvitationDetailSerializer
    permission_classes = [IsAuthenticated]
    renderer_classes = [JSONRenderer]
    object_label = 'invitations'

    def get_queryset(self):
        team_id = self.kwargs['team_id']
        team = get_object_or_404(Team, id=team_id)

        permission = IsOwner()
        if not permission.has_object_permission(self.request, self, team):
            raise PermissionDenied(detail=permission.message)

        return Invitation.objects.filter(team=team)

```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2025 р.

**ВЕБЗАСТОСУНОК ДЛЯ ПЛАНУВАННЯ ТА НАГАДУВАННЯ ПРО
ВАЖЛИВІ ПОДІЇ**

Програма та методика тестування

КПІ.ІІ-1219.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Ярослав ОРИЩЕНКО

Київ – 2025

ЗМІСТ

1	ОБ’ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	5
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ	6

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є вебзастосунок для планування подій та нагадувань, для якого необхідно провести тестування користувацької та серверної частини додатку для забезпечення коректного функціонування в різних умовах, адекватного реагування на некоректні дані.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- перевірка збереження даних;
- перевірка сумісності вебзастосунка з останніми версіями сучасних браузерів (Chrome, Opera, Firefox);
- знаходження проблем, помилок і недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу;
- перевірка надійності та безпеки роботи програмного забезпечення.

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється програма разом з усією документацією, яка аналізується на предмет дотримання стандартів програмування;
- функціональне тестування – полягає у перевірці відповідності реальної поведінки програмного забезпечення очікуваній;
- мануальне тестування – тестування без використання автоматизації, тест-кейси пише особа, що тестує програмне забезпечення;
- тестування «білої скриньки» – об'єктом тестування тут є внутрішня поведінка програми. Перевіряється коректність побудови всіх елементів програми та правильність їхньої взаємодії один з одним.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Під час проведення тестування будуть використовуватись наступні допоміжні засоби:

- Postman – ручне тестування API;
- Mailpit – перехоплення та перегляд надісланих електронних листів;
- radon – бібліотека для обчислення метрик якості ПЗ;
- flake0 – бібліотека для аналізу якості коду;
- pytest – бібліотека для написання unit-тестів;
- pytest-django – бібліотека для інтеграції pytest з Django-застосунком;
- pytest-factoryboy – бібліотека для створення тестових одноразових об'єктів бази даних;
- pytest-cov – бібліотека для вимірювання покриття коду тестами;
- faker – бібліотека для генерації фіктивних даних;

Порядок проведення тестування буде наступним:

- статичне тестування на чистоту коду та його складність;
- тестування на відповідність функціональним вимогам;
- unit-тестування коректності роботи функціоналу застосунку;
- тестування роботи API з використанням Postman;
- тестування на коректність відображення веб-застосунку у різних веб-браузерах;
- тестування інтерфейсу користувача;
- тестування зручності використання.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2025 р.

**ВЕБЗАСТОСУНОК ДЛЯ ПЛАНУВАННЯ ТА НАГАДУВАННЯ ПРО
ВАЖЛИВІ ПОДІЇ**

Керівництво користувача

КПІ.ІІ-1219.045440.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

Максим ГОЛОВЧЕНКО

Виконавець:

Ярослав ОРИЩЕНКО

Київ – 2025

ЗМІСТ

1	ПРИЗНАЧЕННЯ ПРОГРАМИ	3
2	ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ.....	4
2.1	Системні вимоги для коректної роботи.....	4
2.2	Завантаження застосунку	4
2.3	Перевірка коректної роботи.....	6
3	ВИКОНАННЯ ПРОГРАМИ	7

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

«КПІ Notify» – це багатофункціональний, зручний та безпечний вебзастосунок, призначений для планування подій та нагадувань про них для працівників університету.

Метою розробки є спрощення управління подіями, нагадуваннями та взаємодії між користувачами за рахунок створення зручного вебзастосунку для планування та організації діяльності у межах університету.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних мінімальних системних вимог:

- Процесор: 1 гігагерц або швидший процесор чи SoC.
- Оперативна пам'ять: 1 гігабайт для 32-розрядної версії або 2 гігабайти для 64-розрядної версії.
- Обсяг дискового простору: 18 гігабайт для 32-розрядної ОС або 22 гігабайти для 64-розрядної ОС.
- Графічна плата: DirectX 9 або новішої версії з драйвером WDDM 1.0 або новіший.

Рекомендовані системні вимоги:

- Процесор: 1 гігагерц або швидший процесор чи SoC.
- Оперативна пам'ять: 4 гігабайти.
- Обсяг дискового простору: 18 гігабайт для 32-розрядної ОС або 34 гігабайти для 64-розрядної ОС.
- Графічна плата: DirectX 10 або новішої версії з драйвером WDDM 1.3 або новіший.

2.2 Завантаження застосунку

Для запуску застосунку необхідно виконати його локальне завантаження із публічного репозиторію на платформі GitHub. На локальному комп'ютері повинні бути встановлені мова програмування Python, система керування версіями Git, платформа Docker, яка забезпечує ізольоване середовище для розгортання та запуску застосунку, а також інструмент ngrok, який створює захищений тунель до локального сервера через публічну HTTPS адресу, що дає змогу тимчасово надати до нього доступ із мережі Інтернет. Створивши директорію, де зберігатиметься проект, необхідно зайти у дану директорію через термінал.

Установити файли застосунку можна із віддаленого репозиторію. Для цього потрібно використати команду **git clone https://github.com/butuz0/EventsReminder.git**.

Для забезпечення роботи Telegram-бота, його необхідно створити за допомогою офіційного інструменту BotFather. Після успішного створення бота, потрібно скопіювати його API Token, та вставити його у файл змінних середовища `.envs/env.django` для ключа `TELEGRAM_BOT_TOKEN`.

Далі необхідно перейти до каталогу проекту, та запустити застосунок із попередньою побудовою контейнерів за допомогою команди **docker-compose -f local.yml up --build -d**. У результаті виконання цієї команди в ізольованому середовищі будуть створені та налаштовані всі необхідні компоненти застосунку, вебзастосунок буде доступний за адресою **http://localhost:8080**. Для повторних запусків потрібно використовувати команду **docker-compose -f local.yml up -d**, яка активує вже налаштовані контейнери.

Наступним етапом є наповнення бази даних. Застосунок не зможе повноцінно функціонувати, якщо у його базі даних не буде інформації про інститути, факультети та кафедри університету. Наповнити базу даних можна за допомогою команди **docker-compose -f local.yml run --rm api python manage.py populate_units**.

Для випадкової генерації користувачів застосунку, можна використати команду **docker-compose -f local.yml run --rm api python manage.py generate_random_users**.

На даний момент застосунок функціонує, однак отримання нагадувань через Telegram-бота буде недоступним доти, доки користувач не пройде авторизацію через Telegram Login Widget. Це офіційний механізм прив'язки Telegram-акаунта до стороннього застосунку, який потребує наявності верифікованого HTTPS домену. Для забезпечення цього функціоналу на локальному сервері, можна скористатись інструментом ngrok.

Запустивши `ngrok.exe`, в терміналі потрібно ввести команду **ngrok http 8080**, у результаті чого буде отримана публічна HTTPS-адреса на кшталт

<https://abcd-123-45-67-890.ngrok-free.app> – за нею тепер буде доступний застосунок. Домен отриманої адреси необхідно вставити у налаштування Telegram-бота через BotFather, після чого авторизація через Telegram Login Widget стане можливою. Важливим є те, що ngrok кожен запуск генерує нову HTTPS-адресу, що робить необхідним повторне оновлення домена Telegram-бота.

2.3 Перевірка коректної роботи

Після виконання дій, описаних у пункті 2.2, застосунок є повністю готовим до роботи. Якщо все було встановлено та налаштовано коректно, то при переході за отриманим через ngrok посиланням на кшталт **<https://abcd-123-45-67-890.ngrok-free.app>** можна буде побачити початкову сторінку вебзастосунку, а Telegram Login Widget у сторінці профілю буде активним, та міститиме текст «Вхід через Telegram». Telegram-бот буде активним та зможе відповідати користувачу.

3 ВИКОНАННЯ ПРОГРАМИ

При запуску вебзастосунку, користувач потрапляє на головну сторінку, де є кнопки для переходу на сторінку реєстрації, або на сторінку входу у вже існуючий акаунт. (рисунок 3.1).

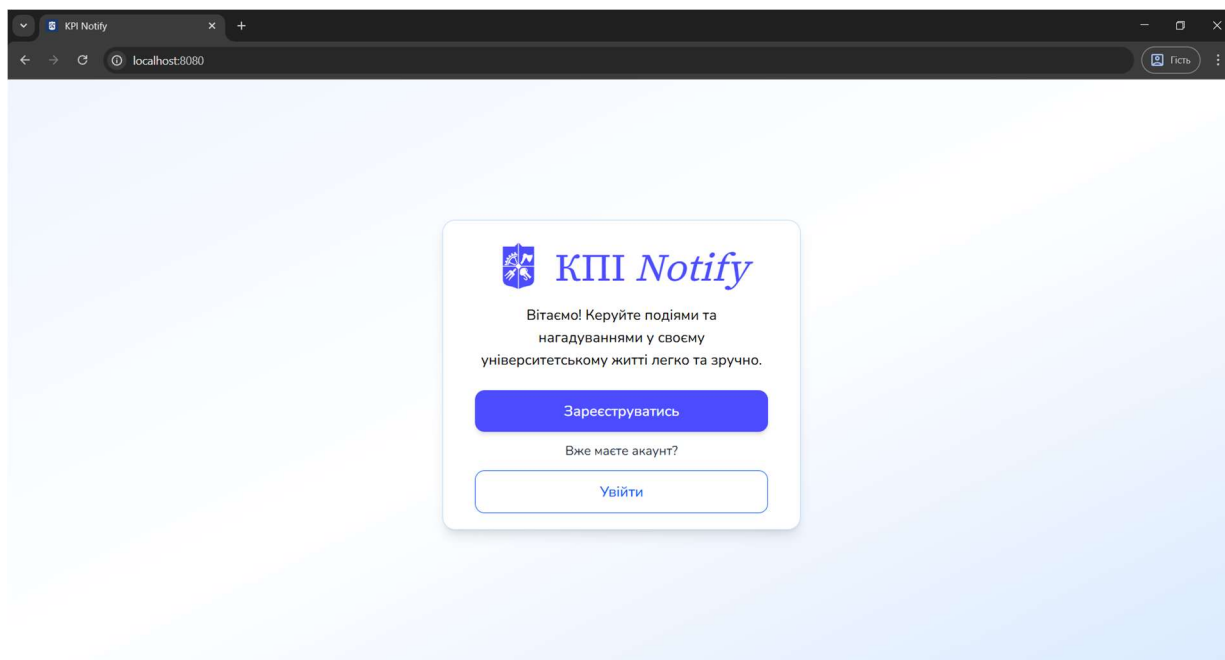


Рисунок 3.1 – Початкова сторінка додатку

Після натиснення на кнопку «Зареєструватись», користувач потрапляє на сторінку створення нового акаунту (рисунок 3.2). У відповідні поля необхідно ввести корпоративну електронну пошту із доментом @kpi.ua, ім'я, прізвище, пароль та повторення паролю. Якщо користувач помилково потрапив на сторінку реєстрації і вже має створений акаунт, то він може натиснути на кнопку «Увійти в акаунт», після чого потрапить на сторінку входу в існуючий акаунт.

Рисунок 3.2 – Сторінка реєстрації

Після коректного заповнення усіх полів форми реєстрації та натиснення на кнопку «Рєєстрація», на вказану електронну пошту користувач отримає лист із підтвердженням реєстрації (рисунок 3.3). У листі надано посилання, при переході по якому обліковий запис користувача буде активовано.

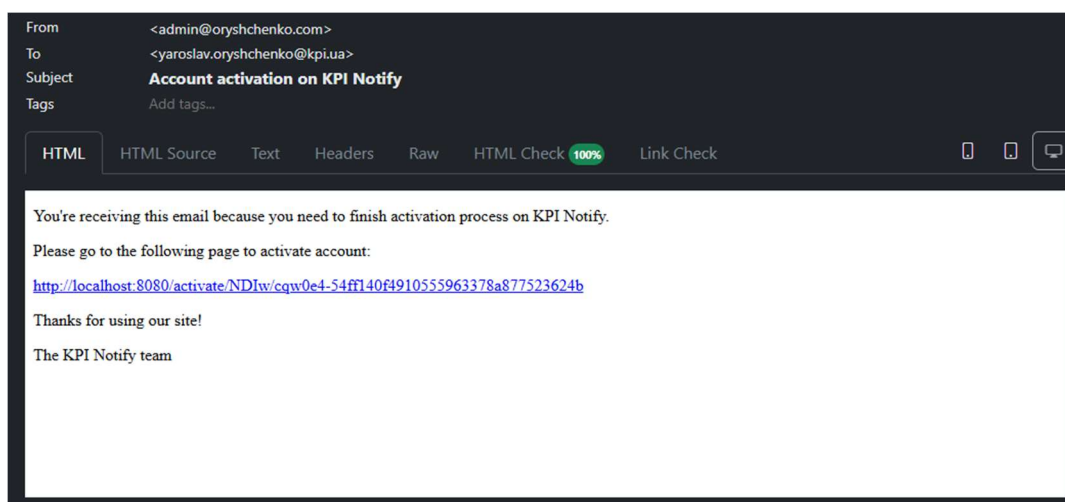


Рисунок 3.3 – Електронний лист із підтвердженням реєстрації

Перейшовши за отриманим посиланням вперше, користувач потрапить на сторінку, де отримає повідомлення про успішну активацію акаунту (рисунок 3.4). Після цього, користувача буде переправлено на сторінку входу в акаунт.

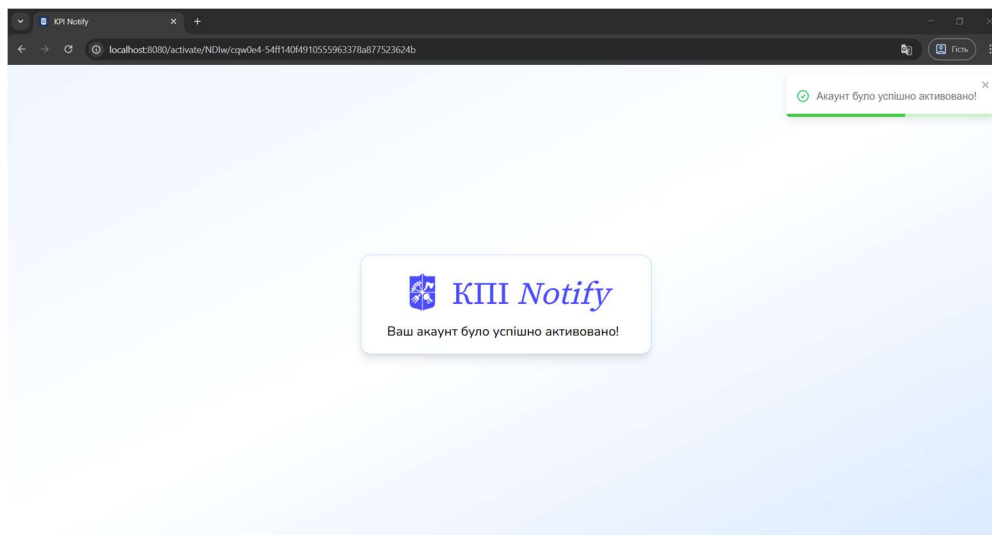


Рисунок 3.4 – Сторінка успішної активації акаунту

На сторінці входу користувачу необхідно ввести електронну пошту та пароль від зареєстрованого акаунту (рисунок 3.5). Якщо користувач ще не створив акаунт, він може натиснути на кнопку «Реєстрація», після чого буде переправлений на сторінку із формою для реєстрації. Користувач може відновити забутий пароль від акаунту, натиснувши на кнопку «Забули пароль?».

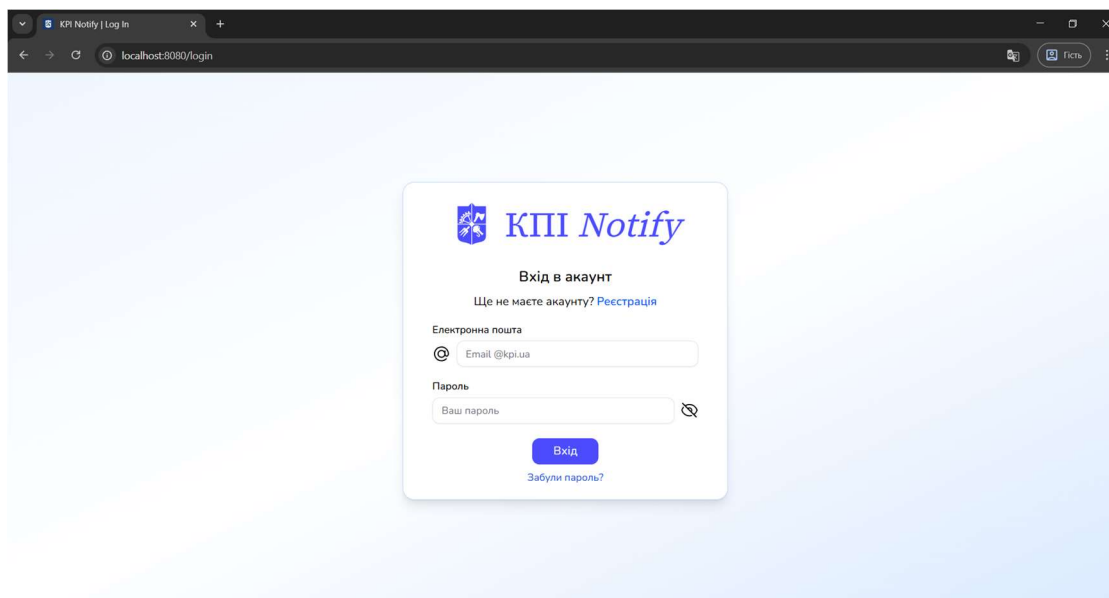
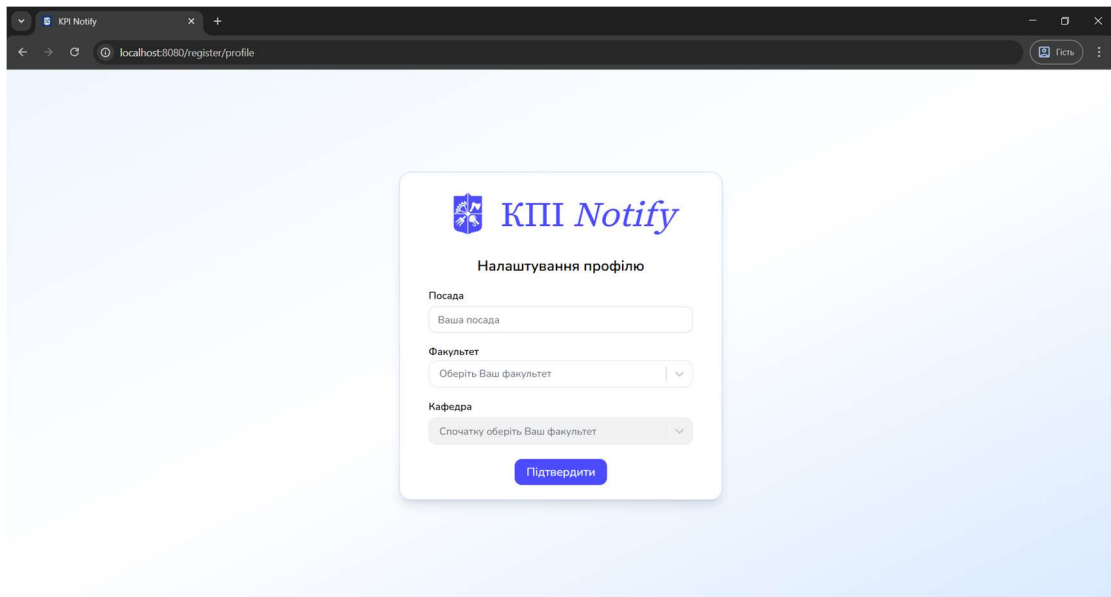


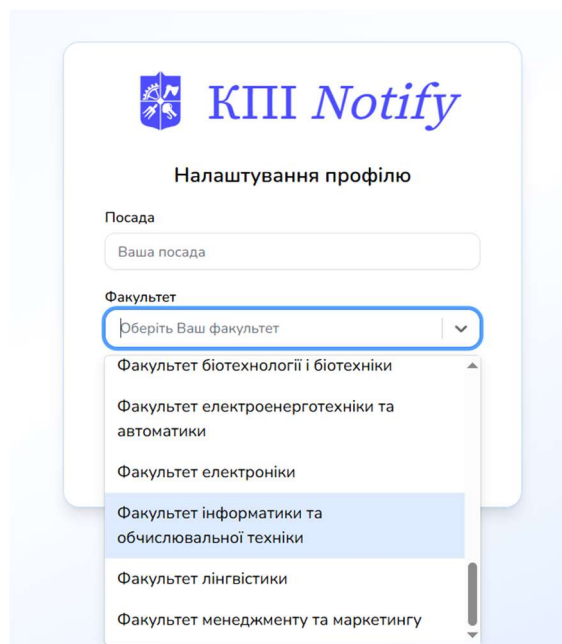
Рисунок 3.5 – Сторінка входу в акаунт

Після введення коректних даних для входу в акаунт та натиснення кнопки «Вхід», користувач потрапить на домашню сторінку застосунку, якщо

він вже заповнив дані свого акаунту. Інакше, якщо користувач входить в акаунт вперше, він потрапить на сторінку створення профілю (рисунки 3.6). Для користування застосунком, необхідно ввести дані про посаду в університеті, із випадаючого списку обрати факультет (рисунки 3.7). Після цього розблокується поле для вибору кафедри із обраного факультету (рисунки 3.8).



Рисунки 3.6 – Сторінка налаштування профілю



Рисунки 3.7 – Вибір факультету на сторінці налаштування акаунту

Рисунок 3.8 – Вибір кафедри на сторінці налаштування акаунту

Якщо користувач натиснув на кнопку «Забули пароль?», то він потрапить на сторінку для створення запиту на зміну паролю (рисунок 3.9). Для створення запиту необхідно ввести електронну пошту, яка є зареєстрованою в застосунку, та натиснути на кнопку «Підтвердити». Користувач може покинути сторінку, натиснувши на кнопку «До сторінки входу».

Рисунок 3.9 – Сторінка створення запиту на зміну паролю

Ввівши коректну електронну пошту та натиснувши на кнопку «Підтвердити» на сторінці створення запиту на зміну паролю, на вказану

електронну пошту надійде лист, що містить посилання для зміни паролю (рисунок 3.10).

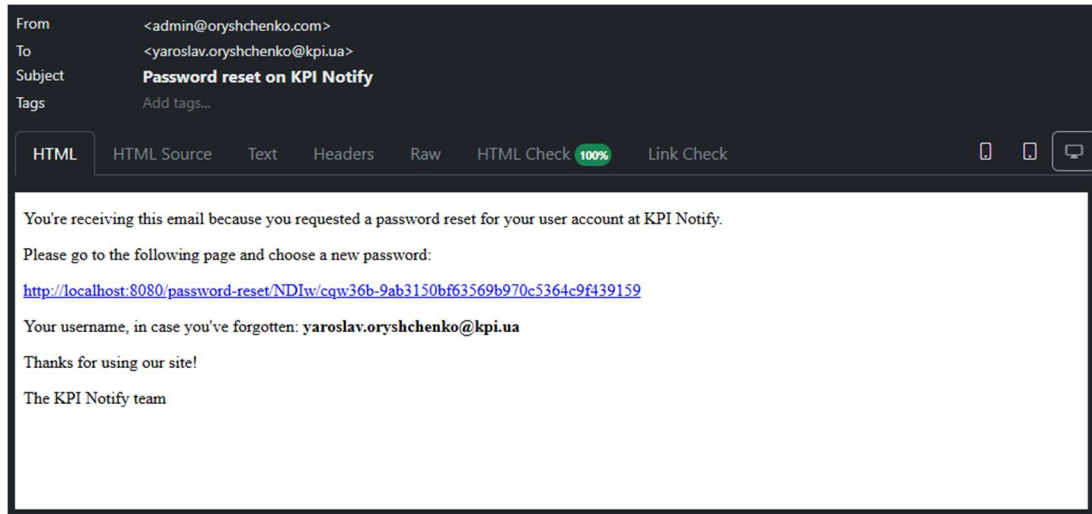


Рисунок 3.10 – Електронний лист із посиланням для зміни паролю

Перейшовши за отриманим посиланням, користувач потрапить на сторінку оновлення паролю, яка містить поля для вводу нового паролю та підтвердження паролю (рисунок 3.11). Після введення коректних даних та натиснення кнопки «Оновити пароль», користувач буде переправлений на сторінку входу в акаунт.

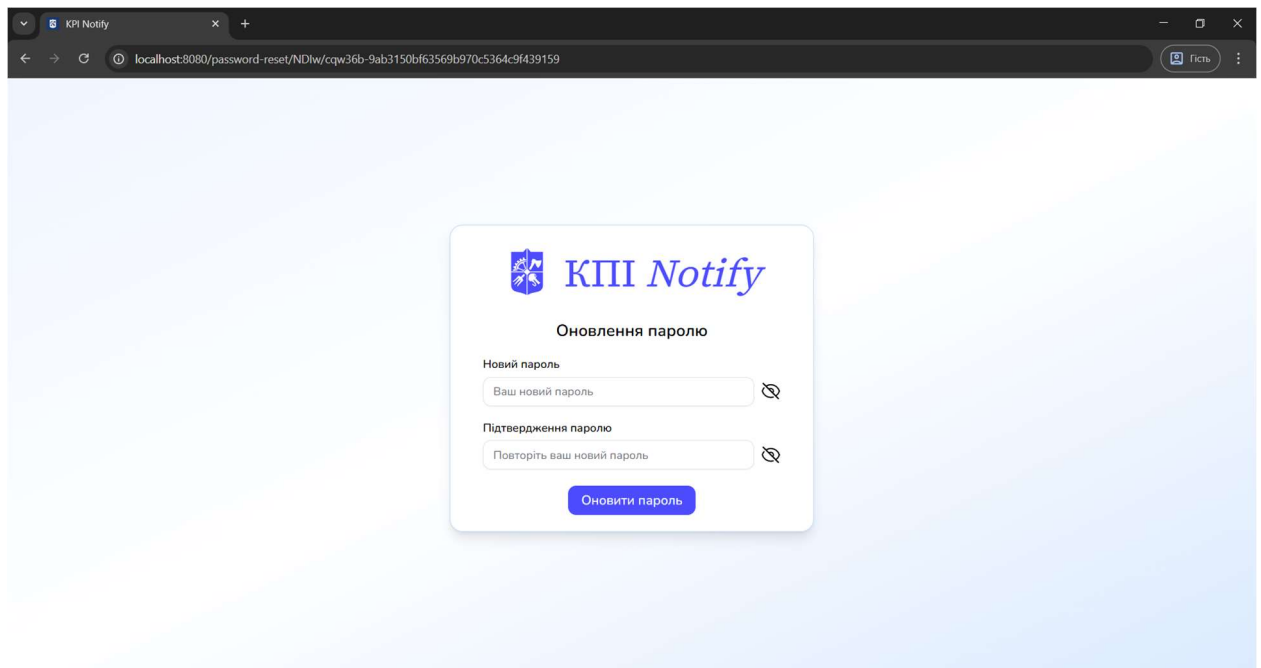


Рисунок 3.11 – Сторінка оновлення паролю

Після входу в акаунт, користувач опиниться на домашній сторінці застосунку (рисунок 3.12). Зліва знаходиться навігаційне меню, у якому виділено сторінку, на якій користувач знаходиться у даний момент. Справа від навігаційного меню міститься основна інформація сторінки. На головній сторінці користувач може бачити блоки із найближчими подіями та нагадуваннями, а також останніми призначеними йому подіями. Натиснувши на «Переглянути всі», користувачеві буде завантажена сторінка із усіма наявними подіями.

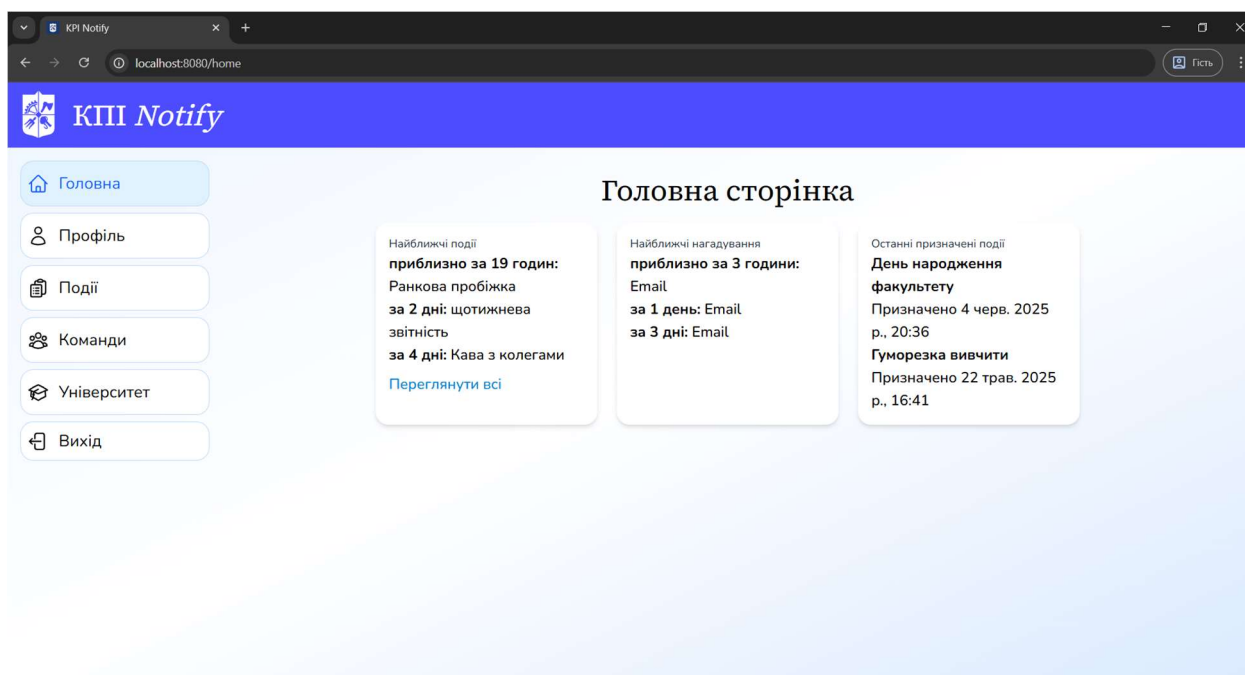


Рисунок 3.12 – Головна сторінка застосунку

Натиснувши на кнопку «Профіль» у навігаційному меню, завантажиться сторінка із усією інформацією про користувача: аватар, ім'я та прізвище, посада, інститут/факультет та кафедра, електронна пошта та інформація про Telegram-акаунт користувача (рисунок 3.13). Іншою вкладкою цієї сторінки є «Картки АЦСК», де можна переглянути приватну інформацію про збережені користувачем реєстраційні картки юридичної особи для отримання кваліфікованого електронного підпису для юридичної особи.

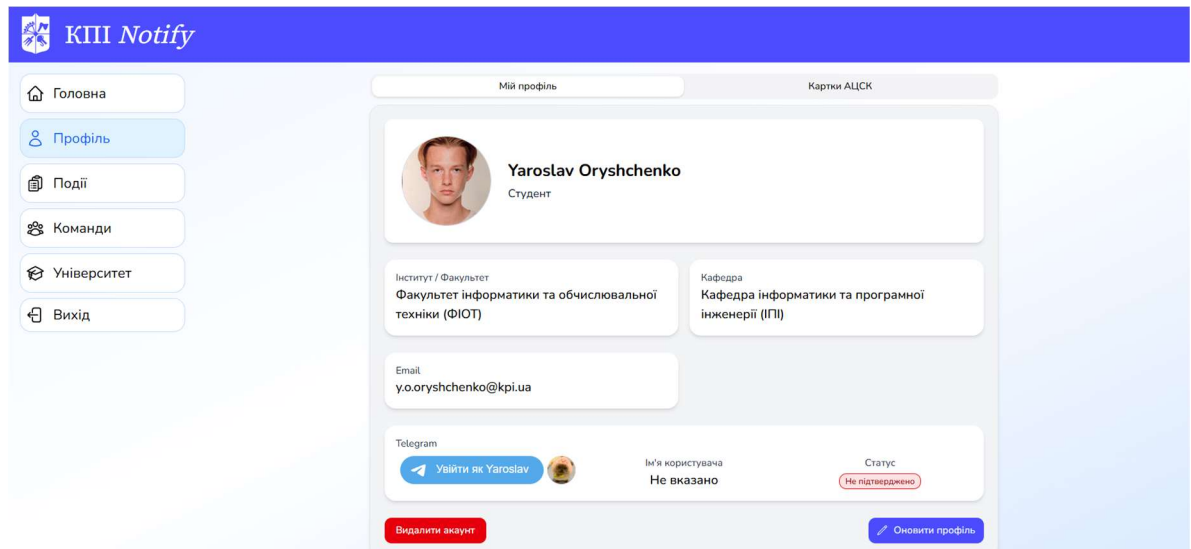


Рисунок 3.13 – Профіль користувача

Після натиснення на кнопку «Оновити профіль», завантажиться форма для оновлення інформації про користувача (рисунок 3.14).

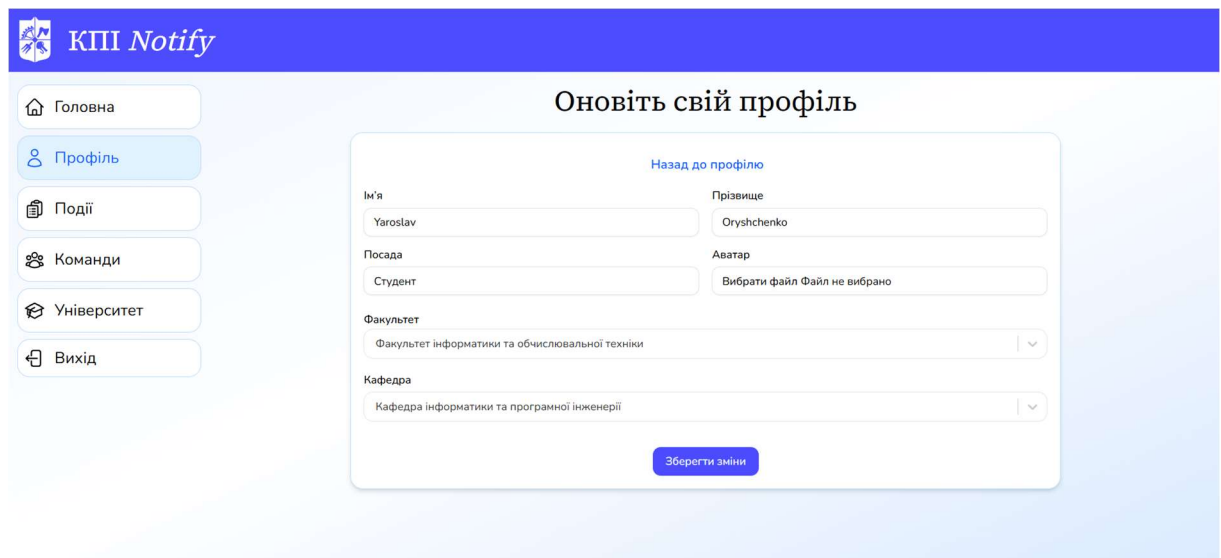


Рисунок 3.14 – Сторінка оновлення інформації профілю

Натиснувши на кнопку «Видалити акаунт», завантажиться вікно, яке потребує підтвердження видалення акаунту, шляхом введення паролю користувача (рисунок 3.15). Скасувати видалення можна натиснувши кнопку «Скасувати», натиснувши на хрестик у верхньому правому кутку вікна, або натиснувши на затемнену зону за межами відкритого вікна. Після введення коректного паролю та натиснення кнопки «Видалити акаунт», обліковий запис

користувача та усі створені ним дані будуть видалені із застосунку, та буде завантажена головна сторінка застосунку.

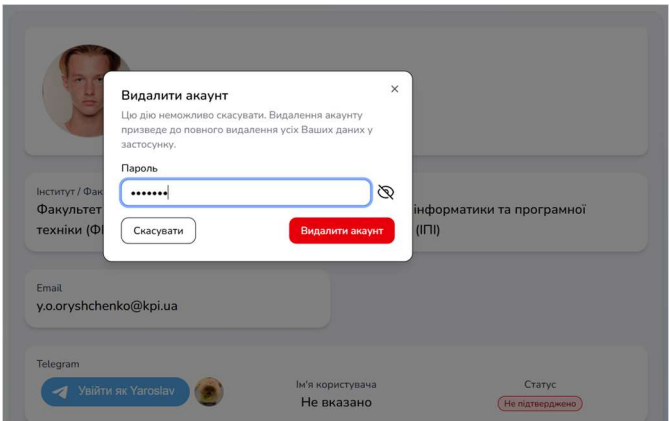


Рисунок 3.15 – Вікно підтвердження видалення акаунту

Натиснувши на вкладку «Картки АЦСК», користувач побачить таблицю із збереженими картками та їх основною інформацією (рисунок 3.16).

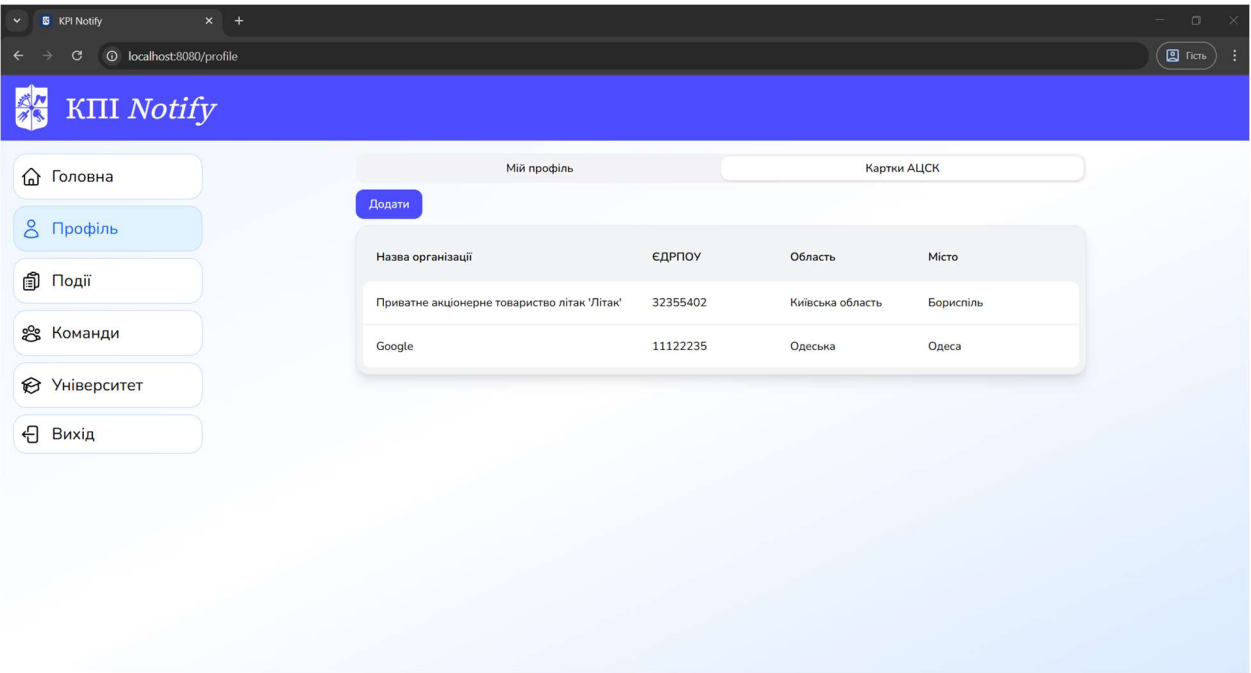


Рисунок 3.16 – Таблиця із збереженими картками АЦСК

Натиснувши на бажану картку, користувач зможе переглянути усю інформацію, яку він зберігав для цієї юридичної особи (рисунки 3.17-3.18).

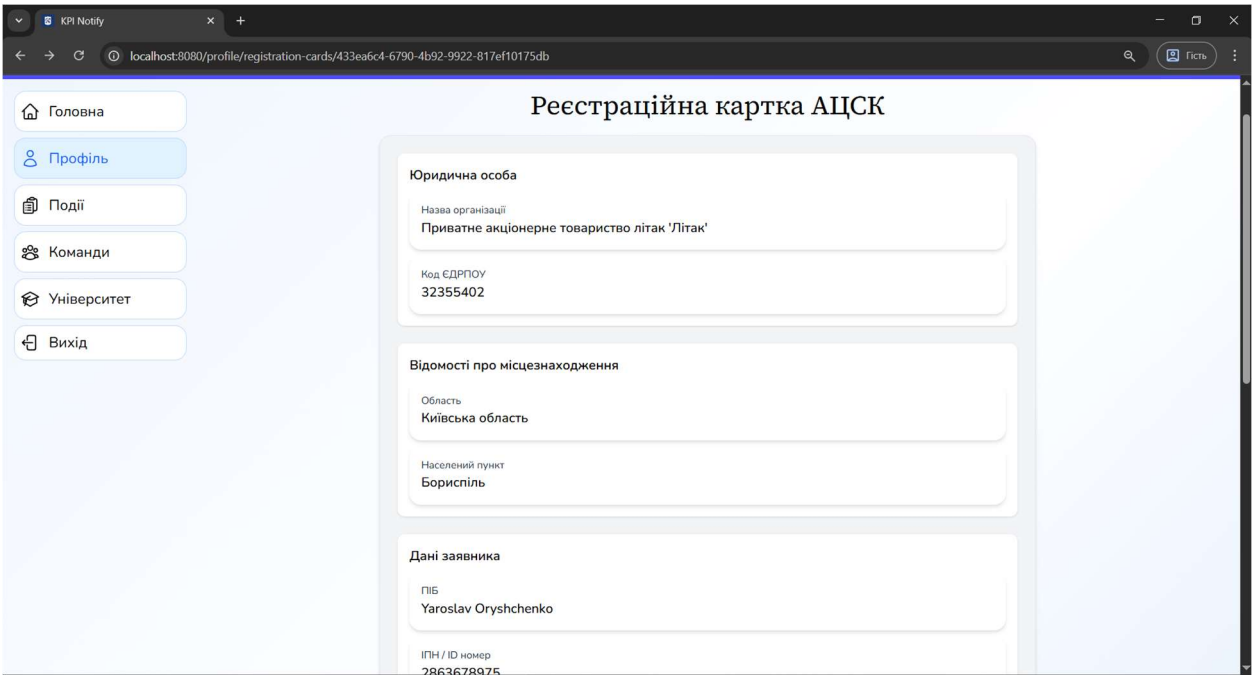


Рисунок 3.17 – Сторінка із повною інформацією реєстраційної картки АЦСК

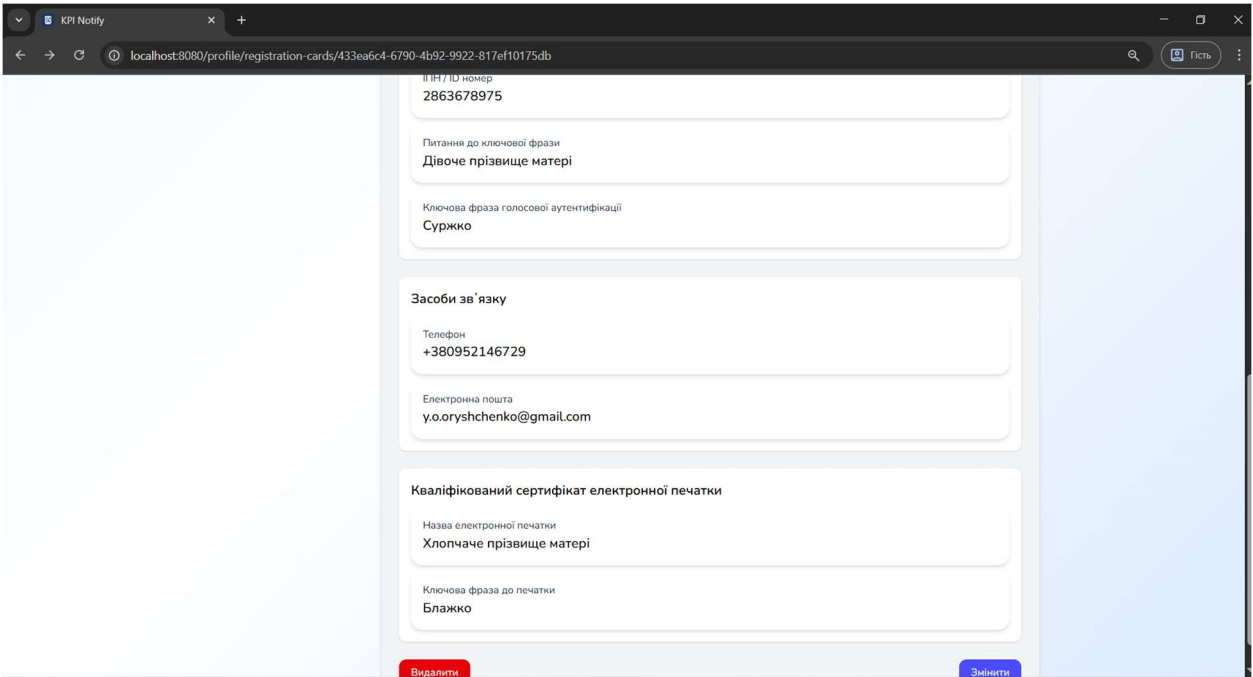


Рисунок 3.18 – Сторінка із повною інформацією реєстраційної картки АЦСК

Натиснувши на кнопку «Змінити», користувач може модифікувати дані існуючої картки у відповідній формі. Натиснувши кнопку «Видалити», користувач зможе видалити створену картку.

Натиснувши на кнопку «Додати» на сторінці із списком усіх карток, що зображена на рисунку 3.16, користувача буде перенаправлено на сторінку із формою для створення нової реєстраційної картки (рисунки 3.19-3.20).

The screenshot shows a web browser window with the URL `localhost:8080/profile/registration-cards/create`. The page has a light blue sidebar on the left with navigation buttons: 'Головна' (Home), 'Профіль' (Profile), 'Події' (Events), 'Команди' (Teams), 'Університет' (University), and 'Вихід' (Logout). The main content area is titled 'Додайте реєстраційну картку АЦСК' (Add registration card ACSC). It contains a vertical stack of input fields for the following information: 'Назва організації' (Organization name), 'Код ЄДРПОУ' (EDRPOU code), 'Область' (Region), 'Місто' (City), 'ПІБ' (Full name), 'Ідентифікаційний номер' (Identification number), 'Email', 'Номер телефону' (Phone number), and 'Ключова фраза' (Key phrase).

Рисунок 3.19 – Сторінка створення реєстраційної картки

This screenshot shows the continuation of the registration card form. The sidebar on the left now shows only the 'Вихід' (Logout) button. The main content area continues the list of input fields: 'ПІБ' (Full name), 'Ідентифікаційний номер' (Identification number), 'Email', 'Номер телефону' (Phone number), 'Ключова фраза' (Key phrase), 'Голосова фраза' (Voice phrase), 'Назва Е-печатки' (E-stamp name), and 'Ключова фраза до печатки' (Key phrase for stamp). At the bottom right of the form, there is a blue button labeled 'Створити' (Create).

Рисунок 3.20 – Сторінка створення реєстраційної картки

Після коректного заповнення бажаних полей та натиснення кнопки «Створити», користувача буде перенаправлено на сторінку із повною інформацією про щойно створену картку.

Натиснувши на кнопку «Події» у навігаційному меню, користувач побачить сторінку із усіма подіями, які були створені ним або призначені йому (рисунок 3.21). Сторінка складається із трьох основних розділів: «Майбутні» - усі події, створені користувачем, які ще не настали; «Призначені» - усі події, призначені користувачу, які ще не настали (рисунок 3.22); «Минулі» - усі події, створені користувачем або призначені йому, які вже минули (рисунок 3.23). За замовчуванням, таблиця подій відсортована від найближчих подій до найпізніших.

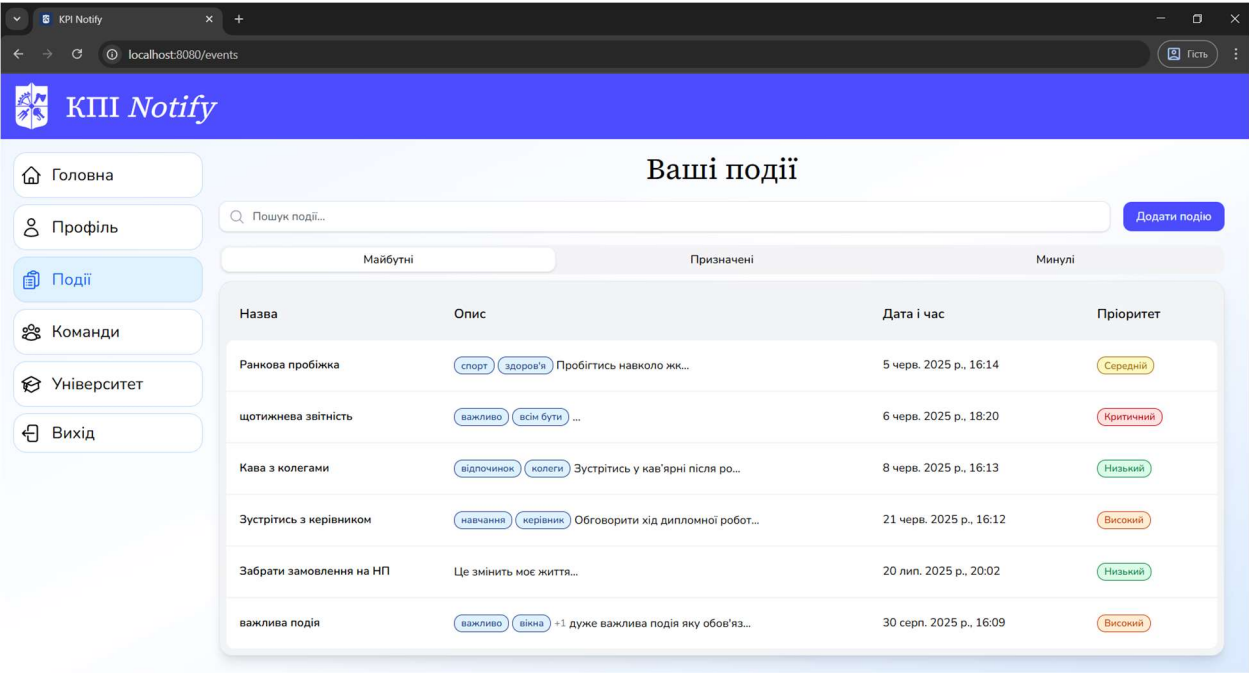


Рисунок 3.21 – Сторінка із подіями користувача

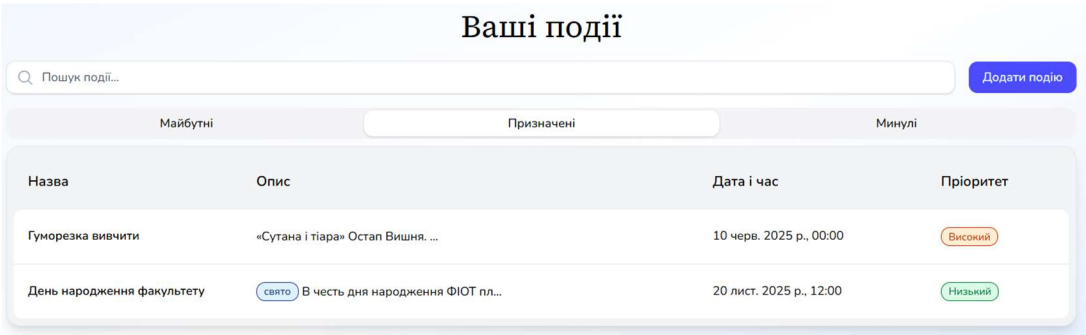


Рисунок 3.22 – Таблиця із призначеними подіями

Ваші події

Пошук події...

Додати подію

Майбутні

Призначені

Минулі

Назва	Опис	Дата і час	Пріоритет
День народження	свято Мій день народження. Відволікт...	29 квіт. 2025 р., 00:00	Низький
Звітність з практики	документи програма ...	16 трав. 2025 р., 23:59	Критичний

Рисунок 3.23 – Таблиця із минулими подіями

Швидко шукати потрібні події можна за допомогою поля пошуку, що знаходиться над таблицею подій (рисунок 3.24). Пошук доступний за текстом у полях назви, опису та тегів події. Також події у таблиці можна сортувати за назвою (рисунок 3.25), за датою (рисунок 3.26), та за пріоритетом (рисунок 3.27). Сортування доступне як у порядку збільшення, так і у порядку зменшення.

Параметри пошуку та сортування зберігаються в посиланні сторінки, що дозволяє зберегти поточний стан таблиці без необхідності повторного пошуку та сортування шляхом збереження поточного посилання.

КПІ Notify

Ваші події

важли

Додати подію

Майбутні

Призначені

Минулі

Назва	Опис	Дата і час	Пріоритет
щотижнева звітність	важливо всім бути ...	6 черв. 2025 р., 18:20	Критичний
важлива подія	важливо вікна +1 дуже важлива подія яку обов'яз...	30 серп. 2025 р., 16:09	Високий

Рисунок 3.24 – Пошук подій

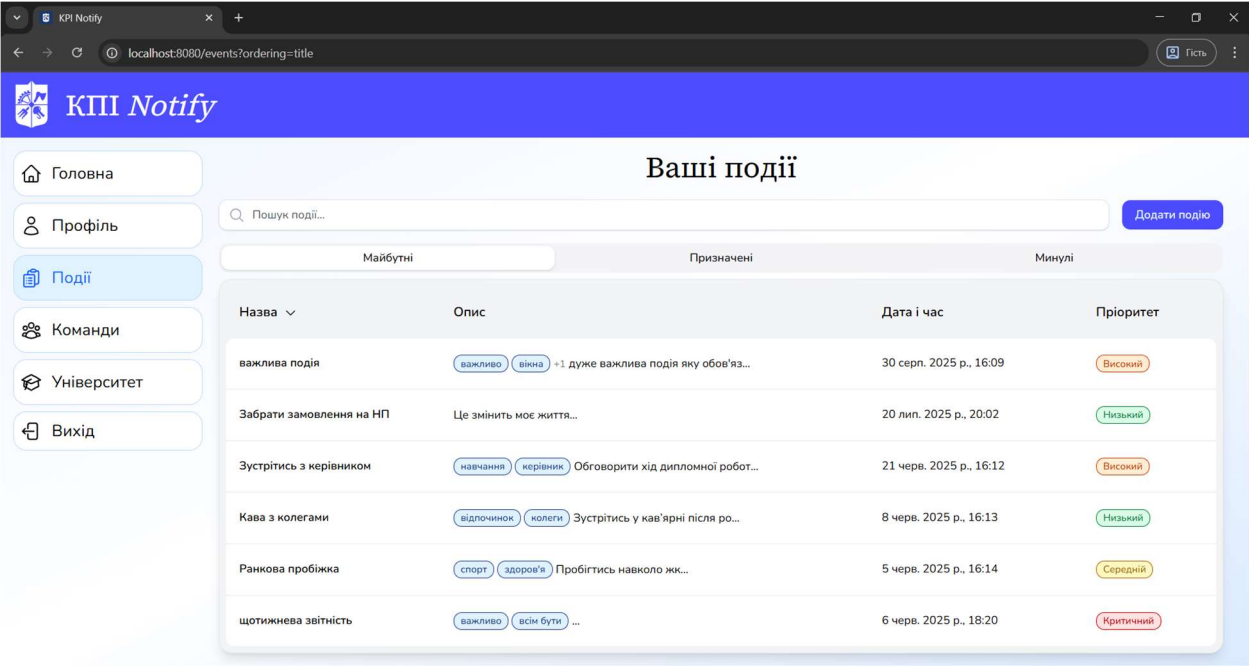


Рисунок 3.25 – Сортуння подій за назвою

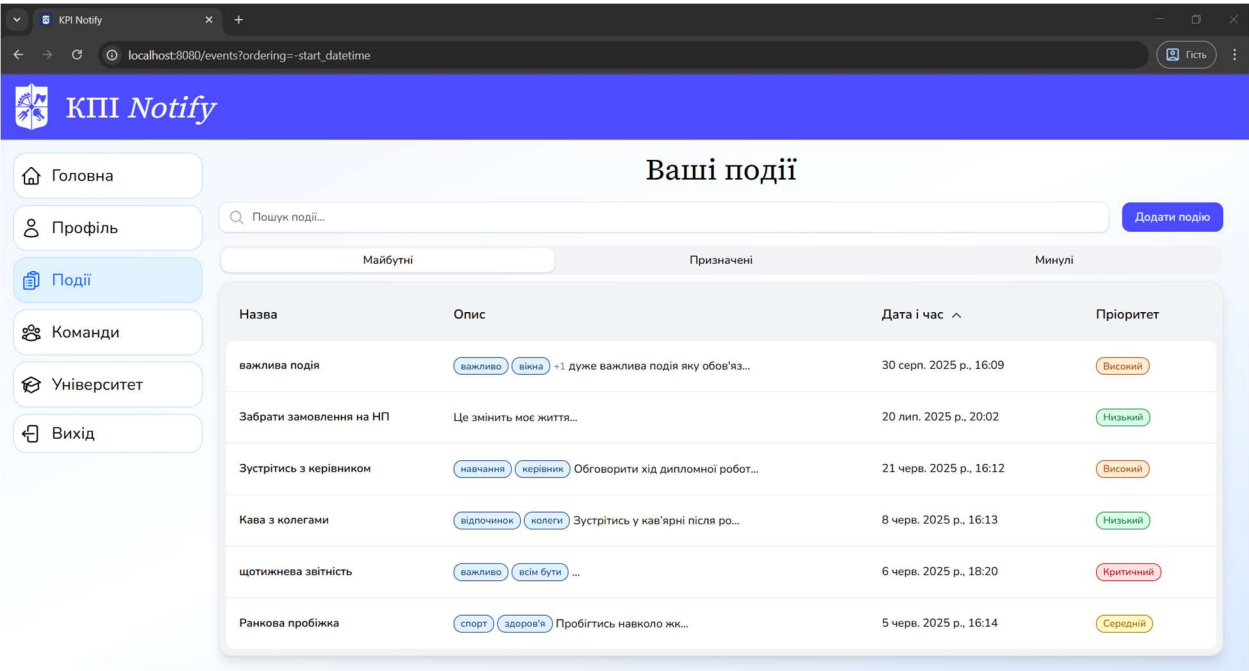


Рисунок 3.26 – Сортуння подій за датою

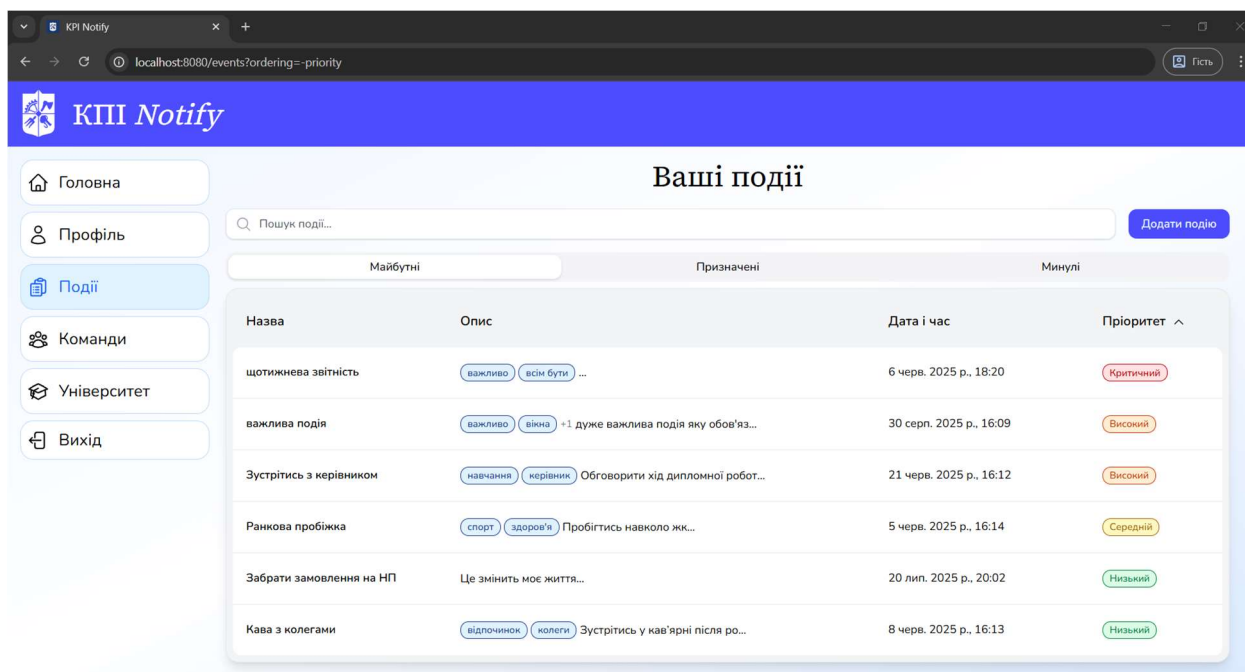


Рисунок 3.27 – Сортуння подій за пріоритетом

Після натиснення на кнопку «Додати подію», буде завантажена сторінка із формою для створення нової події (рисунок 3.28). При створенні події необхідно заповнити основні поля – назва події, дата та час, а також пріоритет, який обирається користувачем із чотирьох можливих опцій: «Низький», «Середній», «Високий» та «Критичний». Можна заповнити додаткову інформацію про подію, таку як опис, місце, посилання, теги, зображення.

Якщо подія може повторюватись, то у формі необхідно обрати пункт «Повторювана подія», після чого обрати періодичність повторення: щодня, щотижня, щомісяця або щороку (рисунок 3.29). Додатково можна обрати дату кінця повторення події.

Також можна одразу запланувати нагадування для події, обравши дату та час нагадування, а також метод, яким воно буде виконано: через електронну пошту, або через Telegram-бота (рисунок 3.30).

The screenshot shows the 'KPI Notify' web application interface. On the left is a sidebar with navigation links: 'Головна' (Home), 'Профіль' (Profile), 'Події' (Events), 'Команди' (Teams), 'Університет' (University), and 'Вихід' (Logout). The main area displays the 'Створіть нову подію' (Create new event) form. The form fields are: 'Назва події' (Event name) with a copy icon, 'Опис' (Description) with a text area, 'Дата та час' (Date and time) with a date/time picker, 'Пріоритет' (Priority) with a dropdown menu set to 'Середній' (Medium), 'Теги' (Tags) with a text input, 'Місце' (Location) with a location picker, 'Посилання' (Link) with a link input, and 'Зображення' (Image) with a file upload button labeled 'Вибрати файл' (Select file).

Рисунок 3.28 – Створення нової події

This screenshot shows the 'Repeat event' configuration section. It features a checked checkbox labeled 'Повторювана подія' (Repeatable event). Below it is the 'Періодичність' (Periodicity) dropdown menu, currently set to 'Щомісяця' (Monthly). At the bottom is the 'Завершення повторень' (End of repeats) date/time picker, showing '13.01.2027 00:13'.

Рисунок 3.29 – Створення повторюваної події

This screenshot shows the 'Нагадування' (Reminders) section. It contains a table with two rows of reminders. The first row has the date '06.06.2025 17:15' and the method 'Email', with a red 'Видалити' (Delete) button. The second row has the date '10.06.2025 00:20' and the method 'Telegram', also with a red 'Видалити' (Delete) button. Below the table is a 'Додати нагадування' (Add reminder) button and a large blue 'Підтвердити' (Confirm) button.

Рисунок 3.30 – Створення нагадувань для події

Коректно заповнивши обов'язкові та додаткові поля про подію та натиснувши кнопку «Підтвердити», користувач потрапить на сторінку із повною інформацією про створену подію (рисунок 2.31).

У нижній частині сторінки можна додати нагадування до події. Вже створені нагадування можна видалити, натиснувши кнопку «Видалити».

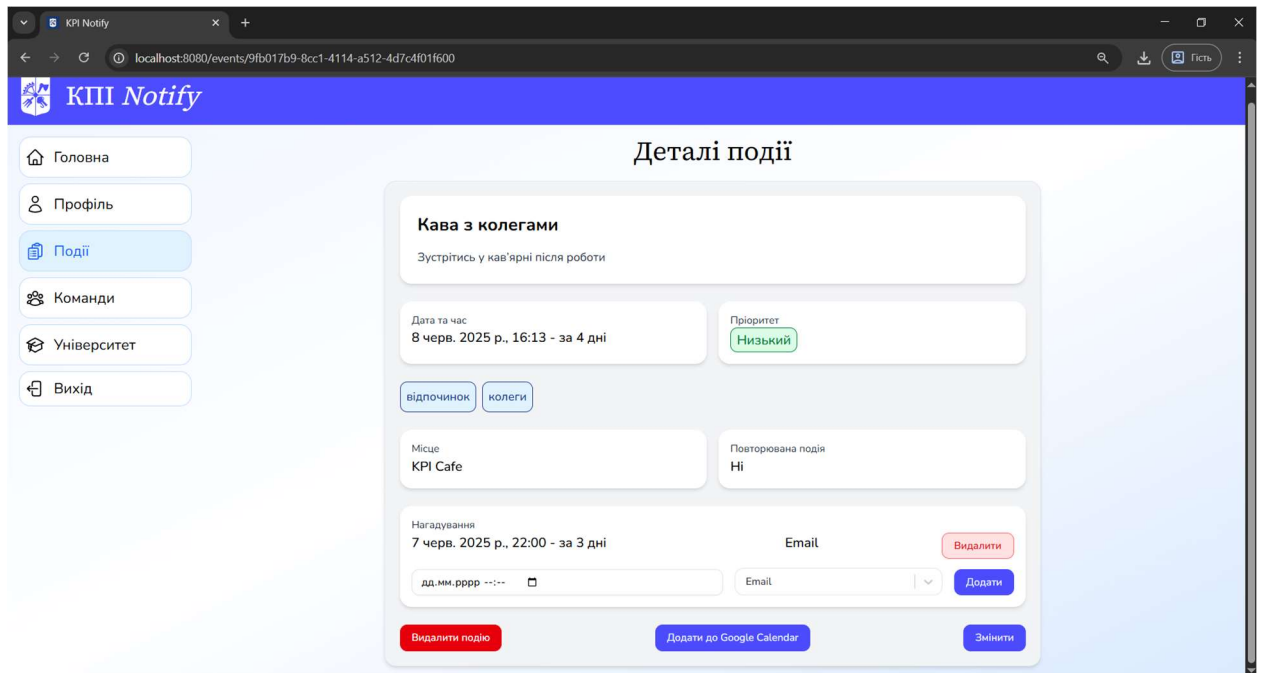


Рисунок 3.31– Сторінка із інформацією про подію

Натиснувши на кнопку «Додати до Google Calendar», користувач зможе перенести створену подію у персональний Google Calendar (рисунок 3.32).

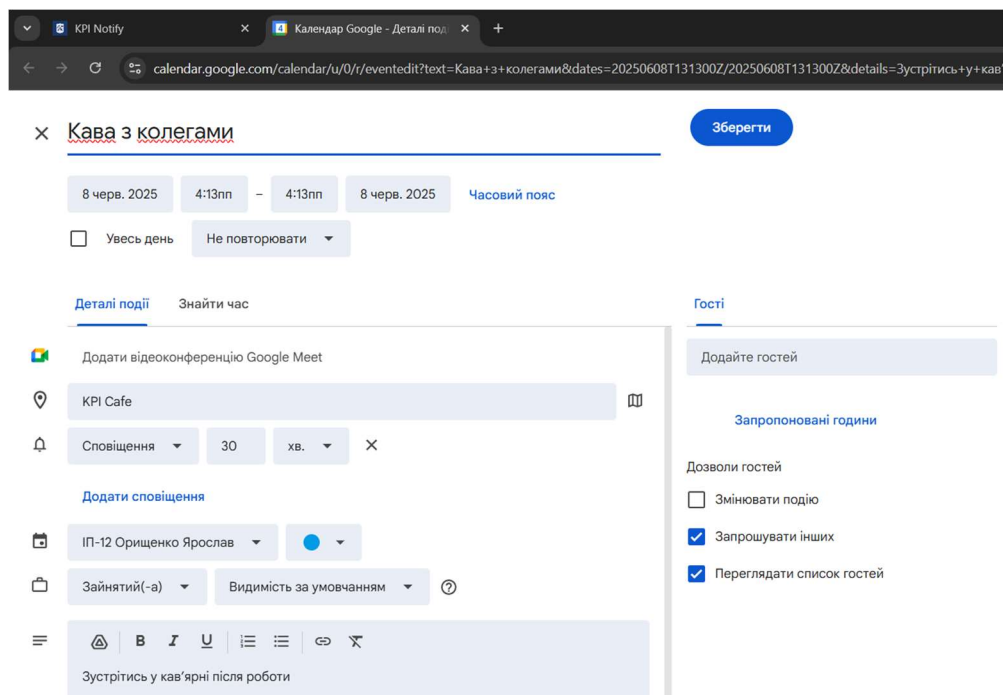
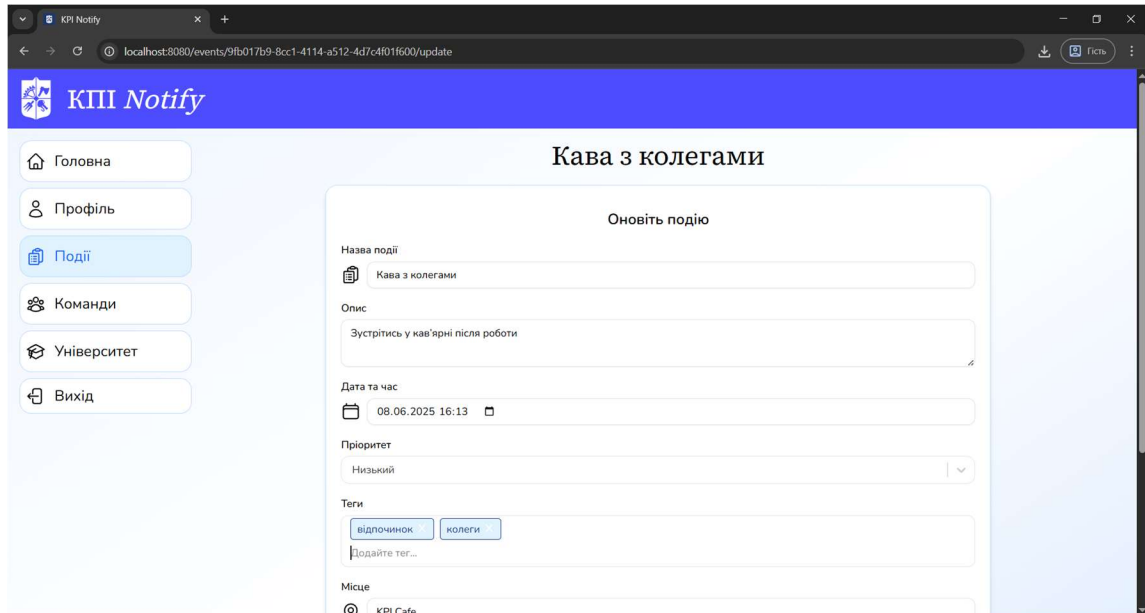


Рисунок 3.32 – Додавання створеної події до Google Calendar

Після натиснення на кнопку «Змінити», буде завантажена форма із інформацією про подію, де користувач може змінити будь-яке бажане поле (рисунк 3.33). Після оновлення бажаних полей та натиснення кнопки «Зберегти зміни», користувач повернеться на сторінку із оновленою повною інформацією про подію.



The screenshot shows a web browser window with the URL `localhost:8080/events/9fb017b9-8cc1-4114-a512-4d7c4f01f600/update`. The application header is blue with the KPI Notify logo. A left sidebar contains navigation links: Головна, Профіль, Події (highlighted), Команди, Університет, and Вихід. The main content area is titled 'Кава з колегами' and 'Оновіть подію'. The form includes fields for:

- Назва події: Кава з колегами
- Опис: Зустрітись у кав'ярні після роботи
- Дата та час: 08.06.2025 16:13
- Пріоритет: Низький
- Теги: відпочинок, колеги
- Місце: KPI Cafe

Рисунок 3.33 – Сторінка оновлення інформації про подію

Запланувавши нагадування через електронну, у обраний час користувач отримає електронний лист із нагадуванням (рисунк 3.34).

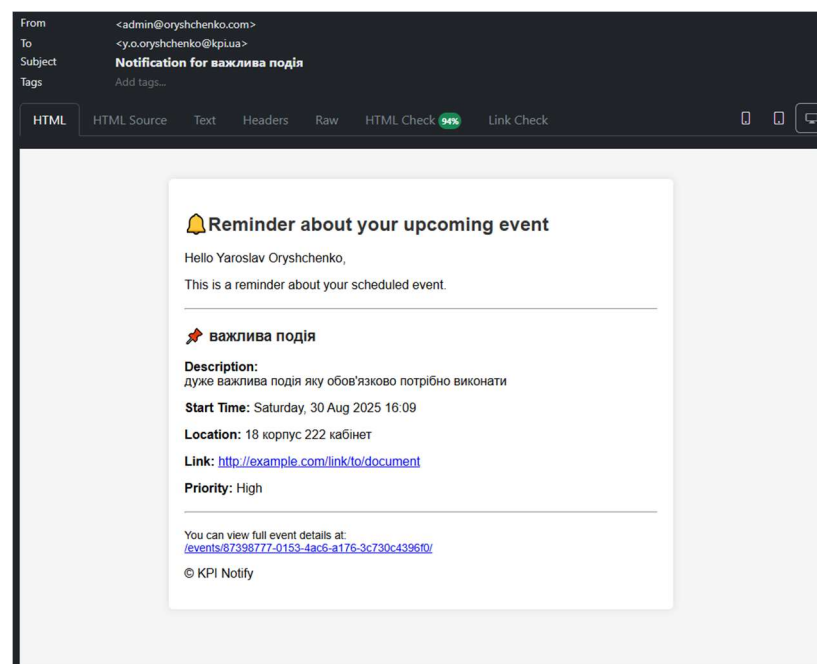


Рисунок 3.34 – Отримане на електронну пошту нагадування

Після натиснення на кнопку «Команди» у навігаційному меню, користувачеві буде завантажена сторінка із списком усіх команд, членом або лідером яких він є (рисунок 3.35). Доступний пошук команд за назвою команди, описом чи ім’ям лідера команди.

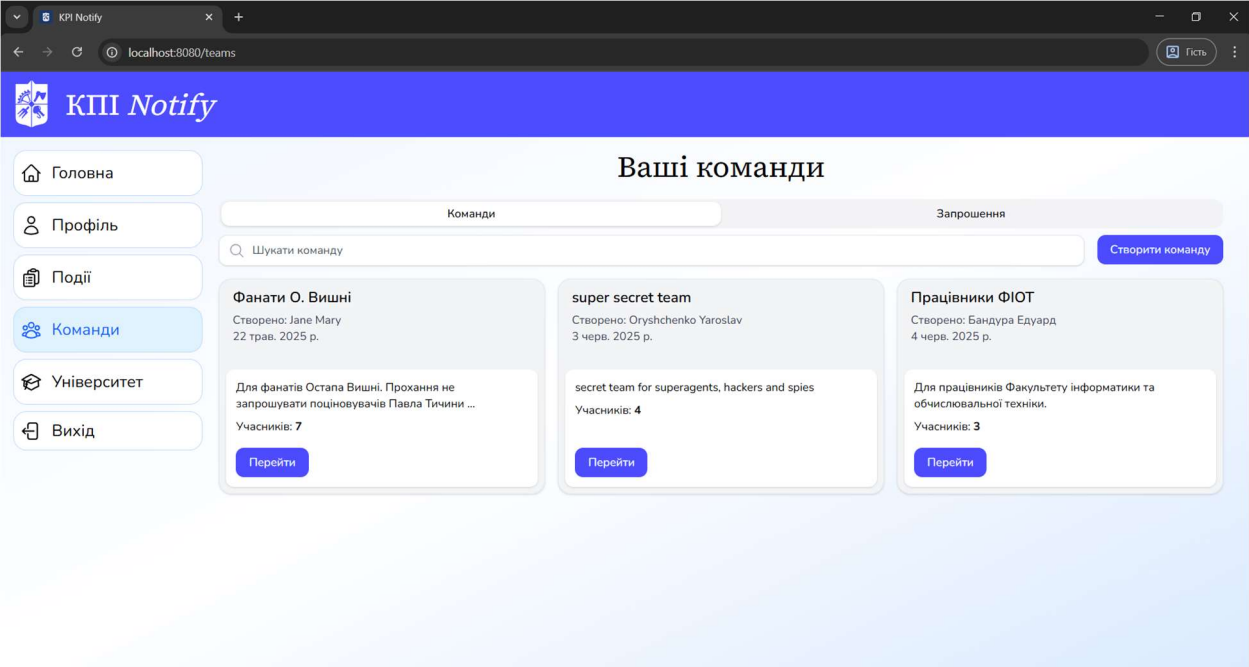


Рисунок 3.35 – Сторінка із командами користувача

У вкладці «Запрошення» можна переглянути отримані запрошення на вступ у інші команди (рисунок 3.36).

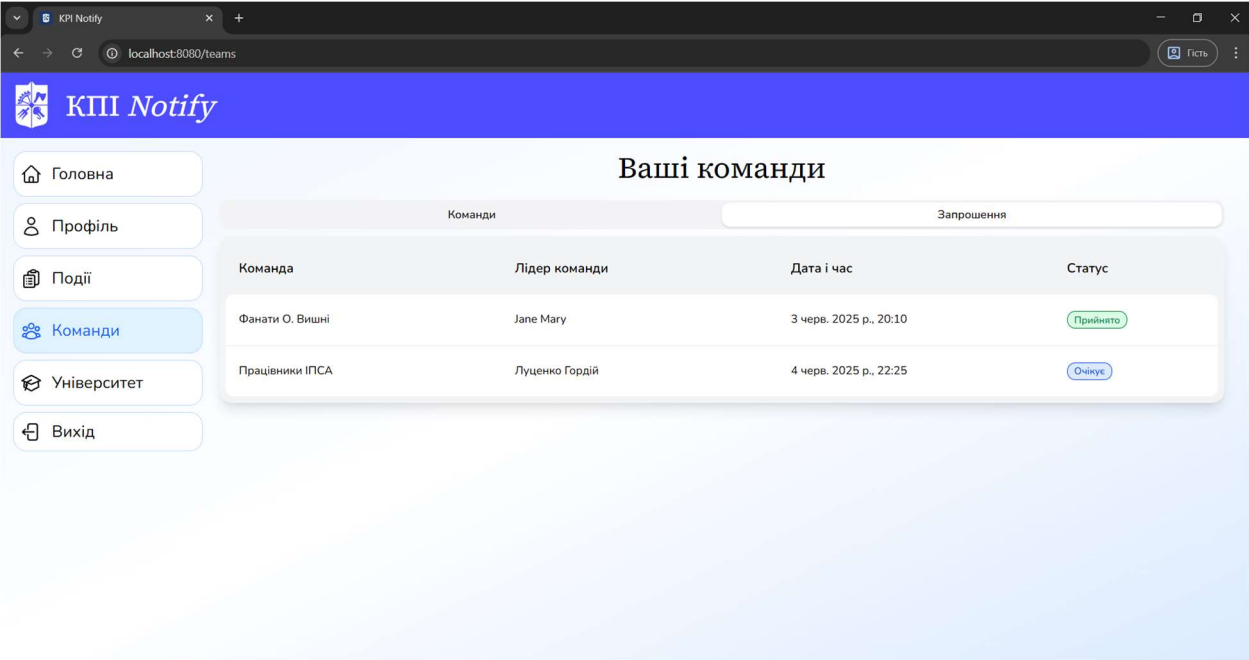


Рисунок 3.36 – Список отриманих запрошень у команди

Натиснувши на запрошення, користувач потрапить на сторінку із інформацією про команду, відправника запрошення, та можливістю прийняти або відхилити запрошення (рисунок 3.37). Натиснувши кнопку «Відхилити», статус запрошення зміниться на «Відхилено», а користувача буде направлено на сторінку із списком команд. Натиснувши на кнопку «Прийняти», статус запрошення буде змінено на «Прийнято», а користувача буде направлено на сторінку із інформацією про команду, до якої він доєднався.

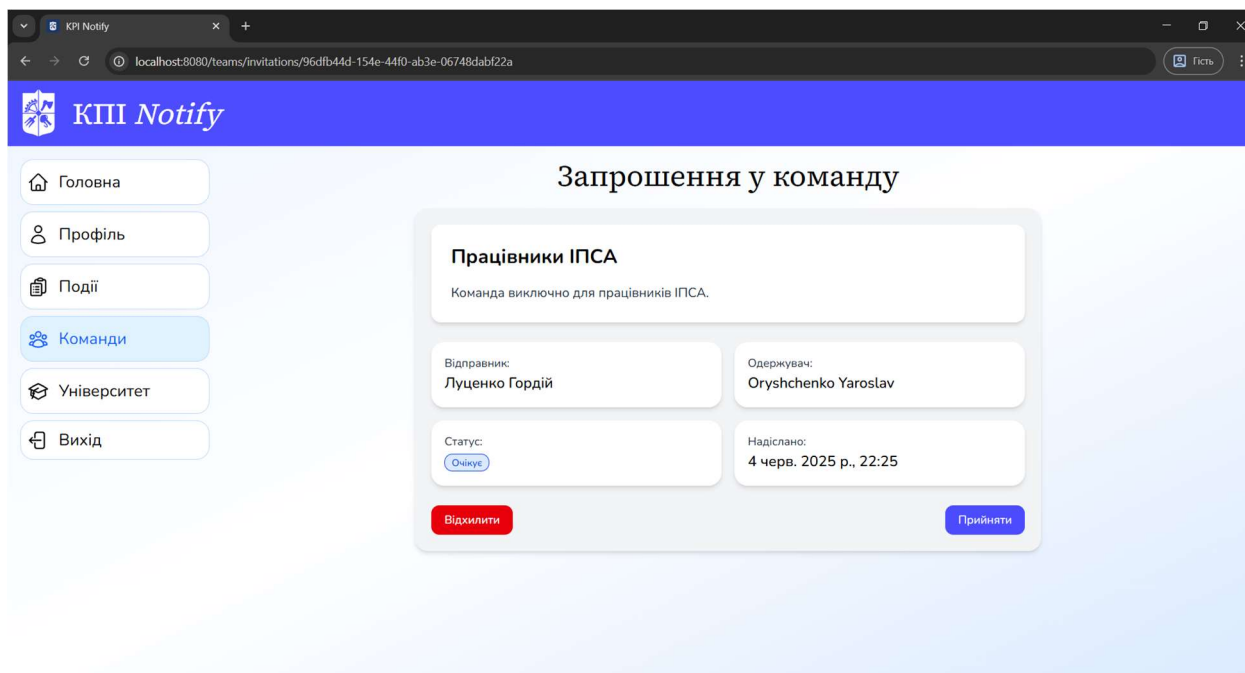


Рисунок 3.37 – Сторінка із інформацією про запрошення у команду для отримувача запрошення

Натиснувши на кнопку «Перейти» біля потрібної команди, користувач потрапить на сторінку команди (рисунок 3.38). Члени команди можуть переглядати події, створені лідером команди для її учасників.

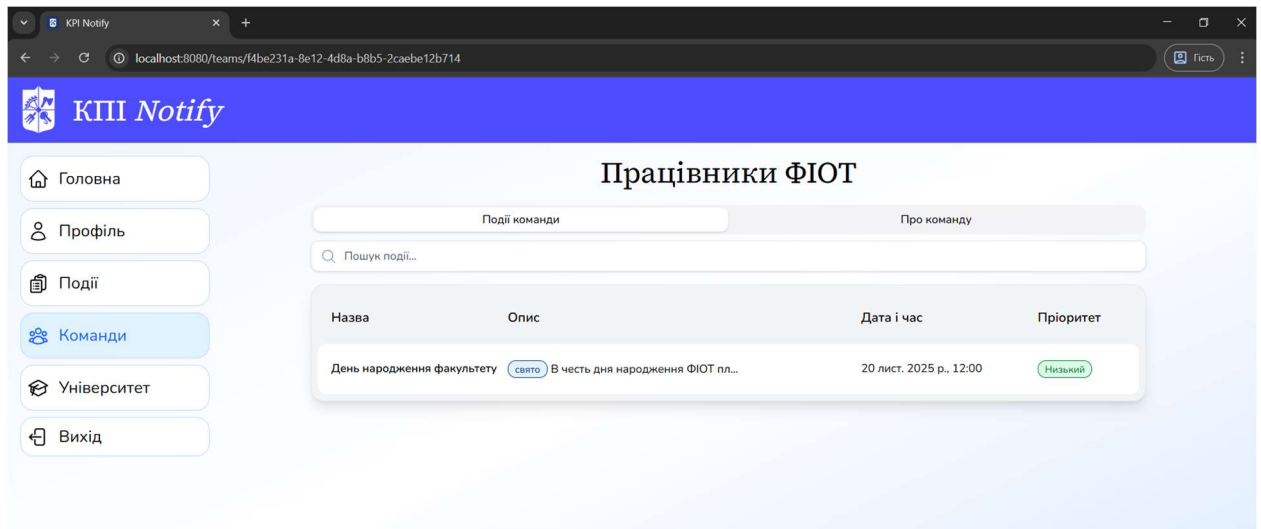


Рисунок 3.38 – Сторінка подій команди

У вкладці «Про команду» користувач може переглянути інформацію про команду та її учасників (рисунок 3.39). Користувач може покинути команду, натиснувши на кнопку «Покинути команду».

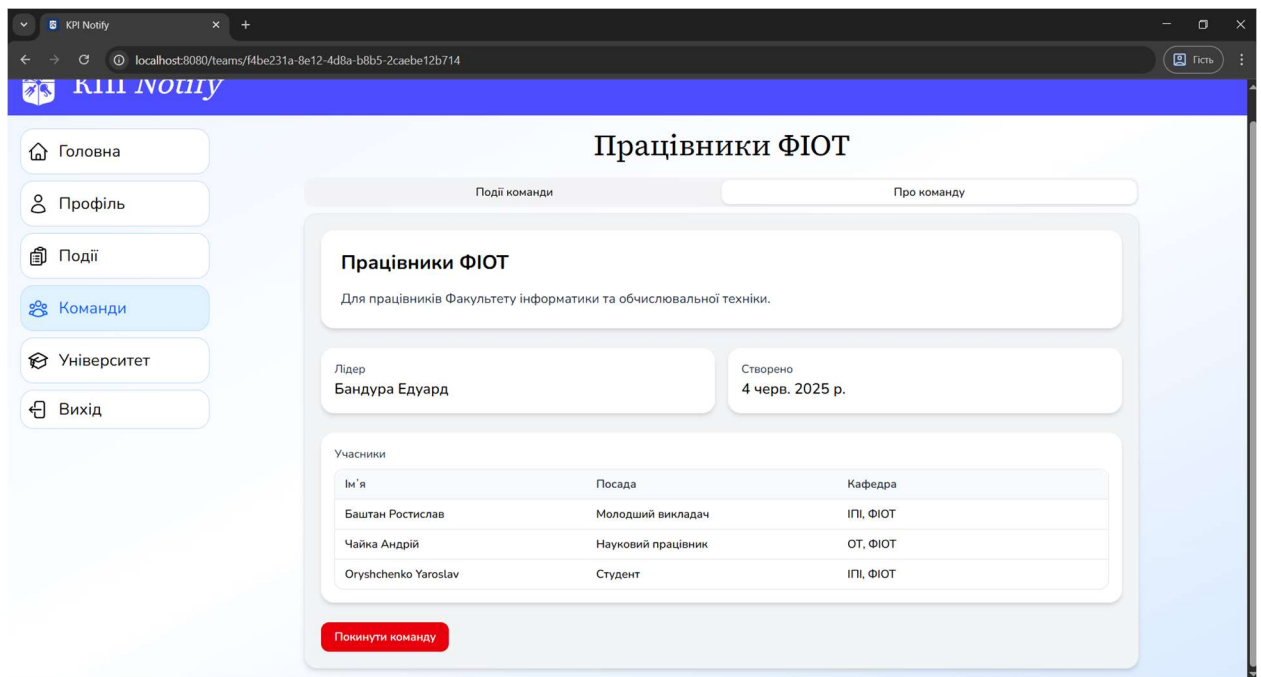


Рисунок 3.39 – Сторінка із інформацією про команду

Якщо користувач є лідером команди, то йому доступні додаткові опції для управління командою (рисунок 3.40).

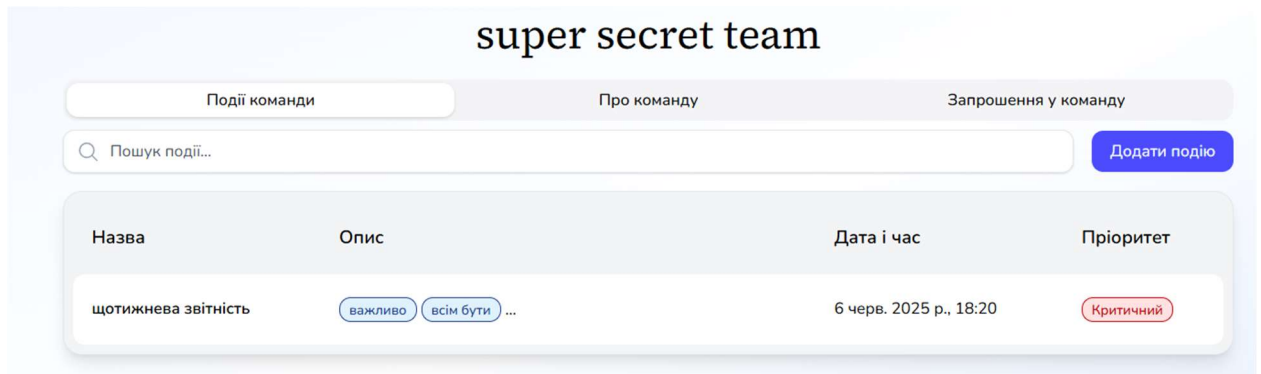


Рисунок 3.40 – Сторінка команди для лідера

Користувач може створити подію для членів команди, натиснувши на кнопку «Додати подію» (рисунок 3.41). Окрім стандартних полів для створення події, лідер команди може обрати яким підлеглим він бажає призначити цю подію.

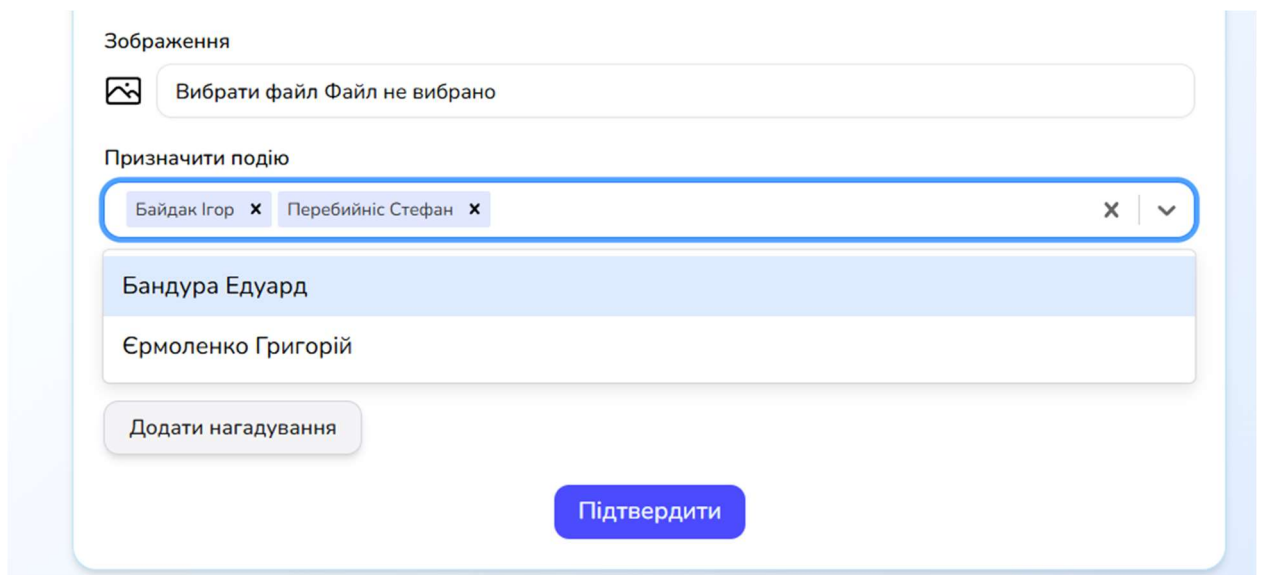


Рисунок 3.41 – Створення командної події

У вкладці «Про команду», лідер має можливість видаляти її членів, змінити інформацію про команду або видалити її (рисунок 3.42).

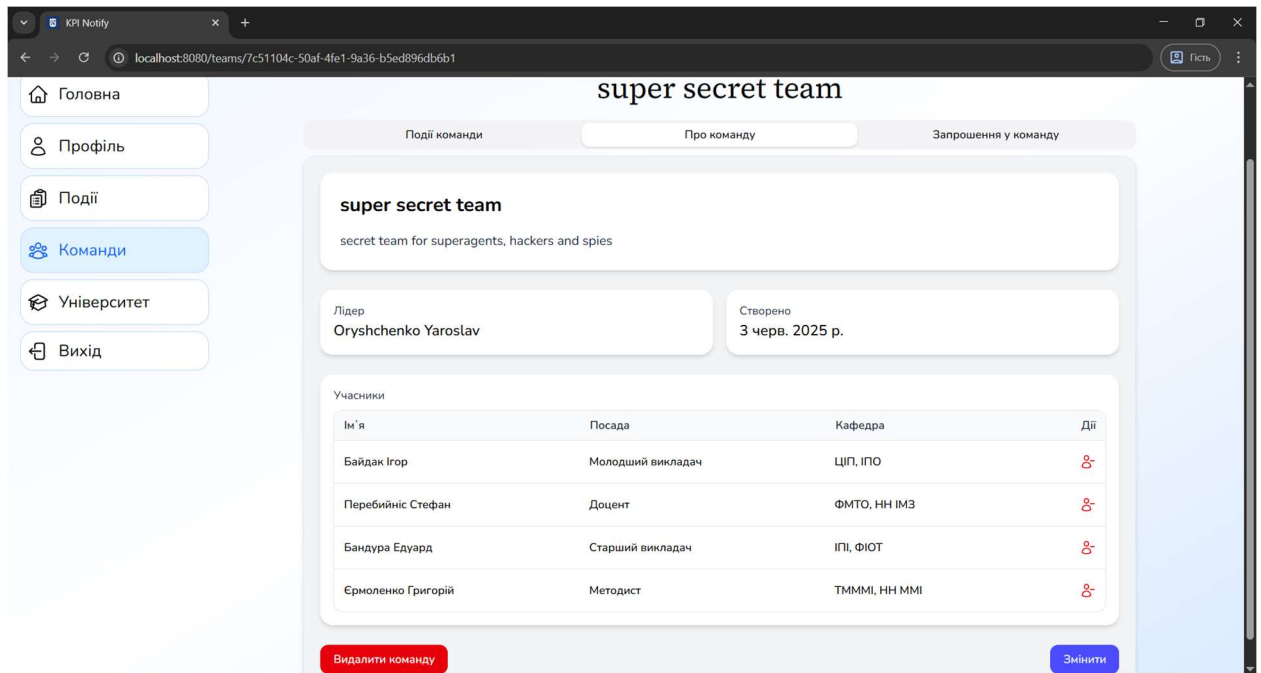


Рисунок 3.42 – Інформація про команду для лідера команди

Натиснувши на іконку біля інформації про учасника команди, з'явиться вікно для підтвердження видалення учасника (рисунок 3.43). Підтвердити дію можна натиснувши на кнопку «Вигнати», після чого обраний користувач перестане бути учасником команди.

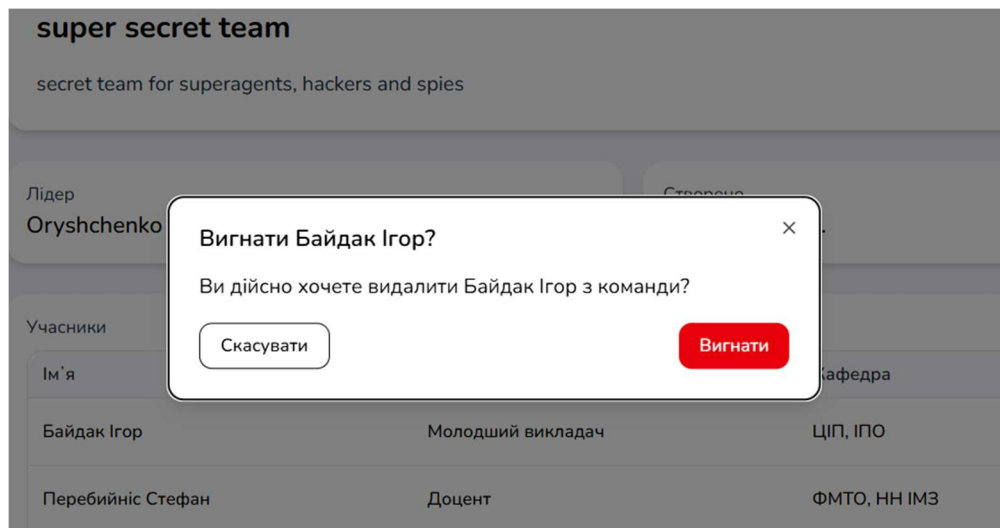


Рисунок 3.43 – Вікно підтвердження видалення учасника команди

Лідер може змінити назву або опис команди, натиснувши на кнопку «Змінити», після чого буде завантажена форма для модифікації інформації про команду (рисунок 3.44).

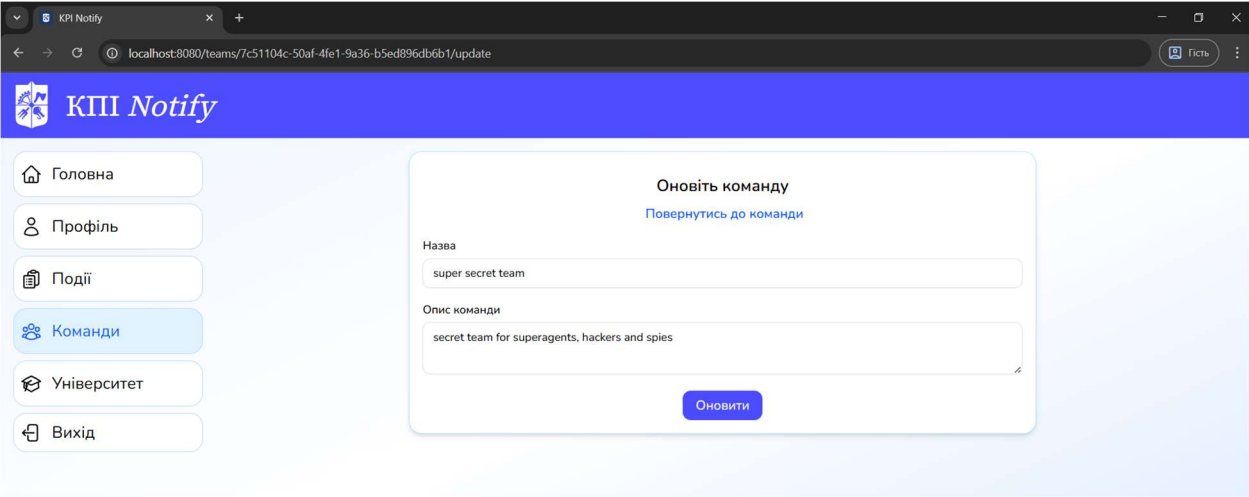


Рисунок 3.44 – Сторінка зміни інформації про команду

У вкладці «Запрошення у команду», лідер має можливість переглянути список створених ним запрошень у цю команду (рисунок 3.45). Натиснувши на запрошення, користувач може переглянути інформацію про нього, а також видалити запрошення, натиснувши на кнопку «Видалити запрошення» (рисунок 3.46).

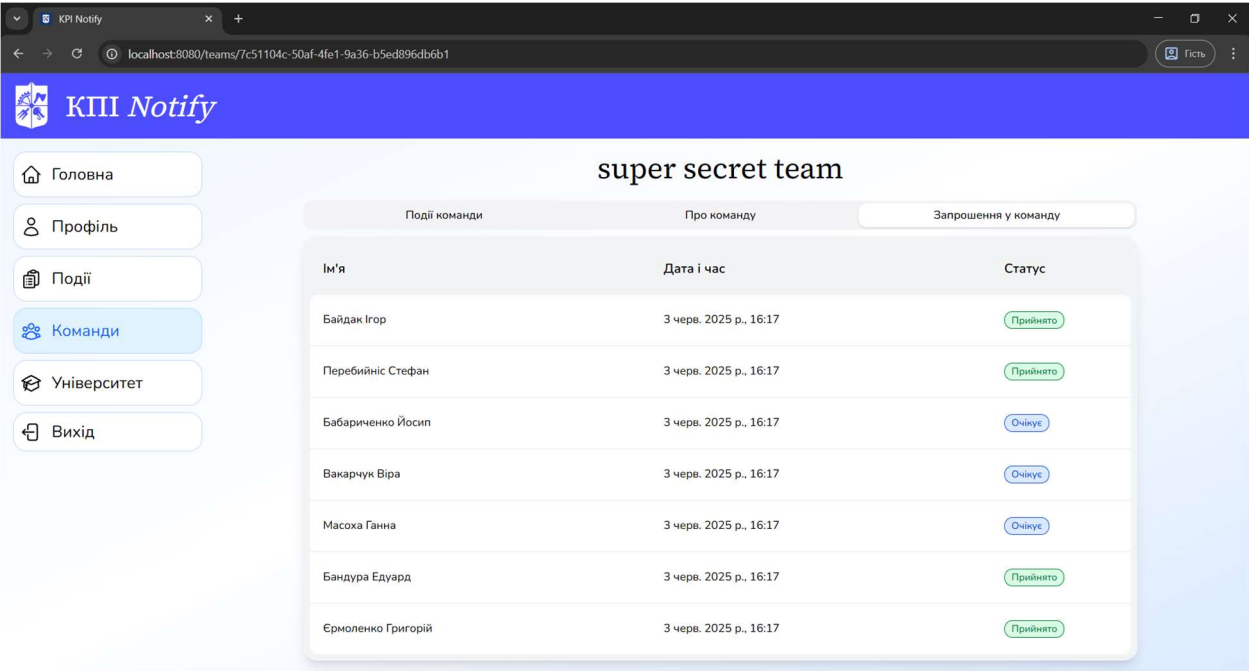


Рисунок 3.45 – Створені лідером команди запрошення

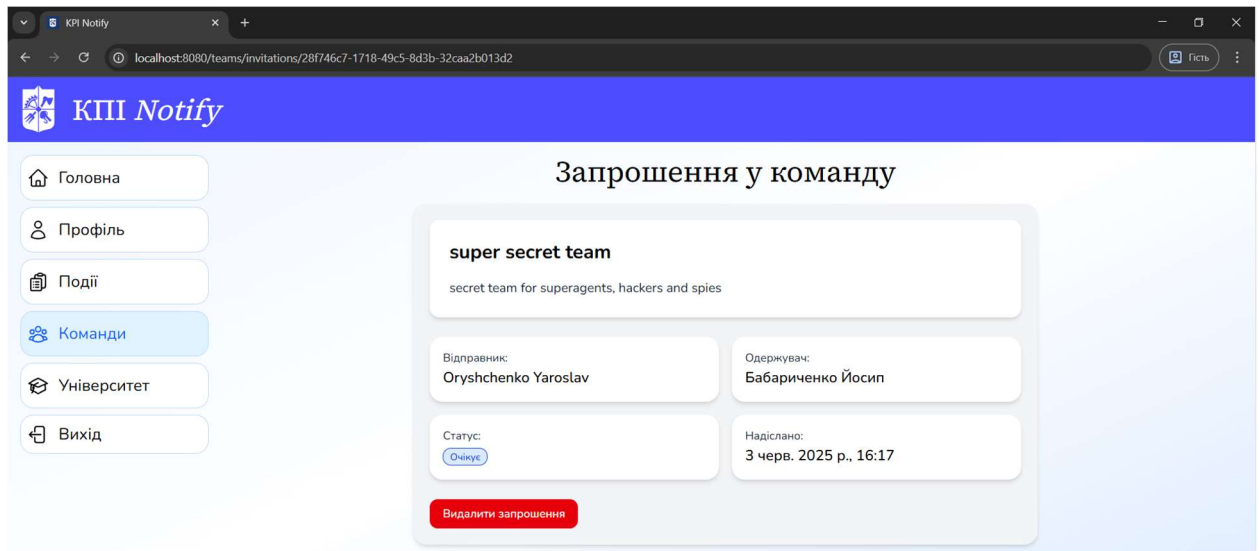


Рисунок 3.46 – Сторінка інформації про запрошення для автора запрошення

Після натиснення на кнопку «Створити команду» на сторінці із списком команд, користувача буде направлено на сторінку із формою для створення нової команди, що містить поля «Назва», «Опис команди» та «Учасники» (рисунок 3.47).

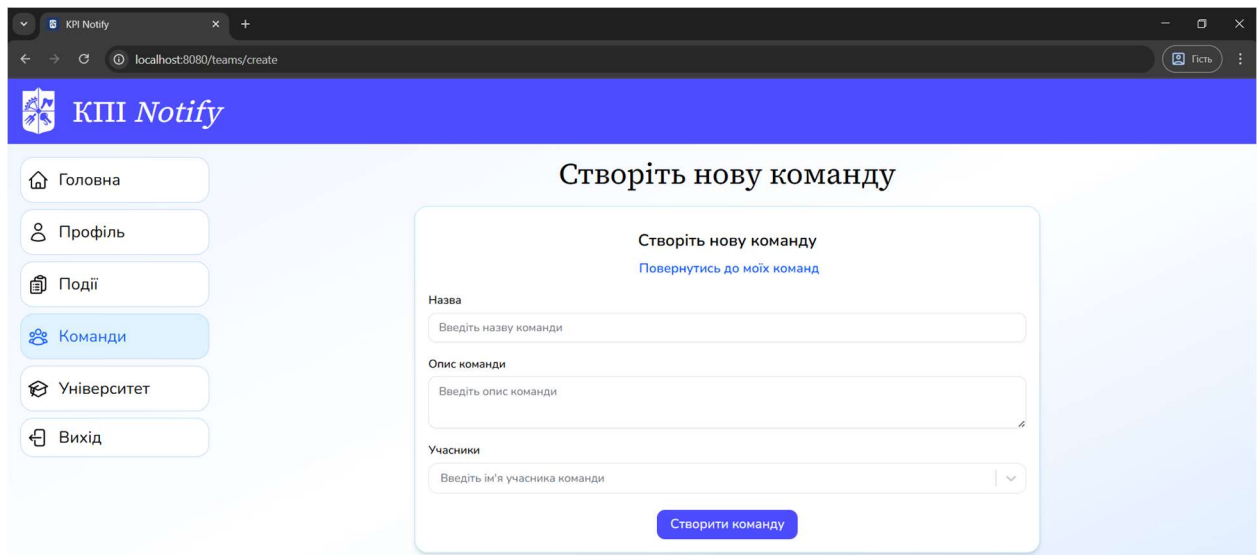


Рисунок 3.47 – Сторінка створення нової команди

Користувач може обрати людей, яких він бажає запросити у цю команду у полі «Учасники» (рисунок 3.48). Поле підтримує пошук, а також вибір багатьох людей одночасно.

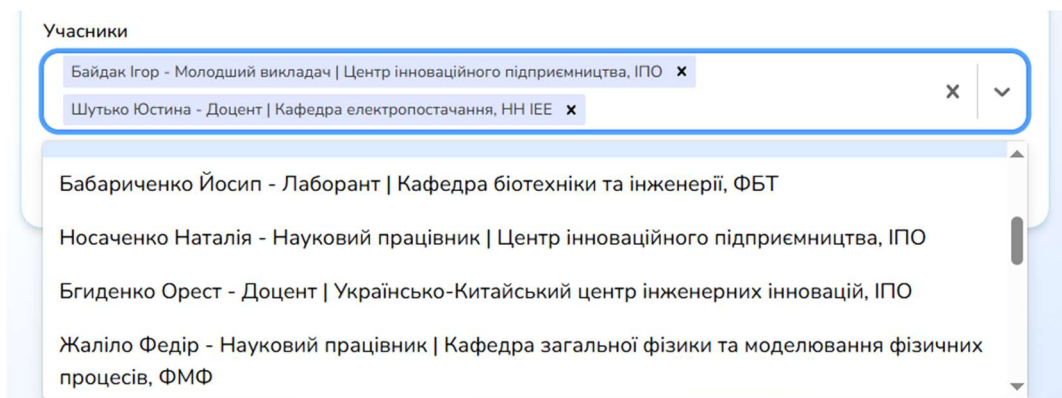


Рисунок 3.48 – Поле вибору учасників команди

Коректно заповнивши усі поля та натиснувши кнопку «Створити команду», користувач отримає повідомлення про успішне створення команди, та буде направлений на сторінку із інформацією про команду. Усі обрані користувачі у полі «Учасники» отримають запрошення із можливістю прийняти його або відмовитись, а також лист на електронну адресу про те, що їх було запрошено у команду (рисунок 3.49).

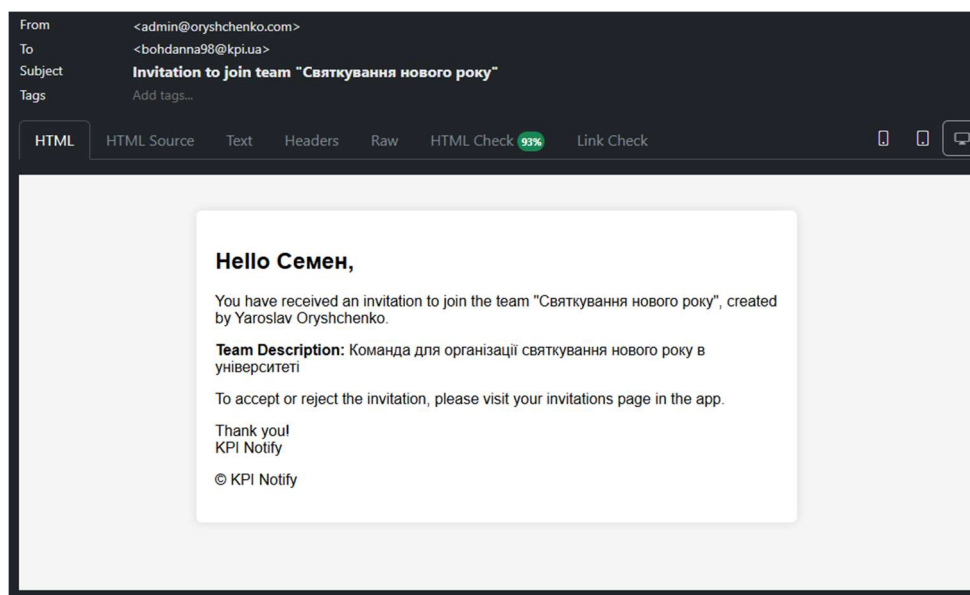


Рисунок 3.49 – Лист про отримане запрошення у команду

Натиснувши на кнопку «Університет» у навігаційному меню, користувач потрапить на сторінку із списком усіх інститутів та факультетів університету (рисунок 3.50). У списку можна побачити інформацію про

підрозділ університету, а також кількість користувачів, які працюють у підрозділі і користуються застосунком КПІ Notify.

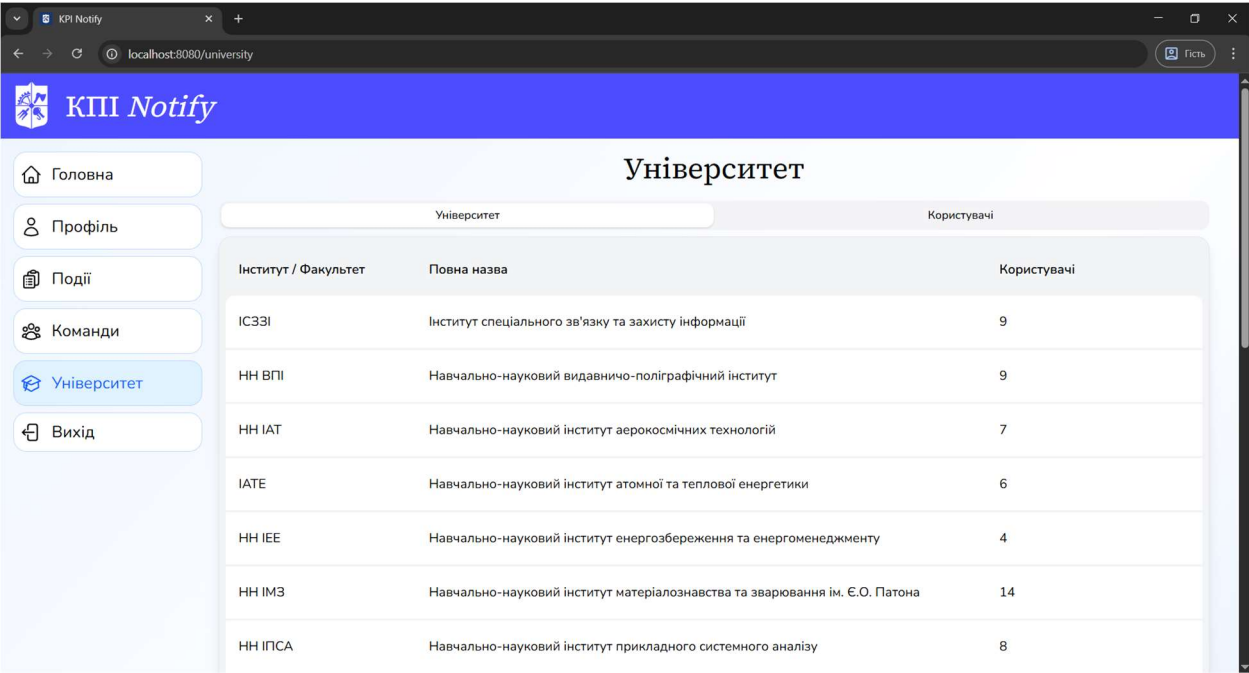


Рисунок 3.50 – Сторінка із підрозділами університету

У вкладці «Користувачі» можна переглянути список усіх користувачів застосунку (рисунок 3.51). Список розділений на сторінки, кожна з яких містить по 20 осіб. Можна виконувати пошук користувачів за їхніми іменами або посадами.

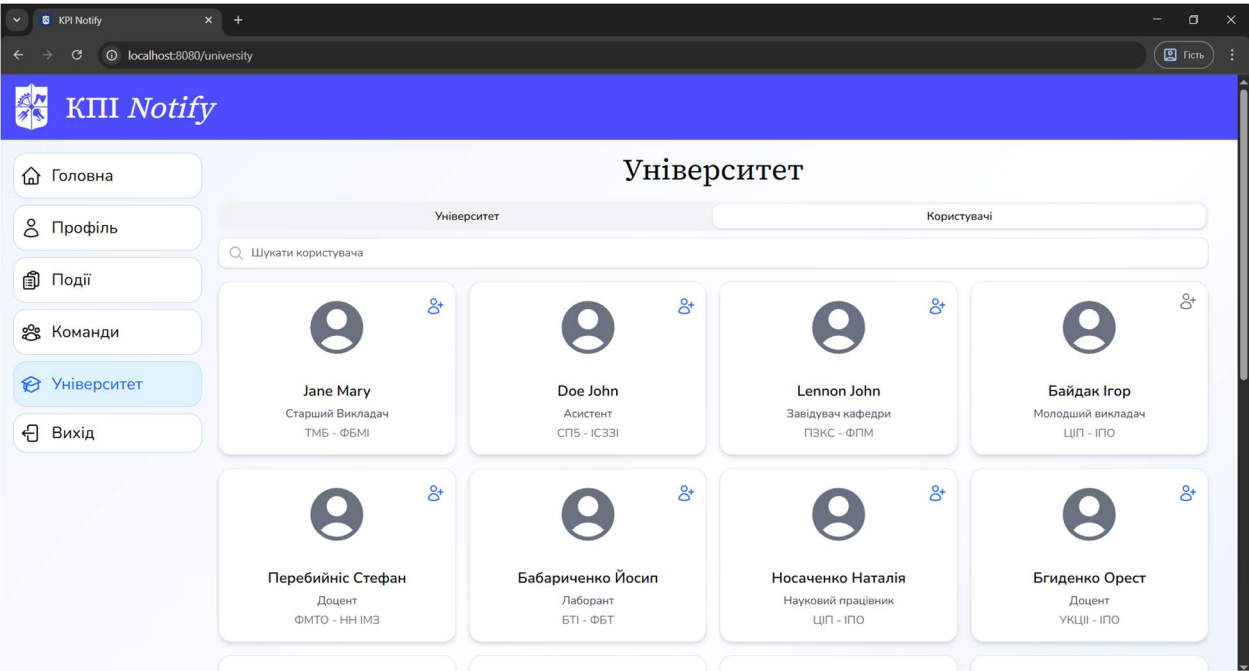


Рисунок 3.51 – Список усіх користувачів

У вкладці «Університет», натиснувши на бажаний інститут чи факультет, користувач може перейти на сторінку обраного підрозділу, де буде відображено список кафедр, а у вкладці «Користувачі» будуть лише ті, хто є працівником обраного підрозділу (рисунки 3.52-3.53).

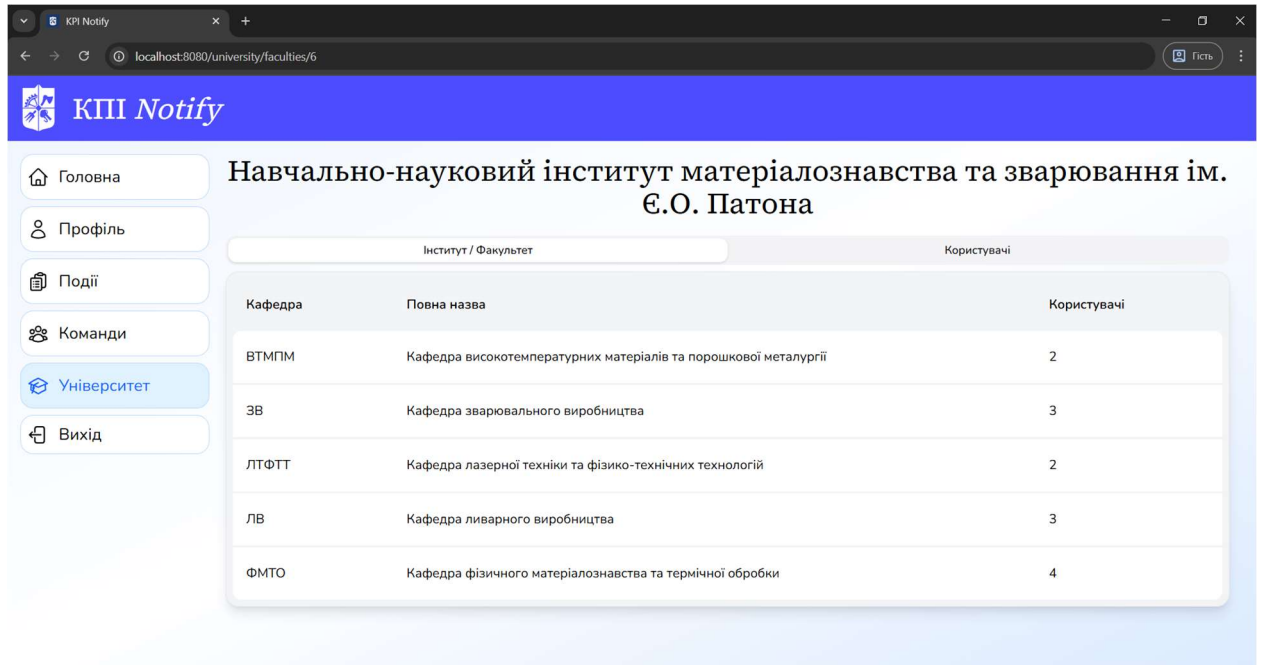


Рисунок 3.52 – Сторінка кафедр обраного підрозділу університету

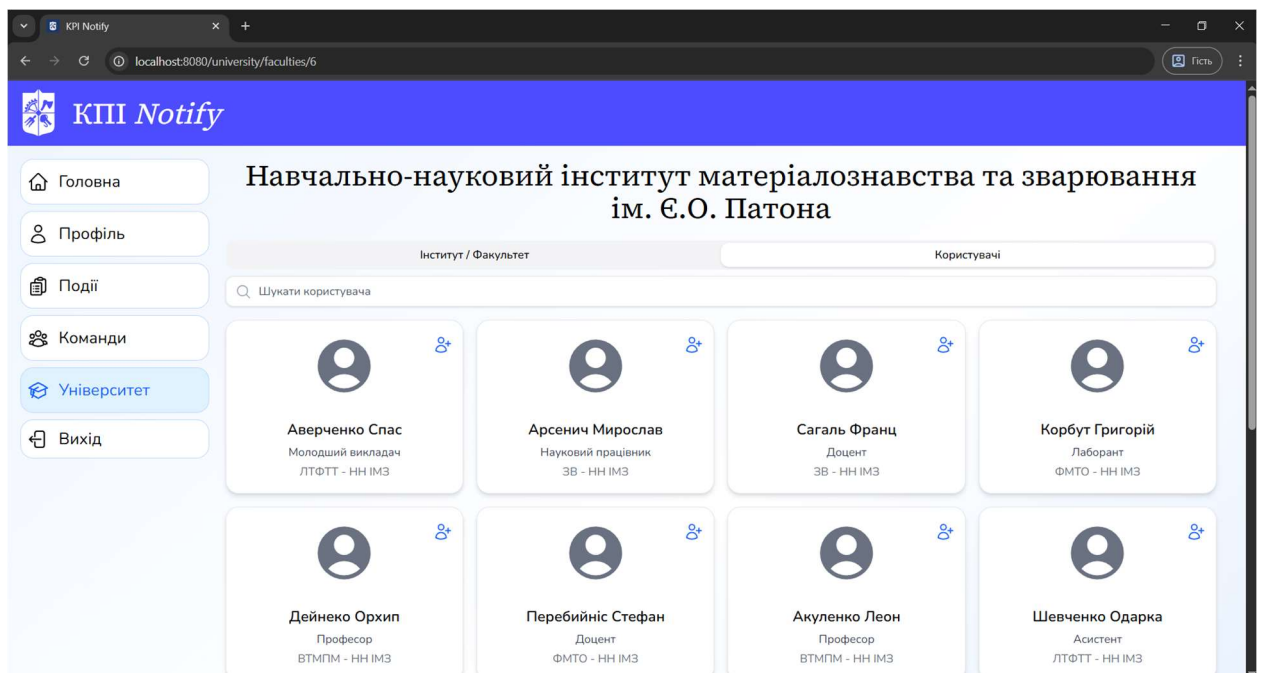


Рисунок 3.53 – Сторінка працівників обраного підрозділу університету

Натиснувши на кафедру обраного інституту чи факультету, можна переглянути користувачів, які є працівниками обраної кафедри (рисунок 3.54).

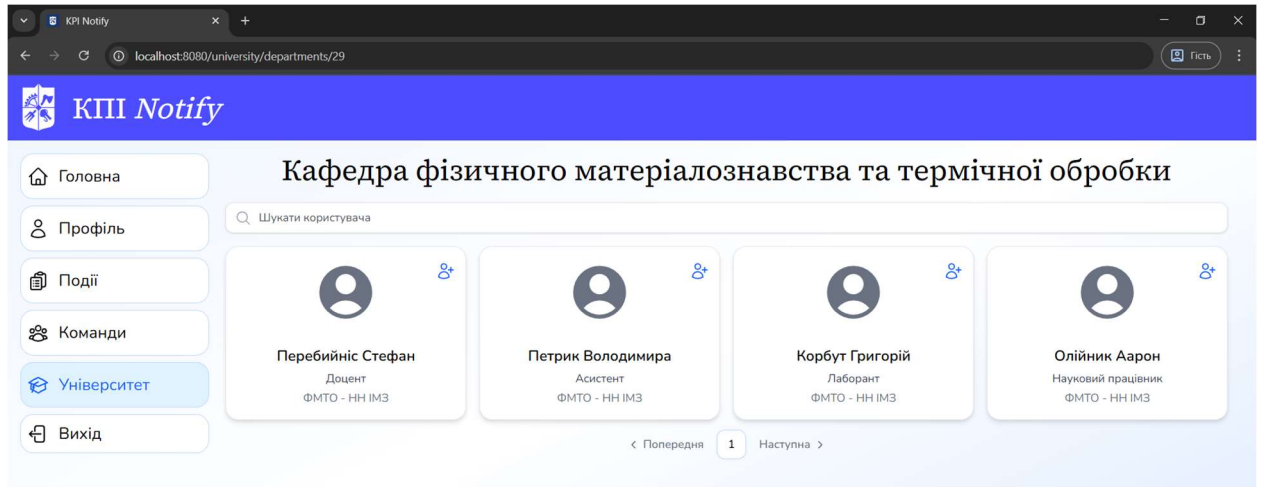


Рисунок 3.54 – Сторінка працівників обраної кафедри

Біля кожного користувача є синя кнопка, яка дозволяє запросити його у команди, лідерами яких є поточний користувач. Натиснувши на кнопку, відкриється вікно із вибором команди, у яку користувач бажає запросити обрану людину (рисунок 3.55). Обравши команду та натиснувши на кнопку «Запросити», буде створено запрошення для обраного користувача, а на його електронну адресу буде надіслано відповідне повідомлення.

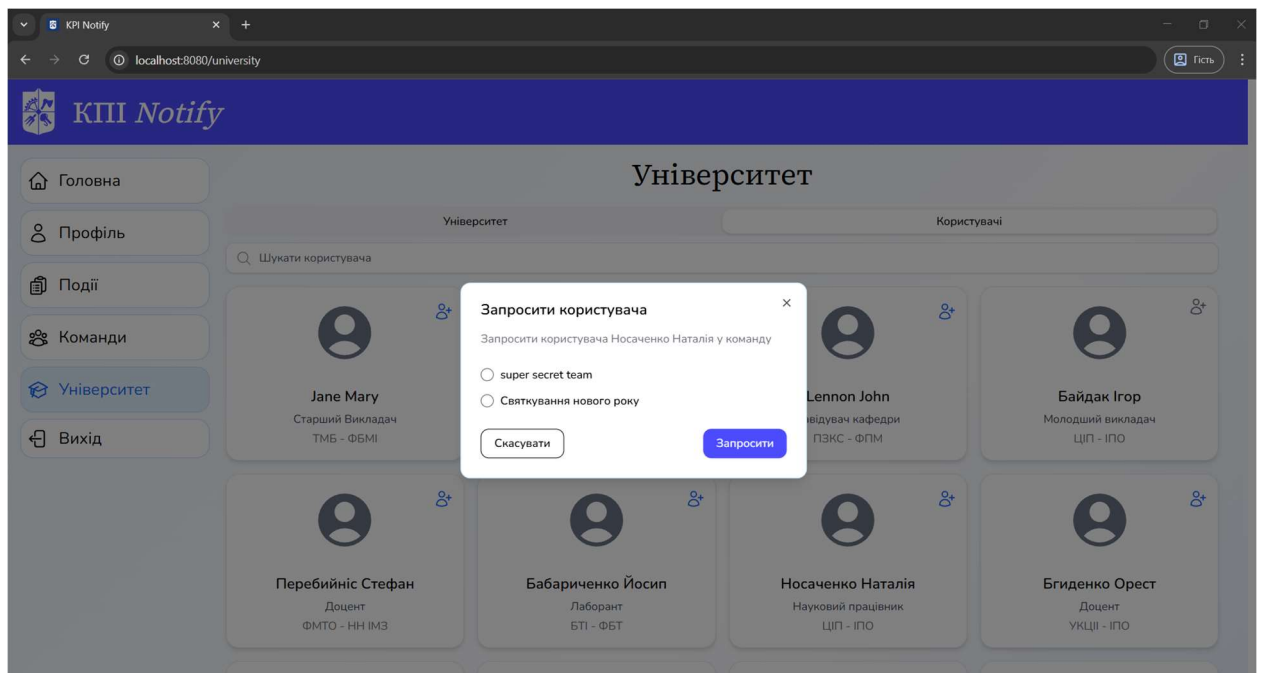


Рисунок 3.55 – Створення запрошення у команду із списку користувачів

Якщо користувач не є лідером жодної команди, або обрана людина вже є членом усіх команд користувача, то кнопка стане недоступною для натискання, а її колір зміниться на сірий (рисунок 3.56).

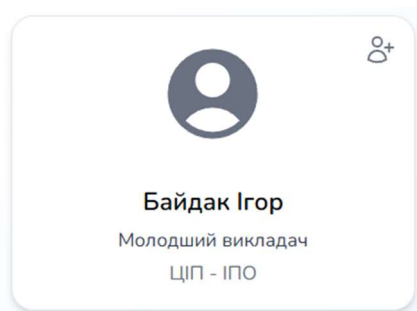


Рисунок 3.56 – Картка користувача із недоступною кнопкою запрошення у команду

Для отримання нагадувань через Telegram-бота, користувачеві необхідно пройти авторизацію за допомогою свого Telegram-акаунта. На сторінці деталей профілю користувача, необхідно натиснути на кнопку «Увійти через Telegram», після чого з’явиться вікно авторизації Telegram (рисунок 3.57).

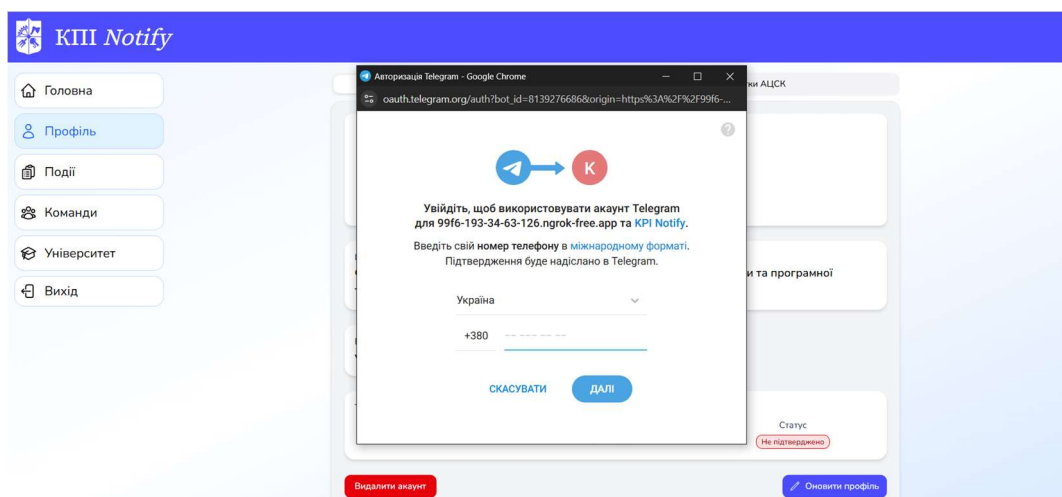


Рисунок 3.57 – Вікно авторизації Telegram-акаунту

Після введення номеру телефону та натиснення кнопки «Далі», Акаунт із вказаним номером телефону отримує повідомлення від Telegram для

підтвердження авторизації (рисунок 3.58), де необхідно натиснути «Підтвердити».

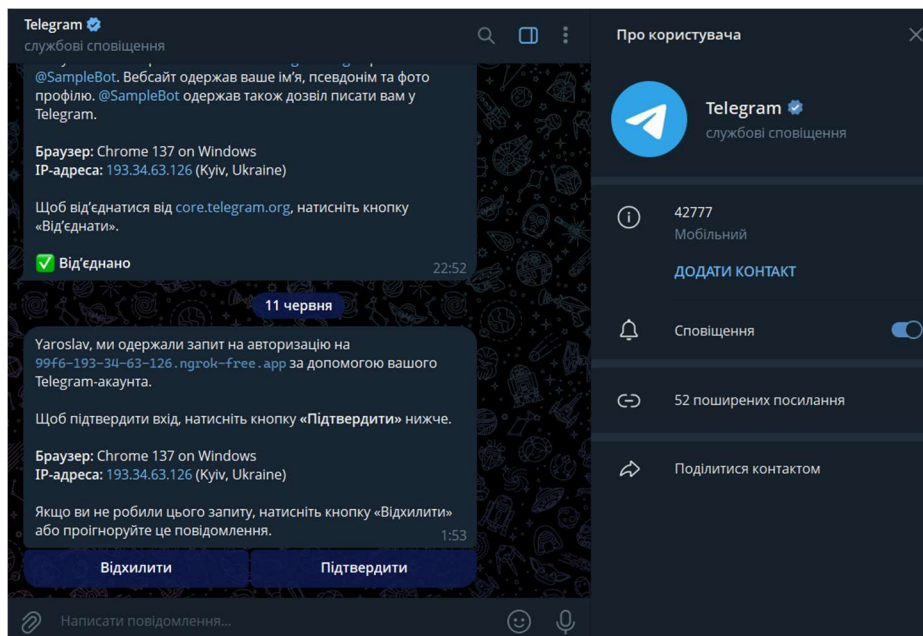


Рисунок 3.58 – Підтвердження авторизації через Telegram

У оновленому вікні авторизації Telegram зв'явиться інформація про необхідність надання даних свого Telegram-акаунту застосунку KPI Notify(рисунок 3.59).

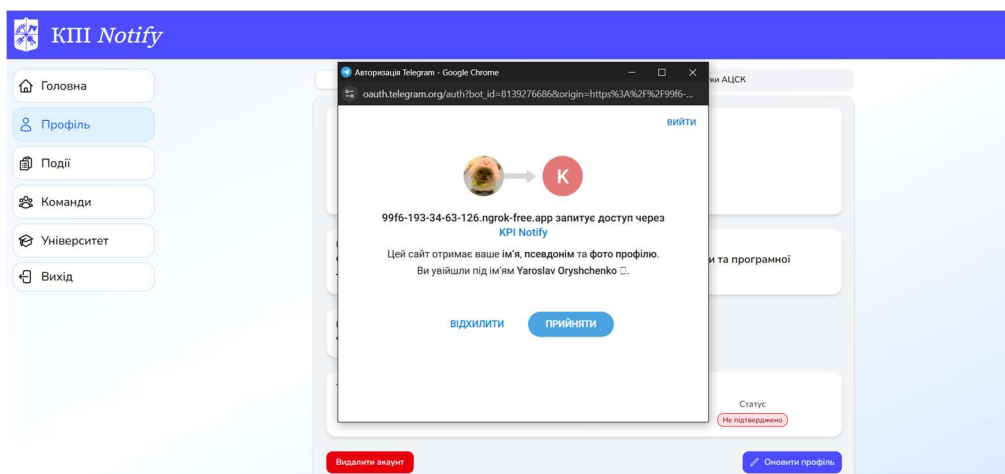


Рисунок 3.59 – Прив'язка Telegram-акаунту до застосунку

Після натиснення кнопки «Прийняти», Користувач отримує повідомлення про успішну авторизацію від Telegram (рисунок 3.60), а також від бота KPI Notify (рисунок 3.61).

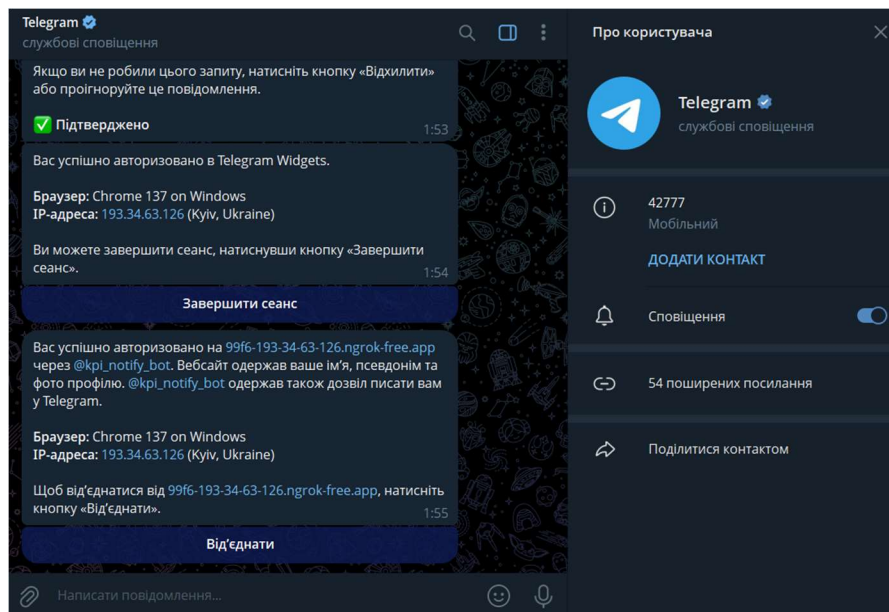


Рисунок 3.60 – Повідомлення про успішну авторизацію від Telegram

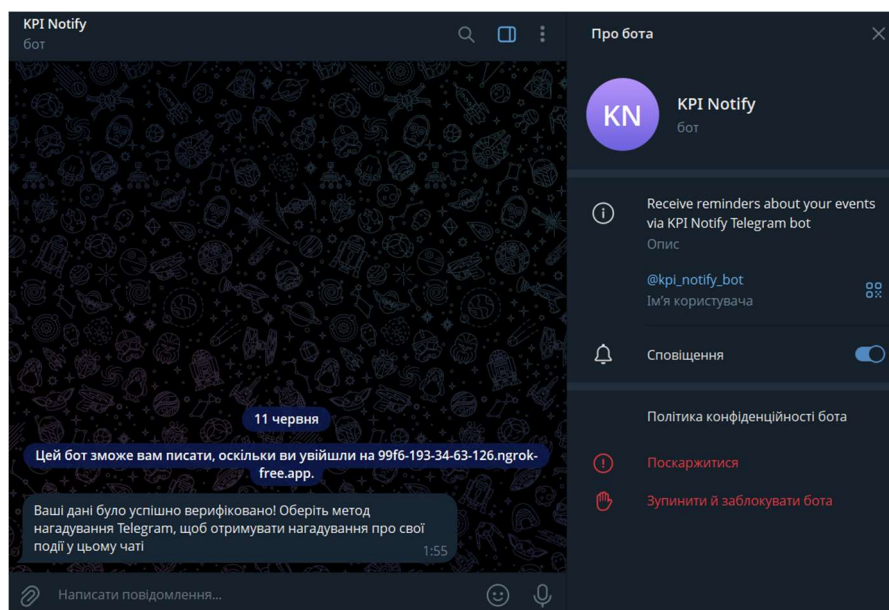


Рисунок 3.61 – Верифікація у Telegram-боті через номер телефону

У разі успішної авторизації, статус Telegram у профілі користувача буде змінено із «Не підтверджено» на «Підтверджено», також з'явиться інформація про ім'я користувача Telegram (рисунок 3.62).

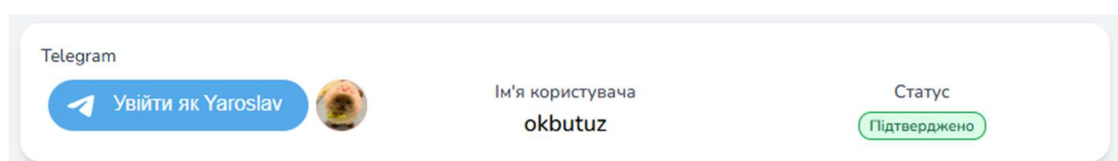


Рисунок 3.62 – Верифікований Telegram-акаунт користувача

Після верифікації акаунту Telegram, користувач може отримувати нагадування через Telegram-бота @kpi_notify_bot. У обраний час, Telegram-бот надішле у чат повідомлення із інформацією про подію (рисунок 3.63).

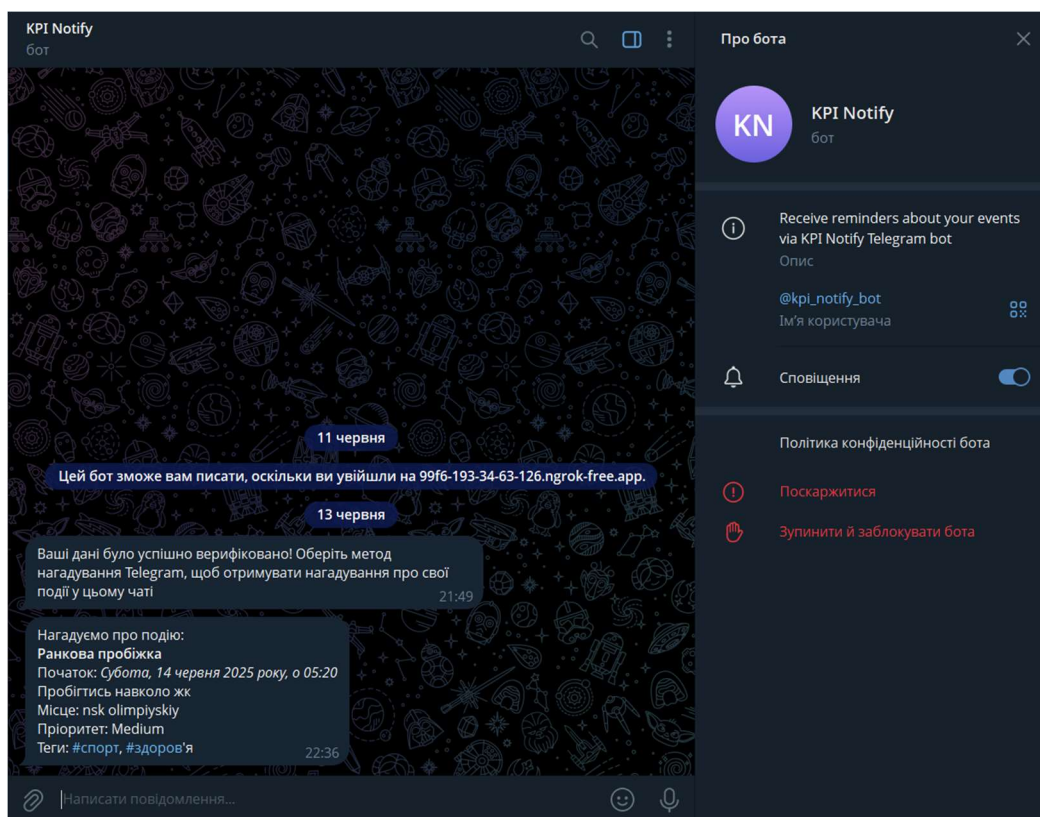


Рисунок 3.63 – Нагадування через Telegram-бота

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2025 р.

**ВЕБЗАСТОСУНОК ДЛЯ ПЛАНУВАННЯ ТА НАГАДУВАННЯ ПРО
ВАЖЛИВІ ПОДІЇ
Графічний матеріал
КПІ.ІП-1219.045440.06.99**

“ПОГОДЖЕНО”

Керівник проєкту:

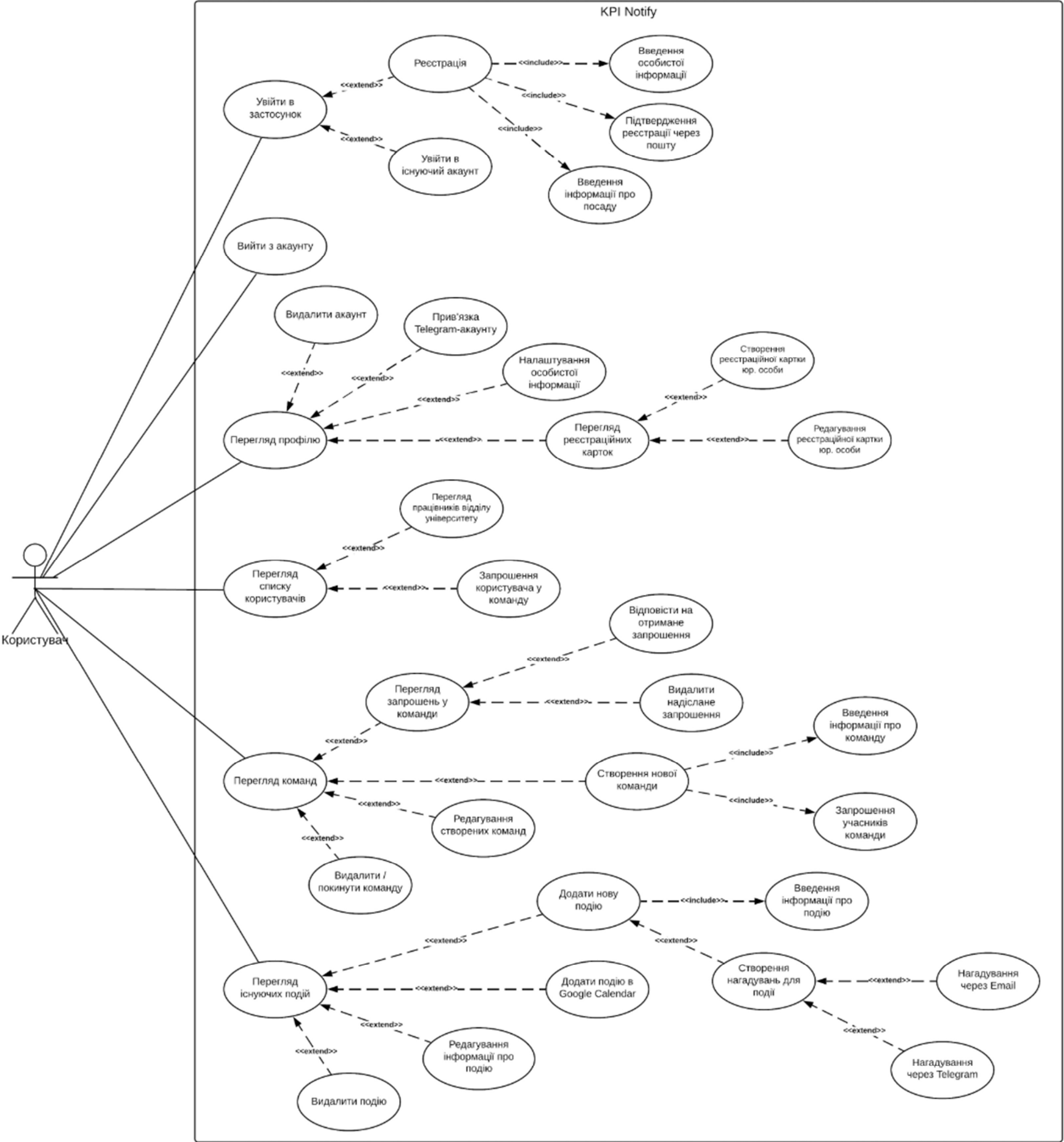
_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

Максим ГОЛОВЧЕНКО

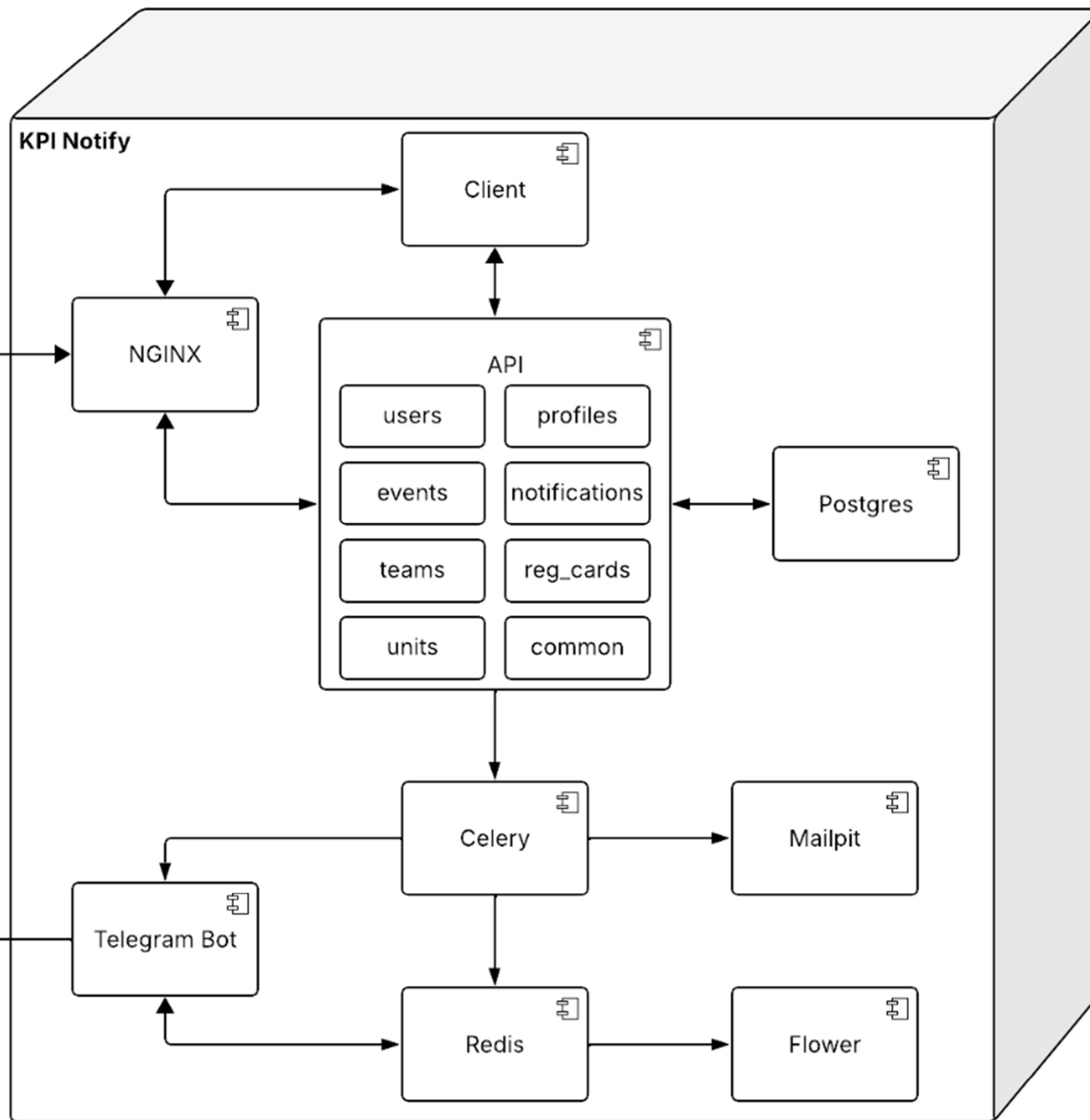
Виконавець:

Ярослав ОРИЩЕНКО

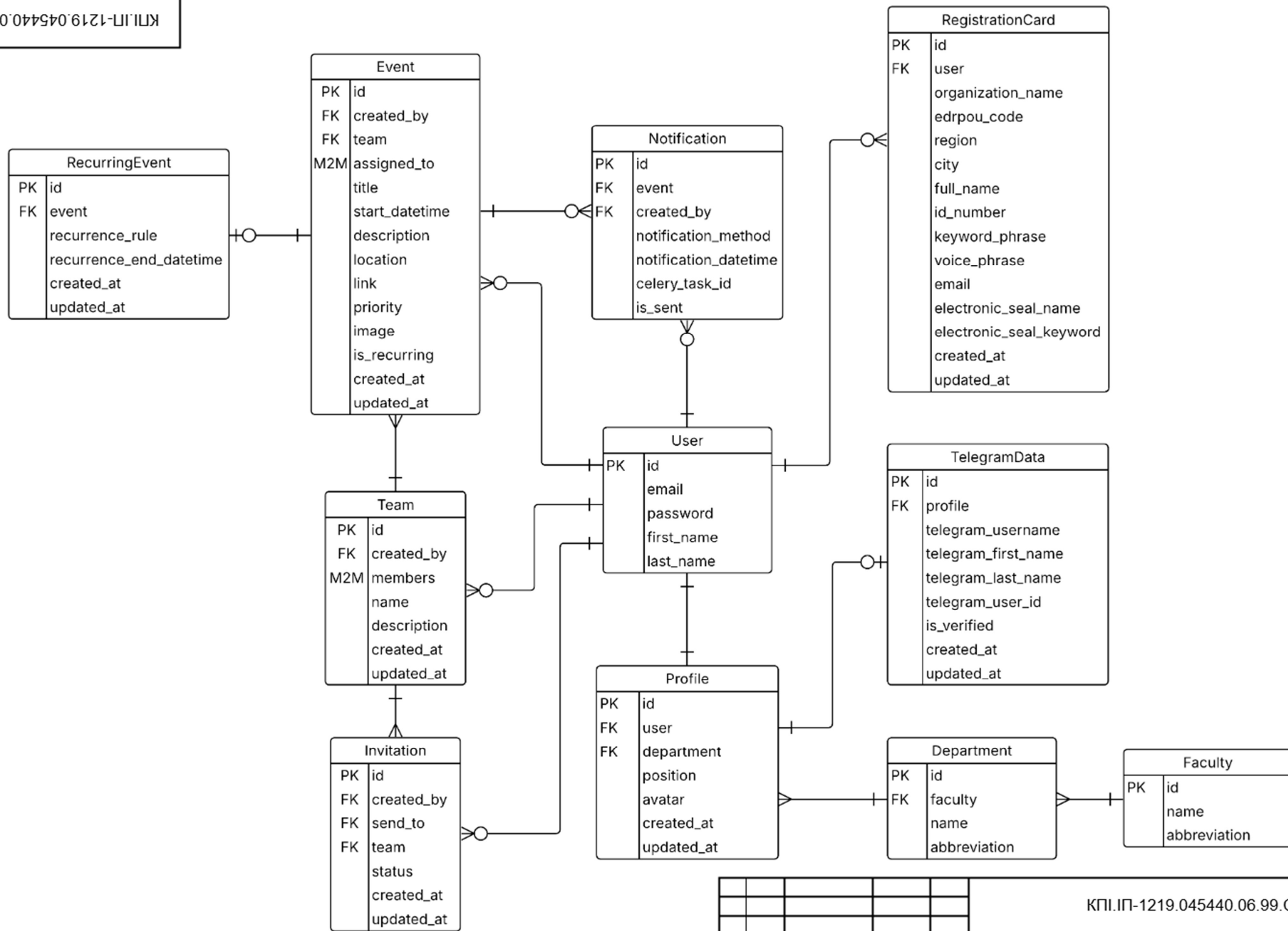


					КПІ.ІП-1219.045440.06.99.ССВ			
					Схема структурна варіантів використання			
Зм.	Арк.	№ докум.	Підп.	Дата				
Розробив		Орищенко Я.О.						
Перевішив		Ковтунець О.В.						
Т. контр.								
					Вебзастосунок для планування та нагадування про важливі події			
Н. контр.		Головченко М.М.						
Затвердив		Жаріков Е.В.						
					Лит. Маса Масштаб			
					Аркуш 1 Аркушів 1			
					КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-12			

Користувач



					КПІ.ІП-1219.045440.06.99.CCM				
					Схема структурна компонентів програмного забезпечення	Літера		Маса	Масштаб
Зм.	Арк.	№ документа	Підпис	Дата					
Розробив		Орищенко Я.О.							
Перевірів		Ковтунець О.В.							
Т. контр.									
						Аркуш 1		Аркушів 1	
Н. контр.		Головченко М.М.			Вебзастосунок для планування та нагадування про важливі події	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-12			
Затвердив		Жаріков Е.В.							



					КПІ.ІП-1219.045440.06.99.СБД			
Зм.	Арк.	№ документа	Підпис	Дата	Схема бази даних			
Розробив		Орищенко Я.О.						
Перевірив		Ковтунець О.В.						
Т. контр.								
Н. контр.		Головченко М.М.						
Затвердив		Жаріков Е.В.			Вебзастосунок для планування та нагадування про важливі події			
						Літера	Маса	Масштаб
						Аркуш 1		Аркушів 1
						КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-12		